

TECHNICAL LEARNING GUIDE

Kubernetes & OpenShift

From Zero to Production

A Practical Learning Guide for Software Engineers

Learn the why, architecture, internals, configuration, troubleshooting, and production thinking behind Kubernetes and OpenShift.

**Kubernetes | OpenShift | GitOps | Production
Troubleshooting**

Problem-first learning • Java/Spring Boot examples • Visual mental models

2026 Edition

About This Book

This book follows a problem-first philosophy. Before asking what YAML should I write?, it asks why does this concept exist? The explanations began as a progressive learning journey and have been reconstructed into a continuous technical guide. Repeated discussions are consolidated, genuine beginner questions are retained as learning prompts, and later topics are placed after the concepts on which they depend.

The recurring example is a Java/Spring Boot microservice system containing an API Gateway, Employee Service, Payment Service, Kafka, and a database. The same application evolves across workload management, networking, configuration, storage, reliability, security, OpenShift, GitOps, observability, and troubleshooting.

The learning path is:

```
Physical server → Virtual machine → Container → Kubernetes  
→ Production operations → OpenShift → GitOps and observability
```

The intended outcome is understanding, application, and production thinking—not memorizing isolated definitions.

Contents

About This Book	1
Why Kubernetes Exists	5
Chapter 1 — From Physical Servers to Kubernetes	6
Kubernetes Foundations	39
Chapter 2 — Kubernetes Big Picture	40
Chapter 3 — What Happens Inside Kubernetes	70
Workloads and Application Lifecycle	95
Chapter 4 — Pods	96
Chapter 5 — ReplicaSets, Labels, Selectors, and Deployments	137
Chapter 6 — Jobs and CronJobs	174
Kubernetes Networking	196
Chapter 7 — Networking Fundamentals and Pod Networking	197
Chapter 8 — Services, DNS, and Service Discovery	230
Chapter 9 — Ingress, Ingress Controllers, and OpenShift Routes	266
Configuration, Storage, and Stateful Systems	297
Chapter 10 — ConfigMaps and Secrets	298
Chapter 11 — Volumes, Persistent Volumes, Claims, and StorageClasses	326
Chapter 12 — Storage Architecture in Practice	355
Chapter 13 — StatefulSets and DaemonSets	382

Health, Resources, Scaling, and Scheduling	411
Chapter 14 — Startup, Liveness, and Readiness Probes	412
Chapter 15 — Graceful Shutdown and Traffic Draining	441
Chapter 16 — CPU, Memory, Requests, Limits, and QoS	469
Chapter 17 — Namespaces, LimitRange, and ResourceQuota	501
Chapter 18 — Horizontal Pod Autoscaling	525
Chapter 19 — Kubernetes Scheduling	541
Security and Multi-Tenancy	569
Chapter 20 — RBAC and Service Accounts	570
Chapter 21 — SecurityContext and Pod Security	591
Packaging, Customization, and GitOps	619
Chapter 22 — Helm	620
Chapter 23 — Kustomize	653
Chapter 24 — GitOps and Argo CD	673
OpenShift	703
Chapter 25 — OpenShift Architecture	704
Chapter 26 — Projects, Routes, Builds, and ImageStreams	728
Chapter 27 — OpenShift Security Context Constraints	743
Chapter 28 — Operators, CRDs, and OLM	763
Observability and Production Troubleshooting	765
Chapter 28 — Monitoring with Prometheus and Grafana	766
Chapter 29 — Centralized Logging and Distributed Tracing	795
Chapter 30 — Observability and Troubleshooting Foundations	820
Chapter 31 — Advanced Production Troubleshooting	864

Availability and Integrated Production Architecture	893
Chapter 32 — High Availability, Upgrades, and Disaster Recovery	894
Chapter 33 — Complete Production Architecture	922
Kubernetes & OpenShift Interview Guide	925
Kubernetes & OpenShift Final Revision Guide	927
Glossary	929
Source Learning Map	931

Why Kubernetes Exists

Chapter 1 — From Physical Servers to Kubernetes

CHAPTER FOCUS

Understand **From Physical Servers to Kubernetes** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

We'll start before Kubernetes, because Kubernetes makes much more sense once you understand the problem it was created to solve.

By the end of this lesson, you should understand this evolution:

```
Application
↓
Physical Server
↓
Virtual Machine
↓
Container
↓
Docker / Container Runtime
↓
Many Containers
↓
Operational Problems
↓
Kubernetes
```

Start with the simplest possible application

Imagine you build a Java Spring Boot application:

```
Employee Service
```

```
GET /employees
POST /employees
```

Your code might ultimately become:

```
employee-service.jar
```

To run it, you need an environment containing things such as:

```
Operating System
  ↓
Java/JVM
  ↓
Application dependencies
  ↓
employee-service.jar
```

For example:

```
java -jar employee-service.jar
```

The application starts:

```
Spring Boot Application
  ↓
Port 8080
```

If your laptop is running the application:

```
Your Laptop
-
├─ macOS
├─ Java 21
├─ employee-service.jar
  ↓
  :8080
```

This is easy.

But your laptop obviously isn't where a production banking, e-commerce, or enterprise application should run.

So historically, companies used servers.

Physical Servers

A physical server is simply a powerful computer.

Imagine a machine in a data center:

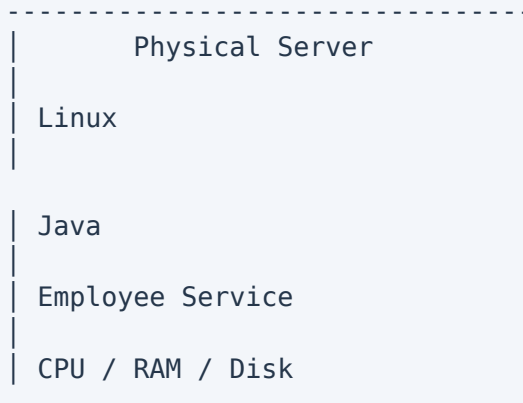
```
Physical Server

CPU:      32 cores
RAM:      128 GB
Storage:  4 TB
Network:  10 Gbps
```

You install:

```
Linux
Your application
```


Architecture:



This works.

But suppose your company has:

Employee Service
Payment Service
Order Service
Notification Service
Inventory Service

You could run all of them on one server:

```
Physical Server
-
├─ Employee Service
├─ Payment Service
├─ Order Service
├─ Notification Service
└─ Inventory Service
```

Now problems appear.

Problem: Applications interfere with each other

Imagine:

Server RAM = 32 GB

Applications consume:

Employee Service	2 GB
Payment Service	4 GB
Order Service	3 GB
Notification Service	2 GB
Inventory Service	3 GB

Everything is fine.

Then Payment Service gets heavy traffic and suddenly consumes:

```
Payment Service → 20 GB
```

Now:

```
Employee      2 GB
Payment       20 GB
Order         3 GB
Notification  2 GB
Inventory     3 GB
Linux         3 GB
-----
Total         33 GB ×
```

The entire server becomes unstable.

Payment Service's problem can affect completely unrelated applications.

This is called poor isolation.

Another problem: dependency conflicts

Suppose:

```
Employee Service → Java 17
Payment Service → Java 21
Legacy Service   → Java 8
```

All three applications share the same operating system.

You now have to manage:

```
Java 8
Java 17
Java 21
different libraries
different configurations
different environment variables
```

This becomes messy.

You may have heard developers say:

“It works on my machine.”

This happens partly because environments differ.

For example:

```
Developer Laptop
Java 21
Library X version 4
Linux library Y
```

```
Production
Java 17
Library X version 3
Different Linux configuration
```

Application:

```
Works locally '
Fails in production ×
```

| One possible solution — One server per application

We could say:

```
Employee Service
      ↓
Server 1

Payment Service
      ↓
Server 2

Order Service
      ↓
Server 3
```

Great isolation.

But now imagine:

```
Server 1:
CPU capacity = 32 cores
Employee Service uses = 2 cores
```

You're paying for:

```
32 cores
```

while using:

```
2 cores
```

That is a lot of wasted hardware.

This problem led to widespread use of virtualization.

| Virtual Machines

A Virtual Machine, or VM, is a software-created computer running inside another physical computer.

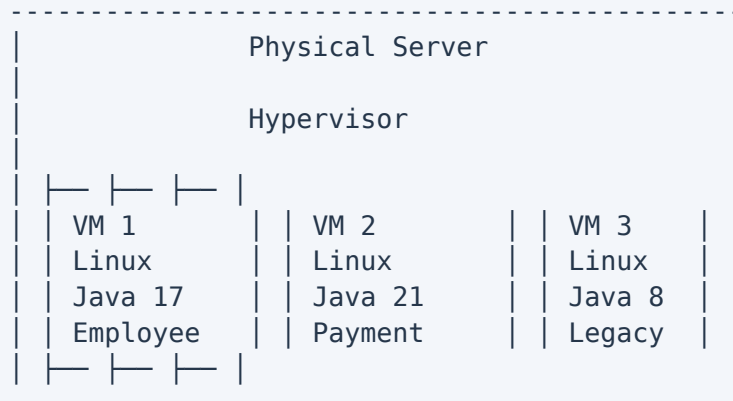
Instead of:

```
1 physical machine
=
1 operating system
```

we can do:

```
graph TD; A[1 physical server] --> B[Hypervisor]; B --> C[Multiple Virtual Machines]
```

Example:



Now applications have better isolation.

Employee Service can't easily interfere with Payment Service.

7. What is a Hypervisor?

The hypervisor manages virtual machines.

Examples include technologies such as:

- VMware
- Hyper-V
- KVM

Conceptually:

```

graph TD
    Hardware --> Hypervisor
    Hypervisor --> VM1[VM 1]
    Hypervisor --> VM2[VM 2]
    Hypervisor --> VM3[VM 3]

```

Each VM behaves almost like an independent computer.

VM 1 might have:

```
2 CPU
4 GB RAM
Ubuntu
Java 17
```

VM 2:

```
4 CPU
8 GB RAM
RHEL
Java 21
```

But VMs have a cost

Look carefully:

```
VM 1
├─ Operating System
├─ Libraries
├─ Java
└─ Application
```

```
VM 2
├─ Operating System
├─ Libraries
├─ Java
└─ Application
```

```
VM 3
├─ Operating System
├─ Libraries
├─ Java
└─ Application
```

Every VM carries an entire operating system.

Suppose:

```
Operating system = 2 GB
Application       = 200 MB
```

For ten applications:

```
10 operating systems
+
10 applications
```

That is significant overhead.

Booting a VM may also take considerably longer than starting a lightweight application process.

This created demand for something lighter.

That something is:

Containers.

9. What is a Container?

One-sentence definition

A container is an isolated environment that packages an application together with the runtime and dependencies it needs to run.

For our Java application:

```
Container
-
├─ Java Runtime
├─ Libraries
├─ Configuration requirements
└─ employee-service.jar
```

Instead of telling production:

Install Java 21, install these dependencies, copy this JAR, configure everything...

you package the environment together.

Conceptually:

```
-----
| Employee Service          |
|                            |
| Java Runtime              |
| Libraries                 |
| employee-service.jar      |
|                            |
|-----|
```

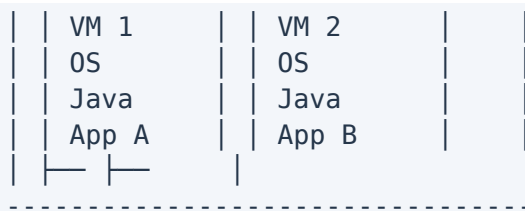
Now the package can run consistently across machines with a compatible container runtime.

VM vs Container

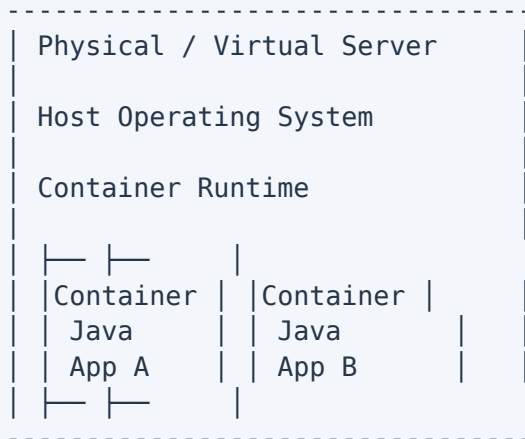
This distinction is extremely important.

Virtual Machines

```
-----
| Physical Server          |
|                            |
| Hypervisor                |
|                            |
|-----|
| └─ └─ └─ └─ └─ └─ └─ └─ |
```



Containers



The major difference:

VM

→ includes its own full guest OS

Container

→ shares the host operating system kernel

This is why containers are usually much lighter.

Important misconception

You may hear:

“A container is a lightweight VM.”

That is useful as a beginner analogy, but technically it isn’t completely accurate.

A VM virtualizes hardware.

A container isolates processes using operating-system features.

Think of it this way:

VM

Physical Hardware

↓

Virtual Hardware

↓

```
Guest Operating System
  ↓
Application
```

versus:

```
Container

Host Operating System
  ↓
Isolated Process Environment
  ↓
Application
```

We'll revisit namespaces and cgroups later.

For now, remember:

Containers share the host kernel. VMs normally have separate guest operating systems.

12. What is Docker?

This causes a lot of confusion.

Docker and containers are not exactly the same thing.

Think:

```
Container = concept / technology
```

```
Docker = popular tooling/platform used to build and run containers
```

Similar analogy:

```
Document
  " \
Microsoft Word
```

Word is a tool for working with documents.

Likewise:

```
Container
  " \
Docker
```

Docker became a very popular tool for working with containers.

Container Image vs Container

This is one of the first concepts you should memorize.

Consider Java:

```
Class
  ↓ instantiate
Object
```

Very roughly:

```
Container Image
  ↓ run
Container
```

An image is the packaged blueprint.

A container is a running instance of that image.

Example:

```
employee-service:1.0
```

is an image.

You run it:

```
Image
employee-service:1.0
  ↓
Container A
```

Run it again:

```
employee-service:1.0
-
  ├── Container A
  ├── Container B
  └── Container C
```

Same image.

Three running containers.

Java analogy

Because you're familiar with Java, this analogy is useful:

```
Class
  ↓
Object
```

Container world:

```
Image
↓
Container
```

Not technically identical, but an excellent mental model.

For example:

```
Employee e1 = new Employee();
Employee e2 = new Employee();
Employee e3 = new Employee();
```

Similarly:

```
employee-service image

Container 1
Container 2
Container 3
```

| Dockerfile

How do we tell Docker how to build our image?

Using a Dockerfile.

A simplified Java example:

```
DOCKERFILE

FROM eclipse-temurin:21-jre

COPY target/employee-service.jar app.jar

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Read it conceptually:

```
FROM Java 21 environment

COPY my application into it

WHEN container starts:
run java -jar app.jar
```

Then:

```
docker build -t employee-service:1.0 .
```

Result:

```
employee-service:1.0
```

That is your container image.

Then:

```
docker run employee-service:1.0
```

creates:

```
Running Container
```

| Why images are useful

Without containers:

Developer gives Operations:

```
employee-service.jar
```

Operations asks:

```
Which Java version?  
Which environment variables?  
Which libraries?  
Which OS packages?  
What command should I run?
```

With containers:

```
Application  
+  
Runtime  
+  
Dependencies  
+  
Startup instructions  
  
↓  
  
Image
```

This dramatically improves portability and consistency.

| Container Registry

Now imagine you built the image on your laptop:

```
Laptop  
↓  
employee-service:1.0
```

Your production server is somewhere else.

How does production get the image?

We store images in a container registry.

Architecture:

```
Developer
  ↓
docker build
  ↓
Image
  ↓
docker push
  ↓
Container Registry
  ↓
Production Server
  ↓
docker pull
  ↓
Run Container
```

Popular examples include:

```
Docker Hub
Amazon ECR
Azure Container Registry
Google Artifact Registry
Red Hat Quay
Private enterprise registries
```

You can think of the registry as:

GitHub/Maven repository, but for container images.

A Java analogy:

```
Maven Central
stores:
JAR libraries

Container Registry
stores:
Container images
```

Now imagine our microservice architecture

Suppose your company has:

```
API Gateway
Employee Service
Payment Service
Order Service
Notification Service
```

Inventory Service

You containerize everything:

```
gateway:1.0
employee-service:1.0
payment-service:1.0
order-service:1.0
notification-service:1.0
inventory-service:1.0
```

Production might run:

Gateway Container

Employee Containers

Employee-1

Employee-2

Employee-3

Payment Containers

Payment-1

Payment-2

Payment-3

Order Containers

Order-1

Order-2

Now containers have solved one problem.

But they have created another problem:

Who manages all of these containers?

And this is the point where Kubernetes becomes necessary.

19. Problem 1 — What if a container crashes?

Suppose:

Employee Service

Container 1 '

Container 2 '

Container 3 '

Container 2 crashes:

Container 1 '

Container 2 x

Container 3 '

Who notices?

Someone could manually do:

```
docker run employee-service
```

But production cannot depend on a human doing this at 3 AM.

We want:

```
Container crashes
      ↓
System detects failure
      ↓
System starts replacement
```

This is called self-healing.

Kubernetes provides this.

Problem 2 — Scaling

Normally:

```
Employee Service
```

```
Container A
```

```
Container B
```

Traffic increases dramatically.

You need:

```
A
B
C
D
E
F
```

Who creates these additional containers?

Later traffic drops.

Who removes unnecessary ones?

Desired system:

```
Traffic increases
      ↓
Scale application

2 containers → 8 containers
```

Kubernetes can help automate this.

21. Problem 3 — Where should containers run?

Imagine your company has five servers:

```
Server A
CPU: 80| used
```

```
Server B
CPU: 20| used
```

```
Server C
CPU: 50| used
```

```
Server D
CPU: 90| used
```

```
Server E
CPU: 10| used
```

You need to launch another Payment Service instance.

Where should it run?

Probably:

```
Server E
```

But imagine thousands of containers and hundreds of servers.

Humans cannot manually decide placement for each container.

You need a scheduler.

Kubernetes has one.

Problem 4 — Server failure

Imagine:

```
Server A
-
|— Employee Container
|— Payment Container
|— Notification Container
```

Server A crashes.

You just lost three application instances.

Desired system:

```
Server A x
  ↓
Detect failure
  ↓
Move/recreate workloads
```

```
Server B → Employee  
Server C → Payment  
Server D → Notification
```

Kubernetes handles this type of recovery.

Problem 5 — Containers don't have permanent identities

Suppose:

```
Payment Container
```

```
IP:  
10.1.5.14
```

The container dies.

A new one gets created:

```
New Payment Container
```

```
IP:  
10.1.9.27
```

But Order Service was calling:

```
10.1.5.14
```

Now it doesn't work.

We need something stable.

Instead of:

```
Order Service  
  ↓  
10.1.5.14
```

we want:

```
Order Service  
  ↓  
payment-service  
  ↓  
Any healthy Payment container
```

This is service discovery and load balancing.

Kubernetes provides this through objects such as Services.

| Problem 6 — Application updates

Suppose production currently runs:

```
employee-service:1.0
```

```
Container A
```

```
Container B
```

```
Container C
```

You release:

```
employee-service:2.0
```

Naive approach:

```
Stop A
```

```
Stop B
```

```
Stop C
```

```
Deploy new version
```

For some period:

```
Employee Service unavailable x
```

Instead we'd like:

```
v1 v1 v1
```

```
↓ update slowly
```

```
v2 v1 v1
```

```
↓
```

```
v2 v2 v1
```

```
↓
```

```
v2 v2 v2
```

Users continue receiving traffic during deployment.

This is a rolling update.

Kubernetes supports this.

| Problem 7 — Configuration

Development:

```
DB_HOST=localhost
```

Testing:

```
DB_HOST=test-db.company.internal
```

Production:

```
DB_HOST=prod-db.company.internal
```

We should not build three completely different application images.

Instead:

```
Same image
+
Different configuration
```

Kubernetes provides:

```
ConfigMaps
Secrets
```

We✓I learn them later.

Problem 8 — Resource management

Suppose:

```
Payment Service
```

has a bug and tries to consume:

```
30 GB RAM
```

You don't want it destroying an entire worker server.

You'd like to say:

```
Payment container:
```

```
Minimum:
512 MB
```

```
Maximum:
2 GB
```

Kubernetes allows:

```
Resource Requests
Resource Limits
```

We✓I go deep into these later.

| We have arrived at Kubernetes

Now the definition will make sense.

Kubernetes in one sentence

Kubernetes is a container orchestration platform that automates deploying, scheduling, scaling, networking, and recovering containerized applications across a cluster of machines.

The key word is:

Orchestration

28. What does orchestration mean?

Think about an orchestra.

There may be:

Violins
Drums
Trumpets
Piano
Cello

Each musician knows how to play their own instrument.

But someone coordinates everyone:

Conductor

Similarly:

Containers know how to run applications.

Kubernetes coordinates containers.



Kubernetes decides:

Where should it run?
How many copies?
Is it healthy?
Should it restart?
How does traffic reach it?
Should it scale?
How should it update?

| Container Engine vs Kubernetes

Another important distinction.

Docker/container runtime can do:

```
Run container
Stop container
Delete container
Build image
Pull image
```

Kubernetes does things like:

```
Run 5 instances.
Make sure 5 remain running.
Replace failed instances.
Spread them across servers.
Expose them through networking.
Scale them.
Perform rolling upgrades.
```

So:

```
Container Runtime
  ↓
Runs containers

Kubernetes
  ↓
Manages containers at scale
```

| Kubernetes doesn't normally manage containers directly

Here is something important we'll study in chapter 3.

You might expect:

```
Kubernetes
  ↓
Container
```

But Kubernetes introduces another object:

```
Kubernetes
  ↓
Pod
  ↓
Container
```

So the fundamental deployment unit is:

Pod Not a raw container.

For now simply remember:

```
Pod
├─ Container
```

Later we✓I answer:

Why did Kubernetes invent Pods instead of managing containers directly?

There is a very good architectural reason.

Kubernetes Cluster

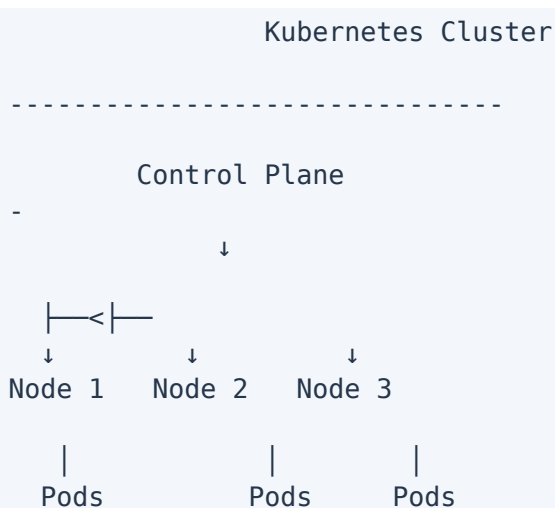
Kubernetes doesn't normally run everything on one machine.

It manages a cluster.

A cluster means:

A collection of machines working together.

Example:



You could have:

```
3 nodes
30 nodes
300 nodes
thousands of nodes
```

depending on scale.

32. What is a Node?

A Node is a machine that participates in the Kubernetes cluster.

It can be:

Physical machine

or more commonly:

Virtual Machine

For example:

Node 1
8 CPU
32 GB RAM

Node 2
8 CPU
32 GB RAM

Node 3
16 CPU
64 GB RAM

Kubernetes distributes application workloads across them.

The most important mental model so far

Imagine telling Kubernetes:

I want three instances of Employee Service.

You are not saying:

Run Employee Service on Server B.

You are saying:

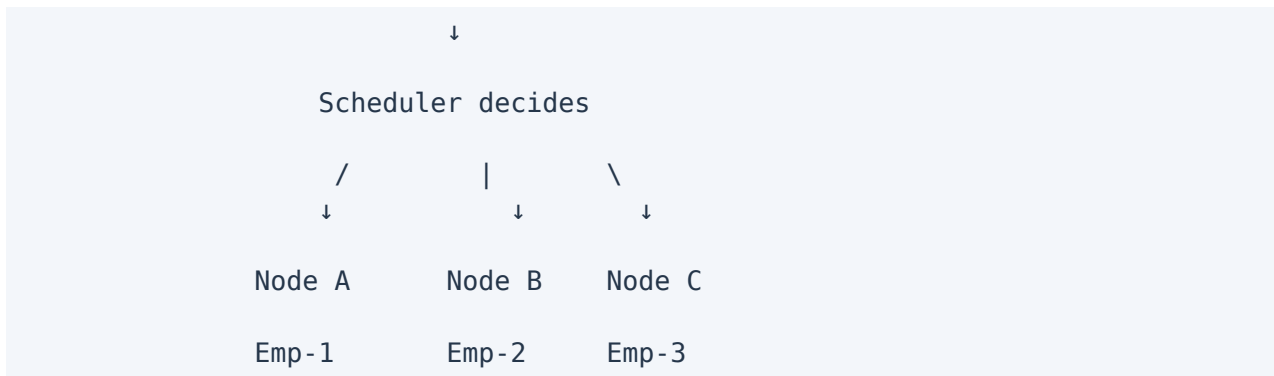
Desired State:

Employee Service instances = 3

Kubernetes figures out where they should run.

Kubernetes

Desired:
Employee Service = 3



This concept is called:

Desired State It is one of the deepest ideas in Kubernetes.

Desired state vs current state

Suppose:

Desired State
Employee replicas = 3

Current:

Employee-1 ' '
Employee-2 ' '
Employee-3 ' '

Everything is good.

Then Employee-2 dies.

Current state:

Employee-1 ' '
Employee-2 ×
Employee-3 ' '

Kubernetes sees:

Desired = 3
Current = 2

Therefore:

$3 - 2 = 1$ missing

Kubernetes creates another instance:

Employee-1 ' '
Employee-3 ' '
Employee-4 ' '

Now:

```
Desired = 3  
Current = 3
```

This process happens continuously.

This idea is called reconciliation.

We✓I study it deeply later.

| Imperative vs Declarative thinking

Traditional administration often looks like:

```
Start container A.  
Start container B.  
Restart container C.  
Move container D.
```

You're giving exact instructions.

That is imperative.

Kubernetes encourages a declarative model:

```
I want:  
  
Employee Service  
replicas = 3  
image = employee-service:2.0  
memory = 1 GB
```

Then Kubernetes continually tries to make reality match that declaration.

Think:

```
Imperative  
  
"Do these steps."
```

versus:

```
Declarative  
  
"This is what the final world should look like."
```

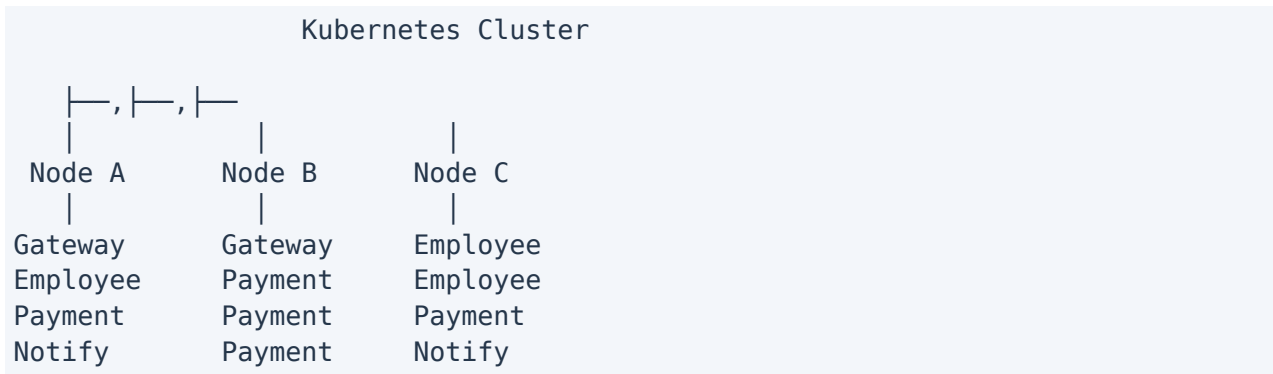
This distinction is fundamental.

| Example from Java development

Imagine your application requirements are:

Gateway	→ 2 instances
Employee	→ 3 instances
Payments	→ 5 instances
Notifications	→ 2 instances

Instead of manually creating 12 containers, Kubernetes can maintain:



Kubernetes manages placement and replacement.

Your job as an application developer becomes more like:

Tell Kubernetes WHAT you need.

Kubernetes decides HOW to maintain it.

37. Where does OpenShift fit?

Now we can place OpenShift correctly.

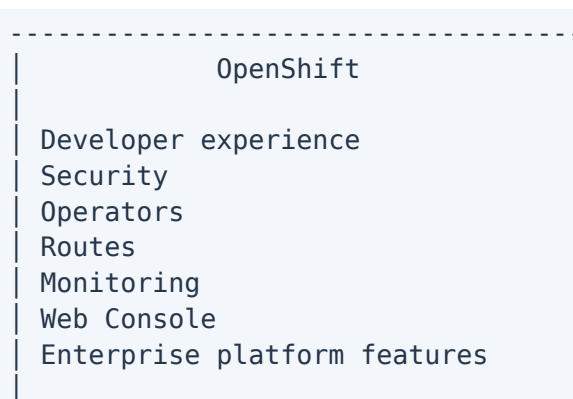
```

Containers
  ↓
Kubernetes
  ↓
OpenShift
  
```

OpenShift is not a completely separate orchestration concept.

It uses Kubernetes and adds enterprise platform capabilities around it.

Simplified:





That is why we're learning Kubernetes first.

Putting the entire evolution together

This is the diagram I want you to remember from today's lesson:

STEP 1
Application
↓
Run directly on Server

Problem:
Dependency conflict
Poor isolation

STEP 2
Virtual Machines

Physical Server
↓
Hypervisor

↓
VM → OS → App
VM → OS → App
VM → OS → App

Benefit:
Strong isolation

Problem:
Heavyweight

STEP 3
Containers

Server
↓
OS
↓
Container Runtime
↓
Container
Container
Container

Benefit:
Lightweight
Portable
Fast
Consistent

STEP 4
Many Containers

100s / 1000s containers

Problems:
Scheduling
Scaling
Failures
Networking
Updates
Configuration
Resource management

STEP 5
Kubernetes

Kubernetes
↓
Orchestrates Containers
↓

Scheduling
Scaling
Self-healing
Networking
Rolling updates
Configuration
Resource management

Terminology learned today

Make sure these words are clear:

Term Meaning

Physical Server Actual hardware computer

Virtual Machine Virtual computer with its own guest OS

Hypervisor Technology managing VMs

Container Isolated environment for running an application

Container Image Packaged blueprint used to create containers

Container Runtime Software responsible for running containers

Term Meaning

Docker Popular container tooling/platform

Dockerfile Instructions used to build an image

Registry Storage/repository for container images

Kubernetes Container orchestration system

Cluster Collection of machines managed together

Node Machine participating in Kubernetes

Pod Kubernetes unit that contains one or more containers

Orchestration Coordinating and managing containers

Desired State What you want the system to look like

Current State What the system currently looks like

Reconciliation Kubernetes trying to make current state match desired state

| A common interview question

Why do we need Kubernetes if Docker can run containers? A good answer:

Docker or another container runtime can run individual containers, but running containers alone doesn't solve large-scale operational problems. Kubernetes provides orchestration capabilities such as scheduling workloads across machines, maintaining the desired number of instances, self-healing failed workloads, service discovery, load balancing, rolling deployments, resource management and scaling.

That is much stronger than saying:

Kubernetes manages Docker containers.

| Another important interview question

VM vs Container? You could explain:

VM:

Virtualizes hardware

Each VM has guest OS

Heavier

Usually slower to start

Strong isolation

Container:

OS-level isolation

Shares host kernel

Lighter

Usually starts quickly

Efficient for application workloads

Don't say:

Containers don't have an OS.

That is an oversimplification.

A container image can contain user-space filesystem components and libraries, but it does not normally carry its own independent kernel.

Check your understanding

Try answering these without looking back.

Question 1 You have:

```
employee-service:1.0
```

Is this most likely an image or container?

Question 2 If I run that image three times, how many images and containers conceptually exist?

Question 3 Why is a container usually lighter than a VM?

Question 4 If a company has 1,000 containers running across 100 servers, name three problems Kubernetes helps solve.

Question 5 What is the difference between:

```
Desired State = 3 replicas
```

```
Current State = 2 replicas
```

What would Kubernetes try to do?

Question 6 Which relationship is correct?

A)

```
Container
  ↓
Image
```

or:

B)

```
Image
  ↓ run
Container
```

Small practical exercise Imagine you've developed:

Order Service

Requirements:

```
Java 21
Port 8080
order-service.jar
```

Your company wants to run five copies of it.

Think through:

1. What would you package into the image?
2. What happens when the image is run?
3. If five copies run, how many containers exist?
4. If one crashes, what should ideally happen?
5. Which system should manage that automatically?

Don't worry about commands yet.

The goal is to make the mental model solid.

chapter 1 summary

The single story behind today's lesson is:

```
Applications needed servers
  ↓
Servers had isolation/resource problems
  ↓
Virtual Machines improved isolation
  ↓
VMs were relatively heavyweight
  ↓
Containers provided lighter application isolation
  ↓
Containers became easy to create
  ↓
Companies started running huge numbers of them
  ↓
Managing them manually became impossible
  ↓
Kubernetes was created for orchestration
```

And remember these three equations:

```
Image
  ↓ run
Container
```

```

Kubernetes
=
Container Orchestration

Desired State
  ↓
Kubernetes continuously reconciles
  ↓
Current State

```

Next: chapter 2 — Kubernetes Architecture Next we'll open the Kubernetes black box and understand:



We'll learn exactly what these components do:

```

API Server
etcd

Scheduler
Controller Manager
kubelet
Container Runtime
kube-proxy / networking components

```

Most importantly, we'll follow one request from `kubectl` apply all the way until a Pod starts running on a worker node. That will give you the core mental model for almost everything that comes afterward.

Kubernetes Foundations

Chapter 2 — Kubernetes Big Picture

CHAPTER FOCUS

Understand **Kubernetes Big Picture** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

Now we'll open the Kubernetes black box.

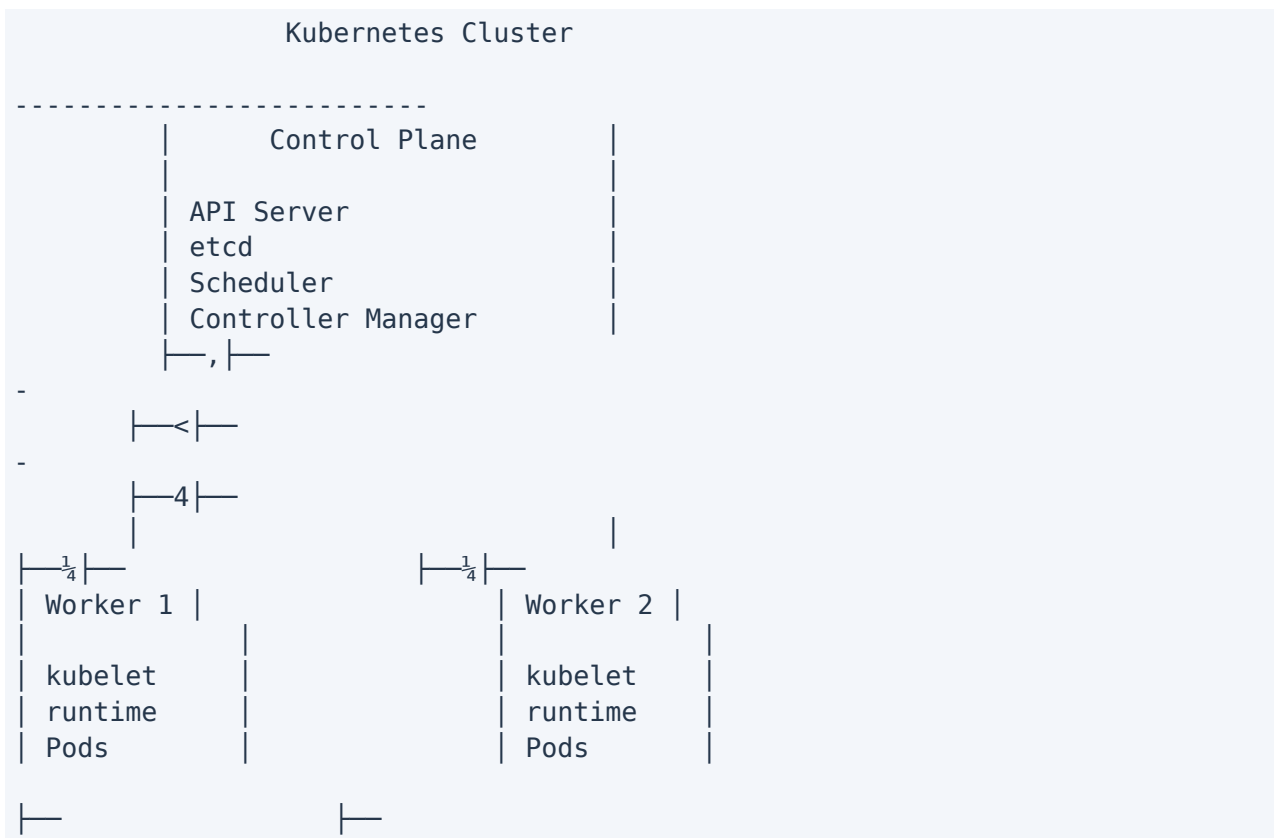
The goal of this lesson is to understand what happens when you say:

```
kubectl apply -f deployment.yaml
```

and how that eventually becomes:

A running application container
inside a Pod
on a Worker Node

The core architecture is:



The most important high-level distinction is:

Control Plane = makes decisions and manages the cluster. Worker Nodes = actually run your application workloads.

1. First: What is a Kubernetes Cluster?

A Kubernetes cluster is a group of machines managed as one system.

Suppose your company has:

```
Machine A
Machine B
Machine C
Machine D
```

Kubernetes can organize them like:

```
Machine A → Control Plane

Machine B → Worker Node
Machine C → Worker Node
Machine D → Worker Node
```

The workers run application workloads.

The control plane decides things such as:

```
Where should a Pod run?
Is the required number of Pods running?
Did a Node fail?
Should a replacement Pod be created?
What configuration did the user request?
```

Control Plane vs Worker Node

Think of a company.

The control plane is roughly like:

```
Management / Planning Team
```

Worker Nodes are roughly like:

```
Execution Team
```

Management doesn't manufacture the product itself.

It decides:

```
What needs to happen?
Who should do it?
Is the work completed?
```

Similarly:

```
Control Plane
  ↓
Makes decisions

Worker Nodes
  ↓
Execute workloads
```

| Main Control Plane Components

We will focus on four components first:

```
Control Plane
-
├─ API Server
├─ etcd
├─ Scheduler
├─ Controller Manager
```

You should understand each one separately.

| API Server

The API Server is the front door of Kubernetes.

Almost every interaction with Kubernetes goes through it.

Suppose you run:

```
kubectl get pods
```

The flow is conceptually:

```
You
  ↓
kubectl
  ↓
API Server
  ↓
Kubernetes Cluster
```

Or:

```
kubectl apply -f deployment.yaml
```

Again:

```
kubectl
  ↓
API Server
```

You generally do not directly talk to:

```
Scheduler
etcd
kubelet
Controller Manager
```

The API Server coordinates access.

5. What does kubectl actually do?

kubectl is a command-line client.

It is not Kubernetes itself.

Think:

```
kubectl
=
remote control
```

Kubernetes itself runs somewhere else.

So:

```
kubectl get pods
```

roughly means:

“API Server, please show me the Pods I am allowed to see.”

And:

```
kubectl delete pod payment-123
```

means:

“API Server, please request deletion of this Pod.”

API Server analogy

Imagine a bank.

You normally don't walk directly into the database and modify your balance.

You interact through:

```
Bank application/API
```

Similarly:

```
User
↓
kubectl
↓
```

```
Kubernetes API Server
↓
Cluster resources
```

The API Server performs things such as:

- authentication
- authorization
- request validation
- API handling
- communication with cluster state

We✓I study security later.

For now remember:

API Server = entry point to Kubernetes.

7. What is etcd?

The next component is extremely important.

etcd is the persistent key-value store Kubernetes uses to store cluster state.

Suppose you tell Kubernetes:

```
I want 3 Employee Service replicas.
```

Where does Kubernetes remember that?

Conceptually:

```
etcd
```

It stores information about objects such as:

```
Pods
Deployments
Services
ConfigMaps
Secrets
Nodes
Namespaces
Desired configuration
```

etcd is the cluster's source of truth

Think:

Kubernetes Brain's Memory

↓
etcd

For example:

Deployment:
employee-service

Desired replicas:
3

Image:
employee-service:2.0

That desired state must survive even if some Kubernetes component restarts.

So the cluster stores it persistently.

| Important misconception about etcd

etcd is not your application database.

Don't think:

Employee records → etcd
Payment transactions → etcd
Customer orders → etcd

No.

Your application still uses systems such as:

PostgreSQL
Oracle
MySQL
MongoDB
etc.

etcd stores Kubernetes cluster configuration and state.

| Scheduler

Now imagine Kubernetes needs to create a Pod.

Suppose we have:

Worker Node A
CPU free: 1 core
RAM free: 2 GB

Worker Node B

CPU free: 6 cores
RAM free: 12 GB

Worker Node C
CPU free: 3 cores
RAM free: 8 GB

New Pod requires:

CPU: 2 cores
RAM: 4 GB

Which node should run it?

Not Node A.

The component responsible for selecting a suitable node is:

Scheduler Conceptually:

```
graph TD
    A[Unscheduled Pod] --> B[Scheduler]
    B --> C[Evaluate Nodes]
    C --> D[Choose Worker Node B]
```

11. Does the Scheduler start the container?

No.

Very important.

The Scheduler does essentially:

“This Pod should run on Node B.”

It does not itself start the container.

Think:

```
graph TD
    A[Scheduler] --> B[Assignment]
    B --> C[Pod X → Node B]
```

After that, another component on Node B actually makes it run.

That component is:

kubelet We✓I reach it shortly.

12. What factors can the Scheduler consider?

Later we'll study this in detail, but examples include:

```
Available CPU
Available memory
Node labels
Affinity rules
Anti-affinity rules
Taints
Tolerations
Resource requests
Topology constraints
```

For now:

Scheduler's main job = choose where a Pod should run.

| **Controller Manager**

Now we reach one of the deepest Kubernetes ideas.

Remember from chapter 1:

```
Desired State = 3 Pods
Current State = 2 Pods
```

Some component must constantly notice:

```
Desired != Current
```

and act.

That is what controllers do.

The Controller Manager runs multiple controllers that continuously watch the cluster and try to move reality toward desired state.

| **Think of a thermostat**

Suppose:

```
Desired temperature = 24°C
Current temperature = 28°C
```

The thermostat detects:

```
28 != 24
```

and activates cooling.

After some time:


```
Current = 24°C  
Desired = 24°C
```

Stop correcting.

Kubernetes does something very similar.

```
Desired Pods = 3  
Current Pods = 2
```

Controller notices:

```
2 != 3
```

and causes another Pod to be created.

This continuous correction is:

Reconciliation

Controller Manager is not one single logical controller

Conceptually, Kubernetes has different controllers for different responsibilities.

Examples include controllers related to:

```
Nodes  
ReplicaSets  
Deployments  
Endpoints  
Namespaces  
ServiceAccounts  
Jobs
```

You don't need to memorize them yet.

The important idea:

Controllers continuously watch and reconcile cluster state.

Worker Node Components

Now let's move from the Control Plane to the Worker Node.

A worker node commonly has:

```
Worker Node  
-  
├─ kubelet  
├─ Container Runtime  
├─ Networking components  
└─ Pods
```

The most important two right now:

```
kubelet  
Container Runtime
```

| kubelet

The kubelet is an agent running on each worker node.

Suppose the Scheduler decides:

```
Employee Pod → Worker Node 2
```

Now Worker Node 2 needs someone to notice:

“I have been assigned this Pod.”

That is the kubelet.

Conceptually:

```
Control Plane  
  ↓  
Pod assigned to Node 2  
  ↓  
kubelet on Node 2  
  ↓  
Make sure Pod runs
```

| kubelet's responsibility

Think of kubelet as:

The Kubernetes manager living on each worker machine.

It makes sure the Pods assigned to its Node are actually running as expected.

Example:

```
Node 2  
  
kubelet receives desired Pod specification  
  
  ↓  
  
Start required container(s)  
  
  ↓  
  
Check status  
  
  ↓
```

Report status back

19. Does kubelet itself run containers?

Not directly.

The kubelet works with a:

Container Runtime The runtime performs the low-level container operations.

Conceptually:

```
kubelet
  ↓
Container Runtime
  ↓
Container
```

| Container Runtime

The container runtime is responsible for actually running containers.

Examples include:

```
containerd
CRI-O
```

OpenShift commonly uses CRI-O.

A simplified chain:

```
Kubernetes
  ↓
kubelet
  ↓
Container Runtime
  ↓
Linux kernel
  ↓
Container process
```

| Important: Docker and Kubernetes

Historically people often said:

```
Kubernetes runs Docker
```

Today that is not the right mental model.

Kubernetes talks to container runtimes through a standardized interface called:

CRI — Container Runtime Interface So conceptually:

```
Kubernetes / kubelet
```

```
↓
```

```
CRI
```

```
↓
```

```
Container Runtime
```

```
Examples:
```

```
containerd
```

```
CRI-O
```

This lets Kubernetes avoid being tightly tied to one container engine.

| OpenShift connection

Since our end goal includes OpenShift, remember:

```
OpenShift
```

```
↓
```

```
Kubernetes
```

```
↓
```

```
CRI
```

```
↓
```

```
CRI-O
```

You don't need to understand CRI-O internals yet.

Just know that OpenShift worker nodes typically use CRI-O as the runtime responsible for container execution.

23. What about kube-proxy?

You✓I often see:

```
kube-proxy
```

mentioned as a node component.

Historically and in many Kubernetes setups, kube-proxy helps implement Service networking rules so traffic can reach appropriate backend Pods.

Conceptually:

```
Client
```

```
↓
```

```
Kubernetes Service
```

```
↓
```

```
Networking rules
```

```
↓
```

```
One of the matching Pods
```

However, modern Kubernetes networking can be implemented in different ways depending on the networking technology in use.

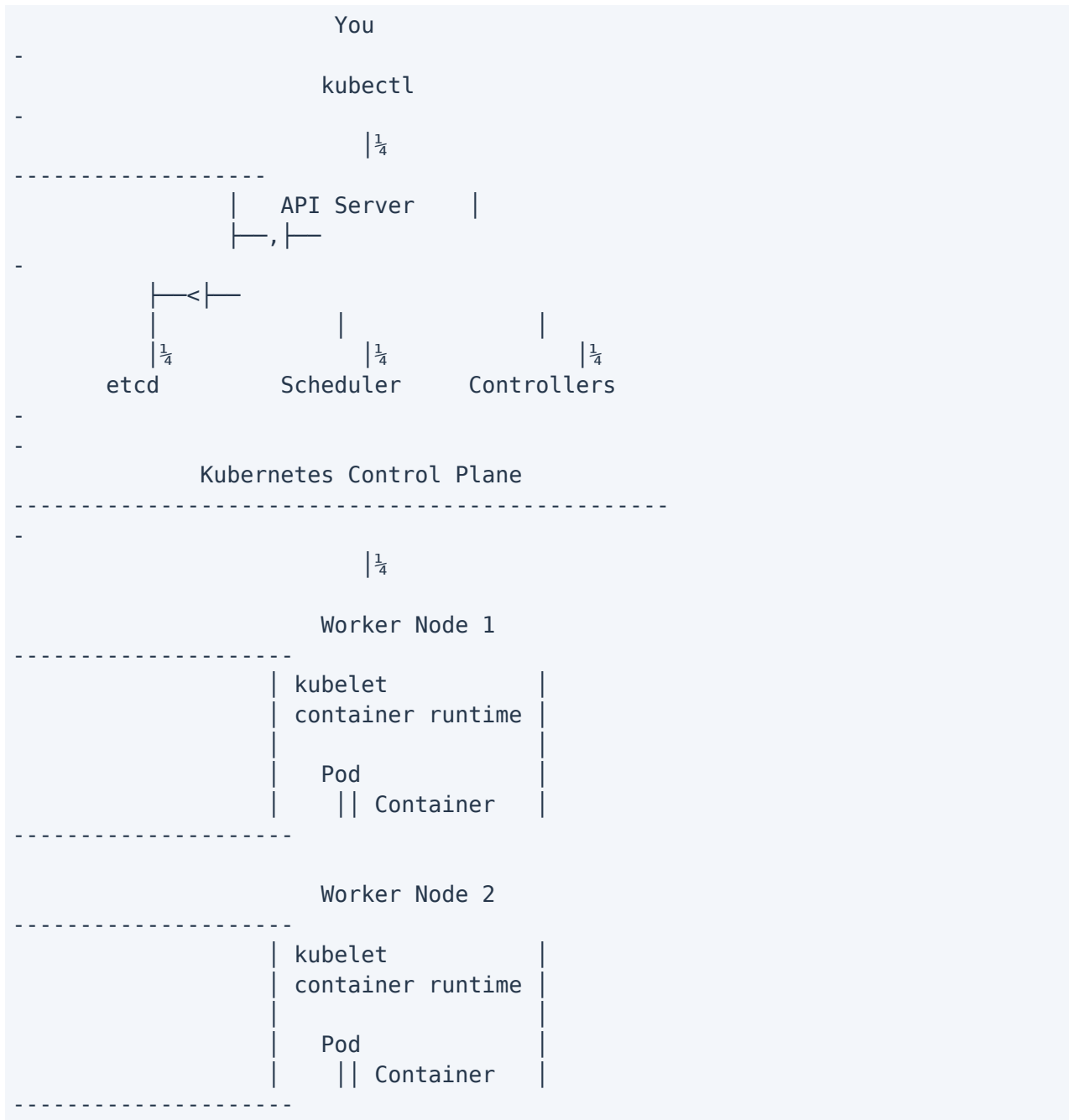
So for now, remember:

Node networking components help implement Kubernetes Service/network connectivity.

We'll handle this properly in the Networking chapter.

Full architecture so far

Now combine everything:



| Now let's follow one real request

Suppose you have a Spring Boot application:

```
employee-service
```

You want Kubernetes to run:

```
3 replicas
```

You prepare a Deployment manifest.

Don't worry about understanding every YAML field yet:

```
apiVersion: apps/v1
kind: Deployment

metadata:
  name: employee-service

spec:
  replicas: 3

  selector:
    matchLabels:
      app: employee

  template:
    metadata:
      labels:
        app: employee

    spec:
      containers:
        - name: employee
          image: myregistry/employee-service:1.0
```

Then:

```
kubectl apply -f deployment.yaml
```

Let's follow what happens.

| Step 1 — kubectl reads your file

kubectl reads:

```
deployment.yaml
```

and understands that you're asking for:

```
Kind:
Deployment

Name:
employee-service

Replicas:
3

Image:
employee-service:1.0
```

It sends this request to:

```
API Server
```

| Step 2 — API Server receives request

The API Server checks things such as:

```
Is the request valid?
Is the user authenticated?
Is the user authorized?
Is the object structurally acceptable?
```

If allowed, Kubernetes accepts the desired configuration.

Conceptually:

```
kubectl
  ↓
API Server
  ↓
Desired Deployment accepted
```

| Step 3 — desired state is persisted

Kubernetes stores the cluster's desired state.

Conceptually:

```
etcd

Deployment:
employee-service

Desired replicas:
3
```

Now Kubernetes remembers:

“There should be three instances.”

Step 4 — Controllers notice the new desired state

The relevant controller observes:

```
Desired Deployment exists
but
required Pods do not yet exist
```

Current state:

```
0 Pods
```

Desired:

```
3 Pods
```

So:

```
Desired = 3
Current = 0
```

Kubernetes starts creating the required lower-level objects.

We'll learn the exact Deployment → ReplicaSet → Pod relationship later.

For now think:

```
Deployment
  ↓
Controller logic
  ↓
Need 3 Pods
```

Step 5 — Pods exist but aren't assigned yet

Kubernetes now conceptually has:

```
Pod A
Pod B
Pod C
```

but initially they may not yet have a Node assignment.

Think:

```
Pod A → ?
Pod B → ?
Pod C → ?
```

The Scheduler sees these unscheduled Pods.

Step 6 — Scheduler chooses Nodes

Suppose:

```
Node 1  
Free RAM: 10 GB
```

```
Node 2  
Free RAM: 4 GB
```

```
Node 3  
Free RAM: 12 GB
```

The Scheduler evaluates the cluster.

It may decide:

```
Pod A → Node 1  
Pod B → Node 2  
Pod C → Node 3
```

Or perhaps:

```
Pod A → Node 1  
Pod B → Node 1  
Pod C → Node 3
```

depending on constraints and resources.

The important thing:

Scheduler selects the Node.

Step 7 — kubelet notices assignment

Suppose:

```
Pod A → Node 1
```

The kubelet on Node 1 sees that Pod A is assigned to it.

It now needs to make that Pod real.

```
kubelet  
↓  
"I need employee-service:1.0"
```

Step 8 — Runtime pulls the image

If the Node doesn't already have:

```
employee-service:1.0
```

the runtime pulls it from the container registry.

Container Registry

-

| pull
| 1/4

Worker Node

For example:

Quay / private registry

↓

employee-service:1.0

↓

CRI-O

Step 9 — Container starts

The runtime creates and starts the container.

Worker Node 1

Pod A

├─ employee-service container

Same for the other assigned Pods.

Eventually:

Node 1

├─ Employee Pod A

Node 2

├─ Employee Pod B

Node 3

├─ Employee Pod C

Step 10 — kubelet reports status

The kubelet monitors the Pod and reports information back to Kubernetes.

For example:

Pod A = Running

Pod B = Running

Pod C = Running

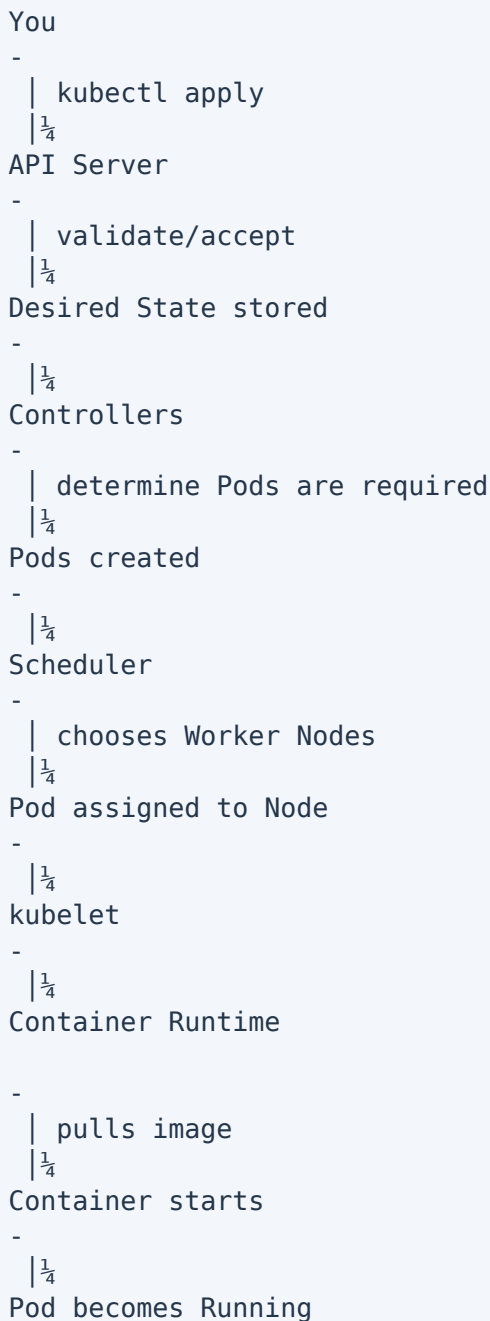
Now:

Desired State = 3
Current State = 3

Cluster is reconciled.

Full request lifecycle

This is probably the most important diagram of this chapter:



If you understand this diagram, you already understand a huge part of Kubernetes architecture.

37. What if a Pod crashes afterward?

Now suppose:

```
Pod A '
Pod B x
Pod C '
```

The container/runtime/kubelet can detect local failure information.

The cluster eventually sees:

```
Desired = 3
Current available replicas = 2
```

Controller logic reacts.

```
Need one replacement Pod
```

Then the cycle happens again:

```
Controller
↓
New Pod
↓
Scheduler
↓
Node
↓
kubelet
↓
Runtime
↓
Container
```

This is why Kubernetes is not just a “container launcher.”

It is a continuously operating control system.

A useful mental model: Kubernetes as control loops

A lot of Kubernetes internally follows:

```
Observe
↓
Compare
↓
Act
↓
Observe again
```

Example:

```
Observe:
2 Pods running
```

```
Compare:
Desired = 3

Act:
Create another

Observe:
3 Pods running

Compare:
Desired = 3

No correction needed
```

This pattern is foundational.

39. What happens if the API Server goes down?

Interesting question.

Existing containers don't immediately disappear.

Your worker nodes may continue running currently existing workloads.

But cluster management becomes severely affected because:

```
kubectl requests cannot be handled normally
new scheduling can't proceed normally
controllers may be unable to coordinate through the API
cluster state changes cannot be processed normally
```

That is why production Kubernetes typically runs control-plane components redundantly.

We✓I discuss high availability later.

40. What happens if etcd is lost?

This is much more serious.

Remember:

```
etcd
=
cluster state
```

If etcd data is irrecoverably lost, Kubernetes loses the authoritative state/configuration for the cluster.

That is why etcd backup and high availability are critical in self-managed Kubernetes environments.

In managed platforms and OpenShift, the platform architecture handles much of this operational complexity.

41. What happens if a Worker Node dies?

Suppose:

```
Node 2 ×  
  
Employee Pod B  
Payment Pod C  
Order Pod A
```

Those workloads disappear with that Node.

Kubernetes notices the Node is unavailable.

If those workloads are controlled by Deployments/ReplicaSets, Kubernetes can create replacement Pods on healthy Nodes.

Conceptually:

```
Node 2 fails  
↓  
Cluster detects failure  
↓  
Desired state no longer satisfied  
↓  
Controllers create replacements  
↓  
Scheduler selects healthy Nodes
```

Control Plane does not normally run your business application

As a beginner, keep this separation clear:

Control Plane

```
API Server  
Scheduler  
Controllers  
etcd
```

versus:

Worker Nodes

```
Your Java applications  
Your APIs  
Your microservices  
Your Pods
```

Production systems normally protect the control plane from ordinary application workloads.

One cluster can have many applications

For example:

Kubernetes Cluster

Control Plane

Worker Node A

- Gateway Pod
- Employee Pod
- Payment Pod

Worker Node B

- Payment Pod
- Order Pod
- Notification Pod

Worker Node C

- Employee Pod
- Gateway Pod
- Order Pod

Kubernetes is managing the entire fleet.

44. Where does the operating system fit?

Each Node is still a machine.

For example:

Worker Node

Physical/Virtual Hardware

↓

Linux OS

↓

kubelet

Container Runtime

Networking

↓

Pods

↓

Containers

Containers share the host's Linux kernel.

So Kubernetes is not replacing the operating system.

It is managing workloads running on top of Node operating systems.

Important distinction: Cluster vs Node vs Pod vs Container

Keep this hierarchy:

```
Kubernetes Cluster
-
├── Control Plane
-
-
├── Worker Node 1
│   ├── Pod A
│   │   └── Container
│   ├── Pod B
│   │   └── Container
-
├── Worker Node 2
│   └── Pod C
│       └── Container
```

So:

Cluster contains Nodes. Nodes run Pods. Pods contain containers.

This hierarchy should become automatic in your mind.

Java enterprise analogy

Since you work with backend systems, imagine:

```
Kubernetes Cluster
=
Company

Control Plane
=
Management

API Server
=
Reception/API gateway for management requests

etcd
=
Authoritative company records

Scheduler
=
Resource allocation manager

Controller Manager
```



```
=  
Operations manager checking expected vs actual  
  
Worker Node  
=  
Execution team  
  
kubelet  
=  
Team lead on each worker team  
  
Container Runtime  
=  
Worker that actually launches processes  
  
Pod  
=  
Assigned unit of work
```

Not technically exact, but useful for remembering responsibilities.

API Server vs Scheduler vs kubelet

This is a very common confusion.

API Server

Receives and exposes Kubernetes API requests

Scheduler

Decides which Node should run an unscheduled Pod

kubelet

Ensures assigned Pods actually run on its Node

So:

API Server:

"Request received."

Scheduler:

"Run it on Node 3."

kubelet:

"I am Node 3. I'll make sure it runs."

That is an excellent simplified model.

| Controller vs Scheduler

Another common confusion.

Suppose:

```
Desired replicas = 3
Current = 2
```

Who notices one is missing?

```
Controller
```

Who chooses where the replacement runs?

```
Scheduler
```

Who starts it on that Node?

```
kubelet + runtime
```

So:

```
Controller
↓
"We need another Pod."

Scheduler
↓
"Put it on Node B."

kubelet
↓
"I'll run it."

Runtime
↓
"Container started."
```

Excellent interview-level understanding.

| Quick troubleshooting connection

Later, when something fails, architecture tells you where to look.

Example:

Pod stuck in Pending Possibilities include:

```
Scheduler cannot find a suitable Node
insufficient CPU
insufficient memory
scheduling constraints
storage dependency unavailable
```

Pod assigned but container won't start Maybe:

```
image pull failure
runtime issue
bad image
configuration problem
```

kubectl cannot communicate with cluster Could involve:

```
API Server connectivity
authentication
authorization
cluster endpoint
```

Understanding architecture makes troubleshooting logical.

Kubernetes architecture summary

Memorize this flow:

```
kubectl
↓
API Server
↓
Desired State
↓
Controllers
↓
Pod needed
↓
Scheduler
↓
Worker Node selected
↓
kubelet
↓
Container Runtime
↓
Container runs
```

And memorize this responsibility table:

Component Main responsibility

kubectl CLI client

API Server Front door to Kubernetes

etcd Stores cluster state

Scheduler Selects a Node for a Pod

Controller Manager Reconciles desired and current state

Worker Node Runs application workloads

kubelet Manages assigned Pods on a Node

Container Runtime Actually runs containers

Networking components Enable Pod/Service networking

Check your understanding

Try answering these mentally.

You run:

```
kubectl apply -f app.yaml
```

What Kubernetes component receives the request first?

Which component remembers:

```
replicas = 3
```

as part of cluster state?

3. There is a Pod waiting to be assigned to a Node. Who chooses the Node?
4. Once the Pod is assigned to Worker Node 4, which component on Node 4 makes sure it runs?
5. Who actually starts the container?

Desired replicas are 5, but only 4 exist. Which kind of component detects and corrects that

mismatch?

7. Does the Scheduler start containers?

Answer:

```
No.
```

It selects a Node.

Mini scenario Suppose:

Cluster:

Node A:

2 GB free

Node B:

10 GB free

Node C:

5 GB free

A Pod needs:

4 GB RAM

Flow:

Controller:

"We need this Pod."

Scheduler:

Node A ×

Node B '

Node C '

Scheduler chooses Node B.

kubelet on Node B:

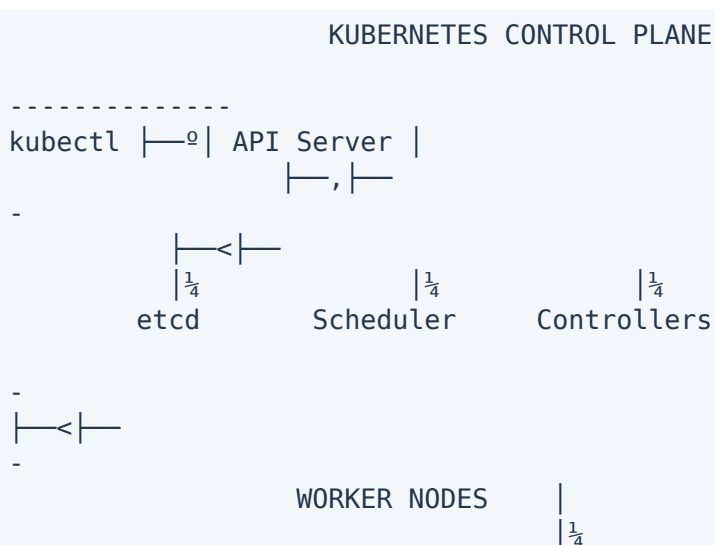
"I see a new Pod assigned to me."

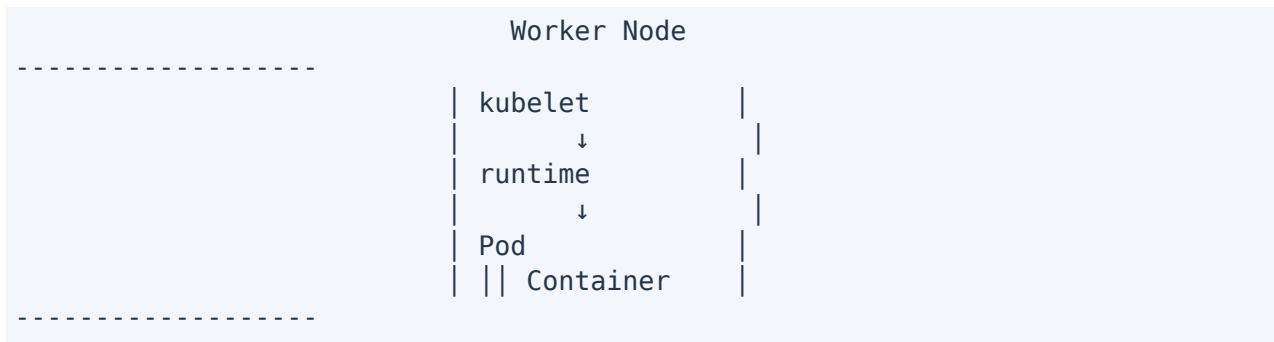
Container runtime:

Pull image → create container → start container.

That single scenario captures most of today's architecture.

At this stage, you should have this picture:





The three lines I want you to retain are:

Controller decides that something needs to exist.

Scheduler decides where it should exist.

kubelet and the container runtime make it exist on that Node.

Next — chapter 3: Pods This will be one of the most important lessons because Kubernetes does not directly deploy raw containers.

We✓I cover:

- Why Pod exists
- Pod vs Container
- Single-container Pods
- Multi-container Pods
- Shared network
- Shared storage
- Pod IP
- Pod lifecycle
- Why Pods are disposable
- Why you should not create production Pods manually
- Sidecar pattern
- Init containers

And we✓I run through a realistic Java example:

```

Spring Boot container
  ↓
  Pod
  ↓
Worker Node

```

Then we✓I start reading our first proper Kubernetes YAML line by line.

Chapter 3 — What Happens Inside Kubernetes

CHAPTER FOCUS

Understand **What Happens Inside Kubernetes** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

Now we're going to move from "I know Kubernetes objects" to "I understand how Kubernetes actually works internally."

This is a very important chapter for you as a Java backend developer because once you understand this flow,

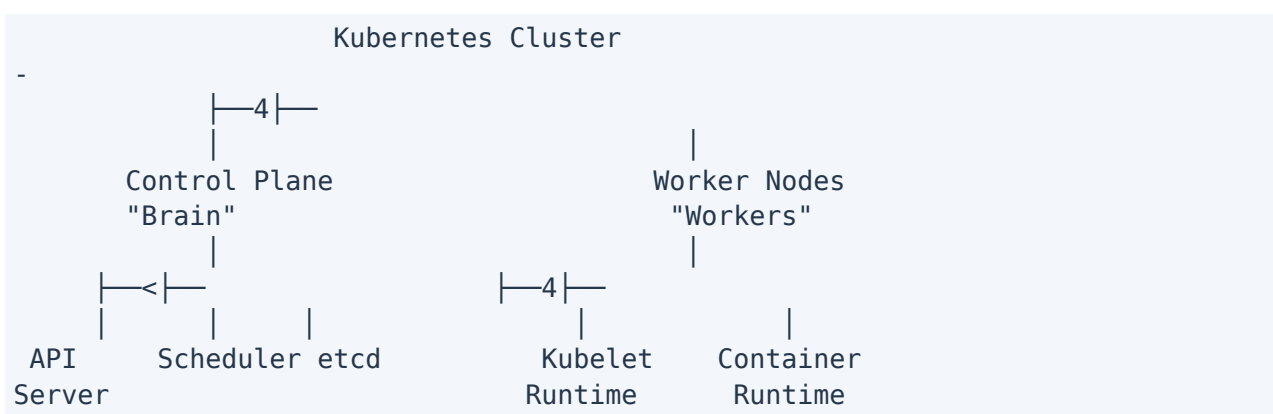
commands like:

```
kubectl apply
kubectl get pods
kubectl describe pod
kubectl logs
```

will stop feeling like magic.

First, the big picture

A Kubernetes cluster has two major sides:



Very simplified:

Control Plane

Decides what should happen.

Worker Node

Actually runs the workload.

| The components

There are several important components.

Control Plane

```
API Server
Scheduler
Controller Manager
etcd
```

Worker Node

```
Kubelet
Container Runtime
kube-proxy
```

Don't worry about memorizing all of them yet.

We✓I understand what each one does.

| The Kubernetes API Server

The most important component to understand first is:

kube-apiserver Almost everything talks through the API Server.

For example, when you run:

```
kubectl get pods
```

your command does not directly ask the worker Node.

Instead:

```
kubectl
  ↓
API Server
  ↓
Kubernetes state
```

| kubectl is basically a client

Think of:

```
kubectl
```

as a command-line client for the Kubernetes API.

For example:


```
kubectl create deployment employee --image=employee:v1
```

Conceptually:

```
You
↓
kubectl
↓
API Server
```

The API Server receives the request and validates it.

5. Where is Kubernetes state stored?

This brings us to:

etcd etcd is Kubernetes' distributed key-value data store.

You can think of it as:

The persistent database of the Kubernetes control plane.

It stores cluster state such as:

```
Pods
Deployments
Services
ConfigMaps
Secrets
Nodes
Namespaces
etc.
```

Conceptually:

```
API Server
↓
etcd
↓
Cluster state
```

Very important distinction

etcd does not run your application.

It stores information about the desired/current Kubernetes state.

For example:

```
Deployment:
employee-service
replicas: 3
```

```
image: employee:v2
```

That information is stored in the cluster's persistent state.

The actual application runs on worker Nodes.

| The Controller Manager

Now imagine you create:

```
replicas: 3
```

Kubernetes needs someone to continuously make sure there are actually three Pods.

That is where controllers come in.

The:

kube-controller-manager runs various controllers.

Think:

```
Desired state
  ↓
Controller
  ↓
Make reality match desired state
```

| Desired state vs actual state

This is perhaps the most important Kubernetes philosophy.

Suppose you say:

```
replicas: 3
```

You are declaring:

I want 3 replicas.

That is:

```
Desired State = 3
```

But currently maybe:

```
Actual State = 2
```

Kubernetes notices:

```
Desired = 3
Actual = 2
```

and takes action.

Create another Pod

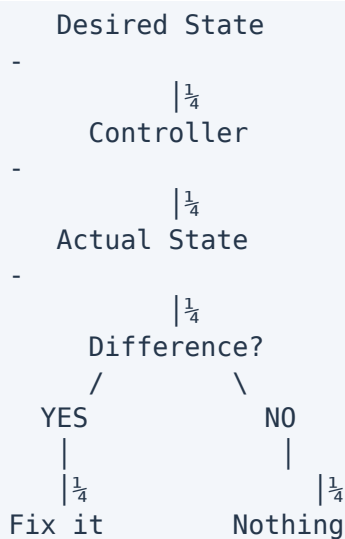
Eventually:

Desired = 3

Actual = 3

This is the reconciliation loop

Kubernetes constantly does something like:



This is called reconciliation.

Kubernetes is fundamentally a declarative system built around reconciliation.

Example: Pod crashes

Suppose Deployment says:

replicas = 3

Current Pods:

Pod A

Pod B

Pod C

Then:

Pod B crashes

Now:

```
Desired = 3  
Actual = 2
```

Controller notices.

It creates:

```
Pod D
```

Eventually:

```
Pod A  
Pod C  
Pod D
```

Back to:

```
3 replicas
```

You didn't manually recreate the Pod.

Kubernetes reconciled the state.

Now let's understand the Scheduler

The scheduler answers:

Which Node should run this unscheduled Pod?

Suppose:

```
Deployment  
replicas: 3
```

creates:

```
Pod A  
Pod B  
Pod C
```

Initially they don't have Nodes assigned.

Scheduler sees them.

```
Pod A → Node 1  
Pod B → Node 2  
Pod C → Node 3
```

The scheduler doesn't actually start containers.

It chooses the Node.

That is its job.

| Scheduler vs Kubelet

This distinction is extremely important.

Scheduler

“Pod A should run on Node 2.”

Kubelet

“Okay, I’ll make Pod A actually run on Node 2.”

So:

```
Scheduler  
→ placement
```

```
Kubelet  
→ execution/management on that Node
```

| Kubelet

Every worker Node has a:

Kubelet Think of the Kubelet as the agent responsible for making sure the Pods assigned to its Node are actually running.

For example:

```
Scheduler  
  ↓  
Pod A assigned to Node 2  
  ↓  
Node 2 Kubelet  
  ↓  
"Got it. I need to run Pod A."
```

| Kubelet doesn't create containers directly

This is another important distinction.

The Kubelet communicates with the:

Container Runtime Examples include:

```
containerd  
CRI-O
```

Historically Docker was commonly used, but Kubernetes removed the built-in Docker Engine integration (dockershim) starting with Kubernetes 1.24.

So today, think primarily:

```
Kubelet
  ↓
Container Runtime
  ↓
Container
```

15. What does the Container Runtime do?

The container runtime is responsible for things such as:

```
Pull image
Create container
Start container
Stop container
Delete container
Manage container lifecycle
```

For example:

```
Kubelet
  ↓
containerd

  ↓
Pull employee:v2
  ↓
Create container
  ↓
Start container
```

| Let's trace a complete deployment

Suppose you run:

```
kubectl create deployment employee \
  --image=employee:v1 \
  --replicas=3
```

Let's see what happens.

| Step 1 — kubectl

You execute:

```
kubectl create deployment employee \
  --image=employee:v1 \
  --replicas=3
```

kubectl sends an API request.

```
kubectl  
-  
  |  
API Server
```

Step 2 — API Server

API Server:

```
receives request  
  ↓  
authenticates  
  ↓  
authorizes  
  ↓  
validates request  
  ↓  
persists state
```

The desired Deployment is stored through the control plane's persistence layer, backed by etcd.

Conceptually:

```
API Server  
  ↓  
etcd  
  ↓  
Deployment desired state
```

Step 3 — Deployment Controller

The Deployment controller notices:

```
Desired:  
Deployment employee  
replicas = 3
```

It needs something to manage those Pods.

So it creates a:

```
ReplicaSet
```

Conceptually:

```
Deployment  
  ↓  
ReplicaSet  
  ↓  
3 Pods
```

This connects directly to what you learned earlier.

Step 4 — ReplicaSet Controller

The ReplicaSet says:

```
I need 3 Pods.
```

But initially:

```
Actual Pods = 0
```

Therefore:

```
Desired = 3  
Actual = 0
```

It creates:

```
Pod A  
Pod B  
Pod C
```

Step 5 — Pods are unscheduled

At this point you might have:

```
Pod A → no Node  
Pod B → no Node  
Pod C → no Node
```

They're waiting for the Scheduler.

Conceptually:

```
Pod A  
Pod B  
Pod C  
-  
  |  
  1/4  
Scheduler
```

Step 6 — Scheduler chooses Nodes

Suppose the cluster has:

```
Node 1  
Node 2  
Node 3
```

Scheduler evaluates:


```
CPU
Memory
Taints
Tolerations
Affinity
Anti-affinity
Topology
etc.
```

Then perhaps:

```
Pod A → Node 1
Pod B → Node 2
Pod C → Node 3
```

| Step 7 — Kubelet notices

Node 1's Kubelet sees:

```
Pod A assigned to me.
```

Node 2's Kubelet sees:

```
Pod B assigned to me.
```

Node 3's Kubelet sees:

```
Pod C assigned to me.
```

Each Kubelet now works to make its assigned Pod actually run.

| Step 8 — Kubelet talks to runtime

For Pod A:

```
Kubelet
  ↓
Container Runtime
  ↓
Pull image employee:v1
  ↓
Create container
  ↓
Start container
```

The same happens for Pods B and C.

| Step 9 — Application starts

Now your Spring Boot application starts:

```
JVM
```

```
↓
```

```
Spring Boot
```

```
↓
```

```
Beans
```

```
↓
```

```
Database connection
```

```
↓
```

```
Application startup
```

Kubernetes doesn't inherently understand Spring Boot.

It just knows about the container and the configured probes.

| Step 10 — Startup Probe

If configured:

```
Startup Probe
```

checks:

```
Is the application finished starting?
```

Until successful:

```
Startup = not complete
```

| Step 11 — Readiness Probe

Once startup succeeds, readiness becomes important:

```
Readiness
```

```
↓
```

```
Can this Pod serve requests?
```

If:

```
SUCCESS
```

the Pod becomes eligible for normal Service traffic.

| Step 12 — Liveness

Meanwhile:

```
Liveness
```

```
↓
```

```
Is this application still functioning?
```

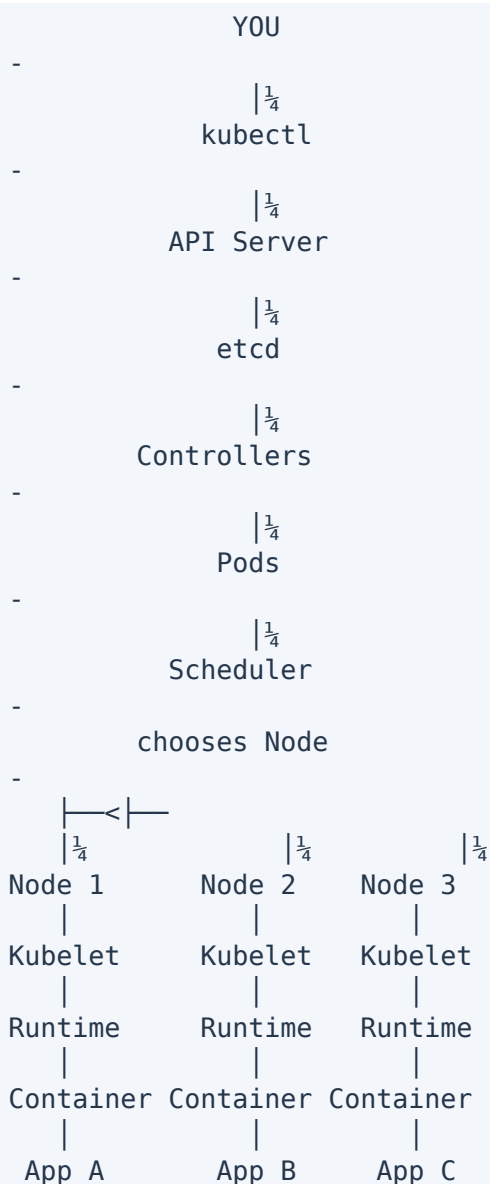
If it repeatedly fails:

```
Kubelet
  ↓
restart container
```

Now you've connected your previous chapter with today's architecture.

Complete flow

Here is the entire process:



That is the Kubernetes machinery behind a simple:

```
kubectl apply -f deployment.yaml
```

| But there's an important detail

You might ask:

How does the Kubelet know what it is supposed to run?

It continuously communicates with the Kubernetes API Server.

Conceptually:



The Kubelet also reports status back to the API Server.

So Kubernetes continuously compares:

Desired State
vs
Actual State

| The Node's Kubelet is not the controller

This distinction matters.

The Kubelet is primarily responsible for:

Pods assigned to THIS Node

It doesn't decide:

How many replicas should exist?

That is handled by controllers.

It doesn't decide:

Which Node should get this Pod?

That is handled by the Scheduler.

| Three components — three jobs

Memorize this:

Component Main responsibility

Controller What should exist?

Scheduler Where should it run?

Kubelet Make it run on that Node

For example:

Controller:

"I need 3 Pods."

Scheduler:

"Pod A → Node 1."

Kubelet:

"I'll make Pod A run on Node 1."

This is one of the most useful Kubernetes mental models.

33. What if the application crashes?

Suppose:

```
Pod A
Spring Boot
  ↓
Crash
```

The Kubelet notices the container has stopped.

Depending on the Pod's restart policy and workload configuration, the container can be restarted.

The Pod itself can remain associated with the same workload.

34. What if the Node crashes?

Now something more interesting happens.

Suppose:

```
Node 1
├── Pod A
└── Pod B
```

Node 1 completely disappears.

The control plane eventually observes:

Node 1 unhealthy/unreachable

Then workload controllers can recreate replacement Pods elsewhere, assuming the workload is managed by something like a Deployment and suitable Nodes exist.

For example:

Node 1 ×

Pod A → replacement Pod C → Node 2

Pod B → replacement Pod D → Node 3

This is a major part of Kubernetes' self-healing behavior.

| Container failure vs Node failure

Very important:

Container failure

Container crashes

↓

Kubelet

↓

restart container

Node failure

Entire Node unavailable

↓

Control plane detects problem

↓

Controllers reconcile workload

↓

Replacement Pods may be created

↓

Scheduler places them elsewhere

Different layers are involved.

36. What is kube-proxy?

You've already learned Services/networking, so now let's connect one more component:

kube-proxy

It runs on Nodes and helps implement Kubernetes Service networking.

Conceptually:

Client

↓

Service

↓

kube-proxy / networking rules

↓

Pod

Modern Kubernetes networking implementations can vary, and some clusters use alternative dataplane technologies, but conceptually kube-proxy is the traditional component responsible for implementing Service traffic rules on Nodes.

You don't need to dive deeply into kube-proxy internals yet.

37. What is CoreDNS?

You also encountered DNS when learning Services.

Kubernetes commonly runs:

```
CoreDNS
```

inside the cluster.

It allows things like:

```
employee-service
```

to resolve to the Service's virtual IP.

Conceptually:

```
employee-service
  ↓
  DNS
  ↓
Service IP
  ↓
Pods
```

So now:

```
CoreDNS
→ service discovery

kube-proxy/network dataplane
→ service traffic implementation
```

Control Plane vs Worker Node

Let's simplify everything.

Control Plane

```
API Server
Scheduler
Controller Manager
etcd
```

Its job:

Manage the cluster.

Worker Node

```
Kubelet  
Container Runtime  
kube-proxy / networking components
```

Its job:

Run workloads.

A real analogy

Imagine an organization.

Control Plane = Management

```
CEO  
Planning  
Database  
Schedulers
```

They decide:

We need 3 employees working on Project X.

Worker Nodes = Offices

```
Office A  
Office B  
Office C
```

Scheduler

Which office should each employee work from?

Kubelet

Make sure the employee assigned to this office actually has everything they need to work.

Container Runtime

Start and manage the employee's actual work environment.

It's not a perfect analogy, but it's useful.

40. What happens when you delete a Pod?

Suppose:

```
kubectll delete pod employee-abc
```

The API Server receives the request.

Then:


```
Pod deleted
  ↓
ReplicaSet notices:
desired = 3
actual = 2
  ↓
creates replacement Pod
  ↓
Scheduler selects Node
  ↓
Kubelet starts it
```

You didn't manually create the replacement.

The controller did it.

This is why you shouldn't manually manage Deployment Pods

Suppose your Deployment has:

```
replicas: 3
```

You delete:

```
employee-abc
```

Kubernetes says:

```
Wait!
I need 3.
Now I have 2.
```

So it creates another.

This is the whole point of declarative infrastructure.

You specify:

```
"I want 3."
```

rather than:

```
"Create these exact three containers and keep them alive."
```

Declarative vs Imperative

This is an important Kubernetes concept.

Imperative You tell Kubernetes exactly what action to perform:

```
kubectl create deployment employee ...
```

Declarative You describe the desired state:

```
replicas: 3
image: employee:v2
```

and:

```
kubectl apply -f deployment.yaml
```

Kubernetes figures out how to reach that state.

In production, declarative configuration is extremely important.

| Why YAML is so important

When you write:

```
apiVersion: apps/v1
kind: Deployment
...
```

you're not writing a script like:

```
do this
then do this
then do this
```

You're describing:

```
"This is what I want my system to look like."
```

Kubernetes then continuously works toward that state.

| One more mental model

Think of Kubernetes as:

```
Desired State
-
  | 1/4
  API Server
-
  | 1/4
  Controllers
-
  | 1/4
  Scheduler
-
  | 1/4
```

```

Node
-
  Kubelet
-
  Container Runtime
-
    | ¼
  Application

```

And feedback travels back:

```

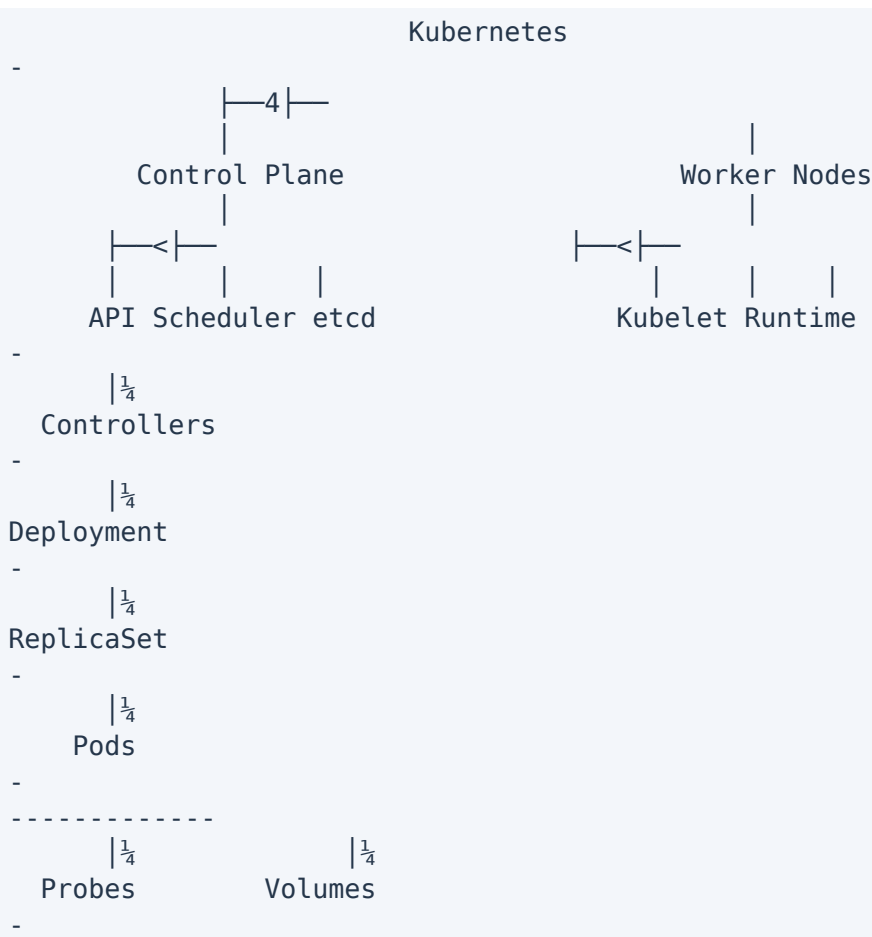
Application
  ↓
Kubelet
  ↓
API Server
  ↓
Cluster state

```

So Kubernetes is constantly observing and reconciling.

Let's connect all modules you've learned

You now have:



$\frac{1}{4}$
Readiness

-

$\frac{1}{4}$
Service

-

$\frac{1}{4}$
Network

Now the different Kubernetes objects aren't isolated topics anymore.

They're pieces of one system.

Your Java/Spring Boot perspective

Imagine you deploy:

```
employee-service.jar
```

to Kubernetes.

The journey is:

```
employee-service.jar
  ↓
Docker/OCI image
  ↓
Deployment
  ↓
ReplicaSet
  ↓
Pod
  ↓
Scheduler
  ↓
Node
  ↓
Kubelet
  ↓
Container Runtime
  ↓
Container
  ↓
JVM
  ↓
Spring Boot
  ↓
Actuator
  ↓
Readiness/Liveness
  ↓
Service
```

↓
Users

That is the entire journey from your Java application to a production Kubernetes workload.

| The most important things from this chapter

If you remember only seven things, remember these:

| API Server

Main gateway to the Kubernetes API

| etcd

Persistent store for Kubernetes cluster state

| Controller

Makes actual state converge toward desired state

| Scheduler

Chooses a Node for unscheduled Pods

| Kubelet

Manages Pods assigned to its Node

| Container Runtime

Actually creates/starts/stops containers

| Reconciliation

Desired state " \ Actual state
↓
Kubernetes acts

One final end-to-end example

You run:

```
kubectl apply -f employee.yaml
```

Kubernetes roughly goes through:

```
1. kubectl
  ↓
2. API Server
  ↓
3. etcd stores desired state
  ↓
4. Controller sees Deployment
  ↓
5. ReplicaSet created/updated
  ↓
6. Pods created

  ↓
7. Scheduler chooses Nodes
  ↓
8. Kubelet sees assigned Pods
  ↓
9. Container Runtime pulls image
  ↓
10. Container starts
  ↓
11. Spring Boot starts
  ↓
12. Startup Probe succeeds
  ↓
13. Readiness Probe succeeds
  ↓
14. Service sends traffic
  ↓
15. Liveness Probe keeps checking
```

And if something breaks:

```
Container crash
  ↓
Kubelet reacts

Pod disappears
  ↓
Controller reconciles
```

```
Node fails
  ↓
Cluster detects failure
  ↓
Controller creates replacement
  ↓
Scheduler chooses another Node
  ↓
Kubelet starts replacement
```

That is Kubernetes.

It's essentially a huge automated desired-state + reconciliation system.

Next chapter — chapter 14 The next logical topic is:

Kubernetes Namespaces, ResourceQuota & LimitRange We✓I answer:

How do large companies run hundreds of applications and teams inside the same Kubernetes cluster without everyone interfering with everyone else?

We✓I build something like:

```
Kubernetes Cluster
-
├─ namespace: payments
│   ├── Deployments
│   ├── Services
│   └── Pods
-
├─ namespace: employee
│   ├── Deployments
│   ├── Services
│   └── Pods
-
├─ namespace: analytics
│   ├── Deployments
│   ├── Services
│   └── Pods
```

Then we✓I introduce resource quotas, default resource limits, namespace isolation, RBAC, and eventually move into the security side of Kubernetes/OpenShift.

Workloads and Application Lifecycle

Chapter 4 — Pods

CHAPTER FOCUS

Understand **Pods** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

Now we come to one of the most important Kubernetes concepts:

Kubernetes does not run containers directly. Kubernetes runs Pods, and Pods contain containers.

The core hierarchy is:

```
Cluster
  ↓
Node
  ↓
Pod
  ↓
Container
```

If this relationship becomes clear, many later concepts—Deployments, Services, health checks, scaling, networking—become much easier.

1. Why did Kubernetes invent Pods?

You might ask:

If the actual thing I want to run is a container, why doesn't Kubernetes just manage containers directly?

Good question.

Let's start with a simple Spring Boot application:

```
employee-service
```

We containerize it:

```
employee-service:1.0
```

Without Kubernetes:

```
Container Runtime
  ↓
employee-service container
```

But Kubernetes adds another layer:

```

Kubernetes
  ↓
Pod
  ↓
employee-service container

```

Why?

Because Kubernetes sometimes needs to manage a group of tightly related containers as one unit.

So Kubernetes created an abstraction called a Pod.

2. What is a Pod?

One-sentence definition

A Pod is the smallest deployable unit in Kubernetes and represents one or more containers that run together and share certain resources.

Most Pods contain:

```

1 Pod
  ↓
1 Container

```

Example:

```

Employee Pod
├─ employee-service container

```

But a Pod can also contain multiple containers:

```

Employee Pod
├─ employee-service
├─ helper-container

```

We✓I understand why shortly.

Pod vs Container

This distinction is critical.

Container A container is where the actual application process runs.

Example:

```

Java 21
employee-service.jar

```

Pod The Pod is the Kubernetes wrapper around that container.

Think:

Pod



So:

Container = application runtime

Pod = Kubernetes execution unit

Java analogy

A rough analogy:

Container
=
Your Java application process

while:

Pod
=
The execution environment Kubernetes creates around it

Don't treat a Pod as another operating system.

A Pod is not:

Mini VM ×

It's a Kubernetes abstraction around one or more containers.

Single-container Pod

This is the most common design.

Example:

```

Pod: employee-service-pod
-
  Container:
    employee-service:1.0
  
```

Worker Node:

Worker Node

```
-  
├─ Linux  
├─ kubelet  
├─ Container Runtime  
-  
├─ Pod  
  └─ Employee Container
```

In typical microservices architectures:

```
Employee Service → one Pod/container  
Payment Service → one Pod/container  
Order Service   → one Pod/container
```

6. Do multiple microservices go inside one Pod?

Usually:

No.

You generally should not do:

```
Pod  
├─ Employee Service  
├─ Payment Service  
├─ Order Service  
└─ Notification Service
```

Why?

Because these applications have different:

```
Scaling requirements  
Deployment cycles  
Failure patterns  
Versions
```

```
Resource requirements  
Ownership
```

For example:

Employee Service needs 2 replicas

Payment Service needs 10 replicas

If both were in the same Pod, they would scale together.

That is undesirable.

Instead:

```
Employee Pod
├─ Employee Container

Payment Pod
├─ Payment Container

Order Pod
├─ Order Container
```

Each service can now scale independently.

7. Then why support multiple containers in a Pod?

Because sometimes two containers are so closely related that they should:

- run together,
- start on the same Node,
- share networking,
- share storage,
- and have the same lifecycle.

For example:

```
Application Container
+
Logging Helper Container
```

Conceptually:

```
Pod
-
├─ Main Application
-
├─ Helper / Sidecar
```

This is often called a:

Sidecar pattern

Sidecar analogy

Imagine a politician or executive traveling with a personal security officer.

Wherever the main person goes:

```
Executive
+
Security Officer
```

They travel together.

Similarly:

```
Main Application Container
+
Supporting Container
```

are placed in the same Pod.

They are:

```
scheduled together
run on same Node
share Pod networking
can share volumes
```

Example sidecar

Suppose your Java application writes logs to:

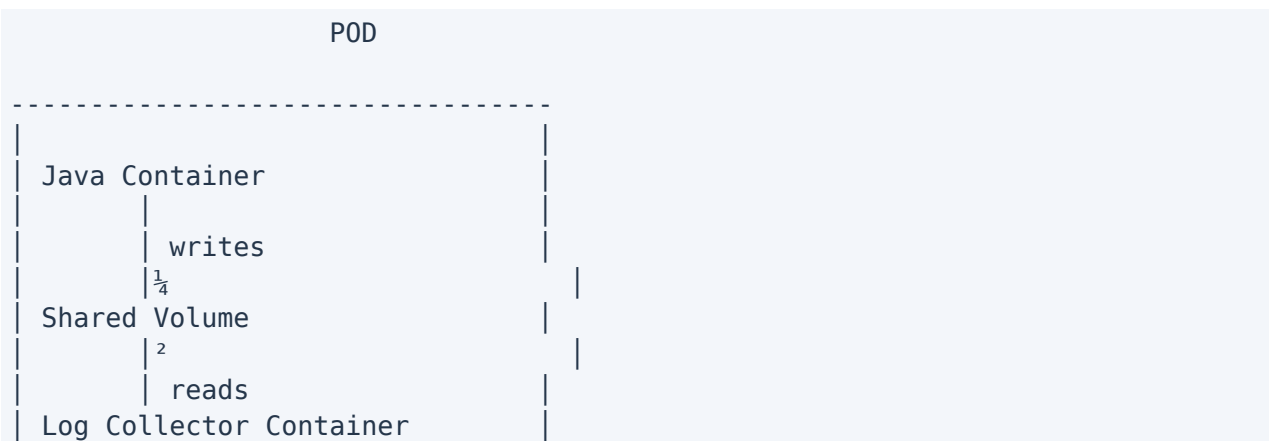
```
/var/log/app.log
```

You could have:

```
Pod
-
|— Java Application Container
|   ↓
|   writes app.log
-
|— Log Collector Container
|   ↓
|   reads app.log
|   ↓
|   sends logs elsewhere
```

Both containers can share a volume.

Architecture:



```
|  
-----|
```

That is a valid reason for multiple containers in one Pod.

10. What do containers inside a Pod share?

Two important things:

```
Networking  
Storage, when configured
```

Let's understand networking first.

| Every Pod gets an IP address

Suppose Kubernetes creates:

```
Employee Pod
```

The Pod may receive an IP like:

```
10.244.1.23
```

Another:

```
Payment Pod  
10.244.2.15
```

Another:

```
Order Pod  
10.244.3.8
```

So:

```
Employee Pod → 10.244.1.23  
Payment Pod → 10.244.2.15  
Order Pod    → 10.244.3.8
```

Important:

The IP belongs conceptually to the Pod, not independently to each container inside that Pod.

| Multiple containers inside one Pod share networking

Suppose:

```
Pod IP:
10.244.1.23
```

Inside it:

```
Container A
Container B
```

Both share the Pod's network namespace.

Conceptually:

```
Pod
IP = 10.244.1.23

├ Container A
├ Container B
```

If Container A listens on:

```
8080
```

and Container B listens on:

```
9090
```

they are reachable conceptually as:

```
10.244.1.23:8080
10.244.1.23:9090
```

Containers inside one Pod can use localhost

This is very useful.

Imagine:

```
Pod
├ Java App → port 8080
├ Helper   → port 9000
```

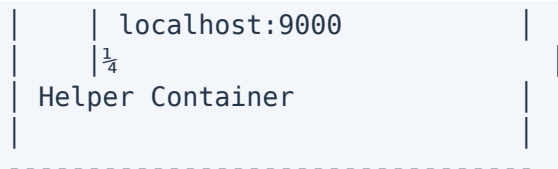
Java App can communicate with the helper using:

```
localhost:9000
```

because they share the same Pod-level network environment.

Conceptually:

```
├ POD ─┬─
        │
        │ Application
        │
        └─┬─
```

Important implication: Port conflicts

Because containers in the same Pod share networking, they cannot both normally bind to the exact same IP/port combination.

For example:

```
Container A → 8080
Container B → 8080
```

would conflict.

Better:

```
Container A → 8080
Container B → 9090
```

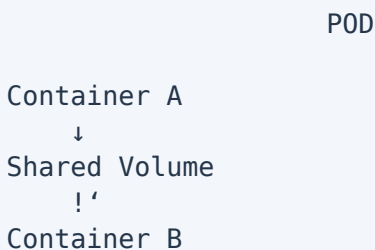
Pods can share storage

Containers have their own filesystem layers.

But Kubernetes can attach a volume to a Pod.

Then multiple containers inside that Pod can mount the same volume.

Example:



We'll go much deeper into volumes later.

For now:

Pod containers can share storage when you configure a common volume.

16. Why Pods—not just groups of containers?

Because Kubernetes wants one unit it can:

```
Schedule
Start
Stop
Replace
Give an IP
Attach volumes to
Apply lifecycle rules to
```

That unit is:

Pod So the Scheduler doesn't typically say:

```
Container A → Node 1
```

It says:

```
Pod A → Node 1
```

Then every container belonging to Pod A goes together onto Node 1.

Pod scheduling

Suppose:

```
Pod A
├─ Container X
├─ Container Y
```

Kubernetes does not do:

```
Container X → Node 1
Container Y → Node 2
```

No.

They belong to one Pod.

Therefore:

```
Pod A → Node 1
```

means:

```
Node 1

Pod A
├─ Container X
├─ Container Y
```

They stay together.

Pod lifecycle

Now we reach another important idea:

Pods are disposable.

Don't think of Pods as permanent servers.

Traditional mindset:

```
Server A  
runs for years
```

Kubernetes mindset:

```
Pod X  
may live
```

```
minutes  
hours  
days
```

```
and be replaced
```

If a Pod dies, Kubernetes often doesn't "repair the same Pod."

Instead:

```
Old Pod x  
  ↓  
New Pod created '
```

Example: Pod replacement

Suppose:

```
employee-pod-abc  
IP = 10.244.1.15
```

It crashes.

Kubernetes may create:

```
employee-pod-xyz  
IP = 10.244.3.27
```

Notice:

```
Different Pod name  
Different IP
```

Therefore you should not build systems assuming:

```
Pod identity is permanent ×  
Pod IP is permanent ×
```

This is very important when we later learn Services.

Why Pod IP changes matter

Imagine Payment Service directly calls:

```
Employee Pod  
10.244.1.15
```

Employee Pod dies.

Replacement:

```
10.244.3.27
```

Payment Service still tries:

```
10.244.1.15 ×
```

Problem.

Kubernetes solves this using a stable abstraction:

```
Service
```

Later we✓I get:

```
Payment Service  
  ↓  
employee-service  
  ↓  
Healthy Employee Pods
```

So applications don't need to track individual Pod IPs.

Pod lifecycle phases

You✓I often see Pod statuses such as:

```
Pending  
Running  
Succeeded  
Failed  
Unknown
```

Let's understand them.

Pending Kubernetes accepted the Pod, but it is not fully running yet.

Possible reasons:

```
waiting for scheduling
pulling image
waiting for storage
waiting for resources
```

Example:

```
Pod
Status = Pending
```

Running

```
Status = Running
```

means the Pod has been assigned to a Node and its containers have been created; at least one container is running, starting, or restarting.

Important:

Running does not automatically mean your application is healthy and ready for traffic.

That distinction becomes important when we learn:

```
Readiness Probe
Liveness Probe
```

Succeeded

Typical with short-running workloads.

Imagine:

```
Generate monthly report
  ↓
Job finishes
```

The Pod runs its work and exits successfully.

Then:

```
Pod = Succeeded
```

This is common for Kubernetes Jobs.

Failed

The Pod's containers terminated and at least one ended unsuccessfully in a way that will not be restarted within that Pod.

Conceptually:

```
Application
  ↓
Exit code failure
  ↓
Pod eventually Failed
```

Pod vs Container status

Be careful.

A Pod may contain:

```
Container A
Container B
```

Each container has its own state.

For example:

```
Pod

Container A → Running
Container B → Waiting
```

Pod-level status summarizes what's happening, but troubleshooting often requires looking at individual container states.

Container restart vs Pod replacement

This distinction is important.

Sometimes:

```
Container crashes
```

and kubelet can restart the container inside the same Pod.

Example:

```
Pod A
  ↓
Container crashes
  ↓
Container restarted
```

↓
Still Pod A

But in other situations, such as the Pod being deleted or a Node failure, Kubernetes may create an entirely new Pod:

Pod A ×
↓
Pod B ' '

We'll differentiate these cases in depth later.

Restart count

You may eventually run:

```
kubectl get pods
```

and see:

NAME	READY	STATUS	RESTARTS
employee-abc	1/1	Running	0
payment-xyz	1/1	Running	4

What does:

```
RESTARTS = 4
```

mean?

The container inside that Pod has restarted four times.

This can indicate:

```
application crashes
health probe failures
configuration issues
runtime problems
```

28. What does READY 1/1 mean?

Suppose:

```
READY = 1/1
```

Generally:

```
1 ready container
/
1 total container
```

For a two-container Pod:

2/2

means both are ready.

You could see:

1/2

meaning one of two containers is not ready.

Our first Pod YAML

Now let's create a simple Pod definition.

```
apiVersion: v1
kind: Pod

metadata:
  name: employee-pod
  labels:
    app: employee

spec:
  containers:
    - name: employee-container
      image: myregistry/employee-service:1.0
      ports:
        - containerPort: 8080
```

Let's go line by line.

apiVersion

```
apiVersion: v1
```

Kubernetes exposes resources through APIs.

This says:

Use API version v1 for this resource definition.

Different Kubernetes resource types belong to different API groups and versions.

Examples you'll later see:

```
apiVersion: v1
```

for things such as Pods and Services.

While Deployments use:


```
apiVersion: apps/v1
```

Don't memorize all of these yet.

| kind

```
kind: Pod
```

This tells Kubernetes what resource we want to create.

Examples:

```
Pod
Service
Deployment
ConfigMap
Secret
Job
```

So Kubernetes reads:

```
kind: Pod
```

and understands:

“The user wants a Pod.”

| metadata

```
metadata:
  name: employee-pod
```

Every Kubernetes object has metadata.

Here we give it a name:

```
employee-pod
```

Then:

```
labels:
  app: employee
```

Labels are key-value metadata attached to Kubernetes resources.

Example:

```
app = employee
```

We'll later use labels to select groups of Pods.

Labels are extremely important.

| spec

spec:

spec describes the desired state of the object.

You are essentially saying:

“This is how I want this Pod to look.”

Inside:

containers:

we specify which containers should exist inside the Pod.

| Container definition

- **name:** employee-container

The container inside the Pod is named:

employee-container

Then:

image: myregistry/employee-service:1.0

means:

Run a container using this image.

Flow:

```
Pod
↓
employee-container
↓
employee-service:1.0
```

| containerPort

ports:

- **containerPort:** 8080

This documents the application port used by the container.

Your Spring Boot app might listen on:

```
8080
```

Important beginner point:

containerPort: 8080 by itself does not expose your application to the internet.

It doesn't automatically create external networking.

Later, Services and Ingress/Routes handle access.

| Apply the Pod

You could save the YAML as:

```
employee-pod.yaml
```

and execute:

```
kubectl apply -f employee-pod.yaml
```

Flow:

```
kubectl
↓
API Server
↓
Pod definition accepted
↓
Scheduler
↓
Worker Node selected
↓
kubelet
↓
Container Runtime
↓
employee-service container starts
```

This connects chapter 2 directly to chapter 3.

| Check Pods

You can then run:

```
kubectl get pods
```

Example:

NAME	READY	STATUS	RESTARTS	AGE
employee-pod	1/1	Running	0	30s

Interpretation:

```
NAME
employee-pod

READY
1 of 1 containers ready

STATUS
Running

RESTARTS
0

AGE
30 seconds
```

| Get more information

Run:

```
kubectl describe pod employee-pod
```

This provides much more detail.

You✓I commonly see information such as:

```
Node
Pod IP
Container image
Container state
Events
Volumes
Conditions
Restart count
```

This command is one of the most important troubleshooting tools in Kubernetes.

| Check logs

Because your Java application is inside the container:

```
kubectl logs employee-pod
```

might show:

```
Starting EmployeeServiceApplication
Tomcat started on port 8080
Started EmployeeServiceApplication
```

Think:

```
kubectl logs
  ↓
Pod
  ↓
Container stdout/stderr
```

40. What about a multi-container Pod?

Suppose:

```
Pod
├─ employee-container
└─ log-agent
```

Then:

```
kubectl logs employee-pod
```

may require specifying which container you want:

```
kubectl logs employee-pod -c employee-container
```

or:

```
kubectl logs employee-pod -c log-agent
```

Because one Pod can contain multiple containers.

Run a command inside the container

You✓I often use:

```
kubectl exec
```

For example:

```
kubectl exec -it employee-pod -- /bin/sh
```

Conceptually:

```
Your terminal
  ↓
Kubernetes API
  ↓
Pod
  ↓
Container shell
```

This is useful for troubleshooting.

However, not every container image has `/bin/sh` or other shell utilities, especially minimal production images.

| Delete the Pod

You could run:

```
kubectll delete pod employee-pod
```

Since this is a directly created standalone Pod:

```
employee-pod x
```

and that's it.

Kubernetes will not necessarily create it again.

Why?

Because you only told Kubernetes:

“Create this Pod.”

You didn't tell it:

“Always ensure one Employee Pod exists.”

That is a massive distinction.

| Standalone Pod vs Deployment

Suppose you directly create:

```
Pod
```

Then delete it:

```
Pod x
```

There is no higher-level controller saying:

```
Desired replicas = 1
```

So nothing recreates it.

But suppose we use a Deployment:

```
Deployment
Desired replicas = 1
  ↓
ReplicaSet
  ↓
Pod
```

Delete the Pod:

```
Pod x
```

Controller sees:

```
Desired = 1  
Current = 0
```

and creates:

```
New Pod '
```

This is why real production applications generally use Deployments, not manually created standalone Pods.

Very important principle

When learning Kubernetes, you will often create raw Pods for practice.

But in production:

```
Directly create Pods
```

is usually not the normal deployment approach.

Instead:

```
Deployment  
  ↓  
ReplicaSet  
  ↓  
Pods
```

We✓I cover this deeply in the next chapter.

45. Why are Pods disposable?

Kubernetes is designed around replacing workloads instead of treating machines/processes as pets.

Traditional approach:

```
Server-01 has problem
```

```
SSH into it  
repair it  
protect it  
keep it alive
```

Kubernetes philosophy is closer to:

```
Pod has problem
```

```
Replace it
```

This is sometimes described using the analogy:

Pets vs Cattle

A pet has a unique identity.

A cattle-style workload is replaceable.

It's only an analogy, but the architectural lesson is:

Design application instances to be replaceable whenever possible.

This is why stateless applications work very well

Imagine:

Employee REST API

It receives:

GET /employees/100

and reads data from an external database.

The Pod itself doesn't contain permanent business data.

Therefore:

Pod A dies

New Pod B starts

Application continues

This is called a stateless application.

Kubernetes handles stateless applications particularly naturally.

47. What about databases?

A database is different.

Imagine:

Database Pod

storing permanent data only inside the container filesystem.

If that Pod disappears:

Pod ×
Data ×

That would be disastrous.

So stateful workloads need:

```
Persistent storage
PersistentVolumes
PersistentVolumeClaims
StatefulSets
```

We✓I cover those later.

For now:

```
Stateless REST service
→ easier to replace

Stateful database
→ needs persistence
```

| Init Containers

Another useful Pod feature is an init container.

An init container runs before the main application containers.

Example:

```
Pod starts
↓
Init Container
↓
prepare configuration
↓
finish
↓
Main Java Container starts
```

Suppose your application requires some startup preparation:

```
download config
wait for prerequisite

initialize files
change permissions
```

An init container can perform that work.

| Init container architecture

POD START

↓



Init containers run sequentially before the ordinary application containers start.

Sidecar vs Init Container

Don't confuse them.

Init Container

```
Runs BEFORE application
  ↓
Completes
  ↓
Main app starts
```

Sidecar-style supporting container

```
Runs alongside application
```

Think:

```
Init Container
=
setup worker
```

while:

```
Sidecar
=
ongoing companion
```

Example

Imagine a Java app needs a configuration file generated before startup.

You could have:

Pod

Step 1:

```
Init Container  
creates:  
/shared/application.properties
```

```
Step 2:  
Java Container  
reads:  
/shared/application.properties
```

Both use a shared volume.

Architecture:

```
Init Container  
  ↓ write  
  
Shared Volume  
  
  ↓ read  
Java Container
```

Pod naming in real Deployments

When Kubernetes controllers create Pods, names may look like:

```
employee-service-7c9b6f97c8-f6k22  
employee-service-7c9b6f97c8-jms9d  
employee-service-7c9b6f97c8-rx4x7
```

Don't worry about the generated suffixes yet.

The important thing is:

```
Deployment  
employee-service
```

↓

```
multiple generated Pods
```

You generally don't care about the individual Pod names as permanent identities.

Labels are more important than Pod names

Suppose:

```
employee-service Pod A  
employee-service Pod B  
employee-service Pod C
```

All have:

Labels:**app:** employee

Kubernetes can refer to the group using that label:

`app = employee`

rather than individually managing:

`pod-abc
pod-def
pod-xyz`

This becomes fundamental for Services and Deployments.

Think:

`Pod name
=
individual identity``Label
=
group/category`

Example of labels

Pods:

`Pod 1
app=employee
env=prod``Pod 2
app=employee
env=prod``Pod 3
app=payment
env=prod``Pod 4
app=employee
env=test`

You could logically select:

`app=employee
env=prod`

and get:

```
Pod 1
Pod 2
```

Labels are Kubernetes' powerful grouping mechanism.

| Pod and Node relationship

Remember:

One Pod runs on exactly one Node at a time.

Example:

```
Node A
├── Pod 1
└── Pod 2
```

```
Node B
├── Pod 3
├── Pod 4
└── Pod 5
```

A single Pod does not span multiple Nodes.

So this is impossible:

```
Pod A
Container 1 → Node 1
Container 2 → Node 2
```

×

Instead:

```
Node 1
Pod A
├── Container 1
└── Container 2
```

,

56. What if the Node dies?

Suppose:

```
Node A ×

Pod A
Pod B
Pod C
```

Those Pods are no longer running there.

Kubernetes does not teleport those same Pod instances to Node B.

Instead, controllers may create replacement Pods on healthy Nodes.

Conceptually:

Node A
Pod-123 ×

↓

Node B
Pod-789 '

New identity.

Possibly new IP.

Kubernetes identity hierarchy

Let's consolidate:

Cluster

```
-  
├── Node A  
│   ├── Pod 1  
│   │   └── Container A  
│   ├── Pod 2  
│   │   └── Container B  
└── Node B  
    ├── Pod 3  
    │   ├── Container C  
    │   └── Container D
```

If I ask:

Which object does the Scheduler schedule?

Answer:

Pod

Not individual containers.

Real Spring Boot example

Suppose you build:

```
payment-service.jar
```

Container image:

```
company/payment-service:1.0
```

Then Kubernetes architecture is:

```
Kubernetes Cluster
      ↓
Worker Node
      ↓
Payment Pod
      ↓
Payment Container
      ↓
Java JVM
      ↓
payment-service.jar
```

This is the complete stack:

```
Physical / Virtual Server
      ↓
Linux
      ↓
Container Runtime
      ↓
Pod
      ↓
Container
      ↓
JVM
      ↓
Spring Boot
```

59. Is the JVM shared between containers?

No.

Suppose:

```
Employee Container → Java 17
Payment Container → Java 21
```

Each container can contain its own required user-space runtime.

They may still share the host's Linux kernel, but the Java runtime within each container can differ.

Conceptually:

```
Linux Kernel
```

```
-  
  ├── Employee Container  
  |   ├── Java 17  
-  
  ├── Payment Container  
  |   ├── Java 21
```

This is one reason containers solve dependency/version conflicts nicely.

60. Does each Pod have its own operating system?

No.

This is worth repeating.

You might see:

```
Pod  
├── Ubuntu-based image
```

and assume:

“This Pod has its own Ubuntu OS.”

Not exactly.

The image may contain Ubuntu user-space files and libraries.

But the container still uses the host Node’s kernel.

So:

```
Node Linux Kernel  
    !  
Pod containers use it
```

They don’t boot an independent kernel like a VM does.

Pod resource boundaries

Later you may define resources such as:

```
resources:  
  requests:  
    memory: "512Mi"  
    cpu: "500m"  
  limits:  
    memory: "1Gi"  
    cpu: "1"
```

These are actually specified at the container level within a Pod.

Kubernetes then reasons about Pod resource needs when scheduling.

For now, simply remember:

```
Pod needs resources
  ↓
Scheduler uses resource requirements
  ↓
Select suitable Node
```

| Common misconception: Scaling a container

In Kubernetes we normally say:

```
Scale the Deployment
```

which ultimately creates more Pods.

For example:

```
1 Pod
  ↓
scale to 3
  ↓

Pod A
Pod B
Pod C
```

Each Pod normally contains one application container.

So people informally say:

“We’re scaling containers.”

But Kubernetes’ actual workload abstraction is usually:

```
more Pods
```

| Pod is not the same as application

Suppose Employee Service has:

```
3 replicas
```

Then:

```
Employee Service
  ↓
3 Pods
```

So:

Application " Pod

Instead:

One application/service
may have many Pod replicas

This becomes important when discussing Services.

| Application architecture example

Imagine:

Gateway
Employee Service
Payment Service

Deployment might eventually look like:

Gateway
↓
2 Pods

Employee
↓
3 Pods

Payment
↓
5 Pods

Total:

10 Pods

These may be distributed across several Worker Nodes.

| A common interview question

What is a Pod in Kubernetes? A good answer:

A Pod is the smallest deployable and schedulable unit in Kubernetes. It contains one or more closely related containers that run on the same Node and share the Pod's network namespace and can share storage volumes. That is much stronger than:

"Pod contains Docker container."

| Another interview question

Why not run each container directly without Pods? Good answer:

Kubernetes uses Pods as a higher-level execution abstraction so one or more tightly coupled containers can be scheduled together, share networking and volumes, and be managed through a common lifecycle.

| Another interview question

Can containers inside one Pod run on different Nodes? Answer:

No. All containers in a Pod are scheduled together onto the same Node.

| Another interview question

Does every container inside a Pod get a separate Pod IP? Normally no.

They share the Pod's network namespace and therefore the same Pod-level IP address, while listening on different ports.

| Another interview question

Are Pod IPs permanent? No.

Pods are ephemeral.

When a Pod is replaced, it can receive:

```
new name
new UID
new IP
```

Therefore stable application communication generally uses Kubernetes Services rather than individual Pod IPs.

| Troubleshooting thought exercise

Imagine:

```
employee-service
Status:
CrashLoopBackOff
```

What might this mean at a high level?

Likely:

```
Pod exists
↓
Container starts
↓
Application crashes
```

```
↓
kubelet restarts it
↓
crashes again
↓
repeated restart cycle
```

Hence:

```
Crash
Loop
BackOff
```

We✓I study this properly in troubleshooting.

| Another state you✓I see: ImagePullBackOff

Suppose your YAML says:

```
image: company/employee-service:99.0
```

but that image doesn't exist or the registry requires credentials Kubernetes doesn't have.

Flow:

```
Pod scheduled
↓
kubelet asks runtime to pull image
↓
image pull fails
↓
retry
↓
ImagePullBackOff
```

Notice how your architecture knowledge tells you roughly where the problem occurs:

```
Scheduler '
Node assignment '
kubelet/runtime image pull x
```

| Architecture reasoning is more important than memorization

Instead of memorizing:

```
ImagePullBackOff → run command X
```

think:

What stage failed?

```

kubectll
↓
API Server
↓
Scheduler
↓
kubelet
↓
Image pull !• FAILURE HERE
↓
Container

```

Then investigate logically.

That is how I want you to learn Kubernetes.

Complete Pod mental model

Keep this diagram:



What you should remember from chapter 3

If you remember only ten points, remember these:

- | **Pod is the smallest Kubernetes deployable/schedulable unit.**
- | **A Pod contains one or more containers.**
- | **Most application Pods contain one main application container.**
- | **All containers in a Pod run on the same Node.**
- | **Containers inside a Pod share the Pod network namespace.**
- | **They can communicate using localhost .**
- | **They can share Kubernetes volumes.**
- | **Every Pod gets a Pod-level IP.**
- | **Pods are ephemeral and replaceable.**
- | **Production apps are normally managed using higher-level controllers such as Deployments, rather than manually**

created standalone Pods.

| **Check your understanding**

Try these before looking at the answers.

Q1 Which object does the Kubernetes Scheduler place onto a Worker Node?

A. Container B. Pod C. Docker image D. Deployment

Answer: B — Pod

Q2 If a Pod has two containers, can Kubernetes put one on Node A and one on Node B?

No.

Both run on the same Node.

Q3 Suppose:

Pod IP = 10.10.2.25

Container A = port 8080

Container B = port 9000

Can Container A call Container B using:

```
localhost:9000
```

Yes, because they share the Pod's network namespace.

Q4 A Pod dies and a replacement is created. Can you assume it will have the same IP?

No.

Q5 Why don't we normally place Employee Service and Payment Service in the same Pod?

Because they should be independently:

```
deployed
scaled
updated
restarted
managed
```

Mini exercise Suppose your application architecture is:

```
Gateway
Employee Service
Payment Service
```

Requirements:

```
Gateway          → 2 replicas
Employee Service → 3 replicas
Payment Service  → 5 replicas
```

Assume one application container per Pod.

How many Pods do we need?

```
2 + 3 + 5 = 10 Pods
```

Possible distribution:

```
Node A
├─ Gateway Pod
├─ Employee Pod
├─ Employee Pod
└─ Payment Pod
```

```
Node B
├─ Gateway Pod
├─ Employee Pod
├─ Payment Pod
└─ Payment Pod
```

```
Node C
```

```
|— Payment Pod
|— Payment Pod
```

The Scheduler decides the actual placement based on resources and constraints.

We now have:

```
chapter 1
Why Kubernetes exists

      ↓

chapter 2
Kubernetes architecture

      ↓

chapter 3
Pod – smallest workload unit
```

And our mental model has grown to:

```
User
  ↓
kubectl
  ↓
API Server
  ↓
Controllers
  ↓
Scheduler
  ↓
Worker Node
  ↓
kubelet
  ↓
Container Runtime
  ↓
Pod
  ↓
Container
  ↓
Java Application
```

Next — chapter 4: ReplicaSet & Deployment This is where Kubernetes starts feeling like a real orchestration system.

We✓I answer:

Why shouldn't we create Pods manually?

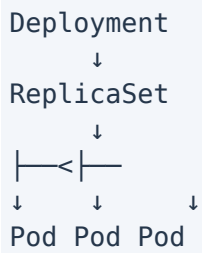
Who recreates a Pod when it disappears?

What exactly is a ReplicaSet?

What is a Deployment?

Why do we normally deploy applications using Deployments?

We✓I build this relationship carefully:



Then we✓I simulate an application upgrade from:

employee-service:v1

to:

employee-service:v2

and see how Kubernetes performs a rolling update without taking every Pod down at once.

Chapter 5 — ReplicaSets, Labels, Selectors, and Deployments

CHAPTER FOCUS

Understand **ReplicaSets, Labels, Selectors, and Deployments** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

This chapter answers one of the most important Kubernetes questions:

If Pods are disposable, who makes sure the right number of Pods always exist?

The answer comes in two layers:

```
Deployment
  ↓
ReplicaSet
  ↓
Pods
```

This relationship is fundamental in Kubernetes.

1. Why not create Pods manually?

Suppose you create a Pod directly:

```
apiVersion: v1
kind: Pod

metadata:
  name: employee-pod

spec:
  containers:
    - name: employee
      image: company/employee-service:1.0
```

You apply it:

```
kubectl apply -f employee-pod.yaml
```

Result:

```
employee-pod '
```

Everything looks good.

But then somebody runs:

```
kubectl delete pod employee-pod
```

Now:

```
employee-pod ×
```

What happens?

Nothing recreates it.

Why?

Because Kubernetes only received this instruction:

Create this particular Pod.

It did not receive this instruction:

Always keep one Employee Service Pod running.

That second requirement is where controllers become useful.

Desired-state thinking

Remember our earlier concept:

```
Desired State  
vs  
Current State
```

Instead of saying:

Create Pod A.

we want to say:

I always want 3 Employee Service Pods.

Like:

```
Desired replicas = 3
```

Current:

```
Pod A '  
Pod B '  
Pod C '
```

Everything matches.

If one disappears:

```
Pod A '  
Pod B ×  
Pod C '
```

then:

```
Desired = 3
Current = 2
```

Something should notice the mismatch and create a replacement.

That something is a controller.

One such Kubernetes resource is the:

ReplicaSet

3. What is a ReplicaSet?

One-sentence definition

A ReplicaSet ensures that a specified number of identical Pod replicas are running at any given time.

For example:

```
ReplicaSet
Desired replicas = 3
```

It manages:

```
Pod A
Pod B
Pod C
```

Architecture:

```
ReplicaSet
replicas = 3
-
  └─ Pod 1
  └─ Pod 2
  └─ Pod 3
```

If:

```
Pod 2 x
```

ReplicaSet detects:

```
Desired = 3
Current = 2
```

and creates:

```
Pod 4 '
```

Now:

```
Pod 1 '  
Pod 3 '  
Pod 4 '
```

Again:

```
Current = 3  
Desired = 3
```

| ReplicaSet is a controller

This is another important mental model.

ReplicaSet is not just:

```
"a collection of Pods"
```

It is a controller-driven Kubernetes object.

Its job is essentially:

```
Observe current Pods  
    ↓  
Compare with desired count  
    ↓  
Create/delete Pods when needed
```

That is reconciliation again.

| Example

Suppose:

```
Desired = 5 Payment Pods
```

Current:

```
Payment-1 '  
Payment-2 '  
Payment-3 '
```

ReplicaSet calculates:

```
5 desired  
-  
3 current  
=  
2 missing
```

It creates:

```
Payment-4  
Payment-5
```

Now:

```
5 / 5 '
```

The opposite also works.

Suppose desired replicas change:

```
5 → 2
```

ReplicaSet sees:

```
Current = 5  
Desired = 2
```

and removes three Pods.

6. How does ReplicaSet know which Pods belong to it?

This is where the labels we learned earlier become important.

Suppose our Pods have:

```
Labels:  
  app: employee
```

The ReplicaSet can say:

```
Manage Pods where:  
  
app = employee
```

This is done using a:

Selector

Labels and selectors

Example Pods:

```
Pod A  
app=employee  
  
Pod B  
app=employee  
  
Pod C  
app=payment
```

ReplicaSet selector:

```
app=employee
```

Therefore it manages:

```
Pod A
Pod B
```

Not:

```
Pod C
```

Conceptually:

```
ReplicaSet
selector:
app=employee
```

↓ finds

```
-----
| Pod A app=employee |
| Pod B app=employee |
-----
```

Selectors are absolutely central to Kubernetes.

ReplicaSet YAML

A simplified ReplicaSet could look like:

```
apiVersion: apps/v1
kind: ReplicaSet

metadata:
  name: employee-rs

spec:
  replicas: 3

  selector:
    matchLabels:
      app: employee

  template:
    metadata:
      labels:
        app: employee

    spec:
      containers:
        - name: employee
          image: company/employee-service:1.0
```

Let's understand the structure.

| replicas

```
replicas: 3
```

This means:

```
Desired Pod count = 3
```

ReplicaSet continuously tries to maintain that number.

| selector

```
selector:
  matchLabels:
    app: employee
```

This tells the ReplicaSet:

Consider Pods with the label app=employee as belonging to me.

| template

```
template:
```

This is the Pod template.

It answers:

If I need to create a new Pod, what should that Pod look like?

Inside it:

```
metadata:
  labels:
    app: employee
```

and:

```
spec:
  containers:
    - name: employee
      image: company/employee-service:1.0
```

So think:

```
ReplicaSet
-
```



```
|— Desired count = 3
-
|— Which Pods?
|   app=employee
-
|— If a Pod is missing:
|   use this template
|   to create one
```

| Very important: selector and labels must match

Suppose:

```
selector:
  matchLabels:
    app: employee
```

but your Pod template says:

```
labels:
  app: payment
```

That would be logically inconsistent.

The ReplicaSet is trying to manage:

```
app=employee
```

but it's creating:

```
app=payment
```

Kubernetes validates these relationships.

The intended model is:

```
Selector
app=employee

      ↓ matches

Pod template label
app=employee
```

| Now the big question

If ReplicaSet is so useful, why do we normally create a Deployment instead?

Because ReplicaSet mainly solves:

How many Pods should exist?

But production applications need more:

```
How do I upgrade v1 → v2?  
How do I roll back?  
How do I perform rolling updates?  
How do I maintain deployment history?
```

That is why Kubernetes provides:

Deployment

14. What is a Deployment?

One-sentence definition

A Deployment is a higher-level Kubernetes controller that manages ReplicaSets and provides declarative application updates, scaling, rollout and rollback capabilities.

Architecture:

```
Deployment  
  ↓  
ReplicaSet  
  ↓  
Pods
```

So normally you create:

```
Deployment
```

and Kubernetes creates/manages:

```
ReplicaSet
```

which creates/manages:

```
Pods
```

| The full hierarchy

Suppose:

```
Deployment:  
employee-service  
  
replicas = 3
```

Then:

```
Deployment  
employee-service  
-  
  | 1/4
```

```
ReplicaSet
employee-service-7c9b...
-
  └─ Pod A
  └─ Pod B
  └─ Pod C
```

You usually interact with the Deployment, not the ReplicaSet directly.

| Java analogy

A rough analogy might help.

Think:

```
Deployment
=
manager

ReplicaSet
=
team supervisor

Pods
=
workers
```

Manager says:

We need 3 workers running version 1.

Supervisor makes sure:

```
Worker 1
Worker 2
Worker 3
```

exist.

Later manager says:

Upgrade the team to version 2 gradually.

That is something a higher-level Deployment handles.

| Deployment YAML

Here is a common Deployment:

```
apiVersion: apps/v1
kind: Deployment
```

```
metadata:
  name: employee-service

spec:
  replicas: 3

  selector:
    matchLabels:
      app: employee

  template:
    metadata:
      labels:
        app: employee

    spec:
      containers:
        - name: employee

          image: company/employee-service:1.0
          ports:
            - containerPort: 8080
```

Notice something interesting.

It looks very similar to the ReplicaSet definition.

That is because the Deployment contains the Pod template that will ultimately be used by its ReplicaSet.

18. What happens when we apply it?

Run:

```
kubectl apply -f employee-deployment.yaml
```

The flow is approximately:

```
kubectl
↓
API Server
↓
Deployment created
↓
Deployment Controller
↓
ReplicaSet created
↓
ReplicaSet Controller
↓
3 Pods created
↓
Scheduler
```

```
↓  
Worker Nodes  
↓  
kubelets  
↓  
containers start
```

This builds on everything we've learned.

Let's trace it step by step

You declare:

```
employee-service  
replicas = 3  
image = v1
```

Deployment controller sees:

```
No corresponding ReplicaSet exists.
```

So it creates one.

ReplicaSet says:

```
Desired replicas = 3  
Current = 0
```

Therefore it creates three Pods.

Then:

```
Pod A → Scheduler → Node 1  
Pod B → Scheduler → Node 2  
Pod C → Scheduler → Node 3
```

Eventually:

```
Deployment  
↓  
ReplicaSet  
↓  
3 Running Pods
```

20. What happens if one Pod is deleted?

Suppose:

```
Pod A  
Pod B  
Pod C
```

You manually delete:

```
kubectl delete pod employee-service-xxxx
```

Now:

```
Current = 2  
Desired = 3
```

ReplicaSet notices.

It creates:

```
Pod D
```

Result:

```
Pod A  
Pod C  
Pod D
```

Still three.

This is Kubernetes self-healing in action.

| Important: the Deployment didn't recreate the Pod directly

For conceptual accuracy:

```
Deployment  
  ↓ manages  
ReplicaSet  
  ↓ manages  
Pods
```

So when a Pod disappears, the ReplicaSet controller is the layer maintaining replica count.

The Deployment manages the ReplicaSet lifecycle.

| Scaling

Suppose currently:

```
replicas: 3
```

You change it to:

```
replicas: 5
```

Apply again:

```
kubectl apply -f employee-deployment.yaml
```

Now:

```
Desired = 5  
Current = 3
```

Two new Pods are created.

Architecture:

Before

```
Deployment  
  ↓  
ReplicaSet  
  ├── Pod 1  
  ├── Pod 2  
  └── Pod 3
```

After:

```
Deployment  
  ↓  
ReplicaSet  
  ├── Pod 1  
  ├── Pod 2  
  ├── Pod 3  
  ├── Pod 4  
  └── Pod 5
```

| Scaling through command line

You can also do:

```
kubectl scale deployment employee-service --replicas=5
```

This changes the desired replica count.

Then Kubernetes reconciles the state.

| But declarative configuration is preferable

Suppose your YAML still says:

```
replicas: 3
```

but you manually run:

```
kubectl scale ... --replicas=5
```

Now the cluster says:

```
5 replicas
```

while your source-controlled YAML says:

```
3 replicas
```

This can cause configuration drift.

In production teams, especially with GitOps, the declarative manifest is often treated as the source of desired configuration.

So you✓I commonly prefer:

```
edit YAML / Git
  ↓
apply / GitOps reconciliation
```

rather than making uncontrolled manual changes.

Now the major Deployment feature: Updates

Suppose your application currently runs:

```
employee-service:1.0
```

Three replicas:

```
Pod A → v1
Pod B → v1
Pod C → v1
```

You release:

```
employee-service:2.0
```

How should Kubernetes upgrade?

A bad strategy would be:

```
Stop A
Stop B
Stop C
```

```
Current running Pods = 0 ×
```

```
Start v2 Pods
```

Your service would be unavailable during the transition.

Instead, Kubernetes Deployments normally support:

Rolling Update

Rolling update

Conceptually:

Initial:

```
v1
v1
v1
```

Start one new Pod:

```
v1
v1
v1
v2
```

Then remove an old one:

```
v1
v1
v2
```

Then:

```
v1
v1
v2
v2
```

Remove another:

```
v1
v2
v2
```

Continue:

```
v1
v2
v2
v2
```

Finally:

```
v2
v2
v2
```

The application remains available through the rollout.

27. What happens internally during a version update?

This is where ReplicaSets become very interesting.

Initial state:

```
Deployment
  ↓
ReplicaSet-v1
├── Pod-v1
├── Pod-v1
└── Pod-v1
```

You update:

```
image:
v1 → v2
```

The Deployment does not simply modify every existing Pod.

Pods are generally immutable for significant template changes.

Instead, Deployment creates:

```
ReplicaSet-v2
```

Now:

```
Deployment
-
├── ReplicaSet-v1
│   ├── v1
│   ├── v1
│   └── v1
-
└── ReplicaSet-v2
```

Then Kubernetes gradually scales:

```
ReplicaSet-v1 ↓
ReplicaSet-v2 !'
```

until:

```
ReplicaSet-v1 → 0 Pods
ReplicaSet-v2 → 3 Pods
```

This is extremely important

Deployment upgrades work roughly like:

```
Old ReplicaSet
  ↓ scale down

New ReplicaSet
  !' scale up
```

Not:

```
Edit running Pod in place
```

This immutable/replacement mindset appears throughout Kubernetes.

| Example update command

You could update the image with:

```
kubectl set image deployment/employee-service \
employee=company/employee-service:2.0
```

or preferably update your YAML:

```
image: company/employee-service:2.0
```

and run:

```
kubectl apply -f employee-deployment.yaml
```

| Check rollout

You can run:

```
kubectl rollout status deployment/employee-service
```

Kubernetes may report that the rollout successfully completed.

You can also inspect:

```
kubectl get pods
```

during rollout and watch old Pods disappear while new ones start.

| Why old ReplicaSets remain

After a successful rollout, you may see older ReplicaSets still present with:

```
DESIRED = 0
CURRENT = 0
```

Why keep them?

Because Kubernetes can use deployment revision history for rollback.

Conceptually:

```
Deployment
Revision 1
ReplicaSet-v1
```

```
0 Pods
```

```
Revision 2  
ReplicaSet-v2  
3 Pods
```

History is useful.

32. What if v2 is broken?

Suppose v2 has a major bug.

You deployed:

```
employee-service:v2
```

and suddenly:

```
500 errors  
Pod crashes  
health checks fail
```

You may need to restore the previous version.

Deployment provides rollback functionality.

Example:

```
kubectl rollout undo deployment/employee-service
```

Conceptually:

```
Current:  
ReplicaSet-v2 = 3
```

```
Rollback
```

```
ReplicaSet-v2 ↓  
ReplicaSet-v1 !'
```

Eventually:

```
v1  
v1  
v1
```

again.

Deployment history

You can inspect rollout history:

```
kubectl rollout history deployment/employee-service
```

Conceptually:

```
Revision 1 → v1  
Revision 2 → v2  
Revision 3 → v3
```

This gives Kubernetes a controlled rollout model.

RollingUpdate strategy

Deployment supports strategies.

The common default is:

```
RollingUpdate
```

Another strategy is:

```
Recreate
```

Let's understand both.

Recreate strategy

With Recreate:

```
v1 v1 v1
```

Kubernetes removes the old version:

```
nothing running
```

then starts:

```
v2 v2 v2
```

This can cause downtime.

So:

```
Recreate
```

is appropriate only when simultaneous old/new versions cannot coexist or downtime is acceptable.

For typical stateless APIs, RollingUpdate is usually preferred.

RollingUpdate configuration

A Deployment can contain:

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 1
    maxSurge: 1
```

These two settings are important:

```
maxUnavailable
maxSurge
```

maxUnavailable

Suppose:

```
Desired replicas = 5
maxUnavailable = 1
```

During rollout, Kubernetes allows at most one desired replica to be unavailable beyond the permitted calculation.

Simplified mental model:

Don't let too many old Pods disappear before replacements are ready.

maxSurge

Suppose:

```
Desired replicas = 5
maxSurge = 1
```

Kubernetes may temporarily run:

```
6 Pods
```

during the rollout.

Why?

Because the extra Pod helps replace old Pods without reducing available capacity too aggressively.

Think:

```
Normal = 5
```

```
During rollout:
temporary maximum = 6
```

Example rolling update

Imagine:

```
replicas = 3  
maxUnavailable = 1  
maxSurge = 1
```

Initial:

```
v1  
v1  
v1
```

Temporary surge:

```
v1  
v1  
v1  
v2
```

When v2 becomes ready, Kubernetes can remove one v1:

```
v1  
v1  
v2
```

Then add another v2:

```
v1  
v1  
v2  
v2
```

Continue until:

```
v2  
v2  
v2
```

The exact sequence is controller-driven, but this is the right conceptual picture.

Readiness will later become critical

You might notice something:

How does Deployment know:

“The new v2 Pod is actually ready to serve customers?”

Simply having the container process running isn’t enough.

That is where:

```
Readiness Probe
```

becomes important.

Without a correct readiness probe, Kubernetes may consider a new Pod available before the application is truly ready.

We✓I cover that in the health-check chapter.

| Self-healing example

Consider:

```
Deployment  
replicas = 3
```

Pods:

```
employee-a  
employee-b  
employee-c
```

employee-b gets deleted.

ReplicaSet sees:

```
Desired = 3  
Current = 2
```

It creates:

```
employee-d
```

This is why Pod names should not matter to clients.

Clients should not call:

```
employee-b
```

directly.

Soon we✓I introduce:

```
Service
```

which gives stable networking over changing Pods.

| Deployment names vs Pod names

Suppose:

Deployment:
employee-service

Generated ReplicaSet might resemble:

employee-service-74d8f7c4db

Generated Pod:

employee-service-74d8f7c4db-xq7ks

The suffixes are generated.

Conceptually:

```
employee-service
    !
Deployment identity

employee-service-74d8...
    !
ReplicaSet revision

employee-service-74d8...-xq7ks
    !
individual Pod instance
```

Don't build logic depending on generated Pod names.

| Let's inspect resources

Common commands:

```
kubectl get deployments
```

short form:

```
kubectl get deploy
```

ReplicaSets:

```
kubectl get replicaset
```

or:

```
kubectl get rs
```

Pods:

```
kubectl get pods
```

You might see:

```
Deployment
employee-service

ReplicaSet
employee-service-abc123

Pods
employee-service-abc123-11111
employee-service-abc123-22222
employee-service-abc123-33333
```

That is the hierarchy made visible.

| **kubectl describe deployment**

Another useful command:

```
kubectl describe deployment employee-service
```

It can show details such as:

```
replicas
strategy
Pod template
conditions
events
image
labels
selectors
```

Again, describe is invaluable when troubleshooting.

| **Scaling vs updating**

Don't mix these two concepts.

Scaling

```
Same version
more/less replicas
```

Example:

```
v1 v1

      ↓ scale

v1 v1 v1 v1 v1
```

Updating

Change version

Example:

```
v1 v1 v1
      ↓ rollout
v2 v2 v2
```

A Deployment handles both.

46. What if replicas = 0?

You can set:

```
replicas: 0
```

Then Deployment remains in Kubernetes, but no application Pods should run.

Conceptually:

```
Deployment exists '
ReplicaSet exists '
Pods = 0
```

Useful when intentionally scaling an application down.

| ReplicaSet vs Deployment

Here's the distinction you should memorize:

ReplicaSet Deployment

Maintains number of Pods Manages application deployment lifecycle

Ensures replicas exist Manages ReplicaSets

Recreates missing Pods Performs rolling updates

Can scale Pods Can scale

No rich rollout abstraction Rollout/rollback support

Usually not created directly Common production choice

The easiest sentence:

ReplicaSet maintains replicas. Deployment manages ReplicaSets and application versions.

| Pod vs ReplicaSet vs Deployment

Let's put all three together.

Pod

Runs the application container

ReplicaSet

Ensures N Pods exist

Deployment

Manages ReplicaSets and application lifecycle

Architecture:

Deployment
"Run Employee Service v2"

↓

ReplicaSet
"Keep 3 copies running"

↓

Pod 1
Pod 2
Pod 3

↓

Container
employee-service:v2

Enterprise example

Suppose your architecture is:

Gateway
Employee Service
Payment Service

You might create:

Gateway Deployment
replicas = 2

Employee Deployment
replicas = 3

Payment Deployment
replicas = 5

Then:

```
Gateway Deployment
```

```
↓
```

```
Gateway ReplicaSet
```

```
↓
```

```
2 Pods
```

```
Employee Deployment
```

```
↓
```

```
Employee ReplicaSet
```

```
↓
```

```
3 Pods
```

```
Payment Deployment
```

```
↓
```

```
Payment ReplicaSet
```

```
↓
```

```
5 Pods
```

Total:

```
10 application Pods
```

| Where OpenShift fits

In OpenShift, Kubernetes Deployments are also core resources.

So commands such as:

```
oc get deployments
```

and:

```
oc get pods
```

follow the same Kubernetes architecture.

This is exactly why learning Kubernetes first pays off.

When you eventually use OpenShift, you✓I already understand:

```
Deployment
```

```
ReplicaSet
```

```
Pod
```

rather than treating oc commands as magic.

| Common interview scenario

Suppose an interviewer asks:

There are 3 replicas in a Deployment. Someone manually deletes one Pod. What happens?

Answer:

The ReplicaSet controller observes that the current number of matching Pods has dropped below the desired replica count and creates a replacement Pod. The Scheduler then selects a Node for the new Pod, and the kubelet/runtime starts it there.

That is an excellent answer because it links multiple components.

| Another interview scenario

Why do we use Deployment instead of creating ReplicaSet directly?

Answer:

A ReplicaSet mainly ensures the desired number of Pods exist, while a Deployment adds higher-level lifecycle management such as rolling updates, rollout history and rollback by managing ReplicaSets for different Pod-template revisions.

| Another interview question

During a Deployment upgrade, does Kubernetes modify the image inside existing Pods?

The better mental model:

No. A Deployment usually creates a new ReplicaSet based on the new Pod template and gradually scales the new ReplicaSet up while scaling the old ReplicaSet down.

| Important misconception: Deployment runs containers

It's okay conversationally to say:

"Deployment runs my application."

But technically, remember the hierarchy:

```
Deployment
  ↓ manages
ReplicaSet
  ↓ manages
Pod
  ↓ contains
Container
  ↓ runs
Application
```

This precision will help you troubleshoot later.

| Another subtle concept: Pod template change

Suppose you modify:

```
image: employee:v1
```

to:

```
image: employee:v2
```

This modifies the Deployment's:

```
Pod template
```

A Pod template change creates a new Deployment revision / ReplicaSet.

But not every metadata change necessarily creates a new rollout.

The important beginner rule:

Changes to the Pod template—such as container image—cause a new rollout.

Visualizing an upgrade end-to-end

Let's put all components together.

You update:

```
employee:v1 → employee:v2
```

Flow:

```
Developer
  ↓
kubectl apply
  ↓
API Server
  ↓
Deployment updated
  ↓
Deployment Controller
  ↓
New ReplicaSet created
  ↓
New Pods required
  ↓
Scheduler
  ↓
Worker Nodes
  ↓
kubelet
  ↓
runtime pulls employee:v2
  ↓
v2 Pods become ready
  ↓
Old ReplicaSet scaled down
```

↓
Rollout complete

That connects Modules 1-4.

| Troubleshooting example

Suppose:

```
Deployment desired replicas = 3

kubectl get pods:

employee-1          Running
employee-2          Running
employee-3          ImagePullBackOff
```

What do we know?

Deployment/ReplicaSet did their job:

```
3 Pods exist
```

Scheduler probably also did its job because the failing Pod was assigned.

The problem is further down:

```
kubelet
  ↓
container runtime
  ↓
pull image ×
```

So you investigate:

```
kubectl describe pod employee-3
```

Potential causes:

```
wrong image name
wrong tag
registry authentication
network access
image unavailable
```

This is exactly why architectural understanding beats command memorization.

| Another troubleshooting scenario

Suppose:


```
replicas = 3
```

but you see:

```
5 Pods
```

Don't immediately assume Kubernetes is broken.

Possible reasons:

```
rolling update is currently happening
maxSurge allows extra Pods
old Pods are terminating
multiple ReplicaSets are involved
```

Check:

```
kubectl get rs
```

You might discover:

```
Old RS → 2
New RS → 3
```

during a rollout transition.

A practical command flow

When checking an application, this is a useful hierarchy:

```
kubectl get deployment
```

then:

```
kubectl get rs
```

then:

```
kubectl get pods
```

Think top-down:

```
Deployment
↓
ReplicaSet
↓
Pods
```

If something looks wrong, drill down.

Mini example YAML

Here's a more realistic Spring Boot Deployment:

```
apiVersion: apps/v1
kind: Deployment

metadata:
  name: payment-service

spec:
  replicas: 3

  selector:
    matchLabels:
      app: payment

  template:
    metadata:
      labels:
        app: payment

    spec:
      containers:
        - name: payment
          image: company/payment-service:1.0

          ports:
            - containerPort: 8080

          env:
            - name: SPRING_PROFILES_ACTIVE
              value: prod
```

We haven't covered environment configuration fully yet, but conceptually:

```
Deployment
  ↓
ReplicaSet
  ↓
3 Payment Pods
  ↓
each Pod
  ↓
Payment container
  ↓
Spring Boot
port 8080
profile=prod
```

| What you should memorize from chapter 4

These four lines are enough to reconstruct most of the concept:

```
Pod
=
Runs application

ReplicaSet
=
Keeps desired number of Pods running

Deployment
=
Manages ReplicaSets and application rollouts

Production hierarchy:

Deployment
  ↓
ReplicaSet
  ↓
Pods
  ↓
Containers
```

| Check your understanding

Q1 You directly create a Pod and delete it.

Who recreates it?

Usually nobody, because there is no higher-level controller managing it.

Q2 A ReplicaSet says:

```
replicas = 4
```

but only two matching Pods exist.

What happens?

ReplicaSet creates two additional Pods.

Q3 Who chooses where those two Pods run?

Scheduler.

Q4 Who actually starts their containers on each selected Node?

```
kubelet
+
container runtime
```

Q5 What manages the ReplicaSet?

Deployment, in the normal Deployment hierarchy.

Q6 During a v1 → v2 upgrade, what typically happens?

New ReplicaSet for v2

↑

scaled up

Old ReplicaSet for v1

↓

scaled down

Small thought exercise

Suppose:

Deployment:

payment-service

replicas = 4

image = payment:v1

Current:

4 v1 Pods

You change:

image = payment:v2

Questions:

1. Does Kubernetes normally modify those four Pods in place?
2. What higher-level object notices the Deployment change?
3. What new object is usually created for the new Pod template?
4. What happens to the old ReplicaSet?
5. Why can the application remain available during the upgrade?

Answers:

1. No.
2. Deployment controller.
3. A new ReplicaSet.
4. It is gradually scaled down.

5. Because old Pods are replaced progressively by ready new Pods instead of stopping all replicas at once.

We can now trace a production application surprisingly far:

```
Developer
↓
Container Image
↓
Deployment
↓
ReplicaSet
↓
Pods
↓
Scheduler
↓
Worker Nodes
↓
kubelet
↓
Container Runtime
↓
Java Application
```

And if a Pod dies:

```
Pod dies
↓
ReplicaSet detects mismatch
↓
Replacement Pod created
↓
Scheduler picks Node
↓
kubelet/runtime starts it
```

Next — chapter 5: Kubernetes Services This is the natural next problem.

We now have three Employee Pods:

```
Employee Pod A
10.244.1.10

Employee Pod B
10.244.2.17

Employee Pod C
10.244.3.21
```

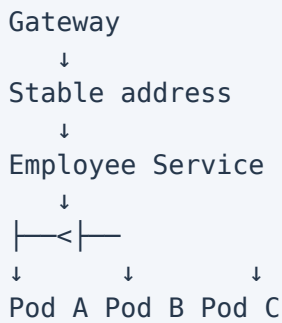
But Pods are disposable.

Tomorrow they may become:

```
10.244.2.41
10.244.4.12
10.244.1.88
```

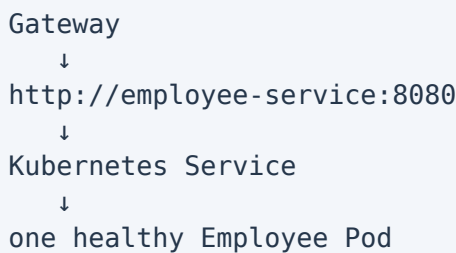
So how does the Gateway reliably call Employee Service?

We need:



That stable abstraction is the Kubernetes Service.

In the next chapter we✓I cover Service discovery, ClusterIP, selectors, endpoints, DNS, load balancing, NodePort and LoadBalancer, and we✓I trace a real request such as:



This chapter is where Kubernetes networking starts to click.

:::writing{variant="standard" id="58321"} employee-service-74d8f7c4db-xq7ks :::

Chapter 6 — Jobs and CronJobs

CHAPTER FOCUS

Understand **Jobs and CronJobs** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

So far, most of the workloads we've discussed are long-running applications:

```
Deployment
  ↓
Pod
  ↓
Application keeps running
```

But many real-world tasks are different.

For example:

- Run a database migration once
- Generate a report
- Process a batch of files
- Clean old records
- Send a daily report
- Run a backup every night

You don't want a Deployment running forever for these.

Kubernetes has:

```
Job      → run a task until it completes
CronJob  → run Jobs on a schedule
```

Deployment vs Job

This distinction is fundamental.

Deployment

```
Deployment
  ↓
Pod
  ↓
Application keeps running
```

Example:

```
employee-api
```

The desired state is:

“Keep this application running.”

Job

```
Job
↓
Pod
↓
Task
↓
Complete
```

The desired state is:

“Make sure this task successfully finishes.”

Real-world example

Imagine your Spring Boot application has a database migration.

You need to execute:

```
Flyway migration
```

You don't want:

```
Deployment
↓
migration Pod
↓
running forever
```

Instead:

```
Job
↓
migration Pod
↓
Flyway
↓
Migration complete
↓
Pod finishes
```

That is exactly what a Job is designed for.

Basic Job

A simple Job:

```
apiVersion: batch/v1
kind: Job

metadata:
  name: employee-migration

spec:
  template:
    spec:
      restartPolicy: Never

      containers:
      - name: migration
        image: employee-api:1.0
        command:
        - java
        - "-jar"
        - "migration.jar"
```

Conceptually:

```
Job
-
  | 1/4
Pod
-
  | 1/4
Run task
-
  | 1/4
Success
-
  | 1/4
Completed
```

4. Why does the Pod remain after completion?

This is an interesting Kubernetes behavior.

After the task completes:

```
kubectl get pods
```

might show:

```
employee-migration-abc12      Completed
```

The Pod isn't actively running.

It remains as historical information associated with the Job until Kubernetes garbage-

collects it according to the configured policies.

You can inspect it:

```
kubectl describe job employee-migration
```

This is useful for troubleshooting.

Job completion

Suppose the application exits with:

```
exit code 0
```

Kubernetes considers the container successful.

Then:

```
Pod
↓
Completed
```

and:

```
Job
↓
Complete
```

You might see:

NAME	COMPLETIONS
employee-migration	1/1

Meaning:

```
1 successful completion
out of 1 required
```

6. What if the task fails?

Suppose:

```
Database unavailable
```

The application exits with:

```
exit code 1
```

Then:

```
Job
↓
Pod
```

↓
FAIL

Kubernetes can retry the workload according to the Job configuration.

For example:

```
spec:  
  backoffLimit: 3
```

means Kubernetes will allow retries before considering the Job failed.

Conceptually:

```
Attempt 1 ×  
  ↓  
Attempt 2 ×  
  ↓  
Attempt 3 ×  
  ↓  
Job failed
```

The exact retry behavior depends on the Job's Pod restart policy and Job controller behavior.

| restartPolicy

Jobs require a Pod restart policy of:

```
restartPolicy: Never
```

or:

```
restartPolicy: OnFailure
```

You don't use:

```
restartPolicy: Always
```

for a Job's Pod template.

Why?

Because a Job is specifically about eventual completion, not maintaining a perpetually running process.

| Example: one-time database migration

Imagine deployment:

```
employee-api:v15
```

Before starting the new application version:

```
Run database migration
```

Architecture:

```

      Release v15
      -
        | ¼
      Migration Job
      -
        | ¼
      Flyway/Liquibase
      -
        | ¼
      Database updated
      -
        | ¼
      Deploy employee-api

```

This is a very common Kubernetes/GitOps deployment pattern.

Job vs initContainer

You may wonder:

Couldn't I use an initContainer for the migration?

You can use initContainers for initialization that is tightly coupled to the startup of a Pod.

But a database migration is often better treated as a separate operational task because:

```

Migration
  " \
Application startup

```

You may want:

```

Migration succeeds
  ↓
Application deployment proceeds

```

rather than every application replica independently attempting the migration.

This becomes especially important when you have:

```
10 application replicas
```

You generally don't want all 10 replicas racing to perform the same migration.

10. What is a CronJob?

Now suppose you want:

Run a cleanup task every night at 2 AM.

You could manually create a Job every night.

Obviously that's inconvenient.

So Kubernetes gives us:

CronJob Architecture:

CronJob



A CronJob creates Jobs according to a schedule.

CronJob example

```
apiVersion: batch/v1
kind: CronJob

metadata:
  name: employee-cleanup

spec:
  schedule: "0 2 * * *"

  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: Never

          containers:
            - name: cleanup
              image: employee-cleanup:1.0
```

The important line:

```
schedule: "0 2 * * *"
```

means:

```
minute = 0  
hour   = 2  
day     = *  
month   = *  
weekday = *
```

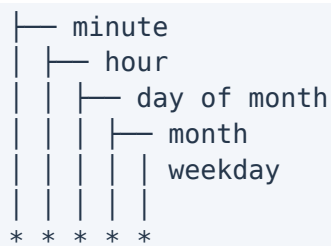
So conceptually:

Every day at 02:00

subject to the CronJob's configured time zone behavior.

Cron expression basics

A Cron schedule has five traditional fields:



Examples:

Every day at midnight

```
0 0 * * *
```

Every day at 2 AM

```
0 2 * * *
```

Every Monday at 9 AM

```
0 9 * * 1
```

Every 15 minutes

```
*/15 * * * *
```

CronJob lifecycle

Suppose:

```
02:00
```

CronJob controller creates:

```
employee-cleanup-291837
```

which is a Job.

Then:

```
Job
↓
Pod
↓
cleanup task
↓
Completed
```

Next day:

```
02:00
↓
New Job
↓
New Pod
```

So:

CronJob doesn't directly run Pods.

It creates Jobs, and Jobs create/manage Pods.

This hierarchy is important:

```
CronJob
↓
Job
↓
Pod
↓
Container
```

CronJob vs Deployment

Let's compare them.

Deployment CronJob

Long-running application Scheduled task

Keeps Pods running Creates Jobs

REST API Nightly cleanup

Web server Daily report

Backend service Scheduled backup

Desired state = running Desired state = successful execution

| CronJob concurrency

Here's a real production problem.

Suppose:

```
cleanup starts at 02:00
```

but the task takes:

```
3 hours
```

Now tomorrow at 02:00, the previous Job is still running.

What should Kubernetes do?

CronJob has:

```
concurrencyPolicy:
```

Common options:

```
Allow  
Forbid  
Replace
```

| Allow

```
concurrencyPolicy: Allow
```

Multiple Jobs can overlap.

Example:

```
02:00 → Job A starts  
03:00 → Job A still running  
02:00 next day → Job B starts
```

Now both can run simultaneously.

This may be dangerous if the task isn't designed for concurrency.

Forbid

concurrencyPolicy: Forbid

If the previous Job is still running, don't start another one.

Conceptually:

```
Job A
-
|— running
-
|— next schedule arrives
  ↓
  Skip/handle according to CronJob scheduling semantics
```

Useful when overlapping executions would be unsafe.

Replace

concurrencyPolicy: Replace

A new scheduled execution can replace the currently running Job.

This is useful only when interrupting the previous run is acceptable.

So choose carefully.

successfulJobsHistoryLimit

CronJobs can create many completed Jobs.

Imagine:

```
Day 1 → Job
Day 2 → Job
Day 3 → Job
...
Day 365 → Job
```

You probably don't want hundreds of old Jobs forever.

You can configure:

successfulJobsHistoryLimit: 3

to retain only a limited number of successful Job histories.

Similarly:

```
failedJobsHistoryLimit: 3
```

controls how many failed Job histories are retained.

| startingDeadlineSeconds

What happens if Kubernetes misses the scheduled time?

For example:

```
02:00
```

but the controller is temporarily unavailable.

You can define a deadline for how late a Job may still be started.

This becomes important for production workloads where:

“Run this task, but not if it’s too late.”

| Time zones

One subtle but important production issue:

What time zone does my CronJob use?

Don’t casually assume:

```
02:00
```

means:

```
02:00 India time
```

unless your cluster/CronJob configuration explicitly establishes that.

Modern Kubernetes supports specifying a time zone using the CronJob’s `timeZone` field.

For example:

```
spec:
  schedule: "0 2 * * *"
  timeZone: "Asia/Kolkata"
```

This makes the intent much clearer.

| Real-world Spring Boot example

Suppose you have:

```
employee-api
```

and every night you want to delete temporary records older than 30 days.

Instead of keeping a special process running 24/7:

```
Deployment
  ↓
cleanup process
  ↓
waiting...
```

use:

```
CronJob
  ↓
02:00
  ↓

Job
  ↓
Spring Boot cleanup application
  ↓
Database cleanup
  ↓
Exit
```

That is much cleaner.

Another real-world example: reports

Your organization needs:

```
Daily employee report
```

at:

```
08:00
```

Architecture:

```
CronJob
  ↓
08:00
  ↓
Job
  ↓
Spring Boot reporting app
  ↓
Query database
  ↓
Generate report
```

```
↓  
Send report  
↓  
Complete
```

No need for a permanently running reporting Pod.

| Another example: database backup

```
CronJob  
↓  
01:00  
↓  
Job  
↓  
Database backup  
↓  
Object storage  
↓  
Complete
```

However, don't assume a CronJob alone gives you a complete backup/DR strategy.

You still need to think about:

```
Retention  
Encryption  
Restore testing  
Off-site storage  
Failure notification  
Recovery objectives
```

| Jobs are not only for “small tasks”

A Job can perform substantial batch processing.

For example:

```
10 million records
```

You could design a batch workload that processes them in chunks.

Kubernetes can also support parallel Jobs.

| Parallel Jobs

Suppose you need:

100 independent tasks

You can configure:

```
completions: 100
parallelism: 10
```

Conceptually:

```
Job
-
  ├── Worker 1
  ├── Worker 2
  ├── Worker 3
  ├── ...
  └── Worker 10
```

As workers finish:

new workers

are created until the required number of completions is reached.

This is useful for batch workloads that can be split into independent units.

| Important distinction: parallelism vs completions

Think:

```
completions
=
How many successful completions do I need?
```

while:

```
parallelism
=
How many can run simultaneously?
```

Example:

```
completions: 100
parallelism: 10
```

means:

I need 100 successful completions, but run at most 10 concurrently.

| Job failure troubleshooting

Suppose:

```
kubectl get jobs
```

shows:

```
employee-migration      0/1    Failed
```

Don't immediately recreate it.

Investigate.

First:

```
kubectl describe job employee-migration
```

Then:

```
kubectl get pods
```

Find the Job's Pod.

Then:

```
kubectl logs <pod-name>
```

Now you might see:

```
Connection refused
```

or:

```
Database authentication failed
```

or:

```
Migration checksum mismatch
```

This gives you the actual application-level reason.

A useful troubleshooting chain

For batch workloads, remember:

```
CronJob
  ↓
Job
  ↓
Pod
  ↓
Container
  ↓
Application
```

Troubleshoot from top to bottom.

CronJob

```
kubectl describe cronjob employee-cleanup
```

Job

```
kubectl describe job <job-name>
```

Pod

```
kubectl describe pod <pod-name>
```

Application

```
kubectl logs <pod-name>
```

This hierarchy will save you a lot of time.

30. What happens if a Job's Pod crashes?

Suppose:

```
Job
↓
Pod
↓
Application crashes
```

The Job controller can retry according to the Job's configuration.

For example:

```
backoffLimit: 4
```

means the Job can tolerate failures before being marked failed.

The exact number of Pod attempts can be influenced by the Pod restart policy and Job failure accounting.

The key concept is:

A Job is responsible for achieving successful completion.

| Job vs initContainer — final comparison

Job initContainer

Independent task Initialization of a specific Pod

Can exist without app Pod Tied to Pod startup

Good for migrations/batch Good for setup before app starts

Has Job lifecycle Runs before app containers

Can be scheduled Runs when Pod starts

Example:

```
Job
→ Run database migration once
```

while:

```
initContainer
→ Download configuration before application starts
```

| One subtle production concern: migrations

Suppose you have:

```
Deployment
replicas: 10
```

and each Pod runs:

```
database migration
```

You could end up with:

```
Pod-1 → migration
Pod-2 → migration
Pod-3 → migration
...
Pod-10 → migration
```

This can create race conditions.

A better architecture is often:

```
Migration Job
  ↓
successful
  ↓
Application Deployment
```

with migration tooling designed appropriately for your database/application.

| Kubernetes now has three major workload patterns

You should now mentally classify workloads as:

Long-running

```
Deployment
```

Example:

Spring Boot REST API

Stateful long-running

StatefulSet

Example:

database / Kafka

Finite batch

Job

Example:

migration

Scheduled batch

CronJob

Example:

nightly cleanup

This classification is very useful when designing Kubernetes workloads.

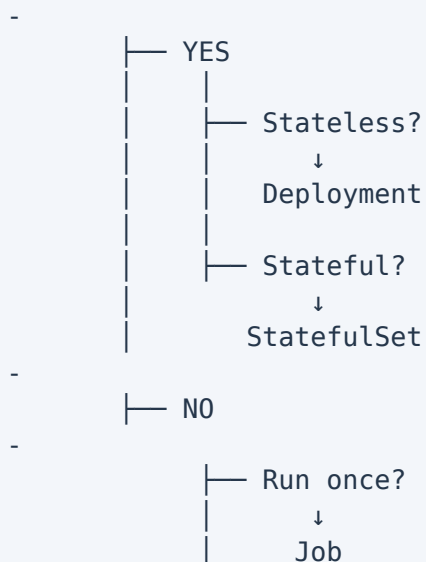
Your Kubernetes workload decision tree

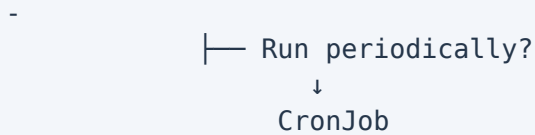
When someone asks:

“What Kubernetes object should I use?”

Think:

Does the application run continuously?





That is a very useful mental model.

We've now covered the major Kubernetes workload objects

At this point:

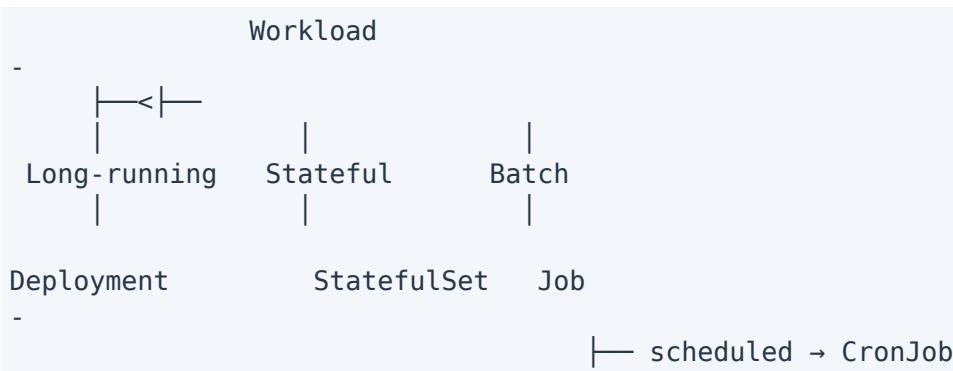
```

Deployment
StatefulSet
Job
CronJob

```

should no longer look like four unrelated YAML objects.

They solve different workload lifecycle problems.



Where we are in the overall roadmap

You've now covered a large portion of the Kubernetes fundamentals:

```

' Containers
' Pods
' Deployments
' ReplicaSets
' Services
' Networking
' Volumes
' PV / PVC
' StorageClass
' StatefulSets
' ConfigMaps
' Secrets
' Jobs
' CronJobs

```

And you’ve also already covered the networking concepts earlier in our sequence.

So I would not repeat networking.

Next Logical chapter — chapter 19: Kubernetes Observability &

Troubleshooting

This is where we shift from:

“How do I create Kubernetes resources?”

to:

“My application is broken. How do I find out why?”

We✓I build a systematic troubleshooting framework:

```
User
↓
Ingress / Route
↓
Service
↓
Pod
↓
Container
↓
Application
↓
Database
```

and learn the commands you✓I actually use as a backend engineer:

```
kubectl get
kubectl describe
kubectl logs
kubectl exec
kubectl events
kubectl top
```

We✓I troubleshoot realistic failures such as:

```
Pod stuck in Pending
CrashLoopBackOff
ImagePullBackOff
Service not reachable
Readiness probe failing
Liveness probe restarting the container
PVC stuck in Pending
Secret/configuration missing
Application can't connect to database
```

This chapter is especially important for your Java backend role, because this is where

Kubernetes knowledge starts becoming useful in real production incidents.

Kubernetes Networking

Chapter 7 — Networking Fundamentals and Pod Networking

CHAPTER FOCUS

Understand **Networking Fundamentals and Pod Networking** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

This chapter is extremely important because Kubernetes networking can feel confusing until one rule becomes clear:

Ingress and egress are always relative to the component you are looking at.

We/I build everything around this architecture:

```
Internet User
  ↓
Gateway Pod
  ↓
Employee Service
  ↓
Employee Pod
  ↓
Database
```

Start with normal networking

Before Kubernetes, imagine two applications running on two servers:

```
Server A
Gateway
10.10.1.10

      ↓ HTTP

Server B
Employee Service
10.10.1.20
```

Gateway calls:

```
http://10.10.1.20:8080
```

The basic idea is:

```
Source
  ↓
Destination
```

Every network connection has:

```
Source IP
Source Port
Destination IP
Destination Port
Protocol
```

For example:

```
Gateway
10.10.1.10:53124
  ↓
Employee
10.10.1.20:8080
```

Kubernetes networking follows the same fundamental principles.

It just adds abstractions such as:

```
Pods
Services
DNS
NetworkPolicy
Ingress
CNI
```

Kubernetes gives Pods IP addresses

Suppose we have two Worker Nodes:

```
Node A
|— Gateway Pod
   10.244.1.10

Node B
|— Employee Pod
   10.244.2.20
```

Even though these Pods are on different Nodes, Kubernetes networking is designed so they can communicate.

Conceptually:

```
Gateway Pod
10.244.1.10
```

↓

Employee Pod
10.244.2.20

Normally, the application shouldn't need to know:

"Employee is on Worker Node B."

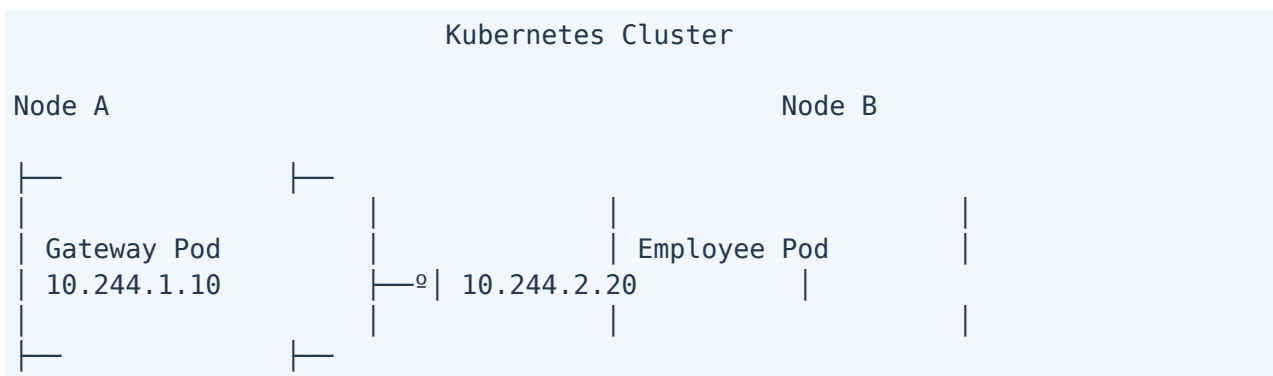
From the application's point of view, it is communicating with another network endpoint.

Kubernetes network model

A useful simplified Kubernetes networking model is:

Each Pod receives its own IP address, and Pods should be able to communicate across Nodes through the cluster networking implementation.

Imagine:



But now we have an obvious question:

Who creates and manages this Pod network?

That is where CNI comes in.

4. What is CNI?

CNI stands for:

Container Network Interface

Think of CNI as a standard interface Kubernetes networking implementations use to configure Pod networking.

Very simplified:

```

Kubernetes
  ↓
CNI networking implementation
  ↓
  
```



```
Create Pod network interface
Assign IP
Configure connectivity
Apply supported network policies
```

The important beginner distinction is:

```
Kubernetes
=
Defines how networking should behave
```

while:

```
CNI plugin / networking implementation
=
Makes that networking work on the Nodes
```

Why Kubernetes doesn't implement every network itself

Different environments have different networking needs:

```
AWS
Azure
Bare metal
Private data center
OpenShift
Large enterprise
Small cluster
```

Rather than Kubernetes hardcoding one networking technology, CNI provides a standard model that different networking implementations can integrate with.

You'll encounter names such as:

```
Calico
Cilium
Flannel
OVN-Kubernetes
```

You don't need to learn them yet.

For now:

CNI handles the underlying Pod networking implementation.

Pod-to-Pod communication

Consider:

```
Node 1
```

```
Gateway Pod
10.244.1.10
```

and:

```
Node 2

Employee Pod
10.244.2.20
```

Gateway technically could send traffic directly:

```
10.244.1.10
  ↓
10.244.2.20:8080
```

The network implementation makes the cross-Node connectivity possible.

But remember the previous chapter:

We normally don't make applications depend directly on changing Pod IPs.

Instead:

```
Gateway
  ↓
Employee Service
  ↓
Employee Pod
```

7. Where does Service fit into networking?

Suppose:

```
Employee Service

ClusterIP:
10.96.30.10
```

Backend Pods:

```
Employee Pod A
10.244.1.20

Employee Pod B
10.244.2.30

Employee Pod C
10.244.3.40
```

Gateway calls:

```
employee-service:8080
```

DNS resolves the name to the Service's virtual IP.

Conceptually:

```

Gateway Pod
  ↓
DNS:
employee-service
  ↓
10.96.30.10
  ↓
Kubernetes Service networking
  ↓
One backend Pod

```

For example:

```
10.244.2.30:8080
```

Important: A Service is not a running server

This is a very common misconception.

You might imagine:

```

Gateway
  ↓
Service process
  ↓
Employee Pod

```

Not really.

A normal ClusterIP Service is primarily a virtual networking abstraction.

There isn't necessarily a container running called:

```
employee-service
```

that receives every packet and forwards it.

Conceptually think:

```

Service
=
Stable virtual network identity + backend selection

```

The actual networking mechanism depends on the cluster networking implementation.

| Now let's understand ingress and egress

These two words are very simple once you remember:

Always ask: ingress/egress relative to WHAT?

General meanings:

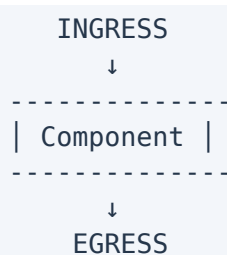
Ingress

Traffic coming into something.

Egress

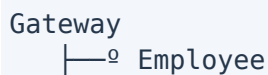
Traffic going out of something.

Think:



| Simple example

Gateway calls Employee Service:



From Gateway's perspective:

```
Gateway
  ↓
Traffic leaves Gateway
= EGRESS
```

From Employee's perspective:

```
Traffic enters Employee
  ↓
Employee
= INGRESS
```

So the same network request is:

```
EGRESS from Gateway
and
```

INGRESS to Employee

This is the most important rule.

Your earlier Gateway example

Consider:

```
Client
  ↓
Gateway
  ↓
Employee Service
```

Let's analyze each hop separately.

Client → Gateway From Gateway's perspective:

```
Client
  ↓
Gateway

INGRESS to Gateway
```

From client's perspective:

```
Client
  ↓

EGRESS from Client
```

Gateway → Employee

```
Gateway
  ↓
Employee
```

For Gateway:

EGRESS

For Employee:

INGRESS

Complete example

Now add a database:

```

Client
  ↓
Gateway
  ↓
Employee Pod
  ↓
Database

```

Traffic directions:

```

Client
-
  | egress from Client
  | 1/4
Gateway
-
  | ingress to Gateway
-
  | egress from Gateway
  | 1/4
Employee
-
  | ingress to Employee
-
  | egress from Employee
  | 1/4
Database
  ingress to Database

```

This answers many firewall and NetworkPolicy questions.

Very important question

Suppose Gateway calls Employee.

Which side requires an ingress rule?

If Employee is isolated by policy and should accept Gateway traffic, the relevant rule is:

```

Employee
INGRESS
allow Gateway

```

Because traffic is entering Employee.

But depending on policy configuration, Gateway may also need:

```

Gateway
EGRESS
allow Employee

```

because traffic is leaving Gateway.

So potentially:



Whether both need explicit policy depends on what NetworkPolicies isolate each side.

Default Kubernetes NetworkPolicy behavior

This is very important.

If there are no NetworkPolicies selecting a Pod, Kubernetes networking is generally:

Allow traffic

for that Pod.

So Kubernetes doesn't automatically say:

Pod A cannot talk to Pod B.

NetworkPolicy is used when you want to introduce network isolation.

15. What is NetworkPolicy?

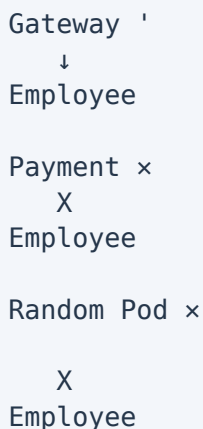
One-sentence definition

A NetworkPolicy defines which network traffic selected Pods are allowed to receive and/or send.

Example requirement:

Only Gateway Pods should be able to call Employee Pods.

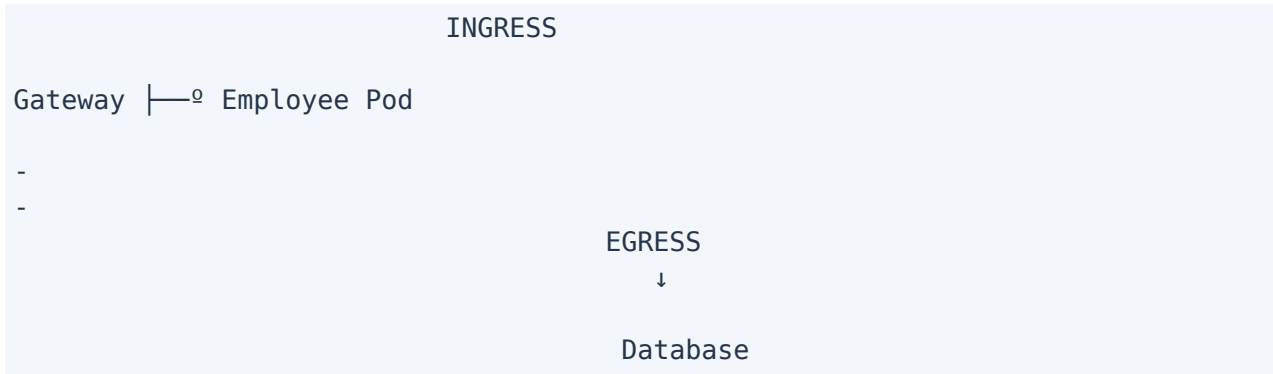
Architecture:



NetworkPolicy can express that type of restriction.

NetworkPolicy operates around Pods

Think:



A policy can control:

Who can enter Employee?

Where can Employee send traffic?

Example labels

Gateway Pods:

```
labels:
  app: gateway
```

Employee Pods:

```
labels:
  app: employee
```

Now we can write policy logic such as:

Select Employee Pods and allow ingress only from Pods labeled app=gateway .

Simplified NetworkPolicy

For learning purposes:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy

metadata:
  name: allow-gateway-to-employee

spec:
  podSelector:
```



```
matchLabels:
  app: employee

policyTypes:
- Ingress

ingress:
- from:
  - podSelector:
      matchLabels:
        app: gateway

  ports:
  - protocol: TCP
    port: 8080
```

Let's decode this carefully.

19. First: Which Pods does this policy protect?

```
podSelector:
  matchLabels:
    app: employee
```

This means:

Apply this NetworkPolicy to Employee Pods.

So:

```
Policy
  ↓
Employee Pods
```

not Gateway Pods.

20. What direction?

```
policyTypes:
- Ingress
```

We're controlling:

```
Traffic coming INTO Employee
```

21. Who is allowed?

```
from:
- podSelector:
    matchLabels:
      app: gateway
```

Meaning:

Allow traffic originating from Gateway Pods.

So:

```
Gateway Pod
app=gateway
-
  | allowed
  | ¼
Employee Pod
app=employee
```

Other sources may not be allowed by this policy/isolation setup.

22. Which port?

```
ports:
- protocol: TCP
  port: 8080
```

Meaning:

```
Gateway
  ↓ TCP
Employee:8080
```

So we're not simply saying:

Gateway can do anything.

We're saying conceptually:

Gateway can reach Employee on TCP 8080.

Important namespace nuance

Our simple example assumes Pods are in the same namespace for straightforward podSelector reasoning.

When applications span namespaces, NetworkPolicy can also use:

```
namespaceSelector
```

and combinations of:

```
namespaceSelector
podSelector
```

We'll cover namespace isolation later.

Egress NetworkPolicy

Now suppose Employee Service should be allowed to communicate only with:

Database

or perhaps certain other services.

You can define egress policy.

Conceptually:

Employee Pod

-

| allowed
| $\frac{1}{4}$

Database

Employee Pod

-

X blocked
| $\frac{1}{4}$

Random destination

The policy is applied to the source Pod because we're controlling traffic leaving it.

Ingress vs Egress policy thinking

Always identify:

SOURCE → DESTINATION

Then:

SOURCE side

=

EGRESS

and:

DESTINATION side

=

INGRESS

Example:

Gateway → Employee

Therefore:

Gateway egress

Employee ingress

Example:

Employee → Kafka

Therefore:

Employee egress
Kafka ingress

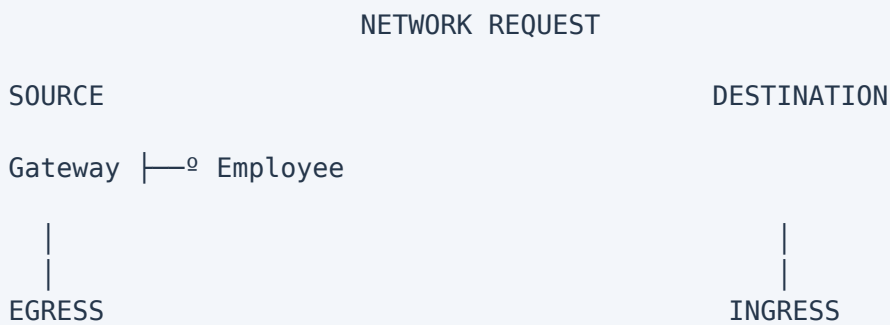
Example:

Employee → Database

Therefore:

```
Employee egress
Database ingress
```

The easiest diagram to remember



If you remember this, you✓I rarely confuse the terms.

Ingress has another Kubernetes meaning

This causes significant confusion.

The word ingress has two meanings in Kubernetes conversations.

General networking term

```
ingress traffic
=
traffic entering something
```

Kubernetes resource

```
Ingress
=
an API resource used for HTTP/HTTPS routing into cluster Services
```

These are related, but not identical.

Kubernetes Ingress resource

Imagine:

Internet
↓
mycompany.com

You want:

```

/api/employee
  ↓
Employee Service

```

and:

```

/api/payment
  ↓
Payment Service

```

An Ingress can define routing rules like:

```

Internet
  ↓
  Ingress
-
  |-----4|-----
  |
  /employee
  |
  | 1/4
Employee
Service

  |
  /payment
  |
  | 1/4
Payment
Service

```

We✓I study this in the next chapter.

For now:

Ingress traffic “” necessarily Ingress resource.

NetworkPolicy vs Ingress resource

Don't mix these.

NetworkPolicy

Who is allowed to communicate with whom?

Example:

```
Allow Gateway → Employee
Block Random Pod → Employee
```

Ingress resource

How should external HTTP/HTTPS requests be routed?

Example:

```
api.company.com/employee
      ↓
Employee Service
```

Completely different jobs.

Firewall analogy

NetworkPolicy is somewhat like an application-level cluster firewall.

Traditional firewall rule:

```
Source:
10.1.1.10

Destination:
10.1.1.20

Port:
8080
```

Kubernetes can make this more dynamic using labels:

```
Source:
app=gateway

Destination:
app=employee

Port:
8080
```

Why is this useful?

Because Pod IPs constantly change.

Instead of maintaining:

```
10.244.1.5
10.244.2.10
10.244.3.55
```

you work with identities like:

```
app=gateway
app=employee
```

Pod replacement and networking policy

Suppose:

```
Employee Pod A  
IP 10.244.2.10  
app=employee
```

dies.

New Pod:

```
Employee Pod B  
IP 10.244.4.90  
app=employee
```

A label-based NetworkPolicy still applies because:

```
app=employee
```

hasn't changed.

That is a major advantage over IP-based configuration.

32. Where does Service sit relative to NetworkPolicy?

Suppose Gateway calls:

```
employee-service:8080
```

Flow conceptually:

```
Gateway  
↓  
Employee Service  
↓  
Employee Pod
```

NetworkPolicy ultimately governs Pod network traffic.

So the important policy relationship is conceptually:

```
Gateway Pod → Employee Pod
```

even though Gateway called a Service name.

This is why network troubleshooting often requires understanding:

```
Service  
selectors  
backend Pods  
NetworkPolicies
```

together.

One subtle point about ports

Suppose Service is:

```
port: 80
targetPort: 8080
```

Gateway calls:

```
employee-service:80
```

But Employee application receives traffic on:

```
8080
```

When thinking about policies, focus carefully on the destination Pod port relevant to the policy/network implementation—in this example, the backend application port is 8080 .

So don't blindly assume:

```
Service port = application port
```

We already learned these can differ.

34. What happens when DNS resolution occurs?

Suppose Gateway calls:

```
http://employee-service:8080
```

Before the actual connection, Gateway needs to resolve:

```
employee-service
```

DNS flow conceptually:

```
Gateway Pod
  ↓
Cluster DNS
  ↓
employee-service → Service IP
```

Then:

```
Gateway
  ↓
Service IP
  ↓
backend Employee Pod
```

This matters for troubleshooting.

If the application says:


```
UnknownHostException: employee-service
```

your problem may be DNS/service naming.

If DNS resolves but connection is refused, it may be something else.

| Java troubleshooting example

Suppose your Spring Boot Gateway logs:

```
java.net.UnknownHostException:  
employee-service
```

Think:

```
Application couldn't resolve service name
```

Investigate:

```
Does Service exist?
```

```
Correct namespace?
```

```
Correct service name?
```

```
Cluster DNS healthy?
```

| Different failure

Suppose:

```
Connection refused
```

That could indicate:

```
Service/backend reached  
but application isn't listening on expected port
```

Possible issue:

```
targetPort wrong  
application not listening  
application startup failure
```

| Different failure again

Suppose:

Connection timed out

Potential areas include:

```
NetworkPolicy
routing/networking
firewall
backend availability
external connectivity
```

The exact cause requires investigation, but the important thing is you're thinking by layers.

Networking troubleshooting layers

A useful mental model is:

```
1. Application
   ↓
2. Port
   ↓
3. Pod
   ↓
4. Service
   ↓
5. DNS
   ↓
6. NetworkPolicy
   ↓
7. Cluster/CNI networking
   ↓
8. External firewall/load balancer
```

Don't jump immediately to:

"Kubernetes networking is broken."

Work layer by layer.

Example: Gateway can't call Employee

Use this debugging approach.

Step 1 — Is Employee Pod healthy?

```
kubectl get pods
```

Step 2 — Is Employee Service present?

```
kubectl get svc employee-service
```

Step 3 — Does selector match?

```
kubectl describe svc employee-service
```

and:

```
kubectl get pods --show-labels
```

Step 4 — Are backend endpoints present? Inspect the Service/EndpointSlice information.

If no backend endpoints exist:

```
Service
  ↓
nothing
```

often investigate selectors and Pod readiness.

Step 5 — Is correct targetPort configured? Example:

```
Spring Boot = 8080
Service targetPort = 9090 ×
```

Step 6 — Can DNS resolve? From an appropriate test Pod/container:

```
employee-service
```

should resolve correctly.

Step 7 — Check NetworkPolicies

```
kubectl get networkpolicy
```

short form:

```
kubectl get netpol
```

Could traffic be blocked?

Namespaces affect networking

Suppose:

```
Namespace:
gateway
```

and:

```
Namespace:
employee
```

Calling only:

```
employee-service
```

from another namespace may not resolve to the intended Service.

You may use:

```
employee-service.employee
```

Conceptually:

```
<Service>.<Namespace>
```

For example:

```
employee-service.employee
```

The full internal DNS form is typically:

```
employee-service.employee.svc.cluster.local
```

Again, we'll go deeper when we learn Namespaces.

Important security principle

Imagine an enterprise cluster containing:

```
Gateway  
Payment  
Employee  
Notification  
Reporting  
Admin  
Kafka
```

Without isolation, workloads may have broad network reachability.

A more secure architecture follows:

Only allow communication that is actually required.

For example:

```
Gateway → Employee '
Gateway → Payment '

Reporting → Payment ×

Employee → Employee DB '
Employee → Payment DB ×
```

This is called:

Least privilege The same principle we use for:

RBAC permissions

also applies to:

network access

| Default-deny concept

In security-sensitive namespaces, teams often establish:

Default deny

and then explicitly allow required communication.

Conceptually:

Initially:

Everything
X
Employee

Then allow:

Gateway
↓
Employee

This creates much stronger isolation than allowing everything and hoping applications behave correctly.

We✓I write proper default-deny examples later when we cover security.

| NetworkPolicy requires networking implementation support

Important:

Kubernetes defines the NetworkPolicy API.

But the cluster networking implementation needs to actually enforce those policies.

So conceptually:

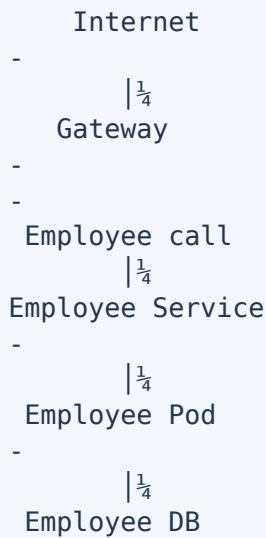
Kubernetes NetworkPolicy
↓
CNI / network implementation
↓
Actual packet enforcement

This is another example of Kubernetes defining desired behavior while infrastructure

components implement it.

Complete example

Let's model a microservice environment:



Traffic perspective:

Internet → Gateway

Gateway ingress

Gateway → Employee

Gateway egress
Employee ingress

Employee → DB

Employee egress
Database ingress

This is the exact way you should reason about firewall and NetworkPolicy rules.

Your earlier question answered precisely

You had previously asked something similar to:

Gateway is calling Service. So is this ingress or egress? Should rules be configured at Gateway or Service? Answer:

For:

Gateway → Employee

the connection is simultaneously:

```
EGRESS from Gateway
```

and:

```
INGRESS to Employee
```

If only Employee has ingress isolation:

```
Employee policy  
allow ingress from Gateway
```

may be sufficient from the Kubernetes NetworkPolicy perspective.

If Gateway is also egress-isolated:

```
Gateway policy  
allow egress to Employee
```

is also required.

So:

Whether you need rules on one side or both depends on whether ingress and/or egress isolation is active for those Pods.

| One more important distinction: Service is not normally where policy

identity lives Suppose:

```
Gateway  
  ↓  
Employee Service  
  ↓  
Employee Pods
```

You generally think about policy on the Pods/workloads, not:

```
"Allow ingress into Service object."
```

The Service is the stable network abstraction.

The actual workloads receiving traffic are Employee Pods.

So your reasoning should be:

```
Gateway Pods  
  ↓  
Employee Pods
```

while the Service provides discovery/routing.

Node networking vs Pod networking

Don't confuse:

Node IP

and:

Pod IP

For example:

Worker Node A

Node IP = 192.168.10.20

```
├─ Pod A
   Pod IP = 10.244.1.5
```

and:

Worker Node B

Node IP = 192.168.10.21

```
├─ Pod B
   Pod IP = 10.244.2.8
```

The CNI/network implementation makes Pod networking function across the Node infrastructure.

48. Why does Kubernetes need both Node IP and Pod IP?

Because these represent different layers.

Node IP

=

machine identity

Pod IP

=

workload network identity

A Node can host many Pods:

Node

192.168.10.20

```
├─ Pod A
   10.244.1.5
```

-

```
├─ Pod B
   10.244.1.6
```

-

```
├─ Pod C
```



```
10.244.1.7
```

| Service IP is yet another IP

Now we potentially have three kinds of IPs:

```
Node IP
Pod IP
Service ClusterIP
```

Example:

```
Node IP:
192.168.10.20
```

```
Pod IP:
10.244.1.5
```

```
Service IP:
10.96.20.10
```

They have different purposes.

IP Represents

Node IP Worker/control-plane machine

Pod IP Individual Pod workload

Service ClusterIP Stable virtual Service endpoint

This table is worth remembering.

| Complete request flow inside cluster

Let's say Gateway calls:

```
http://employee-service:80
```

Employee Service:

```
Service IP = 10.96.10.10
port = 80
targetPort = 8080
```

Employee Pods:

```
10.244.1.10:8080
10.244.2.20:8080
10.244.3.30:8080
```

Flow:

```

Gateway Pod
10.244.5.10
-
  | DNS lookup
  | 1/4
employee-service
-
  | 1/4
10.96.10.10:80
Service
-
  | backend selection
  | 1/4
10.244.2.20:8080
Employee Pod

```

From traffic-direction perspective:

Gateway = egress

Employee = ingress

A key interview question

What is CNI? A good beginner/intermediate answer:

CNI, or Container Network Interface, is the standard Kubernetes ecosystem uses to integrate networking implementations that configure Pod networking, including interfaces, IP addressing and connectivity. Depending on the implementation, it may also enforce NetworkPolicies.

52. Interview question: What is ingress and egress?

Ingress is traffic entering a workload or network boundary, while egress is traffic leaving it. The same request can simultaneously be egress from the source and ingress to the destination.

That is the important last sentence.

53. Interview question: Gateway calls Employee. Is it ingress or egress?

Strong answer:

From Gateway's perspective it is egress. From Employee's perspective it is ingress. NetworkPolicy rules must therefore be evaluated based on which Pods are ingress- or egress-isolated.

Excellent answer.

54. Interview question: What is NetworkPolicy?

NetworkPolicy is a Kubernetes resource used to control allowed ingress and egress traffic for selected Pods, typically using Pod and namespace selectors and allowed ports/protocols. Enforcement depends on the cluster networking implementation.

Interview question: Does Kubernetes block Pod communication by

default? A careful answer:

In the absence of NetworkPolicies selecting and isolating a Pod, traffic is generally allowed by default. Once applicable ingress or egress policies select a Pod, allowed traffic for that direction is determined by those policies.

Check your understanding

Consider:

Gateway → Payment

Q1 For Gateway, this is:

Egress.

Q2 For Payment, this is:

Ingress.

Consider:

Payment → Kafka

Q3 For Payment:

Egress.

Q4 For Kafka:

Ingress.

Q5 Which technology implements Pod networking?

Conceptually:

CNI-compatible networking implementation.

Q6 What gives an application stable access to changing Pods?

Kubernetes Service.

Q7 What controls which Pods may communicate?

NetworkPolicy, when used and enforced by the networking implementation.

Mini exercise

Consider:

```
Internet
  ↓
Gateway
  ↓
Order Service
  ↓
Payment Service
  ↓
Database
```

Identify directions:

Internet → Gateway

Gateway ingress

Gateway → Order

Gateway egress
Order ingress

Order → Payment

Order egress
Payment ingress

Payment → Database

Payment egress
Database ingress

If this becomes automatic in your mind, Kubernetes NetworkPolicy becomes dramatically easier.

chapter 6 Summary

We now understand:

```
Node
  ↓
Pods have IPs
  ↓
CNI makes Pod networking possible
```

and:

Service
 ↓
 provides stable access
 ↓
 changing Pod IPs

and most importantly:

SOURCE —^o DESTINATION

EGRESS

INGRESS

Our architecture now looks like:

```

  Internet
-
-
  INGRESS
  | 1/4
Gateway Pod
-
  EGRESS
  | 1/4
Employee Service
-
  | 1/4
Employee Pod
-
  EGRESS
  | 1/4
Database
```

NetworkPolicy lets us place security controls around those workload communication paths.

Next — chapter 7: Ingress & OpenShift Route Now we have solved internal communication:

```

Gateway
↓
Employee Service
↓
Employee Pods
```

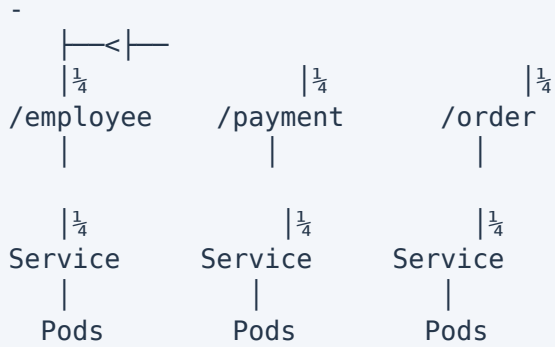
But there's still a major unanswered question:

How does an external user on the internet reach our application in the first place?

Next we✓I build:

```

  INTERNET
-
  | 1/4
Ingress / Router
```



We✓I cover Ingress, Ingress Controller, host-based routing, path-based routing, TLS/HTTPS, LoadBalancer vs Ingress, and then the OpenShift Route equivalent.

Chapter 8 — Services, DNS, and Service Discovery

CHAPTER FOCUS

Understand **Services, DNS, and Service Discovery** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

Now we reach the next major Kubernetes problem:

Pods are disposable and their IP addresses can change. So how does one application reliably communicate with another?

This is exactly why Kubernetes has a Service.

Start with the problem

Suppose Employee Service has 3 Pods:

Employee Pod A
IP = 10.244.1.10

Employee Pod B
IP = 10.244.2.17

Employee Pod C
IP = 10.244.3.21

Your Gateway wants to call Employee Service.

You could technically call:

```
http://10.244.1.10:8080
```

But remember:

Pods are temporary.

Suppose Pod A crashes.

ReplicaSet creates a replacement:

Old Pod A ×
10.244.1.10

New Pod D '
10.244.4.35

Now Gateway still knows:

```
10.244.1.10
```

which no longer exists.

So directly calling Pod IPs is unreliable.

| What we really need

We want a stable address:

```
Gateway
  ↓
employee-service
  ↓
Employee Pods
```

Even if individual Pods change:

Yesterday

```
Pod A → 10.244.1.10
Pod B → 10.244.2.17
Pod C → 10.244.3.21
```

Today:

```
Pod D → 10.244.4.35
Pod E → 10.244.5.11
Pod F → 10.244.2.48
```

Gateway should continue calling:

```
employee-service
```

without caring which Pods currently exist.

That stable abstraction is a Kubernetes:

Service

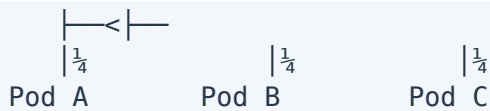
3. What is a Service?

One-sentence definition

A Kubernetes Service provides a stable network endpoint for a dynamic group of Pods.

Think:

```
Service
employee-service
-
```

Clients communicate with:

employee-service

instead of individual Pod IPs.

Service vs Pod

Very important:

Pod

=

Runs your application

Service

=

Provides stable access to matching Pods

For example:

Employee Deployment

↓

ReplicaSet

↓

3 Employee Pods

!'

-

Employee Service

The Service does not run your application.

It routes traffic to the Pods that run the application.

Service is not the Java service itself

This naming can be confusing.

You may already call your Spring Boot application:

Employee Service

Then Kubernetes also has an object called:

Service

These are different concepts.

You might have:

```
Business application:
Employee Microservice
```

```
Kubernetes object:
employee-service Service
```

So when we say:

Create a Service.

we mean the Kubernetes networking resource.

6. How does Service know which Pods belong to it?

Once again:

Labels and selectors Suppose your Employee Pods have:

```
labels:
  app: employee
```

The Service contains:

```
selector:
  app: employee
```

Then Kubernetes connects the Service to matching Pods.

Conceptually:

Service

```
selector:
app=employee
```

```
-
  | matches
  | 1/4
```

```
Pod A
app=employee
```

```
Pod B
app=employee
```

```
Pod C
app=employee
```

A Payment Pod:

```
app=payment
```

does not match.

This is why labels are so important

Remember our Deployment:

```
template:
  metadata:
    labels:
      app: employee
```

Every Pod created by that Deployment receives:

```
app=employee
```

Then the Service can select them:

```
selector:
  app: employee
```

So the relationship becomes:

```
Deployment
  ↓
ReplicaSet
  ↓
Pods
app=employee
! '
  | selector
Service
app=employee
```

Notice something interesting:

The Service does not need to know the Deployment name.

It primarily discovers backend Pods through labels/selectors.

First Service YAML

Here's a simple Service:

```
apiVersion: v1
kind: Service

metadata:
  name: employee-service

spec:
  selector:
    app: employee

  ports:
```

```
- port: 80
  targetPort: 8080

type: ClusterIP
```

Let's understand every part.

| kind: Service

```
kind: Service
```

This tells Kubernetes:

Create a networking Service resource.

| Service name

```
metadata:
  name: employee-service
```

This is important because other applications can use this name for service discovery.

For example, Payment Service may call:

```
http://employee-service
```

instead of:

```
http://10.244.2.17
```

Much better.

| Selector

```
selector:
  app: employee
```

This means:

Send traffic to Pods with app=employee .

Suppose Kubernetes currently has:

```
Pod A → app=employee
Pod B → app=employee
Pod C → app=payment
Pod D → app=employee
```

The Service targets:

Pod A
Pod B
Pod D

Not Pod C.

Understanding port and targetPort

This confuses many beginners.

Our Service says:

```
ports:
  - port: 80
    targetPort: 8080
```

Meaning:

```
Client
  ↓
Service port 80
  ↓
Pod/container port 8080
```

Suppose your Spring Boot application listens on:

8080

Clients can call the Service on:

80

and Kubernetes forwards traffic to:

8080

on the selected backend Pod.

Visualizing it

```
Gateway
-
  | employee-service:80
  | 1/4
  -----
  | Kubernetes Service   |
  | port = 80            |
  |-----|
  |
  | targetPort = 8080
  | 1/4
```

Employee Pod

```
| Spring Boot |  
| listens on 8080 |  
|-----|
```

| containerPort vs targetPort vs port

Let's connect everything.

Deployment:

```
containers:  
- name: employee  
  image: employee:v1  
  ports:  
    - containerPort: 8080
```

Service:

```
ports:  
- port: 80  
  targetPort: 8080
```

Conceptually:

```
Service port  
80  
↓  
targetPort  
8080  
↓  
Application listens on  
8080
```

Important:

containerPort mainly documents the intended application port in the Pod spec.

The Service's targetPort tells networking where backend traffic should go.

15. Can Service port also be 8080?

For example:

```
ports:  
- port: 8080  
  targetPort: 8080
```

Then:

```
employee-service:8080
```

↓

```
Pod:8080
```

That is very common.

There's no requirement for them to be different.

16. What is ClusterIP?

Our YAML said:

```
type: ClusterIP
```

This is the default Kubernetes Service type.

A ClusterIP Service is intended primarily for communication inside the Kubernetes cluster.

For example:

```
Gateway Pod
```

↓

```
Employee Service
```

↓

```
Employee Pods
```

Another application inside the cluster can use it.

An external internet user normally cannot directly access a ClusterIP Service.

Example ClusterIP

Suppose Kubernetes creates:

```
Service:
```

```
employee-service
```

```
ClusterIP:
```

```
10.96.20.50
```

Pods:

```
10.244.1.10
```

```
10.244.2.17
```

```
10.244.3.21
```

Then:

```
Gateway
```

↓

```
10.96.20.50
```

↓

```
Service
  ↓
One of:
10.244.1.10
10.244.2.17
10.244.3.21
```

The Service's virtual IP remains stable while backend Pod IPs change.

18. Do developers normally call the ClusterIP directly?

Usually no.

You normally use the Service's DNS name:

```
employee-service
```

instead of remembering:

```
10.96.20.50
```

Because Kubernetes provides built-in service discovery.

Kubernetes DNS

Suppose you have a Service named:

```
employee-service
```

Another Pod in the same namespace can commonly access it using:

```
employee-service
```

For example:

```
http://employee-service:8080
```

Kubernetes DNS resolves:

```
employee-service
  ↓
Service ClusterIP
```

Then Service networking directs traffic to matching Pods.

Why DNS matters

Without DNS:

```
Gateway
  ↓
```



```
10.96.20.50
```

You would still depend on an IP address.

With DNS:

```
Gateway
↓
employee-service
```

Much cleaner.

This is service discovery.

21. What is Service Discovery?

Simple definition

Service discovery allows applications to find other services using stable names rather than dynamic Pod addresses.

Instead of:

```
10.244.2.17
```

you use:

```
employee-service
```

Think of your phone contacts.

You don't memorize someone's phone number every time.

You save:

```
John
```

and your phone resolves:

```
John
↓
+91-xxxx...
```

Similarly:

```
employee-service
↓
Kubernetes DNS
↓
Service
```

| Namespace-aware DNS

Suppose you have:

```
Namespace: payroll  
Service: employee-service
```

and another Service with the same name:

```
Namespace: testing  
Service: employee-service
```

That is allowed because namespaces provide separation.

A more complete DNS name can look conceptually like:

```
employee-service.payroll
```

and a fully qualified Kubernetes DNS name is typically structured like:

```
employee-service.payroll.svc.cluster.local
```

You don't always need the full name.

Within the same namespace:

```
employee-service
```

is normally enough.

| We✓I cover namespaces later

For now, remember:

```
Same namespace:  
employee-service
```

Across namespaces:

```
employee-service.payroll
```

is a common form.

This will become much clearer in the Namespace chapter.

| Load balancing

Suppose the Service has 3 backend Pods:

```
Employee A  
Employee B  
Employee C
```

Requests arrive:

```
Request 1
Request 2
Request 3
Request 4
```

Kubernetes networking distributes connections across available backend Pods.

Conceptually:



So Service provides both:

```
stable access
+
traffic distribution
```

25. Does Service itself contain Pods?

No.

A Service references a dynamic backend set through selectors.

Think:

```
Service
-
  | selector
  | 1/4
Matching Pods
```

not:

```
Service
contains Pods
```

The Pods are separate Kubernetes objects.

26. What happens when a Pod dies?

Suppose:

```
Service
↓
Pod A
Pod B
Pod C
```

Pod B dies:

```
Pod B x
```

ReplicaSet creates:

```
Pod D '
```

Because Pod D has:

```
app=employee
```

it becomes part of the Service's backend set.

Conceptually:

Before

Service

```
├─ Pod A
├─ Pod B
└─ Pod C
```

After:

Service

```
├─ Pod A
├─ Pod C
└─ Pod D
```

The client continues using:

```
employee-service
```

Nothing changes from the client's perspective.

That is the beauty of the abstraction.

27. How does Kubernetes track Service backends?

Modern Kubernetes uses objects called:

```
EndpointSlices
```

to represent network endpoints backing Services.

Conceptually:

Service

↓

Endpoint information

↓

Pod IPs

For beginner learning, it's enough to understand:

Kubernetes continuously maintains the backend endpoints corresponding to the Service's selected Pods. Historically you✓I also hear the term Endpoints .

Example

Pods:

```
employee-a → 10.244.1.5
employee-b → 10.244.2.7
employee-c → 10.244.4.2
```

Service selector:

```
app=employee
```

Kubernetes endpoint information may effectively point toward:

```
10.244.1.5:8080
10.244.2.7:8080
10.244.4.2:8080
```

Pod B disappears.

Endpoint set becomes:

```
10.244.1.5:8080
10.244.4.2:8080
```

New Pod D appears:

```
10.244.7.8:8080
```

It gets added.

Clients don't need to know any of this.

Very important connection with labels

What happens if your Service has:

```
selector:
  app: employee
```

but your Pods have:

```
labels:
  app: employees
```

Notice the difference:

```
employee
vs
employees
```

Then:

Service selects zero Pods

Your Service exists, but has no matching backend.

This is a very common Kubernetes troubleshooting problem.

| Typical symptom

You may see:

```
Service exists '  
Pod exists '  
Application running '  
  
But request fails ×
```

One thing to check immediately:

Does Service selector match Pod labels?

This question is extremely important.

| Complete Deployment + Service example

Deployment:

```
apiVersion: apps/v1  
kind: Deployment  
  
metadata:  
  name: employee-deployment  
  
spec:  
  replicas: 3  
  
  selector:  
    matchLabels:  
      app: employee  
  
  template:  
    metadata:  
      labels:  
        app: employee  
  
    spec:  
      containers:  
        - name: employee  
          image: company/employee-service:1.0
```

```
ports:
  - containerPort: 8080
```

Service:

```
apiVersion: v1
kind: Service

metadata:
  name: employee-service

spec:
  selector:
    app: employee

  ports:
    - port: 8080
      targetPort: 8080

  type: ClusterIP
```

Notice:

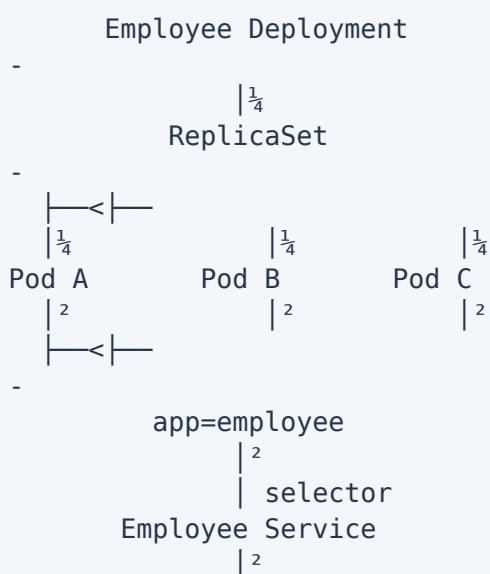
Deployment Pod label
app=employee

! • MATCHES

Service selector
app=employee

That is the connection.

Architecture



-

Gateway

| Now let's understand Service types

The main Service types you'll commonly encounter are:

```
ClusterIP
NodePort
LoadBalancer
ExternalName
```

For now we'll focus mostly on the first three.

| ClusterIP

We've already covered this.

```
type: ClusterIP
```

Purpose:

Internal cluster communication.

Example:

```
Payment Pod
  ↓
employee-service
  ↓
Employee Pods
```

Typical use:

```
microservice-to-microservice communication
```

| NodePort

What if you want someone outside the cluster to reach a Service?

One option is:

```
NodePort
```

NodePort opens a port on the Kubernetes Nodes.

Example:

```
Node IP:
192.168.1.20
```


NodePort:

30080

Then an external client might call:

192.168.1.20:30080

Flow:

External Client



NodeIP:NodePort



Service



Pod

NodePort architecture

Suppose:

Service Port = 80
Target Port = 8080
NodePort = 30080

Flow:

Client



WorkerNodeIP:30080



Service:80



Pod:8080

Three different ports can exist:

NodePort = 30080
Service = 80
TargetPort = 8080

Why NodePort isn't usually the preferred public production interface

Imagine a user needing to know:

10.10.5.17:30080

That is not a friendly public architecture.

And exposing arbitrary Node ports directly isn't normally how sophisticated enterprise HTTP routing is designed.

NodePort is useful for:

```
learning
testing
some infrastructure patterns
underlying implementations
```

But production web traffic commonly uses:

```
LoadBalancer
Ingress
OpenShift Route
```

depending on platform and architecture.

| LoadBalancer Service

In supported cloud/infrastructure environments, you can use:

```
type: LoadBalancer
```

Conceptually:

```
Internet
  ↓
External Load Balancer
  ↓
Kubernetes Service
  ↓
Pods
```

For example:

```
Public IP
34.x.x.x
  ↓
LoadBalancer
  ↓
employee-service
  ↓
Employee Pods
```

This provides external access through infrastructure integration.

39. But do we create one LoadBalancer per microservice?

Sometimes, but often that's inefficient or unnecessary for HTTP applications.

Imagine 50 services:

```
Service A → external LB
Service B → external LB
Service C → external LB
...
Service 50 → external LB
```

This can become expensive and cumbersome.

Instead, HTTP routing commonly uses:

```
Ingress
```

or in OpenShift:

```
Route
```

so multiple applications can share a common ingress layer.

We'll cover that separately.

| Internal vs external access

Think:

```
ClusterIP
=
Inside cluster

NodePort
=
Expose through Node port

LoadBalancer
=
Provision/use external load balancer
```

Later:

```
Ingress / OpenShift Route
=
HTTP/HTTPS routing to internal Services
```

| Very important: Service usually targets Pods, not Deployment

You might imagine:

```
Service
↓
Deployment
↓
```

Pods

Conceptually this seems reasonable, but Kubernetes networking selection is based on Pod labels.

More accurately:

```
Deployment ──0 Pods
              |2
-
Service |selector||
```

Service and Deployment are not directly dependent on each other.

They are connected because they agree on labels.

| Interesting consequence

Suppose you manually create a Pod:

```
labels:
  app: employee
```

and it matches the Service selector.

That Pod can become a backend for the Service even if it wasn't created by the Employee Deployment.

Because the Service cares about:

```
labels
```

not:

```
who created the Pod
```

This is powerful, but also means labels must be managed carefully.

| Service and rolling updates

Now remember Deployment rolling updates:

```
v1 Pods
  ↓
v2 Pods gradually replace them
```

Suppose both versions have:

```
app=employee
```

Service selects:

```
app=employee
```

During rollout, temporarily:

```
Service
```

```
↓
```

```
v1
```

```
v1
```

```
v2
```

Later:

```
Service
```

```
↓
```

```
v1
```

```
v2
```

```
v2
```

Finally:

```
Service
```

```
↓
```

```
v2
```

```
v2
```

```
v2
```

This is part of how users continue receiving traffic through a rolling deployment.

44. But what about unhealthy Pods?

This introduces another connection.

Suppose:

```
Pod A → healthy
```

```
Pod B → healthy
```

```
Pod C → application not ready
```

Should Service send traffic to Pod C?

Ideally:

```
No.
```

That is why Kubernetes uses readiness information.

When we configure a:

```
Readiness Probe
```

a Pod that isn't ready can be excluded from normal Service traffic.

Conceptually:

```
Service
  ↓
Ready Pod A '
Ready Pod B '

Not-ready Pod C ×
```

We✓I study this deeply later.

| Example Spring Boot architecture

Suppose you have:

```
Gateway
Employee
Payment
```

Each application uses a Deployment and Service.

```
Gateway Deployment
  ↓
Gateway Pods
  ! '
Gateway Service

Employee Deployment
  ↓
Employee Pods
  ! '
Employee Service

Payment Deployment
  ↓
Payment Pods
  ! '
Payment Service
```

Gateway can call:

```
http://employee-service:8080
```

and:

```
http://payment-service:8080
```

instead of tracking individual Pods.

| Request flow

Suppose a client request reaches Gateway somehow.

Gateway needs Employee data:

```
Gateway Pod
  ↓
DNS lookup:
employee-service
  ↓
Employee Service
  ↓
One ready Employee Pod
  ↓
Spring Boot :8080
```

The Gateway knows:

```
employee-service
```

It does not know or care:

```
employee-pod-abc
employee-pod-def
employee-pod-xyz
```

That is exactly what we want.

kubectl get services

To see Services:

```
kubectl get services
```

short form:

```
kubectl get svc
```

Example:

NAME	TYPE	CLUSTER-IP	PORT(S)
employee-service	ClusterIP	10.96.20.50	8080/TCP
payment-service	ClusterIP	10.96.30.21	8080/TCP

Interpretation:

```
employee-service
```

```
Type:
ClusterIP
```

```
Stable virtual IP:
10.96.20.50
```

```
Port:
8080
```

Describe Service

Use:

```
kubectl describe svc employee-service
```

You can inspect information such as:

```
Selector
Type
IP
Port
TargetPort
Endpoints
Events
```

One particularly useful thing:

```
Endpoints
```

If you expect Pods but see no usable backends, check labels and readiness.

Useful troubleshooting sequence

Suppose:

Gateway cannot call Employee Service.

Start logically.

Step 1 Does Service exist?

```
kubectl get svc employee-service
```

Step 2 Are Employee Pods running?

```
kubectl get pods
```

Step 3 Do labels match?

```
kubectl get pods --show-labels
```

Step 4 Inspect Service:

```
kubectl describe svc employee-service
```

Step 5 Are endpoints/backend endpoints available?

Then investigate application port/network policies/DNS as needed.

Don't immediately restart everything.

Classic selector bug

Deployment:

```
labels:  
  app: employee-service
```

Service:

```
selector:  
  app: employee
```

These are different.

Result:

```
Service  
  ↓  
No matching Pods
```

Even though:

```
Pods = Running  
Service = Exists
```

the application won't receive traffic through that Service.

This is one of the first real Kubernetes troubleshooting lessons.

Another classic problem: wrong targetPort

Spring Boot listens:

```
8080
```

but Service says:

```
targetPort: 9090
```

Flow becomes:

```
Service  
  ↓  
Pod:9090  
  ↓  
Nothing listening ×
```

The Pod may show:

```
Running ' '
```

while client requests fail.

Again, Kubernetes isn't necessarily broken.

Your networking configuration is wrong.

| Service port does not have to match application port

Example:

```
port: 80
targetPort: 8080
```

perfectly valid.

Think:

```
Clients see:
80
```

```
Application internally listens:
8080
```

This is similar conceptually to port mapping/proxying.

| targetPort can also be named

Instead of:

```
targetPort: 8080
```

you can name a Pod port.

Pod:

```
ports:
- name: http
  containerPort: 8080
```

Service:

```
targetPort: http
```

This can make manifests more descriptive.

For beginner learning, numeric ports are fine.

| Service does not fix application-level problems

Suppose Service routing works perfectly, but Employee application returns:

```
HTTP 500
```

That is an application problem.

Service responsibilities are networking-related.

Think:

Can traffic reach the application?

↓

Kubernetes networking

What does application return?

↓

Application behavior

Keeping boundaries clear makes troubleshooting much easier.

Service + Deployment relationship summary

You now have two independent concerns:

Deployment

How many application instances?

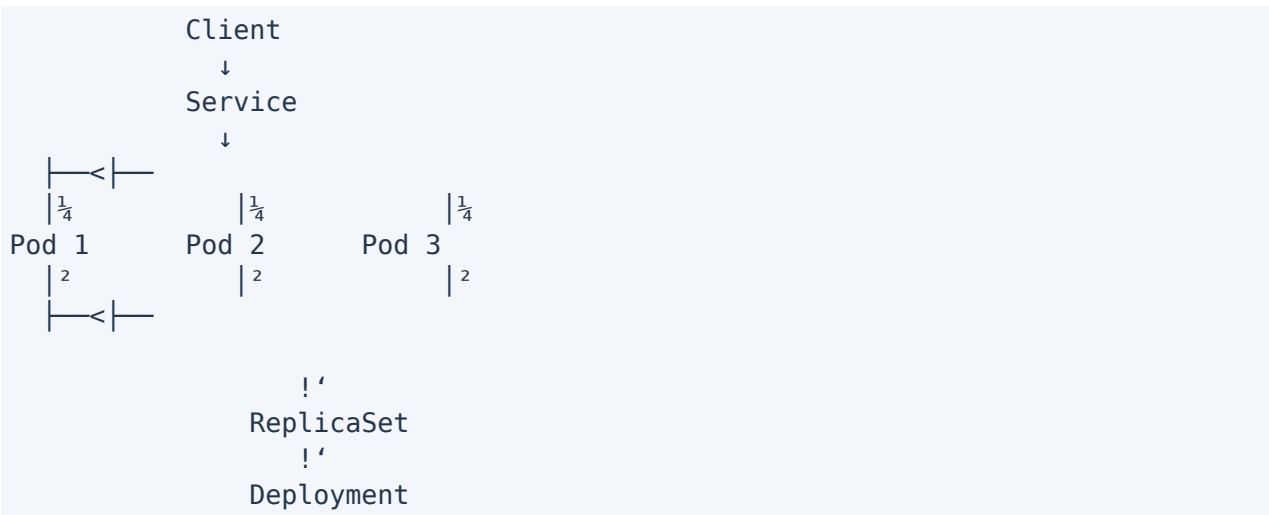
Which image version?

How should updates happen?

Service

How do clients find and reach those application instances?

Together:



That is a very important Kubernetes architecture.

What Service does NOT do

A Service does not:

create Pods
restart Pods
build images

```
manage ReplicaSet  
perform rolling updates
```

Those responsibilities belong elsewhere.

A Service mainly gives:

```
stable networking  
service discovery  
backend selection  
traffic forwarding/load distribution
```

| Service vs ReplicaSet selector

Interesting question:

Both ReplicaSet and Service use selectors.

Why?

Because they use them for different purposes.

ReplicaSet selector:

```
Which Pods do I maintain?
```

Service selector:

```
Which Pods should receive traffic?
```

Example:

```
Pod:  
app=employee
```

ReplicaSet:

```
app=employee  
→ maintain 3
```

Service:

```
app=employee  
→ send traffic
```

Same label system, different responsibilities.

| Full production mental model so far

We now understand this:



And all these Pods run across:

Worker Nodes

selected by:

Scheduler

OpenShift connection

This concept carries directly into OpenShift.

You will use:

`oc get svc`

Services remain Kubernetes Services.

But OpenShift adds another commonly used resource:

Route

A typical OpenShift architecture becomes:

Internet
↓
OpenShift Route

```
↓  
Kubernetes Service  
↓  
Pods
```

This will be very familiar when we reach the OpenShift chapter.

| Common interview question

Why do we need a Service if Pods already have IP addresses? Good answer:

Pod IP addresses are ephemeral and can change whenever Pods are replaced. A Kubernetes Service provides a stable network identity and discovers matching backend Pods using selectors, allowing clients to reach the application without tracking individual Pod IPs.

| Interview question: Service vs Deployment

What's the difference between Deployment and Service?

Answer:

A Deployment manages the lifecycle and desired replica count of application Pods through ReplicaSets, while a Service provides stable network access and service discovery for a set of matching Pods.

62. Interview question: How does Service find Pods?

Answer:

Typically through label selectors. Kubernetes maintains backend endpoint information for Pods matching the Service selector.

| Interview question: ClusterIP vs NodePort vs LoadBalancer

Simplified answer:

```
ClusterIP  
→ internal cluster access
```

```
NodePort  
→ exposes Service on a port of cluster Nodes
```

```
LoadBalancer  
→ integrates with external load-balancer infrastructure
```

Check your understanding

Q1 You have:

```
Pod A → 10.244.1.5  
Pod B → 10.244.2.6
```

Should Payment Service normally store these addresses?

No.

It should use the stable Kubernetes Service name.

Q2 Pods have:

```
app=employee
```

Service selector:

```
app=payment
```

Will traffic reach the Employee Pods?

No.

Selectors don't match.

Q3 Application runs on:

```
8080
```

Service has:

```
port: 80  
targetPort: 8080
```

Which port does the client call?

```
Service port 80
```

Traffic is forwarded to backend application port:

```
8080
```

Q4 Which Service type is primarily for internal cluster communication?

ClusterIP.

Q5 Does a Service create replacement Pods?

No.

ReplicaSet/other workload controllers handle that.

Mini scenario

Suppose:

```
Employee Deployment  
replicas = 3
```

Pods:

```
employee-a  
IP 10.1.1.10
```

```
employee-b  
IP 10.1.1.11
```

```
employee-c  
IP 10.1.1.12
```

All have:

```
app=employee
```

Service:

```
name = employee-service  
selector = app=employee  
port = 80  
targetPort = 8080
```

Gateway calls:

```
http://employee-service
```

Pod B dies.

ReplicaSet creates:

```
employee-d  
IP 10.1.1.25
```

Question:

Does Gateway configuration need to change?

No.

Gateway continues calling:

```
employee-service
```

Kubernetes updates the backend set automatically.

That is the central idea of this chapter.

chapter 5 Summary

Our Kubernetes stack has now grown considerably:

```
Deployment
  ↓
ReplicaSet
  ↓
Pods
  ↓
Containers
```

handles:

```
Application lifecycle
Scaling
Self-healing
Updates
```

While:

```
Service
  ↓
Pods
```

handles:

```
Stable network identity
Service discovery
Traffic forwarding
Load distribution
```

Put together:

```
Client
  ↓
Service
  ↓
Pods
  ! '
ReplicaSet
  ! '
Deployment
```

The one sentence to remember is:

Deployments make sure the application instances exist; Services make sure other applications can reliably find and reach them.

Next — chapter 6: Kubernetes Networking Fundamentals Now we'll go one level deeper and answer questions like:

How do Pods on different Nodes communicate?

Does every Pod really get an IP?

What is the Kubernetes network model?

What is CNI?

What is ingress traffic?

What is egress traffic?

If Gateway calls Employee Service:
who has ingress and who has egress?

What is a NetworkPolicy?

Does Service traffic pass through a traditional load balancer?

How does OpenShift networking fit into this?

We✓I use this exact architecture:

```
graph TD
    User[External User] --> Gateway[Gateway Pod]
    Gateway --> Service[Employee Service]
    Service --> Pod[Employee Pod]
    Pod --> Database[Database]
```

and identify ingress and egress from every component’s point of view, because that’s one of the most important networking ideas to get right.

We’ll move into chapter 6: Kubernetes Networking Fundamentals. I’ll focus first on the traffic mental model—Pod IPs, Services, ingress vs egress, CNI, and NetworkPolicy—before touching OpenShift-specific routing.

Chapter 9 — Ingress, Ingress Controllers, and OpenShift Routes

CHAPTER FOCUS

Understand **Ingress, Ingress Controllers, and OpenShift Routes** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

So far, we know how applications communicate inside Kubernetes:

```
Gateway Pod
  ↓
Employee Service
  ↓
Employee Pods
```

Now let's solve the next problem:

How does a user outside the Kubernetes cluster reach our application?

That leads us to Ingress in Kubernetes and Route in OpenShift.

Start with the problem

Imagine we have three applications:

```
Employee Service
Payment Service
Order Service
```

Each application has its own Deployment and ClusterIP Service:

```
Employee Service → employee-service
Payment Service → payment-service
Order Service    → order-service
```

Inside the cluster, everything works:

```
Gateway Pod
  ↓
employee-service
  ↓
Employee Pods
```

But now someone sitting on the internet opens:

```
https://api.company.com/employees
```

How does that request enter Kubernetes?

A ClusterIP cannot normally be directly reached from outside.

We need an entry point.

One solution: LoadBalancer Service

Remember the previous Service types:

```
ClusterIP
NodePort
LoadBalancer
```

We could expose Employee Service as:

```
type: LoadBalancer
```

Then:

```
Internet
  ↓
External Load Balancer
  ↓
Employee Service
  ↓
Employee Pods
```

This works.

But imagine we have 50 microservices.

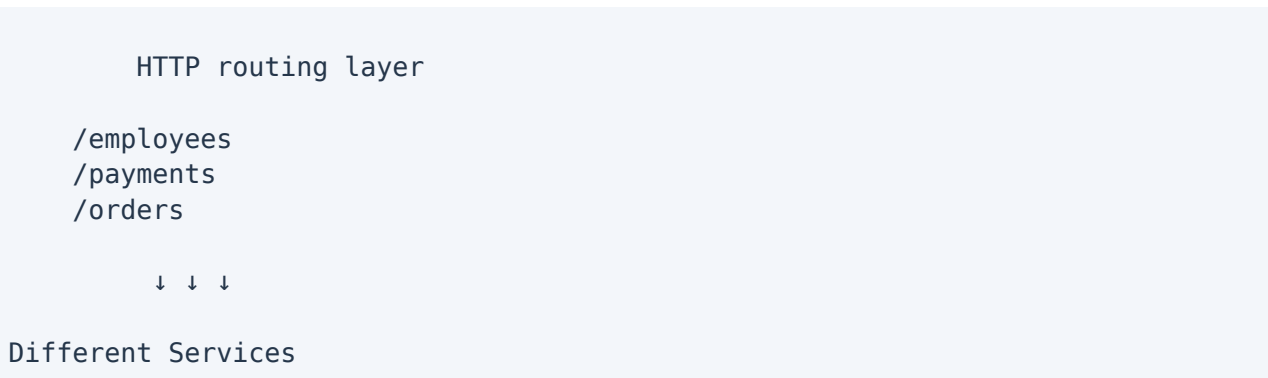
Do we really want:

```
Employee → Load Balancer 1
Payment → Load Balancer 2
Order    → Load Balancer 3
...
Service 50 → Load Balancer 50
```

Usually that's not ideal for HTTP/HTTPS services.

Instead, we'd prefer:

```
One external entry point
  ↓
api.company.com
  ↓
```



That is where Ingress becomes useful.

3. What is Kubernetes Ingress?

One-sentence definition

An Ingress is a Kubernetes API resource that defines HTTP/HTTPS routing rules for external traffic to Kubernetes Services.

For example:

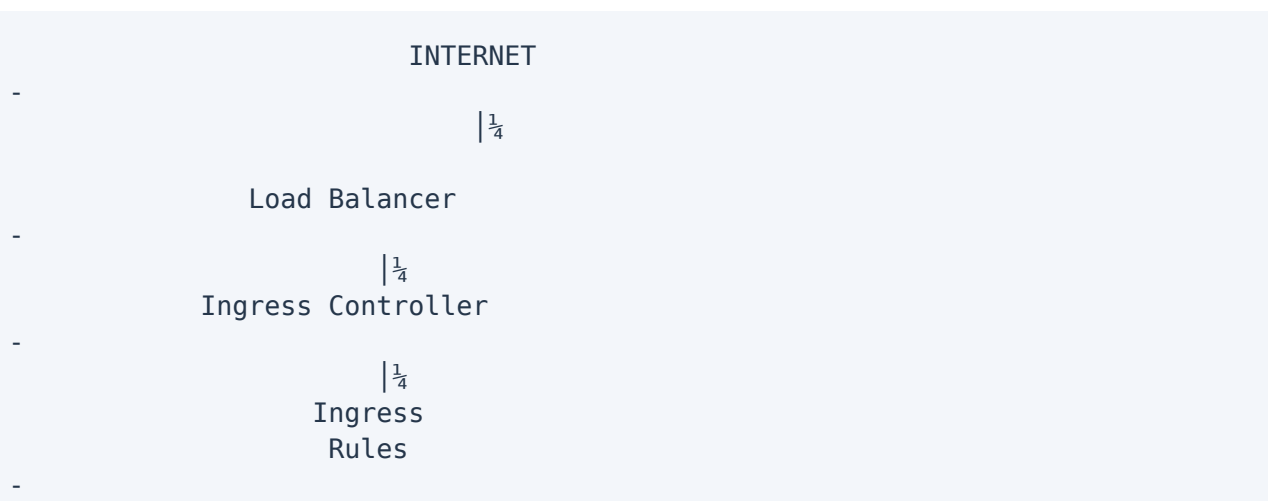
```
api.company.com/employees
      ↓
Employee Service

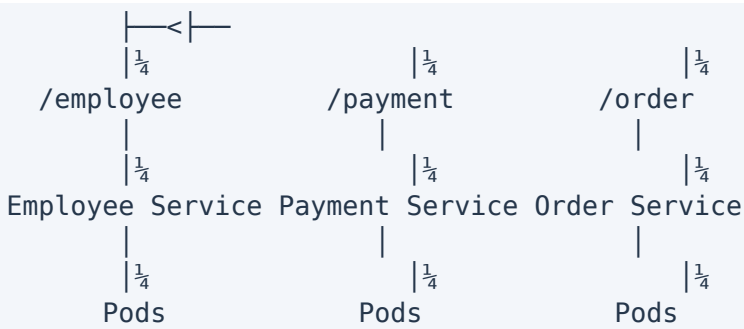
api.company.com/payments
      ↓
Payment Service

api.company.com/orders
      ↓
Order Service
```

Instead of exposing every Service independently.

Basic architecture





This introduces an important distinction:

Ingress resource and Ingress Controller are not the same thing.

Ingress vs Ingress Controller

This is one of the most commonly confused Kubernetes concepts.

Ingress Ingress is basically configuration.

It says:

```
If host = api.company.com
and path = /employee

send traffic to:
employee-service:8080
```

Ingress itself does not normally sit there as a network process forwarding packets.

Ingress Controller The Ingress Controller is the actual software that watches Ingress resources and implements those routing rules.

Think:

```
Ingress
=
Rules

Ingress Controller
=
Component executing those rules
```

Restaurant analogy

Imagine a restaurant.

The seating plan says:

```
Table 1 → Family A
Table 2 → Family B
Table 3 → Family C
```

That is the configuration.

But someone actually needs to guide customers to the tables.

That is the host.

Similarly:

```
Ingress
=
routing instructions

Ingress Controller
=
actual traffic-routing implementation
```

| Very important interview question

Can an Ingress resource work without an Ingress Controller? Generally:

No useful routing happens merely because an Ingress object exists.

You need an Ingress Controller or equivalent platform implementation capable of processing those rules.

Examples historically include controllers built around technologies such as NGINX, HAProxy, Traefik and cloud-provider integrations.

| Path-based routing

Suppose your domain is:

```
api.company.com
```

You want:

```
/api/employees
      ↓
Employee Service

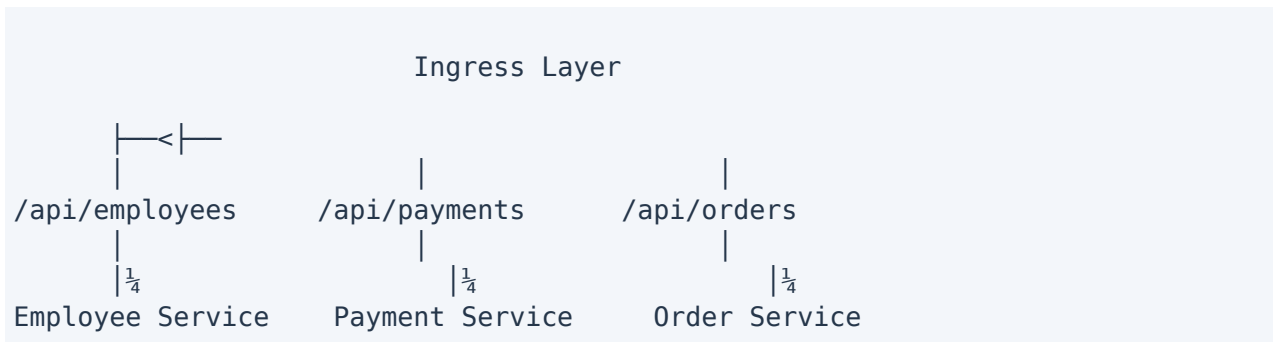
/api/payments
      ↓
Payment Service

/api/orders
      ↓
Order Service
```

Then:

```
api.company.com
```

```
↓
```



This is path-based routing.

Host-based routing

Instead of paths, we can use different hostnames.

For example:

```

employee.company.com
    ↓
Employee Service

payment.company.com
    ↓
Payment Service

orders.company.com
    ↓
Order Service
  
```

Architecture:

```

employee.company.com |—° Employee Service
payment.company.com |—° Payment Service
orders.company.com |—° Order Service
  
```

This is host-based routing.

Basic Ingress YAML

A simplified example:

```

apiVersion: networking.k8s.io/v1
kind: Ingress

metadata:
  name: company-api

spec:
  
```



```
rules:
  - host: api.company.com

    http:
      paths:
        - path: /employees
          pathType: Prefix

          backend:
            service:
              name: employee-service

            port:
              number: 8080
```

Let's decode this.

| kind: Ingress

```
kind: Ingress
```

Means:

Create an HTTP/HTTPS routing configuration.

| Host

```
host: api.company.com
```

Meaning:

This rule applies when the HTTP host is api.company.com .

For example:

```
https://api.company.com/employees
```

| Path

```
path: /employees
```

Meaning:

Requests matching /employees should use this backend rule.

| pathType: Prefix

pathType: Prefix

This means paths beginning with the specified prefix can match.

Conceptually:

```
/employees  
/employees/100  
/employees/search
```

can all match a /employees prefix rule.

| Backend Service

```
backend:  
  service:  
  
    name: employee-service  
    port:  
      number: 8080
```

This is important.

Ingress does not normally send traffic directly to Pods.

It sends traffic to:

Kubernetes Service

Then the Service selects backend Pods.

So:

```
Internet  
  ↓  
Ingress Controller  
  ↓  
Ingress routing  
  ↓  
Employee Service  
  ↓  
Employee Pod
```

| Full request flow

Suppose the user opens:

```
https://api.company.com/employees/100
```

Conceptually:

```
User Browser
  ↓
DNS
  ↓
External IP / Load Balancer
  ↓
Ingress Controller
  ↓
Check Ingress Rules
  ↓
Host = api.company.com
Path = /employees
  ↓
employee-service
  ↓
Service selects backend
  ↓
Employee Pod
  ↓
Spring Boot :8080
```

That is the mental model you should remember.

Multi-service Ingress

You could define several paths:

```
apiVersion: networking.k8s.io/v1
kind: Ingress

metadata:
  name: company-api

spec:
  rules:
    - host: api.company.com

      http:
        paths:
          - path: /employees
            pathType: Prefix
            backend:
              service:
                name: employee-service
                port:
                  number: 8080
```

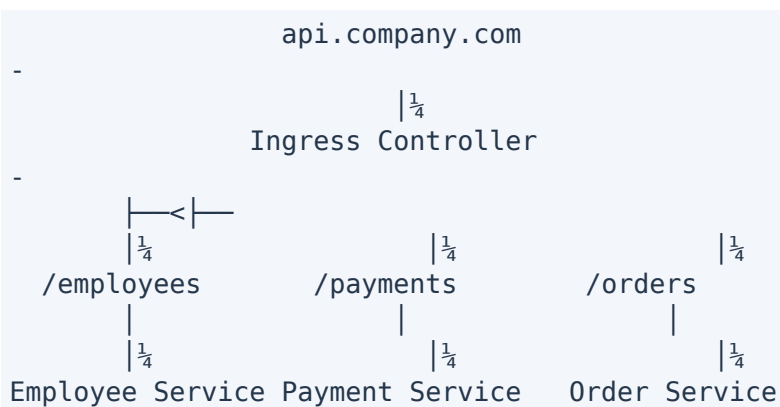
```

- path: /payments
  pathType: Prefix
  backend:
    service:
      name: payment-service
      port:
        number: 8080

- path: /orders
  pathType: Prefix
  backend:
    service:
      name: order-service
      port:
        number: 8080

```

Architecture:



18. Does every Service need Ingress?

No.

This is important.

Imagine:

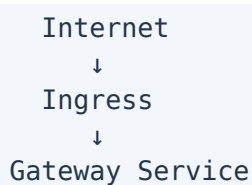
```

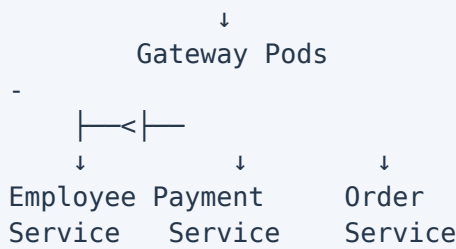
Gateway
Employee
Payment
Order
Notification

```

Only Gateway needs to be reachable from outside.

Then architecture could be:





Employee, Payment and Order remain:

ClusterIP

because only Gateway needs to call them internally.

This is very common in microservice architectures.

This is probably closer to your application architecture

Suppose:



A typical design would be:



So you wouldn't necessarily expose Employee Service to the internet at all.

Ingress vs Service

This distinction is critical.

Service Provides:

stable networking to Pods

Example:

```
employee-service
  ↓
Employee Pods
```

Ingress Provides:

HTTP/HTTPS routing from outside

Example:

```
api.company.com
  ↓
Gateway Service
```

So:

```
Ingress
  ↓
Service
  ↓
Pods
```

21. Does Ingress replace Service?

No.

Typical architecture:

```
Ingress
  ↓
Service
  ↓
Pods
```

Ingress and Service solve different problems.

Ingress decides:

Which Service should receive this HTTP request?

Service decides:

Which Pod should receive traffic for this application?

Ingress vs LoadBalancer Service

Another common interview question.

LoadBalancer Service Conceptually:

```
Internet
  ↓
Load Balancer
  ↓
One Kubernetes Service
  ↓
Pods
```

Good when you need to expose a specific Service.

Ingress

Conceptually:

```
Internet
  ↓
Ingress Controller
  ↓
HTTP routing rules
  ↓
Many Services
```

Better suited for:

```
host routing
path routing
centralized HTTP/HTTPS entry
TLS handling
```

| One LoadBalancer can front an Ingress Controller

A common architecture is:

```
Internet
  ↓
External Load Balancer
  ↓
Ingress Controller
  ↓
Ingress Rules
  ↓
Service A
Service B
Service C
```

So LoadBalancer and Ingress aren't necessarily competing technologies.

They can work together.

24. What about HTTPS?

Production APIs usually use:

HTTPS

not plain HTTP.

That introduces:

TLS TLS provides:

Encryption
Server authentication
Data integrity

So users access:

`https://api.company.com`

rather than:

`http://api.company.com`

TLS termination

One common architecture:

Client

-

| HTTPS
| $\frac{1}{4}$

Ingress Controller

-

| HTTP or HTTPS
| $\frac{1}{4}$

Application Service

The Ingress Controller can terminate TLS.

Meaning:

Encrypted HTTPS connection

↓

Ingress Controller

↓

Decrypt

↓

Route request

This is called:

TLS termination

TLS certificate

To provide HTTPS, the system needs a certificate for something like:

```
api.company.com
```

Kubernetes can reference TLS certificate information through a Secret.

Conceptually:

```
TLS Secret
  ↓
Ingress
  ↓
Ingress Controller
```

Example:

```
spec:
  tls:
    - hosts:
        - api.company.com
      secretName: company-api-tls
```

We✓I cover Secrets separately later.

End-to-end HTTPS is also possible

Depending on architecture, you might have:

```
Client
  | HTTPS
  | ¼
Ingress
  | HTTPS
  | ¼
Application
```

rather than:

```
Client
  | HTTPS
  | ¼
Ingress
  | HTTP
  | ¼
Application
```

The appropriate design depends on security requirements and platform configuration.

Don't worry about those variations yet.

| Ingress resource does not automatically mean public internet access

Important nuance.

Creating:

```
kind: Ingress
```

doesn't magically create:

```
api.company.com
```

You still need infrastructure around it, potentially including:

```
Ingress Controller  
External Load Balancer  
DNS  
Certificates  
Networking/firewalls
```

Think of Ingress as one layer of the solution.

| DNS outside Kubernetes

Previously we discussed internal Kubernetes DNS:

```
employee-service
```

That is cluster-internal service discovery.

External DNS is different.

For:

```
api.company.com
```

external DNS might resolve to:

```
Load Balancer IP
```

Then:

```
api.company.com  
  ↓  
External DNS  
  ↓  
Load Balancer  
  ↓  
Ingress Controller
```

So there are two different DNS concepts.

Internal DNS vs External DNS

Internal Kubernetes DNS Example:

```
employee-service
```

used by:

```
Gateway Pod
```

inside the cluster.

External DNS Example:

```
api.company.com
```

used by:

```
Internet users
```

Outside DNS routes toward your external entry point.

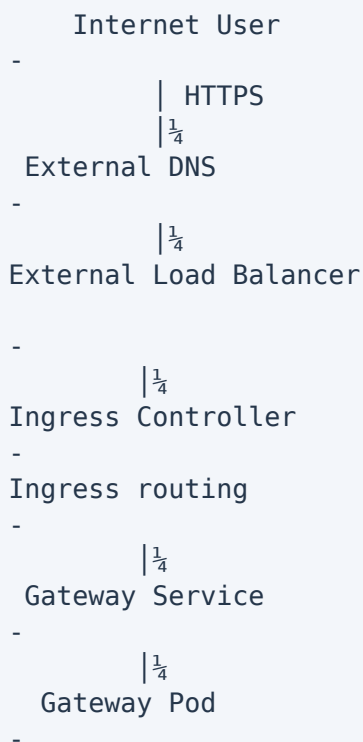
Complete enterprise request path

Let's put everything together.

User calls:

```
https://api.company.com/employees/100
```

Architecture:





You're now seeing how all the previous modules connect.

Let's identify ingress/egress

For:

Internet → Gateway

Gateway sees:

INGRESS

Then:

Gateway → Employee

Gateway:

EGRESS

Employee:

INGRESS

Then:

Employee → Database

Employee:

EGRESS

Database:

INGRESS

Nothing about adding an Ingress resource changes the fundamental direction rule.

Important naming confusion again

We now have:

Ingress traffic

and:

Ingress resource

Example:

Gateway → Employee

is ingress traffic to Employee.

But that does not mean Kubernetes Ingress resource is involved.

This communication may simply be:

```
Gateway Pod
  ↓
ClusterIP Service
  ↓
Employee Pod
```

No external Ingress resource involved.

| Now OpenShift

OpenShift commonly provides another abstraction called:

Route A Route exposes a Service externally, especially for HTTP/HTTPS applications.

Typical architecture:

```
Internet
  ↓
OpenShift Router
  ↓
Route
  ↓
Service
  ↓
Pods
```

This will look very familiar after learning Kubernetes Ingress.

35. What is an OpenShift Route?

Simple definition

An OpenShift Route exposes a Kubernetes/OpenShift Service externally using a host-name and routing configuration.

For example:

```
employee.apps.company.com
  ↓
OpenShift Route
  ↓
employee-service
  ↓
Employee Pods
```

| OpenShift Router

Again, we need to distinguish:

Route

from:

Router

A Route contains routing configuration.

The OpenShift Router implements routing.

Think:

```
Route
=
Instructions

Router
=
Actual component handling traffic
```

Very similar to:

```
Ingress
+
Ingress Controller
```

| Kubernetes vs OpenShift mapping

A useful mental model:

Kubernetes OpenShift

Ingress Route commonly used for external HTTP routing

Ingress Controller OpenShift Router / ingress infrastructure

Service Service

Deployment Deployment

Pod Pod

Don't conclude:

Route completely replaces every Kubernetes Ingress concept.

OpenShift also supports Kubernetes Ingress APIs.

But operationally, OpenShift Routes are very common in OpenShift environments.

| Simple OpenShift Route architecture

Suppose your application has:

```
Service:
gateway-service

port:
8080
```

Create a Route:

```
Route
gateway.apps.example.com
  ↓
gateway-service
  ↓
Gateway Pods
```

Now an external user can call:

```
https://gateway.apps.example.com
```

depending on TLS configuration.

| Example Route YAML

Simplified:

```
apiVersion: route.openshift.io/v1
kind: Route

metadata:
  name: gateway-route

spec:
  host: gateway.apps.example.com

  to:
    kind: Service
    name: gateway-service

  port:
    targetPort: 8080
```

Conceptually:

```
host
gateway.apps.example.com

      ↓

Route

      ↓

gateway-service:8080
```

| oc expose

In OpenShift, you✓I often encounter:

```
oc expose service gateway-service
```

This can create a Route exposing the Service.

Then:

```
oc get routes
```

or:

```
oc get route
```

shows Routes.

This is one area where OpenShift gives developers a convenient platform abstraction on top of Kubernetes.

| Route TLS options

OpenShift Routes commonly support different TLS termination modes.

You✓I hear terms such as:

```
edge
passthrough
reencrypt
```

We don't need to master them yet, but you should know what they broadly mean.

| Edge termination

Conceptually:


```
Client
  | HTTPS
  | ¼
OpenShift Router
-
  | TLS terminates here
  | ¼
Backend
```

Router handles the TLS connection from the client.

| Passthrough

Conceptually:

```
Client
  | HTTPS
  | ¼
Router
-
  | encrypted traffic continues
  | ¼
Application Pod
```

The backend application handles TLS.

Router doesn't terminate that TLS connection in the same way.

| Re-encrypt

Conceptually:

```
Client
  | HTTPS
  | ¼
Router
  | decrypt
-
  | new HTTPS connection
  | ¼
Backend
```

So:

```
HTTPS
Client → Router
```

AND

```
HTTPS
Router → Backend
```

Useful when encryption is required on both network segments.

Don't memorize TLS Route modes yet

At this stage, just retain:

Edge

→ TLS ends at Router

Passthrough

→ TLS passes through to backend

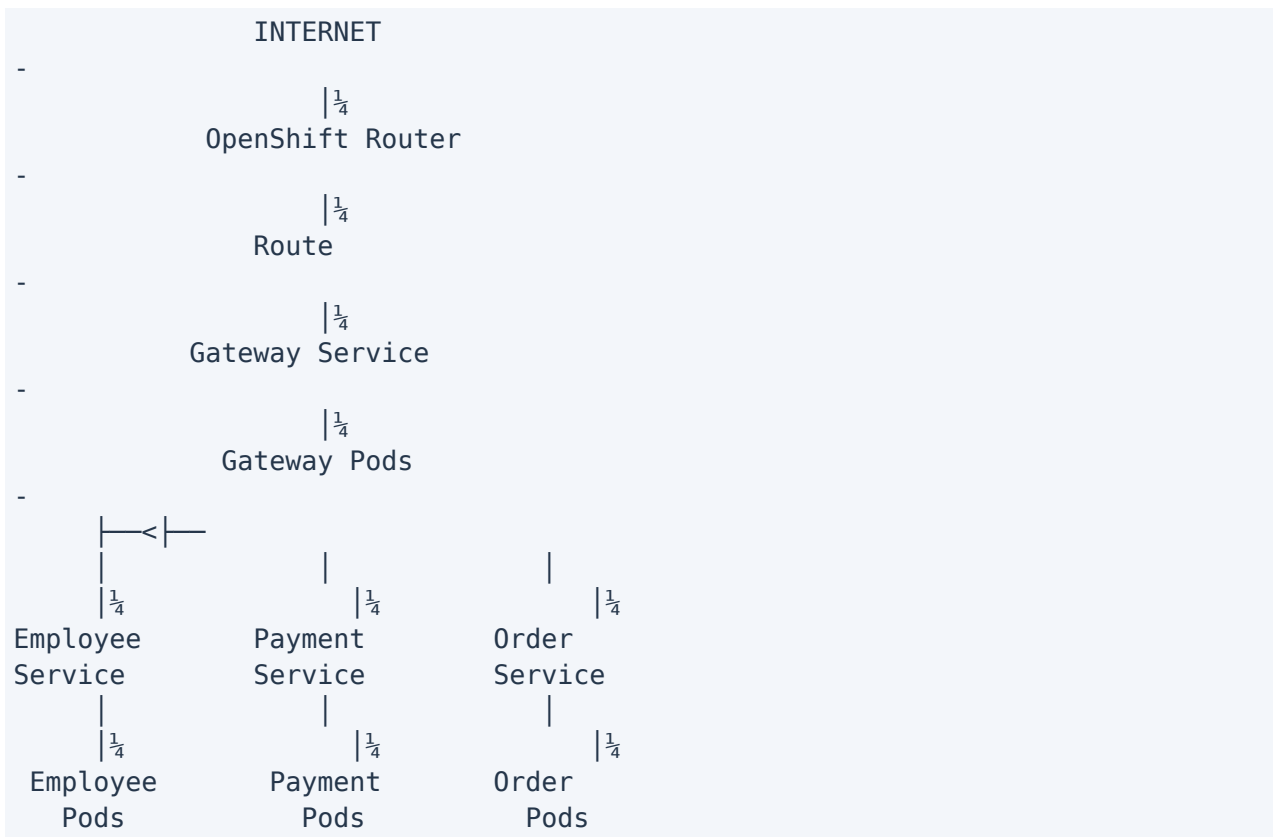
Re-encrypt

→ Router terminates and establishes another TLS connection

We'll revisit them when we do OpenShift security/networking.

Your Gateway architecture in OpenShift

A very realistic design:



Only Gateway is externally exposed.

The internal services remain ClusterIP.

This is a common and clean microservice architecture.

47. Where does NetworkPolicy fit?

Now combine chapter 6 with chapter 7.

External traffic:

```
OpenShift Router
      ↓
Gateway
```

You might allow ingress to Gateway from appropriate router/ingress infrastructure.

Then:

```
Gateway
      ↓
Employee
```

NetworkPolicy could say:

```
Employee ingress:
allow Gateway
```

and if Gateway has egress isolation:

```
Gateway egress:
allow Employee
```

So:

```
Route / Ingress
=
How traffic is routed
```

while:

```
NetworkPolicy
=
Whether workload communication is allowed
```

Again, different responsibilities.

Service vs Route vs NetworkPolicy

This distinction is extremely useful:

Service

```
Which Pods represent this application?
How can they be reached through a stable internal endpoint?
```

Route / Ingress

```
How does HTTP/HTTPS traffic enter and reach a Service?
```

NetworkPolicy

Is this source allowed to communicate with this destination?

Three different networking layers.

Typical request troubleshooting

Suppose:

`https://api.company.com/employees`

doesn't work.

Don't immediately blame the Pod.

Think layer by layer:

```
DNS
↓
External Load Balancer
↓
Ingress/Router
↓
Ingress/Route rule
↓
Service
↓
Service selector
↓
Endpoints
↓
NetworkPolicy
↓
Pod
↓
Application port
↓
Application
```

That gives you a professional troubleshooting methodology.

Example failure: 404 from ingress layer

Suppose:

`api.company.com/employee`

returns:

`404`

Potentially:

```
host rule doesn't match
path rule doesn't match
Ingress configuration wrong
```

The Pod may be perfectly healthy.

Example failure: 503 Service Unavailable

Suppose routing reaches the correct ingress/router, but you receive:

```
503
```

Potential investigation areas:

```
Service has no ready endpoints
Pods aren't ready
selector mismatch
application unavailable
backend port mismatch
```

You need to inspect deeper.

Example failure: DNS doesn't resolve

```
api.company.com
```

cannot be resolved.

Then the request hasn't even reached Kubernetes/OpenShift yet.

Look at:

```
External DNS
```

rather than restarting Pods.

Architecture knowledge tells you where to investigate.

Example failure: Route works, internal Gateway → Employee fails

Suppose:

```
Internet
↓
Route '
↓
Gateway '
```

but Gateway gets an error calling:

```
employee-service
```

Now your external Route is probably irrelevant.

Investigate:

```
Employee Service
DNS
selectors/endpoints
NetworkPolicy
Employee Pod
```

This distinction saves significant debugging time.

| Common interview question

What is the difference between Ingress and Ingress Controller? Good answer:

An Ingress is a Kubernetes resource containing HTTP/HTTPS routing rules, while an Ingress Controller is the actual software component that watches those rules and implements the traffic routing.

| Interview question: Service vs Ingress

A Service provides stable network access to Pods, usually inside the cluster, while Ingress provides HTTP/HTTPS routing from an external entry point to Services.

| Interview question: Ingress vs NetworkPolicy

Ingress routes HTTP/HTTPS requests to Services. NetworkPolicy controls which network traffic selected Pods are allowed to send or receive. Routing and authorization/isolation are different concerns.

57. Interview question: What is OpenShift Route?

An OpenShift Route is an OpenShift resource used to expose a Service externally through the OpenShift routing layer using host-based HTTP/HTTPS routing and optional TLS termination.

| Check your understanding

Suppose:

```
Internet User
  ↓
api.company.com
  ↓
Ingress
```

```

↓
Gateway Service
↓
Gateway Pod

```

Q1 Does Ingress normally route directly to the Deployment?

No.

It routes to a Service.

Q2 Does the Service route to a Deployment?

Not directly.

It selects:

Pods using labels/selectors.

Q3 What actually processes Kubernetes Ingress rules?

Ingress Controller.

Q4 What is the OpenShift external-routing resource commonly used?

Route.

Q5 Should Employee Service necessarily have its own external Route if Gateway is the public API?

No.

It can remain internal:

```
ClusterIP
```

and only Gateway may be externally exposed.

Put Modules 1-7 together

We have built a substantial Kubernetes architecture now:

```

INTERNET
-
  | 1/4
Ingress / OpenShift Route
-
  | 1/4
  Gateway Service
-
  | 1/4
  Gateway Pods
-
  | egress
  | 1/4

```

```
Employee Service
-
  | 1/4
Employee Pods
-
  | 1/4
Database
```

The Employee Pods themselves came from:

```
Employee Deployment
  ↓
Employee ReplicaSet
  ↓
Employee Pods
```

and:

```
Scheduler
  ↓
selected Worker Nodes
```

while:

```
kubelet
  ↓
container runtime
```

actually started the containers.

We're now connecting nearly all of Kubernetes' basic building blocks.

chapter 7 Summary

Remember this hierarchy:

```
External User
  ↓
Ingress / OpenShift Route
  ↓
Service
  ↓
Pods
```

And these responsibilities:

```
Ingress / Route
=
External HTTP/HTTPS routing

Service
=
```


Stable network identity + backend Pod selection

NetworkPolicy

=

Control permitted Pod communication

Deployment

=

Manage application versions and replicas

The biggest architectural takeaway is:

Ingress/Route gets traffic into the application boundary; Service finds the correct application Pods; NetworkPolicy controls whether workload communication is permitted.

Next — chapter 8: ConfigMaps & Secrets So far our Spring Boot container might need:

PROPERTIES

spring.datasource.url=... spring.datasource.username=... spring.datasource.password=...
kafka.bootstrap.servers=... feature.enabled=true

logging.level.root=INFO

The next problem is:

Should all of this be baked into the container image?

No.

We✓I learn:

ConfigMap

Secret

Environment variables

Volume-mounted configuration

ConfigMap vs Secret

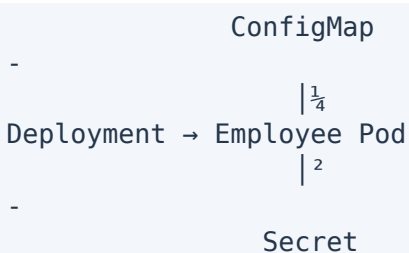
How Pods consume them

What happens when configuration changes

Spring Boot examples

Why Kubernetes Secrets are not magically encrypted just because they're called
↪ Secrets

And we✓I build:



This is another major concept for running real Java applications in Kubernetes.

Configuration, Storage, and Stateful Systems

Chapter 10 — ConfigMaps and Secrets

CHAPTER FOCUS

Understand **ConfigMaps and Secrets** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

Now we move from running applications to configuring applications correctly.

Suppose your Spring Boot app needs:

PROPERTIES

```
spring.datasource.url=jdbc:postgresql://db:5432/employee spring.datasource.username=employee
spring.datasource.password=super-secret logging.level.root=INFO feature.new-ui=true
```

The question is:

Should all of these values be hardcoded inside the container image?

Usually, no.

Why external configuration exists

Imagine you build one image:

```
employee-service:1.0
```

You want to run the same image in:

```
DEV
TEST
UAT
PROD
```

But each environment has different configuration.

For example:

```
DEV DB:
dev-db.internal

PROD DB:
prod-db.internal
```

If configuration is baked into the image, you'd need:

```
employee-service-dev:1.0
employee-service-test:1.0
employee-service-prod:1.0
```

That is not ideal.

A better model is:

```
Same Application Image
      +
Environment-specific configuration
```

Kubernetes gives us two important objects

```
ConfigMap
Secret
```

A simple mental model:

```
ConfigMap
=
non-sensitive configuration

Secret
=
sensitive configuration
```

For example:

```
ConfigMap:
DB_HOST
LOG_LEVEL
FEATURE_FLAG
KAFKA_HOST
```

and:

```
Secret:
DB_PASSWORD
API_TOKEN
CLIENT_SECRET
CERTIFICATE
```

3. What is a ConfigMap?

One-sentence definition

A ConfigMap stores non-confidential configuration data separately from the container image.

Example:

```
apiVersion: v1
kind: ConfigMap

metadata:
  name: employee-config

data:
  DB_HOST: employee-db.internal
  LOG_LEVEL: INFO
  FEATURE_ENABLED: "true"
```

This object exists independently of the Pod.

Architecture

```
ConfigMap
employee-config
-
  | ¼
Employee Pod
-
  | ¼
Employee Container
-
  | ¼
Spring Boot Application
```

The application image stays unchanged.

Only configuration changes.

Why ConfigMap is useful

Imagine:

```
employee-service:1.0
```

running in two environments.

DEV:

```
DB_HOST=dev-db
LOG_LEVEL=DEBUG
```

PROD:

```
DB_HOST=prod-db
LOG_LEVEL=INFO
```

Same image:

```
employee-service:1.0
```

different ConfigMaps.

That is clean separation.

6. How can a Pod consume ConfigMap values?

There are two common methods:

```
Environment variables
```

or:

```
Mounted files / volumes
```

Let's start with environment variables.

| Single ConfigMap value as environment variable

Suppose ConfigMap:

```
apiVersion: v1
kind: ConfigMap

metadata:
  name: employee-config

data:
  LOG_LEVEL: INFO
```

Deployment:

```
env:
- name: LOG_LEVEL
  valueFrom:
    configMapKeyRef:
      name: employee-config
      key: LOG_LEVEL
```

Conceptually:

```
ConfigMap
LOG_LEVEL=INFO
  ↓
Pod
  ↓
Environment Variable
LOG_LEVEL=INFO
  ↓
Application
```

| Spring Boot example

Your Java app could read:

```
System.getenv("LOG_LEVEL")
```

or Spring Boot can map environment variables into configuration.

For example:

```
SPRING_DATASOURCE_URL
```

can be used as Spring configuration.

So Kubernetes doesn't need to understand Spring Boot.

It simply injects environment variables.

| Inject all ConfigMap values

Instead of mapping each key individually, you can do:

```
envFrom:  
  - configMapRef:  
      name: employee-config
```

Suppose ConfigMap contains:

```
data:  
  DB_HOST: db.internal  
  DB_PORT: "5432"  
  LOG_LEVEL: INFO
```

Container receives:

```
DB_HOST=db.internal  
DB_PORT=5432  
LOG_LEVEL=INFO
```

Very convenient.

| But don't put passwords into ConfigMap

You could technically write:

```
data:  
  DB_PASSWORD: mypassword
```

inside a ConfigMap.

But that's poor practice.

Sensitive values should be treated separately.

That is why Kubernetes has:

Secret

11. What is a Secret?

One-sentence definition

A Kubernetes Secret stores sensitive configuration data separately from application manifests and container images.

Examples:

```
Database passwords
API keys
Tokens
TLS certificates
Private keys
Credentials
```

Example:

```
apiVersion: v1
kind: Secret

metadata:
  name: employee-secret

type: Opaque

data:
  DB_USERNAME: ...
  DB_PASSWORD: ...
```

Important misconception: Secret does not automatically mean encrypted

This is extremely important.

People often assume:

“It’s a Kubernetes Secret, therefore nobody can read it.”

Not necessarily.

Kubernetes Secret values are commonly represented in manifests using base64 encoding.

Base64 is:

Encoding

not:

Encryption

Anyone who can read and decode the value can recover the original data.

So:

Secret is a Kubernetes resource designed for sensitive data, but secure handling also depends on RBAC, etcd encryption, external secret management, access controls and operational practices.

| Base64 example

Suppose password is:

```
hello123
```

Base64 might look something like:

```
aGVsbG8xMjM=
```

That does not make it secret.

It is trivially reversible.

So don't think:

```
base64 = encryption
```

It is not.

| stringData

For easier authoring, Kubernetes Secrets can also use:

```
stringData:
```

Example:

```
apiVersion: v1
kind: Secret

metadata:
  name: employee-secret

type: Opaque

stringData:
  DB_USERNAME: employee_user
  DB_PASSWORD: super-secret
```

Kubernetes handles conversion internally.

But be careful:

If you store this YAML in Git:

```
DB_PASSWORD: super-secret
```

your password is now sitting in source control.

That is usually not acceptable.

| Secret as environment variable

Example:

```
env:
  - name: DB_PASSWORD
    valueFrom:
      secretKeyRef:
        name: employee-secret
        key: DB_PASSWORD
```

Flow:

```
Secret
DB_PASSWORD
  ↓
Pod
  ↓
Environment Variable
DB_PASSWORD
  ↓
Spring Boot
```

| Complete application configuration

You might have:

```
ConfigMap
employee-config

DB_HOST
DB_PORT
LOG_LEVEL
FEATURE_FLAG
```

and:

```
Secret
employee-secret

DB_USERNAME
DB_PASSWORD
```

Both feed the same Pod:

```
ConfigMap
-
  | 1/4
Employee Pod
  | 2
-
  Secret
```

| Full Deployment example

```
apiVersion: apps/v1
kind: Deployment

metadata:
  name: employee-service

spec:
  replicas: 3

  selector:
    matchLabels:
      app: employee

  template:
    metadata:
      labels:
        app: employee

    spec:
      containers:
        - name: employee
          image: company/employee-service:1.0

          envFrom:
            - configMapRef:
                name: employee-config

            - secretRef:
                name: employee-secret
```

Now every Pod created by this Deployment receives configuration from both.

18. What happens when replicas = 3?

Suppose:

```
Deployment
replicas = 3
```

Then:

```
Pod A
Pod B
Pod C
```

Each Pod gets the same ConfigMap/Secret values, unless you've deliberately designed otherwise.

Conceptually:

```
employee-config
```

```
-
```

```
├── Pod A
├── Pod B
└── Pod C
```

```
employee-secret
```

```
-
```

```
├── Pod A
├── Pod B
└── Pod C
```

ConfigMap as mounted file

Not every application wants environment variables.

Suppose your app expects:

```
/config/application.properties
```

A ConfigMap can be mounted as files.

For example:

```
apiVersion: v1
kind: ConfigMap

metadata:
  name: employee-properties

data:
  application.properties: |
    server.port=8080
    logging.level.root=INFO
    feature.enabled=true
```

Then mount it into the Pod.

Conceptual architecture

```

ConfigMap
application.properties
-
  | ¼
Mounted Volume
-
  | ¼
/config/application.properties
-
  | ¼
Spring Boot

```

This is useful when applications naturally consume configuration files.

Example volume mount

Simplified:

```

spec:
  containers:
    - name: employee
      image: employee:1.0

      volumeMounts:
        - name: config
          mountPath: /config

  volumes:
    - name: config
      configMap:
        name: employee-properties

```

The Pod volume connects the ConfigMap to the container filesystem.

Secrets can also be mounted as files

This is common with things such as:

```

TLS certificate
Private key
Token

Credential file

```

Example concept:

```
Secret
tls.crt
tls.key
-
  |1/4
Pod Volume
-
  |1/4
/etc/certs/
```

The application can then read the files.

| Environment variable vs mounted file

Both are useful.

Environment variables Good for:

```
DB_HOST
LOG_LEVEL
PORT
FEATURE_FLAG
```

Simple configuration values.

Mounted files Good for:

```
application.properties
JSON configuration
Certificates
Private keys
Configuration files
```

24. Important question: what happens when ConfigMap changes?

Suppose you have:

```
LOG_LEVEL=INFO
```

and you update it to:

```
LOG_LEVEL=DEBUG
```

Do running Pods automatically restart?

Generally:

No.

This distinction is important.

| ConfigMap through environment variables

If a Pod received the ConfigMap as environment variables at startup:

```
Pod started
  ↓
LOG_LEVEL=INFO
```

then ConfigMap changes to:

```
LOG_LEVEL=DEBUG
```

the existing process generally continues with:

```
INFO
```

because environment variables were established when the container started.

To pick up the new value, you'd typically restart/roll out the Pods.

| Mounted ConfigMap updates behave differently

Mounted ConfigMap files can be updated in the mounted volume after the ConfigMap changes, subject to Kubernetes propagation behavior.

But:

That still doesn't mean your application automatically reloads the file.

Your Spring Boot application may only read the file at startup.

So:

```
ConfigMap changes
  ↓
mounted file may update
  ↓
application may or may not reload
```

Application behavior matters.

| Common production pattern

Teams often change configuration and trigger a Deployment rollout.

Conceptually:

```
Config changes
  ↓
New Pods created
  ↓
New Pods read new configuration
```

↓
Old Pods removed

This gives predictable behavior.

Immutable container image philosophy

A strong container practice is:

Build once
Deploy many

Example:

employee-service:1.5.2

Same exact image goes through:

DEV
↓
QA
↓
UAT
↓
PROD

Only configuration changes.

This improves confidence because:

The artifact tested is the artifact deployed.

Bad approach

Imagine:

DEV image
employee-service-dev:1.0

TEST image
employee-service-test:1.0

PROD image
employee-service-prod:1.0

Now you haven't truly tested the exact same artifact.

External configuration avoids much of that problem.

Spring Boot example

Your app may have:

PROPERTIES

```
spring.datasource.url=DB_URLspring.datasource.username={DB_USERNAME}
spring.datasource.password=DB_PASSWORDlogging.level.root={LOG_LEVEL:INFO}
```

Then Kubernetes provides:

```
DB_URL
DB_USERNAME
DB_PASSWORD
LOG_LEVEL
```

No Kubernetes-specific Java code is necessary.

That is a clean design.

ConfigMap vs Secret table

ConfigMap Secret

Non-sensitive configuration Sensitive configuration

Feature flags Passwords

Hostnames API keys

Log levels Tokens

URLs Certificates/keys

Can inject as env/file Can inject as env/file

The key distinction is sensitivity.

32. Where are these objects stored?

Remember chapter 2:

```
API Server
  ↓
etcd
```

ConfigMaps and Secrets are Kubernetes API objects whose cluster state is persisted through Kubernetes' storage architecture.

This is why etcd security matters.

Sensitive Secrets need strong cluster controls.

33. Who should be allowed to read Secrets?

Not everyone.

This is where RBAC becomes critical.

For example:

```
Developer A  
may read ConfigMaps '
```

```
Developer A  
may read production Secrets x
```

or perhaps:

```
Application ServiceAccount  
can access required Secret
```

```
Other applications  
cannot
```

We✓I cover RBAC deeply later.

| Secret access is a security boundary

Suppose someone can run:

```
kubectl get secret
```

and has permission to retrieve Secret contents.

Then Kubernetes calling it a Secret does not protect you from that authorized reader.

Security therefore depends on:

```
RBAC  
Least privilege  
etcd encryption at rest  
Audit controls  
External secret management  
Git hygiene  
Workload identity
```

| Don't commit plain Secrets to Git

Bad:

```
stringData:  
  DB_PASSWORD: ProdPassword123
```

committed to:

```
Git repository
```

Even if deleted later, Git history may preserve it.

This is why production environments commonly use dedicated secret-management approaches.

| Where Vault fits

Since enterprise environments often use HashiCorp Vault, conceptually:

```
Vault
  ↓
secret integration
  ↓
Application / Pod
```

instead of manually keeping sensitive credentials in Git.

There are multiple integration patterns, for example:

```
External Secrets-style synchronization
CSI secret mounts
Vault agents/injectors
Application fetching from Vault
Platform-native integrations
```

We'll discuss these later after Kubernetes Secrets are fully understood.

| ConfigMap doesn't know who consumes it

Suppose:

```
employee-config
```

exists.

It doesn't automatically attach itself to Employee Pods.

The Deployment must reference it:

```
envFrom:
- configMapRef:
  name: employee-config
```

Same with Secret.

So:

```
ConfigMap exists
```

does not mean:

```
Pod automatically receives it
```

38. What if ConfigMap doesn't exist?

Suppose Deployment references:

```
configMapRef:  
  name: employee-config
```

but there is no such ConfigMap.

Depending on how it's referenced/configured, the Pod may fail to start properly.

You may see errors/events around missing configuration.

This is where:

```
kubectl describe pod ...
```

becomes useful.

39. What if Secret doesn't exist?

Same idea.

Pod requires:

```
employee-secret
```

but it is missing.

The container may not start as expected.

Look at:

```
kubectl describe pod <pod>
```

and inspect Events.

Architecture helps:

```
Deployment ' '  
ReplicaSet '  
Pod created '  
Config dependency ×  
Container start blocked/problematic
```

Troubleshooting configuration

Suppose app crashes with:

```
Could not resolve placeholder 'DB_URL'
```

Think:

```
Application expected environment variable
```

↓

```
Was it provided?
```

Check:

```
Deployment env/envFrom
```

```
ConfigMap
```

```
Secret
```

```
correct key names
```

```
correct namespace
```

| Key-name mismatch

ConfigMap:

```
data:
```

```
  DATABASE_URL: jdbc:...
```

Deployment expects:

```
key: DB_URL
```

Problem:

```
DATABASE_URL
```

```
!=
```

```
DB_URL
```

Simple configuration mistakes often cause application startup failures.

| Namespace matters

Suppose:

```
ConfigMap employee-config
```

```
Namespace A
```

but:

```
Employee Pod
```

```
Namespace B
```

A Pod can't simply reference a ConfigMap from another namespace using a normal local reference.

Kubernetes configuration objects are generally namespace-scoped.

So:

```
Pod
ConfigMap
Secret
```

usually need to exist in the same namespace for direct references.

This is important in OpenShift

Remember:

```
OpenShift Project
"H
Kubernetes Namespace concept
```

You may have:

```
dev project
test project
prod project
```

Each project can contain its own:

```
ConfigMaps
Secrets
Deployments
Services
```

So the same application can have:

```
employee-config in DEV
employee-config in PROD
```

with different values.

Example DEV vs PROD

DEV:

```
data:
  DB_HOST: dev-db
  LOG_LEVEL: DEBUG
```

PROD:

```
data:
  DB_HOST: prod-db
  LOG_LEVEL: INFO
```

Same Deployment conceptually uses:

```
employee-config
```

in each namespace.

Very clean.

| Kubernetes object names can repeat across namespaces

You can have:

```
Namespace dev:  
employee-config
```

```
Namespace prod:  
employee-config
```

No conflict.

Because their full identity includes namespace context.

Same applies to:

```
Services  
Deployments  
Secrets  
Pods
```

This is a major reason namespaces are useful.

| Secret types

You✓I eventually see different Secret types.

For example:

```
Opaque  
kubernetes.io/tls  
kubernetes.io/dockerconfigjson
```

You don't need to memorize these now.

Opaque General-purpose secret data.

TLS Certificate/private key material.

Docker config Registry authentication credentials.

We✓I encounter them naturally.

| ImagePullSecret example

Suppose your image is in a private registry:

```
registry.company.com/employee-service:1.0
```

The worker runtime needs authentication to pull it.

Kubernetes can use registry credentials stored in a Secret.

Conceptually:

```
Private Registry
  |2
  | credentials
ImagePullSecret
  |2
-
  Pod
```

This is different from an application DB password, but both use Secret resources.

| Application Secret vs Image Pull Secret

Application Secret Used inside the running application:

```
DB_PASSWORD
API_TOKEN
```

Image Pull Secret Used by Kubernetes/runtime to:

```
authenticate to container registry
```

Different purpose.

| Configuration should not be confused with persistent data

ConfigMap is not a database.

Don't store:

```
Employee records
Orders
Payment transactions
```

in ConfigMaps.

ConfigMaps are for configuration.

Persistent application data belongs in databases or persistent storage systems.

Size considerations

ConfigMaps and Secrets are intended for relatively small configuration data, not huge files or large datasets.

Think:

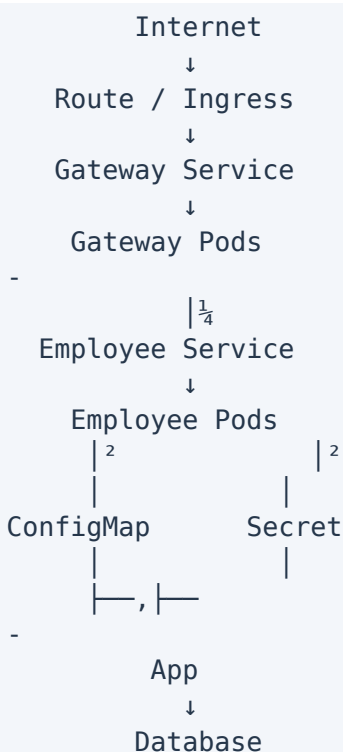
```
Configuration '
Small certificates '
Small config files '

500 MB dataset x
Database dump x
Video file x
```

Use appropriate storage mechanisms for larger data.

Full application architecture now

We've added configuration to our previous model:



This is becoming a real enterprise architecture.

52. Who manages what?

Let's separate responsibilities again.

Deployment

Which image?
How many replicas?
How to roll out?

ConfigMap

Non-sensitive configuration

Secret

Sensitive configuration

Service

Stable network access

Ingress/Route

External HTTP routing

NetworkPolicy

Allowed communication

Kubernetes is made of many small resources, each with a focused job.

Common interview question

What is the difference between ConfigMap and Secret? Good answer:

Both provide externalized configuration to Pods. ConfigMaps are intended for non-sensitive configuration, while Secrets are intended for sensitive values such as credentials and tokens. Secrets require additional security controls because base64 representation itself is not encryption.

54. Interview question: Why not put configuration inside Docker image?

Because environment-specific configuration would require rebuilding or maintaining different images. Externalizing configuration allows the same immutable image to move through environments while configuration changes independently.

Excellent answer.

Interview question: If ConfigMap changes, does the Pod restart

automatically? Good answer:

No, updating a ConfigMap does not inherently restart dependent Pods. Environment-variable values are normally fixed for the lifetime of the container, while mounted

ConfigMap files can be updated over time, but application reload behavior depends on the application. Teams often trigger a rollout when configuration changes.

56. Interview question: Are Kubernetes Secrets encrypted?

Careful answer:

Kubernetes Secrets are API resources intended for sensitive information, but base64 representation is not encryption. Secure deployment requires appropriate RBAC and cluster configuration such as encryption at rest for Secret data, and many organizations integrate external secret-management systems.

Check your understanding

Q1 Where should this go?

```
LOG_LEVEL=DEBUG
```

Usually:

ConfigMap

Q2 Where should this go?

```
DB_PASSWORD=...
```

Secret

Q3 Should Secret YAML containing a production password be committed directly to Git?

Generally:

No.

Q4 Does base64 protect a password cryptographically?

No.

It is encoding, not encryption.

Q5 Can the same image be used in DEV and PROD with different ConfigMaps?

Yes.

That is one of the key benefits.

Mini exercise

Suppose your application needs:

```
DB_HOST
DB_USERNAME
```

```
DB_PASSWORD
LOG_LEVEL
KAFKA_BOOTSTRAP_SERVERS
PAYMENT_API_TOKEN
FEATURE_ENABLED
```

A sensible split:

ConfigMap

```
DB_HOST
LOG_LEVEL
KAFKA_BOOTSTRAP_SERVERS
FEATURE_ENABLED
```

and:

Secret

```
DB_USERNAME
DB_PASSWORD
PAYMENT_API_TOKEN
```

You could argue about whether usernames need secrecy, but credentials/tokens/passwords should clearly be treated as sensitive.

Common mistake

Suppose you have:

```
ConfigMap:
DB_HOST=prod-db
```

but application logs show:

```
DB_HOST=null
```

Don't immediately blame the ConfigMap.

Trace:

```
ConfigMap exists?
  ↓
Correct namespace?
  ↓
Deployment references ConfigMap?
  ↓
Correct key?
  ↓
New Pods created after configuration?
  ↓
Application reads correct environment/property?
```

Always troubleshoot through the dependency chain.

chapter 8 Summary

The core architecture is:

```

Container Image
  +
ConfigMap
  +
Secret
  ↓
Pod
  
```

Remember:

```

ConfigMap
=
non-sensitive configuration

Secret
=
sensitive configuration
  
```

and the most important design principle:

Build the application image once; inject environment-specific configuration at deployment time.

Our complete application model is now:

```

Deployment
  ↓
ReplicaSet
  ↓
Pod
├─ Container Image
├─ ConfigMap configuration
└─ Secret configuration
  
```

Next — chapter 9: Kubernetes Storage — Volumes, PV, PVC & StorageClass Now we'll solve a completely different problem.

Imagine:

```

Database Pod
  ↓
writes 100 GB data
  
```

Pod dies.

ReplicaSet/StatefulSet creates a new Pod.

Question:

Where did the 100 GB of data go?

That leads to:

```
Volume
PersistentVolume (PV)
PersistentVolumeClaim (PVC)
StorageClass
Dynamic Provisioning
```

We build this relationship:

```
Pod
↓
PVC
↓
PV
↓
Actual Storage
```

and then distinguish stateless Spring Boot services from stateful workloads such as databases.

Chapter 11 — Volumes, Persistent Volumes, Claims, and StorageClasses

CHAPTER FOCUS

Understand **Volumes, Persistent Volumes, Claims, and StorageClasses** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

Now we move into stateful workloads.

So far, most examples were stateless Spring Boot services:

```
Gateway
Employee Service
Payment Service
```

If one of these Pods disappears, Kubernetes can simply create a replacement.

But databases are different.

Suppose:

```
Database Pod
↓
stores 100 GB of data
```

If the Pod disappears, we cannot allow the data to disappear with it.

That is the problem Kubernetes storage solves.

Start with the basic problem

Imagine this Pod:

```
MySQL Pod
↓
Container filesystem
↓
100 GB customer data
```

Now the Pod is deleted:

```
MySQL Pod ×
```

Kubernetes may create another Pod:

New MySQL Pod '

But remember:

Pods and containers are disposable.

The new Pod is a new workload instance.

If your important data existed only in the old container's writable filesystem, you cannot treat that as durable storage.

So we need:

Application lifecycle

"`

Data lifecycle

The Pod can die.

The data must survive.

| Stateless vs Stateful

Let's distinguish these first.

Stateless application Example:

Employee REST API

Request:

GET /employees/100

The actual employee data lives somewhere else:

Employee Pod

↓

Database

If Employee Pod dies:

Old Pod ×

New Pod '

No important business data is lost.

That is stateless.

Stateful application Examples:

PostgreSQL

MySQL

Kafka


```
Elasticsearch
MongoDB
```

They maintain persistent state.

Example:

```
Database Pod
  ↓
/var/lib/database
  ↓
Customer data
```

If that storage disappears, you have a serious problem.

That is stateful.

First concept: Volume

A Volume gives containers storage that can exist outside their individual writable container layer.

Conceptually:

```
Pod
-
| Container
|   |
|   | 1/4
|   | /data
-
| Volume
```

The container mounts the volume somewhere inside its filesystem.

For example:

```
/data
```

But not every Kubernetes volume is automatically persistent forever.

Some volume types are still tied to the Pod's lifetime.

So we need to separate:

```
temporary storage
```

from:

```
persistent storage
```

Example: emptyDir

A common temporary volume is:

```
emptyDir
```

Conceptually:

```
Pod created
  ↓
emptyDir created
  ↓
containers use it
  ↓
Pod removed
  ↓
emptyDir removed
```

It is useful for:

```
temporary files
caches
scratch space
sharing files between containers in one Pod
```

But not:

```
critical database data
```

because it follows the Pod's lifetime.

Example multi-container Pod

Suppose:

```
Pod
├─ Java Application
└─ Log Processor
```

Both need access to:

```
/shared
```

You could attach:

```
emptyDir
```

Architecture:

```
Java Container
-
  | ¼
  Shared Volume
```

```
|2
-
Log Container
```

That is a valid use.

But if the Pod is deleted:

```
shared temporary data disappears
```

So this isn't yet the persistent-storage solution.

| Persistent storage problem

We want something like:

```
Database Pod A
-
|1/4
Persistent Storage
-
|1/4
100 GB data
```

Database Pod A dies:

```
Database Pod A x
```

New Pod:

```
Database Pod B
-
|1/4

Same persistent storage
-
|1/4
100 GB data still exists '
```

This is the fundamental requirement.

| Kubernetes separates storage into two perspectives

Kubernetes uses two especially important abstractions:

```
PersistentVolume
PersistentVolumeClaim
```

abbreviated:

PV
PVC

The core relationship:

Pod
↓
PVC
↓
PV
↓
Actual Storage

This diagram is worth memorizing.

8. What is a PersistentVolume?

One-sentence definition

A PersistentVolume, or PV, is a Kubernetes storage resource representing storage available to the cluster. Think:

PV
=
Actual usable storage resource

Examples underneath it could involve:

cloud block storage
SAN
NFS
CSI-provisioned storage
enterprise storage systems

The application doesn't necessarily need to know the storage vendor.

Kubernetes abstracts it.

9. What is a PersistentVolumeClaim?

One-sentence definition

A PersistentVolumeClaim, or PVC, is a request for persistent storage made by a workload/user.

Think:

PV
=
storage available

PVC
=

```
storage requested
```

A useful analogy:

```
PV
=
apartment

PVC
=
tenant saying:
"I need a 2-bedroom apartment."
```

Kubernetes tries to match the request to suitable storage.

10. Why not let the Pod directly use a PV?

You might ask:

Why do we need PVC in the middle?

Why not:

```
Pod
↓
PV
```

Because Kubernetes wants to separate:

```
Application requirement
```

from:

```
Storage implementation
```

Application developer says:

I need 100 GiB storage with these characteristics.

Infrastructure/platform handles:

Which actual storage system provides that?

So:

```
Developer
  ↓
PVC

Platform
  ↓
PV / storage backend
```

This separation is powerful.

| Java analogy

Think of JDBC.

Your Java code says:

```
I need a DataSource.
```

You don't want your business code managing:

```
disk controller
storage array
block device
filesystem allocation
```

Similarly:

```
Application:
"I need 20 GiB persistent storage."
```

Kubernetes/platform:

```
"I'll provide suitable storage."
```

| Basic PVC example

A PVC might look like:

```
apiVersion: v1
kind: PersistentVolumeClaim

metadata:
  name: employee-data

spec:
  accessModes:
    - ReadWriteOnce

  resources:
    requests:
      storage: 10Gi
```

Meaning:

I need 10 GiB of persistent storage, with this access mode.

| Pod consuming PVC

The Pod can reference:

```
volumes:
  - name: data
    persistentVolumeClaim:
      claimName: employee-data
```

Then mount it:

```
volumeMounts:
  - name: data
    mountPath: /data
```

Architecture:

```
Container
|
|_ /data
|_ -
|   | 1/4
|   Volume
|   -
|       | 1/4
|       PVC
|       employee-data
|       -
|           | 1/4
|           PV
|           -
|               | 1/4
|               Physical / cloud storage
```

| Complete simplified Pod example

```
apiVersion: v1
kind: Pod

metadata:
  name: database-pod

spec:
  containers:
    - name: database
      image: postgres

      volumeMounts:
        - name: db-storage
          mountPath: /var/lib/postgresql/data

  volumes:
    - name: db-storage
      persistentVolumeClaim:
        claimName: database-pvc
```

Notice:

```
Container
↓
mountPath

Pod volume
↓
PVC
```

The Pod doesn't directly specify:

```
AWS EBS disk ID
SAN LUN
specific physical disk
```

That is intentionally abstracted.

15. What happens when Pod dies?

Suppose:

```
Pod A
↓
PVC
↓
PV
↓
Data
```

Pod A disappears.

The PVC and underlying persistent storage can remain.

Then a replacement Pod can mount the claim:

```
Pod B
↓
same PVC
↓
persistent storage
↓
same data
```

That is the key difference from storing important data only inside the container filesystem.

Kubernetes PV/PVC resources are designed to have lifecycles separate from individual Pods.

16. But where does PV come from?

There are two broad possibilities:

Static provisioning

and:

Dynamic provisioning

Let's understand both.

| Static provisioning

In static provisioning, an administrator/platform team prepares PVs ahead of time.

Example:

Admin creates:

```
PV-A = 10 GiB
PV-B = 50 GiB
PV-C = 100 GiB
```

Then users create PVCs.

Example:

PVC asks for 50 GiB

Kubernetes finds a suitable PV:

```
PVC
↓
PV-B
```

This works, but managing large environments manually becomes tedious.

| Dynamic provisioning

Modern Kubernetes environments commonly use dynamic provisioning.

Instead of creating PVs manually:

```
User creates PVC
      ↓
Kubernetes/storage system
      ↓
new storage automatically provisioned
      ↓
PV created/bound
```

The resource that describes how storage should be provisioned is called:

StorageClass

19. What is StorageClass?

One-sentence definition

A StorageClass describes a class of storage and the provisioner/configuration Kubernetes should use when dynamically provisioning persistent storage.

Conceptually:

```
PVC
↓
StorageClass
↓
Storage provisioner
↓
Actual storage created
↓
PV
```

This is a central production concept.

| Example StorageClasses

A cluster might provide:

```
fast-ssd
standard
high-iops
encrypted
archive
```

The names are platform-specific.

Then the developer can request:

```
storageClassName: fast-ssd
```

instead of asking:

Give me disk serial number ABC123.

That is infrastructure abstraction again.

| Example PVC with StorageClass

```
apiVersion: v1
kind: PersistentVolumeClaim

metadata:
  name: database-pvc

spec:
```

```
storageClassName: fast-ssd
```

```
accessModes:
```

```
- ReadWriteOnce
```

```
resources:
```

```
requests:
```

```
storage: 100Gi
```

Meaning:

I need:

100 GiB

from:

fast-ssd

with:

ReadWriteOnce

The provisioner may dynamically create suitable storage.

Complete dynamic flow

This is the important diagram:

Developer

-

| creates PVC
| $\frac{1}{4}$

PVC

100 GiB

fast-ssd

-

| $\frac{1}{4}$

StorageClass

-

| $\frac{1}{4}$

CSI / Storage Provisioner

-

| $\frac{1}{4}$

Actual Storage

-

| $\frac{1}{4}$

PersistentVolume

-

| $\frac{1}{4}$

PVC bound

-

| $\frac{1}{4}$

Pod mounts storage

23. What is CSI?

You✓I hear another acronym:

CSI — Container Storage Interface Think of it similarly to CNI.

We learned:

```
CNI
=
networking integration standard
```

Now:

```
CSI
=
storage integration standard
```

CSI allows storage vendors/providers to integrate their storage systems with Kubernetes through a standardized interface.

| CNI vs CSI

Very useful comparison:

Interface Concern

CNI Networking

CSI Storage

Think:

```
CNI
→ Give my Pod networking

CSI
→ Give my workload persistent storage
```

| Access Modes

Remember this PVC field:

```
accessModes :
- ReadWriteOnce
```

You✓I commonly encounter modes such as:

```
ReadWriteOnce
ReadOnlyMany
ReadWriteMany
ReadWriteOncePod
```

They describe how a volume can be mounted/accessed, subject to what the actual storage driver supports.

Let's simplify them.

| ReadWriteOnce — RWO

Conceptually:

The volume can be mounted read-write by a single node.

Important nuance:

This does not necessarily mean only one Pod in all cases; multiple Pods on the same node can sometimes use that volume, depending on workload/storage behavior.

Beginner mental model:

```
RWO
"H
read-write from one Node
```

This is common for block-storage-backed databases.

| ReadOnlyMany — ROX

Conceptually:

```
Multiple Nodes
  ↓
same storage
  ↓
read only
```

Useful when multiple workloads need to consume shared static data but not modify it.

| ReadWriteMany — RWX

Conceptually:

```
Pod A |—
Pod B |—<|—° Same shared storage
Pod C |—      read/write
```

Multiple nodes can mount the volume read-write when supported by the storage backend.

Examples often involve shared/network filesystems.

| ReadWriteOncePod — RWOP

This is stricter:

The volume can be mounted read-write by a single Pod.

It's useful where you want Kubernetes to enforce single-Pod access semantics for supported CSI storage.

| Don't choose access mode randomly

Suppose a database needs:

```
one primary database instance
```

You may use storage suitable for single-writer semantics.

But suppose multiple Pods need shared files:

```
Pod A
Pod B
Pod C
```

You might require a backend supporting:

```
RWX
```

Storage architecture depends on application requirements.

| PV lifecycle state

You may encounter PV states such as:

```
Available
Bound
Released
Failed
```

Simplified:

Available

```
PV exists
not yet claimed
```

Bound

```
PV
!"
PVC
```

They are matched.

Released The previous claim was deleted, but the storage hasn't necessarily been made available for reuse.

Failed The volume experienced provisioning/reclamation problems.

You don't need to memorize every detail yet.

PVC status

A PVC commonly becomes:

Pending

or:

Bound

Pending Kubernetes hasn't yet satisfied the storage request.

Possible reasons:

No suitable PV
StorageClass issue
Provisioner problem
access mode mismatch
capacity mismatch
topology/scheduling conditions

Bound

PVC
↓
suitable storage established

Good.

Typical storage failure

You deploy an application.

Pod status:

Pending

Why?

It may not be a CPU issue.

Suppose the Pod needs:

database-pvc

and the PVC itself is:

Pending

Then:

```
Pod
↓
needs PVC
↓
PVC unavailable
↓
Pod can't fully proceed
```

Storage can therefore affect scheduling/startup.

Troubleshooting flow

Start with:

```
kubectl get pvc
```

Example:

NAME	STATUS	CAPACITY
database-pvc	Pending	

Then:

```
kubectl describe pvc database-pvc
```

Inspect events.

Also:

```
kubectl get pv
```

and:

```
kubectl get storageclass
```

short form:

```
kubectl get sc
```

Useful command hierarchy

```
kubectl get pvc
```

asks:

What storage has my application requested?


```
kubectl get pv
```

asks:

What persistent volumes exist?

```
kubectl get sc
```

asks:

What storage classes/provisioning options exist?

Very useful mental split.

| Reclaim Policy

What happens when the PVC is deleted?

Should the underlying storage also disappear?

That is controlled partly by the PV/StorageClass reclaim policy.

Common policies include:

```
Delete  
Retain
```

| Delete

Conceptually:

```
PVC deleted  
  ↓  
PV/storage reclamation  
  ↓  
underlying dynamically provisioned storage may be deleted
```

This can be convenient.

But obviously dangerous for valuable data if used without understanding lifecycle.

| Retain

Conceptually:

```
PVC deleted  
  ↓  
underlying storage retained
```

This gives administrators an opportunity to manually recover/manage the storage/data.

Useful for critical datasets where automatic deletion is undesirable.

The exact behavior depends on the storage configuration and resource lifecycle.

Very important distinction: Pod deletion vs PVC deletion

Suppose:

```
Pod
↓
PVC
↓
PV
```

Deleting the Pod:

```
Pod ×
```

doesn't inherently mean:

```
PVC ×
PV ×
Data ×
```

That is the whole point of separating storage lifecycle from Pod lifecycle.

But deleting the PVC can have storage lifecycle consequences depending on reclaim policy/provisioning.

So:

Deleting a Pod and deleting its storage claim are very different operations.

Example: PostgreSQL

Architecture:

```
PostgreSQL Pod
-
  | ¼
/var/lib/postgresql/data
-
  | ¼
PVC: postgres-data
-
  | ¼
PV
-
  | ¼
100 GiB persistent disk
```

Pod crashes.

New Pod can attach/mount persistent data as allowed by workload/storage architecture.

Data survives because it doesn't live only in the disposable container layer.

41. Does persistent storage mean database is automatically highly available?

No.

Very important.

You might have:

```
One PostgreSQL Pod
+
Persistent disk
```

If the Pod dies, Kubernetes may restart/recreate it and reuse persistent data.

But that is not the same as:

```
Multi-node database replication
Automatic database failover
Distributed consensus
Point-in-time recovery
Backups
Disaster recovery
```

Persistence and high availability are different problems.

Persistent storage is also not a backup

Another extremely important distinction.

Suppose your persistent disk gets corrupted.

Or someone executes:

SQL

DELETE FROM employee;

The PVC faithfully preserves that deletion.

So:

```
PersistentVolume
" `
Backup
```

You may still need:

```
database backups
snapshots
```

```
replication
disaster recovery
```

depending on the workload.

| Why databases need more than Deployment

Suppose you run:

```
Deployment
replicas = 3
```

with PostgreSQL.

Would that automatically create a proper PostgreSQL cluster?

No.

You might just get:

```
Postgres Pod A
Postgres Pod B
Postgres Pod C
```

with no proper database replication relationship.

Scaling stateless REST applications is easy:

```
Employee Pod × 3
```

Scaling databases requires application-specific state coordination.

This is why Kubernetes has workload patterns such as:

StatefulSet which we will cover next.

| Stateless Deployment vs stateful storage workload

Stateless

```
Employee Deployment
replicas = 3
```

```
Employee Pod A
Employee Pod B
Employee Pod C
```

All use external database

Pods are interchangeable.

Stateful

```
Database-0  
Database-1  
Database-2
```

These may need:

```
stable identity  
stable storage  
ordered startup  
application-specific replication
```

That is what leads us toward StatefulSets.

45. What if two Pods mount the same RWO volume?

Remember:

```
RWO  
=  
read-write from one Node
```

If two Pods are on different Nodes and both attempt incompatible access to the same volume, storage attachment/access constraints can prevent it.

This is another reason stateful scheduling requires more thought than stateless scaling.

Storage topology

Suppose storage physically belongs to:

```
Availability Zone A
```

but Scheduler tries to run the Pod in:

```
Availability Zone B
```

That storage might not be attachable there.

Kubernetes scheduling and storage provisioning can coordinate around topology constraints.

You don't need the internals yet, but remember:

Pod placement and storage location can affect each other.

WaitForFirstConsumer

You may later encounter a StorageClass setting:

```
volumeBindingMode: WaitForFirstConsumer
```

Conceptually it delays certain volume binding/provisioning decisions until a Pod using the PVC is being scheduled.

Why?

So Kubernetes can consider:

```
Pod scheduling requirements
+
storage topology
```

together.

This avoids provisioning storage somewhere the Pod cannot use.

You don't need to memorize the YAML yet.

| OpenShift connection

OpenShift uses the same Kubernetes storage concepts:

```
PV
PVC
StorageClass
CSI
```

So commands such as:

```
oc get pvc
oc get pv
oc get storageclass
```

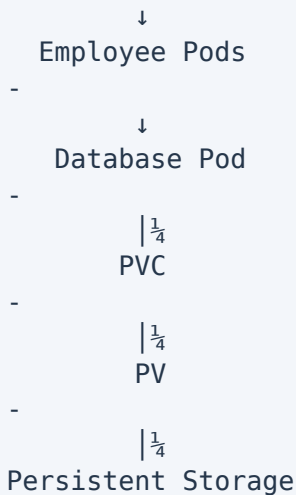
follow the same underlying model.

OpenShift's UI may make storage management easier, but the Kubernetes concepts remain the foundation.

| Complete application architecture now

Let's connect storage to everything we've learned:

```
Internet
  ↓
Route
  ↓
Gateway Service
  ↓
Gateway Pods
  ↓
Employee Service
```



Now the database Pod can be disposable while the data has a separate lifecycle.

StorageClass architecture

If dynamic provisioning is used:



The application developer focuses mostly on:

PVC

The platform abstracts the underlying storage implementation.

Common interview question

What is the difference between PV and PVC? Strong answer:

A PersistentVolume represents persistent storage available to the Kubernetes cluster, while a PersistentVolumeClaim is a workload/user request for storage. Kubernetes binds a suitable PV to a PVC, either from existing volumes or through dynamic provisioning.

52. Interview question: What is StorageClass?

A StorageClass defines a class of storage and the provisioner/configuration used for dynamic persistent-volume provisioning, allowing applications to request storage without knowing the physical storage implementation.

Interview question: What happens to data when a Pod using a PVC is

deleted?

The Pod's lifecycle and persistent storage lifecycle are separate. Deleting the Pod does not inherently delete the PVC or underlying storage, so a replacement workload can often mount the persistent data again, subject to access and scheduling constraints.

54. Interview question: Is a PV a backup?

No.

A PV provides persistence.

Backups protect against:

```
data corruption
accidental deletion
disaster
logical mistakes
```

Persistence alone doesn't provide those protections.

Interview question: CNI vs CSI

Easy answer:

```
CNI
→ networking

CSI
→ storage
```

Check your understanding

Q1 You need 50 GiB storage for a database.

Which object does the application usually request?

PVC

Q2 Which object represents the persistent storage resource?

PV

Q3 Which object can describe dynamic provisioning behavior?

StorageClass

Q4 Which standardized interface integrates storage drivers with Kubernetes?

CSI

Q5 If a Pod using a PVC dies, should you expect the persistent data to automatically disappear?

No.

Q6 If you delete the PVC, can that potentially affect the underlying storage?

Yes.

The outcome depends on provisioning/reclaim behavior.

Mini scenario

Suppose:

Payment DB Pod

PVC:
payment-db-data

Request:
200 GiB

StorageClass:
fast-ssd

Flow:

```
PVC created
  ↓
fast-ssd StorageClass
  ↓
CSI storage provisioner
  ↓
200 GiB disk created
  ↓
PV created/bound
  ↓
Payment DB Pod mounts PVC
  ↓
database writes data
```

Payment DB Pod crashes.

Kubernetes creates/restarts appropriate workload.

The new Pod can use:

```
payment-db-data
```

and continue with persistent storage.

That is the central concept.

chapter 9 Summary

Remember these four lines:

```
Volume
=
storage mounted into a Pod/container

PV
=
persistent storage resource

PVC
=
request for persistent storage

StorageClass
=
how/class of storage to dynamically provision
```

The core flow:

```
Pod
↓
PVC
↓
PV
↓
Actual Storage
```

and with dynamic provisioning:

```
PVC
↓
StorageClass
↓
CSI Provisioner
↓
PV / Actual Storage
```

The biggest architectural takeaway:

Pods are disposable, but application data does not have to be. Kubernetes separates the lifetime of the workload from the lifetime of persistent storage.

Next — chapter 10: StatefulSets & DaemonSets Now that storage makes sense, we can answer a much deeper question:

Why can't we simply run a database using Deployment replicas: 3 ?

We✓I cover:

```
Deployment vs StatefulSet
Stable Pod names
Stable network identity
Stable storage
volumeClaimTemplates
Ordered startup/shutdown
Why Pods become database-0, database-1, database-2
Headless Services
When to use StatefulSet
```

And we✓I also cover DaemonSet, which solves a completely different requirement:

```
"I want exactly one monitoring/logging agent on every Worker Node."
```

Architecture:

```
Node 1 → Log Agent Pod
Node 2 → Log Agent Pod
Node 3 → Log Agent Pod
```

That will complete the three most important workload patterns:

```
Deployment
StatefulSet
DaemonSet
```

Chapter 12 — Storage Architecture in Practice

CHAPTER FOCUS

Understand **Storage Architecture in Practice** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

StatefulSets Together Now let's connect the storage concepts you already learned into one real-world picture.

The most important idea today is:

A Pod is temporary, but application data often needs to survive the Pod.

The problem Kubernetes storage solves

Imagine a PostgreSQL database running in a Pod:

```
PostgreSQL Pod
-
  └─ /var/lib/postgresql/data
```

The database writes:

```
customers
orders
payments
```

Now the Pod crashes.

Kubernetes creates a replacement:

```
Old Pod ×
  ↓
New Pod ' '
```

If the data was stored only inside the Pod's writable filesystem:

```
Old Pod
  ↓
data ×
```

The new Pod may start with an empty filesystem.

That is obviously unacceptable for a database.

So we separate:

```
Application
  ↓
Pod
```

from:

```
Persistent Data
  ↓
Storage
```

The complete storage architecture

The picture you should remember is:

```
Pod
-
  | mounts
  | ¼
PVC
-
  | binds to
  | ¼
PV
-
  | provisioned by
  | ¼
StorageClass
-
  | ¼
Actual Storage
```

Each layer has a different responsibility.

3. What is a Volume?

Let's start from the bottom of the Kubernetes Pod world.

A container has a filesystem.

```
Container
-
|— /app
|— /tmp
|— /data
```

A Kubernetes Volume allows storage to be mounted into containers.

For example:

```
Pod
-
├─ container
-
    └─ /data
```

The container sees:

```
/data
```

but the underlying storage can come from different sources.

| emptyDir

One simple volume type is:

```
volumes:
- name: app-data
  emptyDir: {}
```

Conceptually:

```
Pod starts
  ↓
emptyDir created
  ↓
Container writes data
```

If the container restarts:

```
Container restart
  ↓
emptyDir remains
```

But if the Pod itself is deleted:

```
Pod deleted
  ↓
emptyDir deleted
  ↓
data lost
```

So emptyDir is useful for temporary data.

Examples:

```
temporary files
cache
scratch space
sharing files between containers in one Pod
```

Not normally for PostgreSQL's permanent database data.

PersistentVolume

Now we need something that survives Pod replacement.

That is where:

PersistentVolume — PV comes in.

A PV represents storage that Kubernetes can make available to workloads.

Conceptually:

```

Kubernetes
-
  | ¼
PersistentVolume
-
  | ¼
Actual storage
  
```

The actual storage could be backed by cloud or enterprise storage.

For example:

```

AWS EBS
Azure Disk
Google Persistent Disk
NFS
Ceph
enterprise SAN
etc.
  
```

PersistentVolumeClaim

Now comes the interesting part.

A developer generally shouldn't need to know:

"Give me disk storage-disk-23791 on this particular storage system."

Instead, the application says:

I need 20 GiB of persistent storage.

That is the job of:

PersistentVolumeClaim — PVC Think of PVC as a storage request.

```

Application
-
  | ¼
PVC
"I need 20Gi"
  
```

| PV vs PVC

This distinction is extremely important.

PV

```
PV
=
Available/provisioned storage
```

PVC

```
PVC
=
Request for storage
```

Think about a hotel:

```
Hotel
  ↓
Available rooms
```

is like:

```
PV
```

A customer saying:

```
"I need one room"
```

is like:

```
PVC
```

The hotel assigns a room to the customer.

Similarly:

```
PVC
  ↓
binds to
  ↓
PV
```

| Example PVC

Suppose PostgreSQL needs 20 GiB.

```
apiVersion: v1
kind: PersistentVolumeClaim

metadata:
  name: postgres-data
```



```
spec:
  accessModes:
    - ReadWriteOnce

  resources:
    requests:
      storage: 20Gi
```

The important line is:

```
requests:
  storage: 20Gi
```

Meaning:

Kubernetes, please provide me 20 GiB of persistent storage.

9. What happens next?

Kubernetes tries to satisfy the PVC.

Conceptually:

```
PVC
"20Gi please"
-
  | 1/4
Storage system
-
  | 1/4
PV
"20Gi available"
-
  | 1/4
PVC ! " PV
```

Once bound:

```
PVC
-
  | 1/4 PV
```

The application uses the PVC.

10. Why doesn't the Pod directly use the PV?

This is a very good question.

You generally don't configure your application as:

```
Pod → PV
```

Instead:

Pod → PVC → PV

Why?

Because the application asks for:

storage requirements

rather than knowing:

the exact storage resource implementation.

This provides abstraction.

| The StorageClass

Now we reach an even more powerful concept.

Imagine you have thousands of PVCs.

Do platform administrators manually create thousands of PVs?

No.

We use:

StorageClass StorageClass defines how storage should be dynamically provisioned.

Think:

```
PVC
↓
StorageClass
↓
Storage provisioner
↓
New storage
↓
PV
```

| Static provisioning

You already learned this earlier.

In static provisioning:

```
Admin
↓
creates PV
↓
Developer creates PVC
↓
PVC binds to PV
```

Example:

```
Admin:  
"Here's a 100Gi PV."
```

Then:

```
Developer:  
"I need 20Gi."
```

PVC binds to a suitable PV.

Dynamic provisioning

Modern Kubernetes environments usually prefer dynamic provisioning.

Developer says:

```
PVC:  
"I need 20Gi"
```

Then:

```
PVC  
↓  
StorageClass  
↓  
Provisioner  
↓  
Creates storage  
↓  
Creates/binds PV  
↓  
PVC
```

The developer doesn't need to manually create the PV.

Why dynamic provisioning is powerful

Imagine a company with:

```
5,000 applications
```

If every application needs:

```
20Gi
```

manually creating PVs becomes operationally painful.

Dynamic provisioning allows:

```
Developer
  ↓
PVC
  ↓
StorageClass
  ↓
Storage automatically created
```

This is one of the most important Kubernetes abstractions.

| Example StorageClass

A simplified example might look like:

```
apiVersion: storage.k8s.io/v1
kind: StorageClass

metadata:
  name: fast-storage

provisioner: example.com/storage

reclaimPolicy: Delete
```

The exact provisioner depends on your environment.

For example, cloud providers and enterprise platforms have their own storage integrations.

| PVC using StorageClass

Now your PVC can say:

```
apiVersion: v1
kind: PersistentVolumeClaim

metadata:
  name: postgres-data

spec:
  storageClassName: fast-storage

  accessModes:
    - ReadWriteOnce

  resources:
    requests:
      storage: 20Gi
```

Conceptually:

```
PVC
20Gi
-
  | storageClassName
  | 1/4
fast-storage
-
  | 1/4
Provisioner
-
  | 1/4
20Gi storage
```

Access Modes

You will frequently encounter:

```
ReadWriteOnce
ReadOnlyMany
ReadWriteMany
```

These describe how the volume may be mounted, subject to the capabilities of the underlying storage.

ReadWriteOnce — RWO

```
Read + Write
```

with the volume attached for writing in the manner supported by the storage implementation, commonly from one node at a time.

Typical example:

```
PostgreSQL
```

ReadOnlyMany — ROX Multiple nodes can mount the volume read-only, if supported.

ReadWriteMany — RWX Multiple nodes can mount the volume for read/write, if supported.

Typical use cases can include shared file storage.

Important: Access mode “ application behavior

Don’t interpret:

```
ReadWriteOnce
```

as:

“Only one Pod can ever use this.”

That is an oversimplification.

Access modes describe how the volume can be mounted according to the storage system's capabilities.

The actual behavior also depends on:

```
Storage implementation
Node placement
Pod configuration
```

This distinction becomes especially important with StatefulSets.

| Let's build our PostgreSQL example

We have:

```
PostgreSQL
```

running in Kubernetes.

We want:

```
Database
  ↓
Persistent data
```

Architecture:

```
PostgreSQL Pod
-
  | mounts
  | 1/4
postgres-data PVC
-
  | 1/4
postgres PV
-
  | 1/4
Actual storage
```

Now the Pod can be replaced without necessarily losing its persistent data.

20. What happens when the Pod crashes?

Suppose:

```
postgres-abc
```

crashes.

Kubernetes creates:

```
postgres-def
```

The new Pod mounts:

```
postgres-data PVC
```

which remains associated with persistent storage.

So:

```
Old Pod x
```

```
-
```

```
  X
```

```
-
```

```
PVC
```

```
-
```

```
PV
```

```
-
```

```
Storage
```

```
-
```

```
New Pod '
```

The database data can survive the Pod replacement.

21. But what if the Node dies?

This is where things become more interesting.

Suppose:

```
Node 1
```

```
-
```

```
├─ PostgreSQL Pod
```

Node 1 fails.

Kubernetes may recreate the Pod on:

```
Node 2
```

But whether the storage can be attached to Node 2 depends on the storage implementation and its topology/attachment capabilities.

For example, some block-storage systems can detach from one node and attach to another.

So persistent storage does not magically mean:

“The disk is instantly available everywhere.”

Storage architecture matters.

| This is why StorageClass matters

A StorageClass can encode important storage behavior.

Depending on the platform, it can determine things such as:

```
Storage backend
Performance characteristics
Provisioning behavior
Reclaim policy
Volume binding behavior
Topology
```

For production Kubernetes, the StorageClass is a very important platform abstraction.

| Reclaim Policy

Suppose:

```
PVC
↓
PV
```

is deleted.

What happens to the underlying storage?

That depends partly on the PV's:

```
reclaimPolicy
```

Common values include:

```
Delete
Retain
```

| Delete

With:

```
reclaimPolicy: Delete
```

the dynamically provisioned storage may be deleted when the PVC/PV lifecycle causes it to be released, depending on the storage implementation.

This is convenient for temporary environments.

Example:

```
Development environment
```


Retain

With:

```
reclaimPolicy: Retain
```

the underlying storage is retained after the claim is released.

This is useful when you want administrators to explicitly handle the data.

For critical production databases, retention policies need to be chosen very carefully.

Now we reach StatefulSets

This connects directly to the question you asked earlier:

Why do we need StatefulSets?

A Deployment is designed primarily for stateless replicas.

For example:

```
frontend  
frontend  
frontend
```

It doesn't matter much which Pod is which.

But databases often care about:

```
identity  
storage  
ordering
```

That is where StatefulSet comes in.

Deployment identity

Suppose a Deployment creates:

```
employee-api-7c9d4  
employee-api-7c9d5  
  
    employee-api-7c9d6
```

Pods are replaceable.

If one dies:

```
employee-api-7c9d4  ×
```

a replacement gets a new generated identity.

That is perfectly fine for stateless applications.

StatefulSet identity

A StatefulSet might create:

```
postgres-0
postgres-1
postgres-2
```

These identities are stable.

If:

```
postgres-1
```

is restarted, Kubernetes aims to bring back:

```
postgres-1
```

rather than treating it as a completely interchangeable anonymous replica.

That is extremely useful for stateful systems.

StatefulSet + PVC

Here's where things become powerful.

You can define:

```
volumeClaimTemplates:
- metadata:
  name: data
  spec:
    accessModes:
    - ReadWriteOnce

    resources:

    requests:
      storage: 20Gi
```

The StatefulSet can create claims corresponding to its replicas.

Conceptually:

```
StatefulSet
-
├─ postgres-0
│   └─ data-postgres-0
-
├─ postgres-1
│   └─ data-postgres-1
-
├─ postgres-2
│   └─ data-postgres-2
```

This is one of the most important StatefulSet concepts.

30. Why does each replica need its own PVC?

Imagine three database replicas:

```
postgres-0
postgres-1
postgres-2
```

You generally don't want:



because many database systems require carefully managed storage semantics.

Instead:

```
postgres-0 → PVC-0 → Storage-0
```

```
postgres-1 → PVC-1 → Storage-1
```

```
postgres-2 → PVC-2 → Storage-2
```

Each stateful replica gets its own persistent identity/storage association.

This answers your earlier Deployment question

You asked:

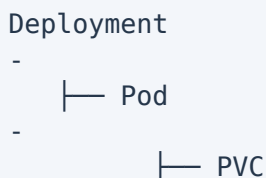
Why can't I apply PVC to a Deployment?

The precise answer is:

You can.

A Deployment can absolutely mount a PVC.

For example:



The problem is not that Deployments can't use PVCs.

The problem is:

A single shared PVC is often not the right storage model for multiple replicas.

Example

Suppose:

```
Deployment replicas = 3
```

and all three Pods use:

```
shared-pvc
```

Then:

```
Pod A ||
Pod B ||<|| shared PVC
Pod C ||
```

Whether this works depends on the volume's access mode and application behavior.

If it's:

```
ReadWriteOnce
```

you may have attachment/mounting constraints.

If it's:

```
ReadWriteMany
```

the storage system may permit multiple nodes to mount it, but the application still has to be designed for shared concurrent access.

StatefulSet gives you a different model

Instead:

```
StatefulSet
-
├─ Pod-0 → PVC-0
├─ Pod-1 → PVC-1
└─ Pod-2 → PVC-2
```

This matches many stateful application architectures much better.

Real-world example: Kafka

Consider Kafka.

You might have:

```
kafka-0
kafka-1
kafka-2
```

Each broker needs stable identity and persistent data.

You don't want:

```
broker A
broker B
broker C
```

to randomly swap identities.

StatefulSet is a natural Kubernetes primitive for workloads requiring:

```
stable network identity
stable storage
ordered lifecycle
```

Kafka, databases, and some clustered systems can use this model.

| Stable network identity

StatefulSets commonly work with a headless Service.

Conceptually:

```
StatefulSet
-
  ├── postgres-0
  ├── postgres-1
  └── postgres-2
-
      | 1/4
  Headless Service
```

This enables stable DNS identities for the Pods.

For example, conceptually:

```
postgres-0.<service>.<namespace>.svc.cluster.local
```

The exact DNS name depends on the StatefulSet and Service names.

| Why stable identity matters

Imagine a distributed database.

Node A needs to know:

“Which database member am I talking to?”

With stable identities:

```
db-0
db-1
db-2
```

the members can discover each other consistently.

With arbitrary Pod names changing after every replacement, this becomes harder.

| StatefulSet ordering

StatefulSets can also provide ordered startup/shutdown behavior depending on the configured pod management policy.

The traditional ordered behavior is:

Create:

```
db-0
  ↓
db-1
  ↓
db-2
```

And termination can happen in reverse:

Delete:

```
db-2
  ↓
db-1
  ↓
db-0
```

Why?

Because some clustered applications have dependencies between members.

38. Why reverse deletion?

Imagine:

```
db-0 = primary/bootstrap member
db-1 = replica
db-2 = replica
```

You may prefer to remove:

```
db-2
```

before:

```
db-1
```

and preserve the lower ordinals longer.

Kubernetes StatefulSet provides predictable identity/order primitives, but the application itself still needs to be designed correctly for clustering.

StatefulSet does not magically turn PostgreSQL/Kafka/etc. into a highly available database.

Very important distinction

This:

StatefulSet

does not mean:

“My database is automatically highly available.”

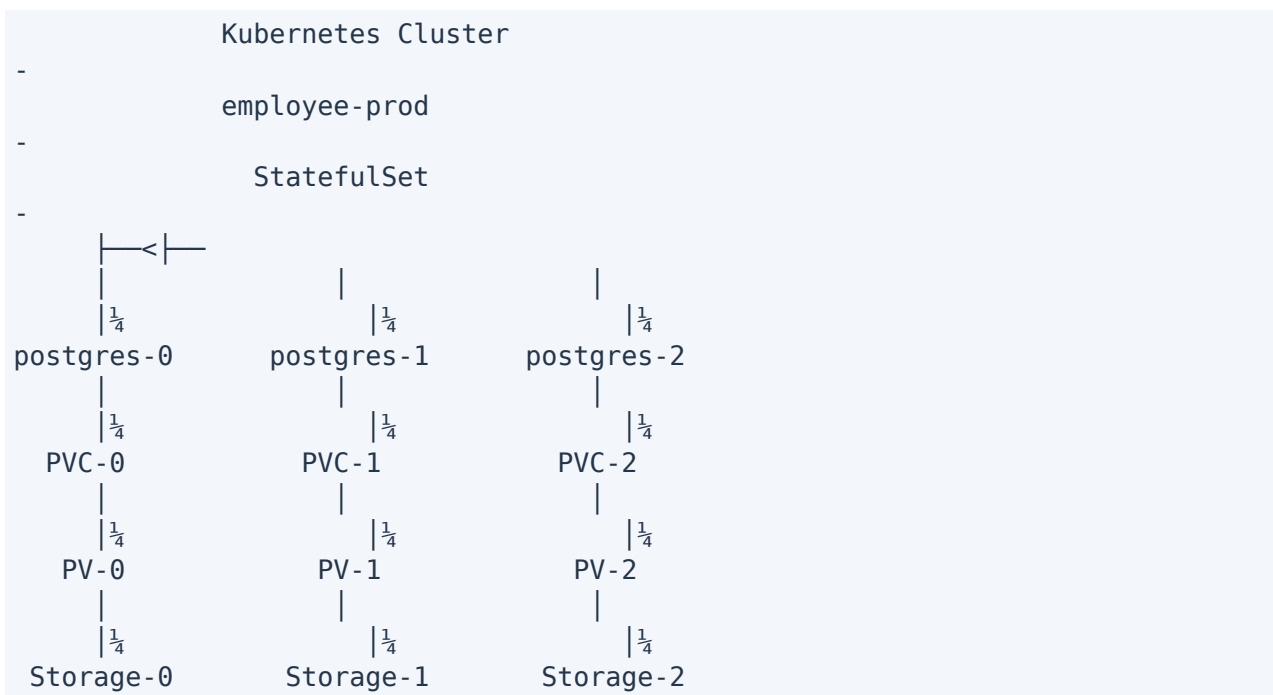
It means Kubernetes provides primitives for:

stable identity
stable storage association
ordered lifecycle

The database software still needs its own replication/failover architecture.

Complete PostgreSQL architecture

Now let's put everything together.



Now you can see why StatefulSets exist.

41. What happens when postgres-1 dies?

Suppose:

```
postgres-1 x
```

Kubernetes recreates:

```
postgres-1 '
```

and associates it with its persistent storage claim:

```
postgres-1
-
  | 1/4
PVC-1
-
  | 1/4
PV-1
-
  | 1/4
Storage-1
```

The important thing is that the identity and storage association are stable.

42. What happens if the Pod moves to another Node?

Suppose:

Before:

```
Node 1
|— postgres-1
-
  |— Storage-1
```

Node 1 fails.

Potentially:

After:

```
Node 2
|— postgres-1
-
  |— Storage-1
```

The exact ability to reattach/move storage depends on the storage backend and topology.

This is why production storage design cannot be separated from the underlying infrastructure.

Kubernetes storage isn't backup

This is a very important production lesson.

Suppose:

```
PVC → PV → disk
```

Your disk is persistent.

That does not automatically mean:

```
backup exists
```

If someone accidentally deletes the database:

```
DELETE FROM customers;
```

the PVC doesn't save you.

You still need:

```
Database backups  
Snapshots  
Replication  
Disaster recovery
```

depending on the application's requirements.

Persistence vs Backup

Remember:

```
Persistent Storage  
=  
survives Pod replacement
```

while:

```
Backup  
=  
allows recovery from data loss/corruption
```

These are completely different concepts.

Kubernetes storage mental model

Whenever you see:

```
Pod needs persistent data
```

think:

```
Pod  
↓  
PVC
```

```

↓
PV
↓
StorageClass / provisioner
↓
Actual storage

```

For stateful workloads:

```

StatefulSet
↓
stable Pod identity
↓
individual PVC
↓
persistent storage

```

Deployment vs StatefulSet

Let's finally make the comparison very clear.

Feature Deployment StatefulSet

Typical use Stateless apps Stateful apps

Pod identity Usually interchangeable Stable ordinal identity

Stable hostname No Yes, with governing Service

Stable storage association Not automatically per replica Yes, via PVC templates

Ordered lifecycle Not the main purpose Supported

Typical examples REST API, frontend DB, Kafka, ZooKeeper-like systems

Can use PVC? Yes Yes

The most important row is:

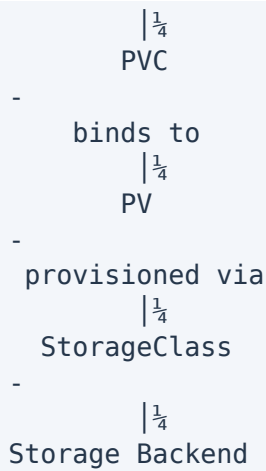
Deployment can use PVC. StatefulSet is designed to manage stateful replicas with stable identity and storage.

Storage architecture you should now know

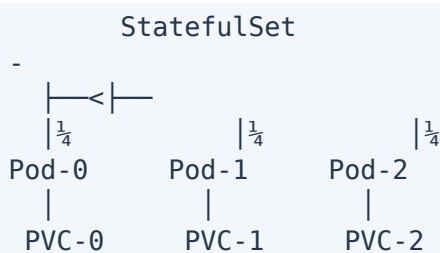
```

Application
-
  | 1/4
  Pod
-
volumeMount
-

```



For stateful applications:



Practical commands

Check PVCs:

```
kubectl get pvc -n employee-prod
```

Check PVs:

```
kubectl get pv
```

Check StorageClasses:

```
kubectl get storageclass
```

Detailed PVC information:

```
kubectl describe pvc postgres-data -n employee-prod
```

See which PVC a Pod uses:

```
kubectl describe pod postgres-0 -n employee-prod
```

Check StatefulSets:

```
kubectl get statefulsets -n employee-prod
```

| One troubleshooting scenario

You create:

```
PVC:  
20Gi
```

but see:

```
STATUS: Pending
```

Don't immediately blame the Pod.

The problem may be:

```
No suitable PV
```

or:

```
StorageClass problem
```

or:

```
Provisioner problem
```

or:

```
Topology/capacity issue
```

So you'd investigate:

```
kubectl describe pvc postgres-data -n employee-prod
```

and:

```
kubectl get storageclass
```

This is how Kubernetes troubleshooting starts: follow the resource chain.

| The most important lesson from this chapter

Don't memorize:

```
PV  
PVC  
StorageClass
```

as three random Kubernetes objects.

Understand the flow:

```
Developer  
-  
  | "I need 20Gi"
```

```

    |  $\frac{1}{4}$ 
PVC
-
    | "Give me storage"
    |  $\frac{1}{4}$ 
StorageClass
-
    | provisions
    |  $\frac{1}{4}$ 
PV
-
    |  $\frac{1}{4}$ 
Actual storage

```

Then:

```

Pod
-
  └─ mounts PVC

```

And for stateful replicas:

```

StatefulSet
-
  └─ Pod-0 → PVC-0
  └─ Pod-1 → PVC-1
  └─ Pod-2 → PVC-2

```

If this flow is clear, Kubernetes storage becomes much less confusing.

Next chapter — chapter 17: ConfigMaps & Secrets The next logical question is:

Where should my application configuration and passwords live?

We'll build a realistic Spring Boot application:

```

employee-api
-
  └─ application settings
  └─ database URL
  └─ database username
  └─ database password

```

and learn:

```

ConfigMap
Secret
Environment variables
Volume-mounted configuration
Secret management
OpenShift Secrets

```

Most importantly, we'll answer:

Why shouldn't you put database passwords directly inside a Deployment YAML?

And we✓I connect this to HashiCorp Vault, which you've encountered before, so you✓I understand where Kubernetes Secrets end and enterprise secret management begins.

Chapter 13 — StatefulSets and DaemonSets

CHAPTER FOCUS

Understand **StatefulSets** and **DaemonSets** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

Now we'll cover two important workload types:

```
Deployment
StatefulSet
DaemonSet
```

They all run Pods, but they solve very different problems.

A useful summary before we begin:

```
Deployment
→ "Run N interchangeable copies of my application."

StatefulSet
→ "Run N stateful Pods with stable identity/storage."

DaemonSet
→ "Run one copy on every eligible Node."
```

1. Why not use Deployment for everything?

Suppose you run a stateless Spring Boot service:

```
Employee Service
```

You don't care whether the Pods are called:

```
employee-abc
employee-xzy
employee-pqr
```

They are interchangeable.

If one dies:

```
employee-abc x
```

a replacement:

```
employee-mno '
```

is perfectly fine.

That is ideal for a:

Deployment

Databases are different

Now imagine a database cluster:

```
Database Node 1  
Database Node 2  
Database Node 3
```

These instances may have distinct identities and roles.

For example:

```
db-0 → primary  
db-1 → replica  
db-2 → replica
```

You may need:

```
stable names  
stable persistent disks  
predictable startup order  
predictable shutdown order
```

A normal Deployment does not provide this identity model.

That leads to:

StatefulSet

3. What is a StatefulSet?

One-sentence definition

A StatefulSet manages stateful Pods that need stable identity, persistent storage and ordered lifecycle behavior. Typical uses include:

```
Databases  
Kafka  
ZooKeeper-style systems  
Distributed stateful applications
```

But remember:

StatefulSet itself does not magically make an application stateful or clustered correctly.

Your application still needs to understand replication, clustering, leadership, etc.

| Deployment vs StatefulSet Pod names

Suppose Deployment replicas = 3.

You might see:

```
employee-79c98c9c8-abc12  
employee-79c98c9c8-def45  
employee-79c98c9c8-ghi78
```

The names are generated.

If one dies, replacement gets a new identity.

With StatefulSet:

```
mysql-0  
mysql-1  
mysql-2
```

These identities are stable.

If:

```
mysql-1
```

dies, Kubernetes recreates:

```
mysql-1
```

not:

```
mysql-random123
```

This is a major difference.

| Stable ordinal identity

StatefulSet Pods get an ordinal number:

```
0  
1  
2
```

Example:

```
database-0  
database-1  
database-2
```

You can think of:

```
StatefulSet name + ordinal
```

This gives each Pod a stable identity.

| Why stable identity matters

Imagine a distributed database where members need to know:

```
node-0  
node-1  
node-2
```

The application can reason about specific cluster members.

For example:

```
node-0  
may initialize cluster  
  
node-1  
joins node-0  
  
node-2  
joins node-0
```

Random replacement names would make this harder.

| Stable storage

This is even more important.

Suppose StatefulSet creates:

```
db-0  
db-1  
db-2
```

Each gets its own PVC:

```
db-0 → pvc-0  
db-1 → pvc-1  
  
db-2 → pvc-2
```

Architecture:

```
StatefulSet  
-  
  |— db-0 |—° PVC-0 |—° Disk-0  
-  
  |— db-1 |—° PVC-1 |—° Disk-1  
-
```

```
|— db-2 |—° PVC-2 |—° Disk-2
```

If db-1 dies:

```
db-1 ×
```

replacement db-1 comes back and uses:

```
PVC-1
```

again.

This preserves its storage identity.

| Important distinction

StatefulSet does not mean:

```
All Pods share one disk
```

Usually, each Pod gets its own persistent storage.

Example:

```
postgres-0 → postgres-data-postgres-0  
postgres-1 → postgres-data-postgres-1  
postgres-2 → postgres-data-postgres-2
```

This is very different from all replicas mounting the same volume.

| volumeClaimTemplates

How does StatefulSet create one PVC per Pod?

Using:

```
volumeClaimTemplates
```

Conceptually:

StatefulSet says:

```
"For every Pod I create,  
create a corresponding PVC from this template."
```

Example:

```
volumeClaimTemplates:  
  - metadata:  
      name: data  
  
    spec:
```

```
accessModes:
  - ReadWriteOnce

resources:
  requests:
    storage: 20Gi
```

If replicas = 3:

```
Pod 0 → PVC 0
Pod 1 → PVC 1
Pod 2 → PVC 2
```

| Simplified StatefulSet YAML

```
apiVersion: apps/v1
kind: StatefulSet

metadata:
  name: database

spec:
  serviceName: database

  replicas: 3

  selector:
    matchLabels:
      app: database

  template:
    metadata:
      labels:
        app: database

    spec:
      containers:
        - name: database
          image: my-database:1.0

          volumeMounts:
            - name: data
              mountPath: /var/lib/database

      volumeClaimTemplates:
        - metadata:
            name: data

            spec:
              accessModes:
```

```
- ReadWriteOnce

resources:
  requests:
    storage: 20Gi
```

Don't memorize the YAML yet.

Understand the architecture first.

11. What gets created?

Suppose:

```
replicas = 3
```

StatefulSet creates:

```
database-0
database-1
database-2
```

And PVCs conceptually like:

```
data-database-0
data-database-1
data-database-2
```

So:

```
database-0
  ↓
data-database-0

database-1
  ↓
data-database-1

database-2
  ↓
data-database-2
```

12. What if database-1 crashes?

Before:

```
database-0 '
database-1 '
database-2 '
```

Then:

```
database-1 x
```

Kubernetes recreates:

```
database-1 '
```

and associates it with its expected persistent storage.

The identity remains:

```
database-1
```

rather than becoming a random new member.

| Ordered Pod creation

StatefulSets commonly create Pods in order.

For example:

```
database-0  
  ↓ ready
```

```
database-1  
  ↓ ready
```

```
database-2
```

Conceptually:

```
0 → 1 → 2
```

Why?

Some stateful systems require earlier members to be available before later members start.

| Ordered scale-down

Scale from:

```
3 → 1
```

StatefulSet can terminate in reverse ordinal order:

```
database-2  
  ↓  
database-1  
  ↓  
database-0 remains
```

So conceptually:

```
2 → 1 → 0
```

This predictability can matter for stateful workloads.

Deployment behavior is different

Deployment doesn't normally care:

```
which Pod starts first
```

because replicas should be interchangeable.

For:

```
Employee Service
```

this is fine.

You just want:

```
3 healthy instances
```

No identity relationship exists.

Stable networking

Stateful applications may also need stable network identity.

Suppose:

```
database-0  
database-1  
database-2
```

They need to reach each other consistently.

Kubernetes commonly combines StatefulSet with a:

Headless Service

17. What is a Headless Service?

Remember normal ClusterIP Service:

```
Service  
ClusterIP = 10.96.x.x  
↓  
multiple Pods
```

A headless Service uses:

```
clusterIP: None
```

It doesn't provide one normal Service virtual IP in the usual way.

Instead, DNS can expose individual Pod identities/endpoints.

This is useful for stateful systems.

18. Why would we want individual Pod identities?

For stateless applications:

```
Client
↓
employee-service
↓
any Employee Pod
```

We don't care which one.

For a clustered database:

```
database-0
```

may need to communicate specifically with:

```
database-1
database-2
```

So stable per-Pod DNS becomes useful.

| Conceptual DNS

With a StatefulSet and headless Service, you can get stable DNS patterns conceptually like:

```
database-0.database
database-1.database
database-2.database
```

Possibly with namespace and cluster domain appended.

So:

```
database-1
```

can consistently refer to a specific stateful member.

| Normal Service vs Headless Service

Normal ClusterIP Service


```
employee-service
  ↓
any matching Pod
```

Good for stateless load balancing.

Headless Service

```
database-0.database
database-1.database
database-2.database
```

Useful when clients or cluster members need to discover individual Pods.

| Important misconception

Do not think:

StatefulSet automatically gives me a highly available database.

Suppose you deploy:

```
postgres image
replicas = 3
```

using StatefulSet.

That doesn't automatically configure:

```
replication
leader election
failover
data consistency
```

PostgreSQL itself or an operator/database clustering solution still needs to implement those behaviors.

StatefulSet provides Kubernetes primitives:

```
stable identity
stable storage
ordered lifecycle
```

It doesn't implement database semantics.

| Database operators

In production Kubernetes/OpenShift, complex databases are often deployed using:

Operators

An Operator understands application-specific operational logic.

For example, a PostgreSQL Operator may understand:

```
create primary
create replicas
configure replication
failover
backup
restore
upgrade
```

while using Kubernetes resources underneath.

We'll discuss Operators in the OpenShift section.

| StatefulSet scaling

Suppose:

```
database replicas = 3
```

You scale to:

```
5
```

New Pods:

```
database-3
database-4
```

and potentially new PVCs:

```
data-database-3
data-database-4
```

But again:

Whether the database application correctly joins those new replicas is application-specific.

Kubernetes only creates the infrastructure.

| StatefulSet and Pod replacement

This distinction is worth memorizing.

Deployment:

```
employee-abc x

replacement:
employee-xyz
```

StatefulSet:

```
database-1 x  
  
replacement:  
database-1
```

Stable identity is the key.

25. Does deleting StatefulSet Pod delete PVC?

Typically, deleting/replacing a StatefulSet Pod does not automatically mean its PVC disappears.

This is intentional.

We want:

```
Pod disposable  
Storage persistent
```

So:

```
database-1 Pod deleted  
    ↓  
PVC remains  
    ↓  
database-1 recreated  
    ↓  
same storage identity reused
```

Exact PVC retention behavior can be configured in modern Kubernetes, but the fundamental design is preservation of state.

Deployment vs StatefulSet

Here's the important comparison:

Deployment StatefulSet

Stateless workloads Stateful workloads

Pods interchangeable Pods have stable identity

Random/generated Pod identity Predictable ordinal identity

No stable per-Pod storage relationship by default Stable per-Pod PVCs

Parallel/unordered replicas generally fine Ordered lifecycle available

Example: REST API Example: clustered database

The simplest memory trick:

```
Deployment
= interchangeable replicas

StatefulSet
= identity matters
```

Now let's switch to DaemonSet

StatefulSet solves stateful identity.

DaemonSet solves a completely different problem.

Imagine a 5-node Kubernetes cluster:

```
Node A
Node B
Node C
Node D
Node E
```

You want a monitoring agent on every Node.

Do you create:

```
replicas: 5
```

using Deployment?

That might create 5 Pods, but Kubernetes could schedule:

```
Node A → 2 Pods
Node B → 1 Pod
Node C → 0 Pods
Node D → 2 Pods
Node E → 0 Pods
```

That is not what we want.

We specifically want:

```
one Pod per Node
```

That leads to:

DaemonSet

28. What is a DaemonSet?

One-sentence definition

A DaemonSet ensures that a copy of a Pod runs on every eligible Node, or on a selected subset of Nodes. Architecture:

Cluster

Node A → Agent Pod
Node B → Agent Pod
Node C → Agent Pod
Node D → Agent Pod

When a new Node joins:

Node E joins

DaemonSet automatically creates:

Agent Pod on Node E

29. Why “Daemon”?

In operating systems, a daemon is a background service.

Examples:

logging service
monitoring agent
network agent
security agent

Likewise, DaemonSet is commonly used for node-level background infrastructure.

| Common DaemonSet use cases

Typical examples:

Log collectors
Node monitoring agents
Security agents
CNI networking agents/components
Storage agents
Node-local proxies

For example:

Fluent Bit

or similar logging agent may run on every Node to collect container logs.

| Logging example

Cluster:

```
Node A
├─ Employee Pod
├─ Payment Pod
└─ Log Agent
```

```
Node B
├─ Order Pod
├─ Gateway Pod
└─ Log Agent
```

```
Node C
├─ Payment Pod
├─ Employee Pod
└─ Log Agent
```

Each node-local log agent can gather logs from workloads on that Node.

This is an ideal DaemonSet pattern.

32. Why not put the logging agent inside every application Pod?

You sometimes can use sidecars, but then:

```
100 application Pods
=
100 logging sidecars
```

If the requirement is truly node-level log collection, DaemonSet may be much more efficient:

```
10 Nodes
=
10 agent Pods
```

Different patterns solve different problems.

DaemonSet behavior when Nodes change

Initial:

```
Node A → agent
Node B → agent
Node C → agent
```

Add Node D:

```
Node D joins
```

DaemonSet controller sees:

```
Node D eligible  
agent missing
```

and creates:

```
Node D → agent
```

No need to manually change:

```
replicas = 4
```

DaemonSet derives desired count from eligible Nodes.

| Important: DaemonSet usually has no normal replica count

With Deployment:

```
replicas: 3
```

means:

Run three Pods.

DaemonSet conceptually says:

Run one Pod on every eligible Node.

So if you have:

```
20 eligible Nodes
```

you may have:

```
20 DaemonSet Pods
```

Add another Node:

```
21
```

Remove a Node:

```
20
```

Desired count follows topology.

| Selected Nodes only

DaemonSet doesn't necessarily have to run on literally every Node.

You can restrict it with scheduling controls.

For example:

Only GPU Nodes

or:

Only worker Nodes with label:
logging=true

Then:

Node A logging=true → Agent '
Node B logging=true → Agent '
Node C logging=false → No Agent

We✓I learn node selectors/affinity later.

| DaemonSet example YAML

Simplified:

```
apiVersion: apps/v1
kind: DaemonSet

metadata:
  name: node-monitor

spec:
  selector:
    matchLabels:
      app: node-monitor

  template:
    metadata:
      labels:
        app: node-monitor

    spec:
      containers:
        - name: monitor
          image: company/node-monitor:1.0
```

Notice something:

There is no:

replicas: 5

The number of eligible Nodes determines how many Pods should run.

| DaemonSet still uses Pods

Don't think DaemonSet creates a process outside Kubernetes.

Hierarchy:

```
DaemonSet
  ↓
Pods
  ↓
Containers
```

The special behavior is placement/count.

| Deployment vs DaemonSet

Suppose there are 3 nodes.

Deployment replicas = 3:

Scheduler may place:

```
Node A → Pod 1, Pod 2
Node B → Pod 3
Node C → none
```

unless additional scheduling constraints prevent it.

DaemonSet:

```
Node A → Pod
Node B → Pod
Node C → Pod
```

That is the critical difference.

| StatefulSet vs DaemonSet

They solve completely different problems.

StatefulSet

I need stable identities:

```
database-0
database-1
database-2
```

DaemonSet

I need one agent per Node.

Do not choose based on whether an application “sounds important.”

Choose based on workload behavior.

| Three workload types together

Now let's compare:

Deployment

Employee Service

Desired:

3 interchangeable replicas

Architecture:

Deployment

↓

Pod

Pod

Pod

StatefulSet

Database Cluster

Desired:

stable members

Architecture:

StatefulSet

↓

db-0 → disk-0

db-1 → disk-1

db-2 → disk-2

DaemonSet

Log Collector

Desired:

one per node

Architecture:

Node A → collector

Node B → collector

Node C → collector

| Real enterprise cluster example

Imagine:

Kubernetes Cluster

Node A

Node B

Node C

Applications:

Employee Deployment

replicas = 3

Payment Deployment

replicas = 4

Database:

Database StatefulSet

replicas = 3

Monitoring:

Node Monitor DaemonSet

Result may look like:

Node A

├─ Employee Pod

├─ Payment Pod

├─ database-0

└─ Monitor Pod

Node B

├─ Employee Pod

├─ Payment Pod

├─ Payment Pod

├─ database-1

└─ Monitor Pod

Node C

├─ Employee Pod

├─ Payment Pod

├─ database-2

└─ Monitor Pod

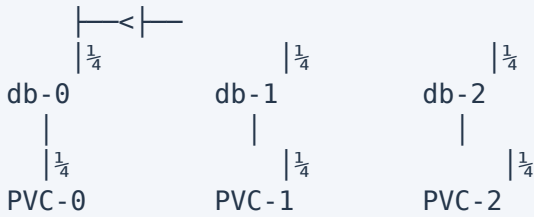
Different controllers manage different workload requirements.

StatefulSet Service architecture

For a stateful cluster:

Headless Service

-



This gives:

```

stable network identity
+
stable storage identity
  
```

That is the heart of StatefulSet.

StatefulSet update behavior

StatefulSets can also update Pod templates.

For example:

```

database:v1
  ↓
database:v2
  
```

Kubernetes can roll updates through Pods according to StatefulSet update strategy.

Because identity/order may matter, rollout behavior differs conceptually from a simple stateless Deployment.

We'll keep this advanced topic for later.

Common misconception: StatefulSet = persistent volume

No.

You can have persistent volumes without StatefulSet.

For example:

```

Deployment
  ↓
Pod
  ↓
PVC
  
```

is possible.

StatefulSet becomes useful when you additionally need:

```
stable Pod identity
stable per-Pod storage relationship
ordered lifecycle
```

PV/PVC and StatefulSet are related but not identical concepts.

Common misconception: DaemonSet = exactly one Pod total

No.

DaemonSet typically means:

```
one Pod per eligible Node
```

So if cluster has:

```
100 Nodes
```

you may get:

```
100 DaemonSet Pods
```

Common interview question

What is the difference between Deployment and StatefulSet? Good answer:

A Deployment manages interchangeable stateless Pod replicas, while a StatefulSet is designed for stateful workloads requiring stable Pod identity, stable per-Pod storage and ordered lifecycle behavior.

47. Interview question: Why doesn't StatefulSet just use random Pod names?

Because stateful systems may require stable member identities.

For example:

```
database-0
database-1
database-2
```

are predictable and can have stable storage/DNS relationships.

48. Interview question: What is a headless Service?

Good beginner answer:

A headless Service is a Service configured without a normal ClusterIP, commonly used when clients need to discover individual Pod endpoints instead of accessing the workload through one virtual Service IP, such as with StatefulSets.

49. Interview question: What is DaemonSet?

A DaemonSet ensures that one instance of a Pod runs on every eligible Node or on a selected subset of Nodes. It is commonly used for node-level infrastructure such as logging, monitoring and networking agents.

50. Interview question: Deployment replicas=5 vs DaemonSet on 5 nodes?

Very different.

Deployment:

```
5 Pods anywhere suitable
```

DaemonSet:

```
1 Pod per eligible Node
```

The number happens to be 5 only because there are 5 eligible Nodes.

Troubleshooting StatefulSet

Suppose:

```
database-0 Running
database-1 Pending
database-2 not created yet
```

Possible reason:

StatefulSet may be waiting for:

```
database-1
```

to become healthy/ready before proceeding.

Why is database-1 Pending?

Investigate:

```
PVC
storage provisioning
scheduling
resources
Node constraints
```

Commands:

```
kubectl get statefulset
kubectl get pods
kubectl get pvc
kubectl describe pod database-1
```

| Troubleshooting DaemonSet

Suppose:

```
10 Nodes
```

but:

```
DaemonSet READY = 8
```

Ask:

```
Which Nodes don't have Pods?
```

Possible reasons:

```
Node selector mismatch  
Taints/tolerations  
Resource problems  
Node not ready  
Image pull failure  
Pod startup error
```

Useful command:

```
kubectl get daemonset
```

short:

```
kubectl get ds
```

| Commands to remember

Deployments:

```
kubectl get deploy
```

StatefulSets:

```
kubectl get statefulset
```

short:

```
kubectl get sts
```

DaemonSets:

```
kubectl get daemonset
```

short:

```
kubectl get ds
```

Pods:

```
kubectl get pods
```

PVCs:

```
kubectl get pvc
```

| OpenShift connection

All three are Kubernetes resources and work in OpenShift too:

```
oc get deployment
oc get statefulset
oc get daemonset
```

OpenShift does not replace these concepts.

It builds additional platform features around them.

| Architecture so far

We've now learned several controller patterns:

```
Deployment
  ↓
ReplicaSet
  ↓
stateless Pods

StatefulSet
  ↓
  stable stateful Pods
    ↓
    PVC per Pod

  DaemonSet
    ↓
    one infrastructure Pod
    per eligible Node
```

This gives Kubernetes flexibility to manage very different workloads.

| Check your understanding

Q1 You have a Spring Boot REST API where every replica is interchangeable.

Use:

Deployment

Q2 You have a database cluster where each instance needs a stable name and its own persistent disk.

Use:

StatefulSet

Q3 You need one log collector on every worker Node.

Use:

DaemonSet

Q4 If:

```
database-1
```

dies in a StatefulSet, should its replacement normally become:

```
database-random-xyz
```

No.

It returns with the stable ordinal identity:

```
database-1
```

Q5 If you add a new Worker Node, what happens to a DaemonSet?

If the Node is eligible, the DaemonSet controller creates its Pod on that Node.

Mini architecture exercise

Suppose your platform contains:

```
Gateway
Employee API
PostgreSQL cluster
Monitoring agent
```

Correct workloads:

```
Gateway
→ Deployment

Employee API
→ Deployment

PostgreSQL
→ StatefulSet
```

Monitoring Agent
→ DaemonSet

Architecture:



with:

postgres-0 → PVC-0
postgres-1 → PVC-1
postgres-2 → PVC-2

That is a realistic use of all three workload types.

chapter 10 Summary

Remember:

Deployment
=
N interchangeable Pods

StatefulSet
=
N stateful Pods
with stable identity/storage

DaemonSet
=
one Pod per eligible Node

The most important comparison is:

```
Deployment
"Which Pod is which doesn't matter."
```

```
StatefulSet
"Pod identity matters."
```

```
DaemonSet
"Node placement matters."
```

Next — chapter 11: Health Checks — Liveness, Readiness & Startup Probes Now we✓I solve another production problem.

Suppose:

```
Pod = Running
```

but Spring Boot is actually:

```
stuck
unable to connect to DB
still initializing
deadlocked
returning failures
```

Should Kubernetes send traffic to it?

Should Kubernetes restart it?

Those are two completely different questions.

That leads to:

```
Readiness Probe
→ "Can I receive traffic?"
```

```
Liveness Probe
→ "Am I alive, or should I be restarted?"
```

```
Startup Probe
→ "Have I finished starting yet?"
```

This chapter is particularly important for real Spring Boot/OpenShift deployments.

Health, Resources, Scaling, and Scheduling

Chapter 14 — Startup, Liveness, and Readiness Probes

CHAPTER FOCUS

Understand **Startup, Liveness, and Readiness Probes** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

Probes This is one of the most important production Kubernetes concepts, especially for a Java/Spring Boot developer.

The key problem is:

A Pod being Running does not necessarily mean the application is healthy or ready to receive traffic.

Let's understand this from the ground up.

A real-world problem

Imagine you deploy:

```
Spring Boot Employee Service
```

Kubernetes says:

```
Pod: Running
```

You might think:

Great, the application is working.

But internally:

```
Spring Boot
  ↓
Application started
  ↓
Database connection ×
  ↓
API requests fail
```

Kubernetes may still see:

```
Pod = Running
```

So Kubernetes needs a way to ask the application:

```
Are you alive?  
Are you ready?  
Have you finished starting?
```

That is what probes are for.

Three types of probes

Kubernetes provides:

```
Liveness Probe  
Readiness Probe  
Startup Probe
```

Think of them as three different questions.

Liveness

“Are you still alive?”

Readiness

“Can you receive traffic right now?”

Startup

“Have you finished starting?”

Memorize those three questions first.

3. Why do we need separate probes?

Consider a Spring Boot application:

```
09:00:00  
Container starts  
  
09:00:02  
JVM starting  
  
09:00:05  
Spring loading beans  
  
09:00:10  
Database connection initializing  
  
09:00:15  
Application ready
```

During:

```
09:00:02 → 09:00:14
```

the application is alive but not ready.

So:

Alive?	YES
Ready?	NO

This distinction is extremely important.

First: Liveness Probe

A liveness probe answers:

Is this application still functioning sufficiently to remain alive?

Suppose your application gets stuck:

```
Spring Boot
  ↓
deadlock
  ↓
no useful responses
```

The process might technically still exist.

Kubernetes can detect repeated liveness failures and restart the container.

Conceptually:

```
Container
  ↓
Liveness Probe
  ↓
FAIL
  ↓
Kubernetes restarts container
```

5. Why restart?

Imagine:

```
Application process
  ↓
stuck forever
```

Without a liveness probe:

```
Pod = Running
Application = useless
```

Kubernetes doesn't automatically know that the application is internally broken.

A liveness probe gives Kubernetes permission to say:

This container isn't healthy enough to continue. Restart it.

| Readiness Probe

Now imagine the application is starting.

```
Container starts
  ↓
Spring Boot initializing
  ↓
Database connection establishing
  ↓
Application not ready
```

Should Kubernetes send user requests to it?

Obviously:

No.

That is what readiness is for.

```
Readiness = FAIL
  ↓
Pod removed from Service endpoints
  ↓
No normal traffic sent to it
```

The container itself does not necessarily restart.

That is the critical difference.

| Readiness does NOT mean restart

This is probably the most important distinction in this chapter.

If:

```
Readiness = FAIL
```

Kubernetes essentially says:

Don't send traffic to this Pod right now.

It doesn't mean:

Kill the container.

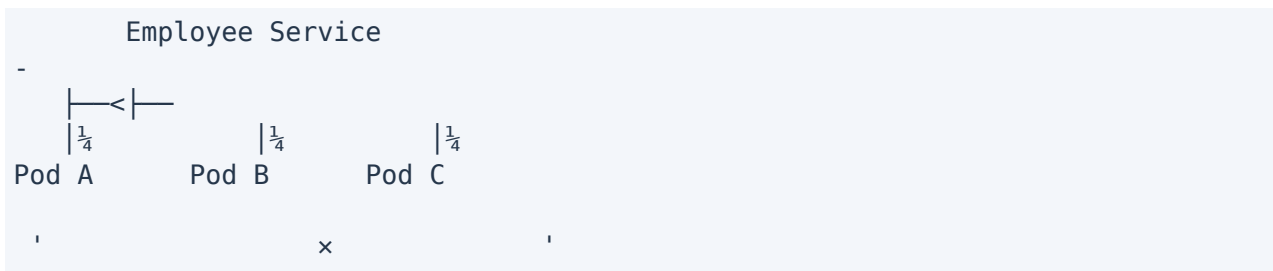
So:

```
Readiness failure
→ remove from traffic

Liveness failure
→ restart container
```

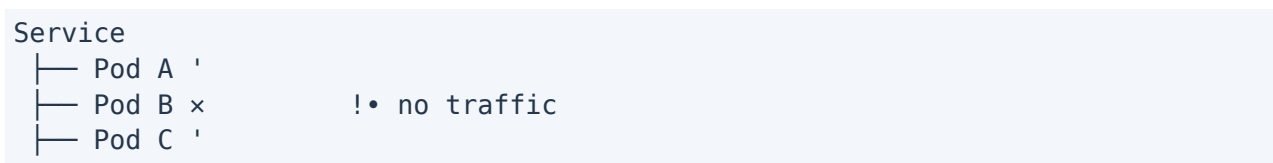

Visualize it

Imagine a Service:



Pod B's readiness probe fails.

Kubernetes effectively routes traffic:



Pod B may still be running.

Liveness vs Readiness

Probe Question Failure behavior

Liveness Is the application alive? Container may be restarted

Readiness Can it receive traffic? Removed from Service endpoints

Startup Has startup completed? Protects slow-starting applications

This table is worth remembering.

Startup Probe

Consider a large Spring Boot application.

Startup takes:

90 seconds

You configure liveness:

check every 10 seconds

Suppose liveness starts failing during startup.

Kubernetes might think:

Application unhealthy!

and restart it.

Then:

```
Application starts
→ liveness fails
→ restart
→ starts again
→ liveness fails
→ restart
```

You get a:

Restart Loop The application never gets enough time to finish starting.

| Startup Probe solves this

Startup probe says:

Don't start evaluating liveness/readiness in the normal way until the application has successfully started.

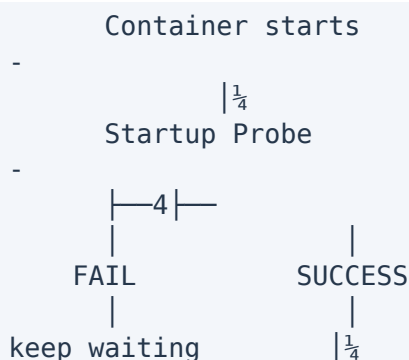
Conceptually:

```
Container starts
  ↓
Startup Probe
  ↓
FAIL
  ↓
Give application more time
  ↓
Startup Probe
  ↓
SUCCESS
  ↓
Liveness/Readiness take over
```

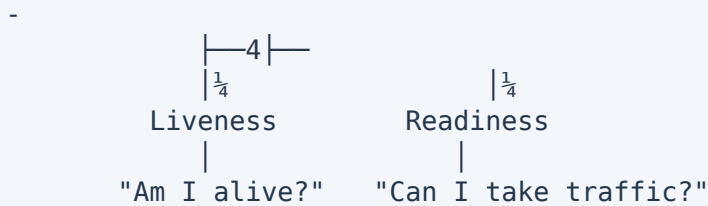
This is particularly useful for slow-starting applications.

| The relationship between the probes

Think about application lifecycle:



Liveness + Readiness



This is the mental model I want you to remember.

Probe mechanisms

Kubernetes can perform health checks in several ways.

Common ones:

```

HTTP GET
TCP Socket
gRPC
Exec command
  
```

Let's understand each.

HTTP Probe

Very common for Spring Boot.

Kubernetes can call:

```
GET /health
```

If the application responds successfully:

```
HTTP 200
```

the probe succeeds.

If it responds with an unhealthy status:

```
HTTP 500
```

the probe fails.

Architecture:

Kubernetes

```

-
  | HTTP GET
  | 1/4
  |
  | /health
  
```

```
-
  | ¼
Spring Boot
```

Example HTTP readiness probe

```
readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 5
```

Meaning roughly:

```
Wait 10 seconds
  ↓
check every 5 seconds
  ↓
GET /actuator/health/readiness
  ↓
port 8080
```

Don't memorize every field yet.

Understand the behavior.

Spring Boot Actuator

For Spring Boot applications, Spring Boot Actuator is commonly used to expose health endpoints.

You may have:

```
/actuator/health
```

and, when health groups are configured appropriately:

```
/actuator/health/liveness
/actuator/health/readiness
```

These provide Kubernetes-friendly health information.

Conceptually:

```
Spring Boot
-
  └─ Actuator
      └─ liveness
      └─ readiness
```

17. Why not use the same endpoint for everything?

Because:

Alive

and:

Ready

are different concepts.

For example:

```
Application process = healthy
Database connection = temporarily unavailable
```

You might want:

```
Liveness → SUCCESS
Readiness → FAIL
```

This means:

Don't kill my application just because the database is temporarily unavailable; but don't send user traffic to it. That can be a much better production behavior.

TCP Probe

Instead of HTTP, Kubernetes can check whether a TCP connection can be established.

Example:

```
livenessProbe:
  tcpSocket:
    port: 8080
```

Conceptually:

```
Kubernetes
  ↓
Can I establish TCP connection?
  ↓
Port 8080
```

Useful when the application doesn't expose a suitable HTTP health endpoint.

Exec Probe

Kubernetes can execute a command inside the container.

Example:

```
LivenessProbe:  
  exec:  
    command:  
      - cat  
      - /tmp/healthy
```

If the command exits successfully:

```
exit code 0
```

the probe succeeds.

Otherwise:

```
failure
```

This is useful for certain custom workloads, although HTTP endpoints are often cleaner for web applications.

| gRPC Probe

For gRPC applications, Kubernetes also supports gRPC health checking.

Conceptually:

```
Kubernetes  
  ↓  
gRPC health check  
  ↓  
Application
```

Useful for services built around gRPC.

| Probe timing

You will see configuration fields such as:

```
initialDelaySeconds  
periodSeconds  
timeoutSeconds  
failureThreshold  
successThreshold
```

Let's understand them.

| initialDelaySeconds

Example:

```
initialDelaySeconds: 20
```

Means:

Wait approximately 20 seconds before starting the probe.

Useful for simple applications that need predictable startup time.

However, for slow or variable startup, `startupProbe` is often a better mechanism than relying only on a large initial delay.

| periodSeconds

Example:

```
periodSeconds: 5
```

Means:

Perform the probe approximately every 5 seconds.

Conceptually:

```
10:00:00 → check  
10:00:05 → check  
10:00:10 → check  
10:00:15 → check
```

| timeoutSeconds

Example:

```
timeoutSeconds: 2
```

Means roughly:

Allow the probe operation this long before considering that individual check unsuccessful.

| failureThreshold

Suppose:

```
failureThreshold: 3
```

A few consecutive failed checks may be required before Kubernetes takes the corresponding failure action.

Conceptually:

```
Check 1 → FAIL
Check 2 → FAIL
Check 3 → FAIL
      ↓
    threshold reached
      ↓
    failure action
```

The exact lifecycle behavior depends on the probe type.

| successThreshold

This controls how many successful checks are required to consider a probe successful after failures, with behavior depending on probe type.

For many common liveness/startup configurations, the normal/default value is effectively one successful check.

Don't worry about memorizing the defaults now.

| Real Spring Boot example

Imagine:

```
Employee Service
Port: 8080
```

Endpoints:

```
/actuator/health/liveness
/actuator/health/readiness
```

Deployment:

```
containers:
- name: employee-service
  image: employee-service:1.0

  ports:
  - containerPort: 8080

  readinessProbe:
    httpGet:
      path: /actuator/health/readiness
      port: 8080

  livenessProbe:
    httpGet:
      path: /actuator/health/liveness
      port: 8080
```


Now Kubernetes understands two different things:

Readiness

→ should I send requests?

Liveness

→ should I restart the container?

| Add startup probe

Suppose startup sometimes takes 60 seconds.

We could add:

```
startupProbe:
  httpGet:
    path: /actuator/health
    port: 8080
  periodSeconds: 5
  failureThreshold: 20
```

Conceptually:

```
20 failures × 5 seconds
"H 100 seconds of startup allowance
```

During this startup-probe phase, liveness/readiness handling is coordinated so that a slow application isn't prematurely killed by liveness.

| A very important production scenario

Suppose:

```
Employee Service
  ↓
PostgreSQL
```

Database becomes temporarily unavailable.

What should happen?

Bad configuration:

```
Liveness checks DB
  ↓
DB unavailable
  ↓
Liveness FAIL
  ↓
Restart Employee Service
```

Then:

```
Restart
↓
DB unavailable
↓
Liveness FAIL
↓
Restart
```

You can create a restart storm.

A better design is often:

```
Liveness
→ Is the application process fundamentally alive?

Readiness
→ Can this instance currently serve requests?
```

If DB dependency is unavailable:

```
Readiness FAIL
Liveness SUCCESS
```

Then:

```
Service stops routing normal traffic
```

without necessarily restarting the application.

This is one of the most important real-world lessons.

Probe vs application health

Kubernetes doesn't magically understand your application.

It only knows what your probe tells it.

Suppose:

```
Pod = Running
```

doesn't mean:

```
Application = healthy
```

Similarly:

```
Pod = Ready
```

means the configured readiness condition is currently successful; it does not guarantee that every possible business operation will succeed.

So probes are signals, not magic.

Pod lifecycle vs probe status

A Pod can be:

```
Running
```

while:

```
Ready = False
```

For example:

```
Pod
├── Running
└── Readiness = False
```

This is completely normal during startup.

You may see:

```
READY
0/1
```

even though:

```
STATUS
Running
```

That is why beginners sometimes get confused.

Example

Run:

```
kubectl get pods
```

You might see:

NAME	READY	STATUS
employee-abc	1/1	Running
employee-def	0/1	Running
employee-ghi	1/1	Running

What does that mean?

```
employee-abc
→ Running + Ready
```

```
employee-def
→ Running but NOT Ready
```

```
employee-ghi
→ Running + Ready
```

The second Pod may still be starting or may have a readiness problem.

| Why Service depends on readiness

Remember our Service architecture:

```
Client
  ↓
Service
  ↓
Pods
```

The Service should normally send traffic to healthy/ready endpoints.

So:

```
Pod A → Ready
Pod B → Not Ready
Pod C → Ready
```

means traffic should effectively target:

```
A + C
```

rather than sending normal Service traffic to B.

| Deployment rollout and readiness

This becomes extremely important during deployments.

Suppose you have:

```
Version 1
Pod A
Pod B
Pod C
```

You deploy:

```
Version 2
```

A new Pod starts:

```
Pod D = v2
```

If Pod D is:

```
Running
but
Not Ready
```

Kubernetes shouldn't immediately consider it a healthy replacement for serving traf-

fic.

Once:

```
Readiness = SUCCESS
```

it can participate in Service traffic.

This is one reason readiness probes are critical for zero/low-downtime rolling deployments.

| Readiness during shutdown

There is another subtle but important scenario.

Suppose Kubernetes wants to terminate:

```
Pod A
```

You don't want new requests arriving while the application is shutting down.

Kubernetes handles endpoint/termination behavior as part of Pod termination, and well-designed applications also support graceful shutdown.

Conceptually:

```
Pod starts termination
      ↓
stop accepting new traffic
      ↓
finish in-flight requests
      ↓
application exits
```

This becomes especially important for Spring Boot services.

We'll cover graceful shutdown more deeply when we study deployments and rolling updates.

| Liveness can actually be dangerous if badly designed

Imagine:

```
livenessProbe:
  check database
```

Database has a 30-second outage.

Then:

```
App healthy enough to recover
      ↓
Liveness FAIL
```

```
    ↓
Restart
    ↓
Reconnect attempt
    ↓
Liveness FAIL
    ↓
Restart
```

You can make an external dependency outage worse by repeatedly restarting every application.

Rule of thumb:

Liveness should generally represent whether the process/application itself is irrecoverably stuck, not simply whether every dependency is currently available.

Readiness can include dependencies

Readiness is often more appropriate for:

```
Database availability
Required downstream service
Critical initialization
Application ability to serve requests
```

For example:

```
DB unavailable
  ↓
Readiness FAIL
  ↓
No traffic
```

while:

```
Liveness SUCCESS
```

allows the application to remain alive and potentially recover when the dependency returns.

The exact health-group design should match the application.

Startup is different from readiness

This distinction is subtle.

Startup

```
"I'm still starting."
```

Readiness

```
"I'm started, but can I serve traffic?"
```

Example:

```
0-60 sec:  
Startup = not complete  
  
60-65 sec:  
Startup complete  
Readiness = false  
because cache warming is happening  
  
65 sec:  
Readiness = true
```

That is perfectly valid.

Real application lifecycle

Imagine a complex Java application:

```
Container starts  
↓  
JVM starts  
↓  
Spring Boot starts  
↓  
Beans initialized  
↓  
Database connection established  
↓  
Kafka consumer initialized  
↓  
Cache warmed  
↓  
Application ready
```

Different probes can represent different stages:

```
Startup  
→ JVM/Spring startup completed  
  
Readiness  
→ application can serve production requests  
  
Liveness  
→ application isn't stuck
```

How Kubernetes reacts

Here's the simplified behavior:

```
Startup Probe FAIL
→ keep waiting / eventually restart if threshold exhausted

Startup Probe SUCCESS
→ normal liveness/readiness evaluation can proceed

Liveness FAIL repeatedly
→ restart container

Readiness FAIL
→ don't route normal Service traffic to Pod
```

That is the core behavior.

41. What happens when liveness fails?

Suppose:

```
Pod
↓
Container
↓
Liveness probe repeatedly fails
```

Kubernetes can restart the container according to the Pod's restart policy and kubelet behavior.

This doesn't necessarily mean:

```
entire Pod object deleted
```

It's important to distinguish:

```
Pod lifecycle
```

from:

```
Container restart
```

Container restart vs Pod replacement

Example:

```
Pod
├─ Spring Boot Container
```

Liveness failure may result in:


```
Spring Boot Container
      ↓
restart
```

The Pod can remain the same Pod object.

But if a controller needs to replace the Pod for other reasons, you may see an entirely new Pod.

This distinction will help you understand Kubernetes troubleshooting later.

| How to troubleshoot probes

Start with:

```
kubectl get pods
```

Look at:

```
READY
STATUS
RESTARTS
```

Then:

```
kubectl describe pod <pod-name>
```

Look at:

```
Events
```

You may see messages indicating:

```
Readiness probe failed
Liveness probe failed
Startup probe failed
```

| Check logs

If liveness is failing:

```
kubectl logs <pod-name>
```

If the container has restarted:

```
kubectl logs <pod-name> --previous
```

The `--previous` option is extremely useful because it can show logs from the previous container instance.

| Test the health endpoint yourself

If your application exposes:

```
/actuator/health
```

you can test it from inside the Pod or through appropriate networking/debugging methods.

For example:

```
kubectl exec -it <pod-name> -- sh
```

Then potentially:

```
curl http://localhost:8080/actuator/health
```

Now you can compare:

```
What Kubernetes sees
```

with:

```
What the application actually returns
```

| Common mistake #1

“The Pod is Running, so why isn’t my Service receiving traffic?” Check:

```
Readiness
```

You may have:

```
STATUS = Running  
READY = 0/1
```

The application is alive but not ready.

| Common mistake #2

“My application keeps restarting.” Check:

```
Liveness probe
```

Run:

```
kubectl describe pod <pod>
```

and inspect:

Events

Then:

```
kubectll logs <pod> --previous
```

Possible problem:

Liveness endpoint failing

| Common mistake #3

“My application takes 90 seconds to start but Kubernetes keeps restarting it.” Likely issue:

Liveness probe starts too early

Solution may involve:

Startup Probe

rather than simply making arbitrary timing values larger.

| Common mistake #4

“Database is down and all my application Pods keep restarting.” Check whether:

Liveness Probe

is incorrectly dependent on the database.

Often:

```
DB unavailable  
→ Readiness FAIL
```

is preferable to:

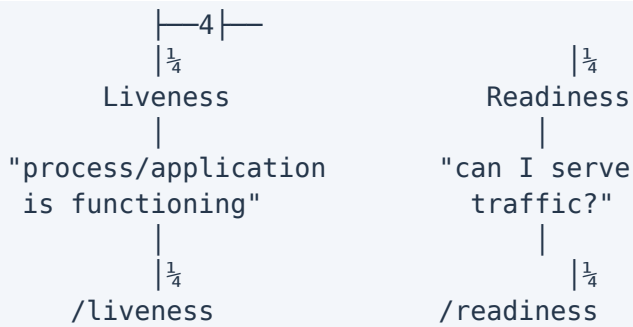
```
DB unavailable  
→ Liveness FAIL  
→ restart everything
```

depending on application architecture.

| Spring Boot production pattern

A reasonable conceptual pattern is:

```
Spring Boot  
-
```



And:

```

Startup Probe
  ↓
protects slow startup
  
```

The exact Actuator health groups and dependency inclusion should be deliberately configured rather than blindly exposing every dependency through every probe.

A complete Deployment example

Here's a more realistic example:

```

apiVersion: apps/v1
kind: Deployment

metadata:
  name: employee-service

spec:
  replicas: 3

  selector:
    matchLabels:
      app: employee-service

  template:
    metadata:
      labels:
        app: employee-service

    spec:
      containers:
        - name: employee-service
          image: employee-service:1.0

          ports:
            - containerPort: 8080

          startupProbe:
            httpGet:
              path: /actuator/health
  
```

```

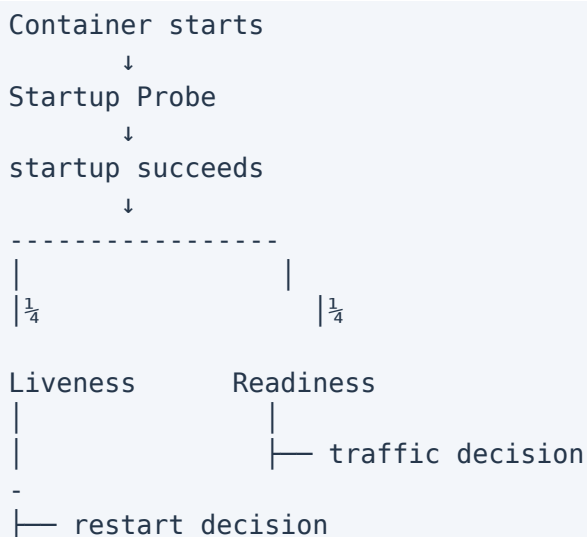
    port: 8080
    periodSeconds: 5
    failureThreshold: 20

    readinessProbe:
      httpGet:
        path: /actuator/health/readiness
        port: 8080
        periodSeconds: 5

    livenessProbe:
      httpGet:
        path: /actuator/health/liveness
        port: 8080
        periodSeconds: 10

```

Conceptually:



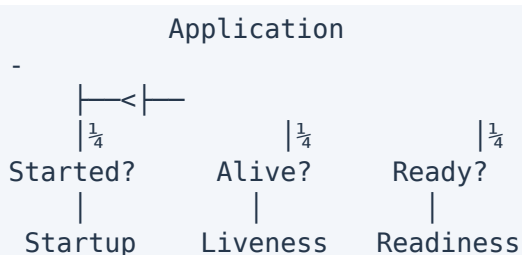
One very important mental model

Don't think:

Healthy

is one single state.

Instead think:



These answer different operational questions.

Quick scenario test

Scenario A Application is still starting.

```
Startup = FAIL
```

What should happen?

Wait for startup, rather than immediately treating it as a normal healthy application.

Scenario B Application is alive but database is temporarily unavailable.

Possible desired state:

```
Liveness = SUCCESS  
Readiness = FAIL
```

Result:

```
Don't send normal traffic  
Don't unnecessarily restart application
```

Scenario C Application has become completely stuck.

```
Liveness = FAIL
```

After configured failure threshold:

```
Container restart
```

Scenario D Application is healthy and ready.

```
Startup = SUCCESS  
Liveness = SUCCESS  
Readiness = SUCCESS
```

Result:

```
Pod can serve traffic normally.
```

The three questions you should always ask

Whenever you deploy an application to Kubernetes, ask:

Question 1

How do I know it has finished starting?

→ Startup Probe

Question 2

How do I know it is alive and not stuck?

→ Liveness Probe

Question 3

How do I know it can currently serve user requests?

→ Readiness Probe

If you can answer these three questions, you understand the purpose of Kubernetes probes.

chapter 11 Summary

The most important table:

Probe Meaning Typical Kubernetes action

Startup Has the app finished starting? Protects slow startup; failure can lead to restart

Liveness Is the app alive/not stuck? Repeated failure can restart container

Readiness Can the app serve traffic? Failure removes it from normal Service traffic

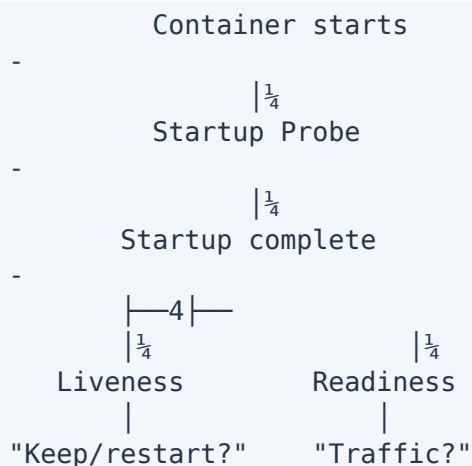
And remember this:

```
Pod Running
  " \
Application Ready
```

and:

```
Readiness failure
  " \
Liveness failure
```

The production mental model:



Next chapter — chapter 12: Kubernetes Networking Now we✓I connect Pods, Ser-

vices, DNS and Ingress/Routes and finally answer a question that confuses almost every Kubernetes beginner:

If every Pod gets its own IP, why do we need a Service?

We✓I build this architecture step by step:

```
Internet
  ↓
Ingress / OpenShift Route
  ↓
Service
  ↓
Pod A
Pod B
Pod C
```

Then we✓I understand:

```
Pod IP
Service IP
ClusterIP
NodePort
LoadBalancer
DNS
kube-proxy
CoreDNS
Ingress
OpenShift Route
```

This is the point where Kubernetes starts to feel like a complete platform rather than just a system that creates containers.

Yes. Based on the conversation history I can see, we have already covered Kubernetes networking concepts earlier, including the basics around:

- Pod networking / Pod IPs
- Services
- ClusterIP
- DNS/service discovery
- Ports and how traffic reaches Pods
- Volumes/PVCs and related Kubernetes fundamentals

So we shouldn't repeat Networking as chapter 12.

Our recent progression has reached:

Kubernetes fundamentals

- | **Pods**
- | **Deployments / ReplicaSets**
- | **Services**
- | **ConfigMaps & Secrets**
- | **Volumes / PV / PVC**
- | **Storage provisioning**
- | **StatefulSets**
- | **DaemonSets**
- | **Health checks / Probes**

And your latest completed topic was chapter 11 — Liveness, Readiness & Startup Probes.

I can continue from the next logical chapter after networking, rather than making you go through networking again.

teaching style.

Absolutely. Since we've already covered the networking fundamentals, let's move to the next logical production concept.

Chapter 15 — Graceful Shutdown and Traffic Draining

CHAPTER FOCUS

Understand **Graceful Shutdown and Traffic Draining** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

Lifecycle Hooks This chapter answers a production problem that is easy to miss:

A Pod is serving a customer request. Kubernetes decides to terminate that Pod during a rollout, scale-down, or Node drain. What happens to the request already running?

A badly terminated Pod can create:

```
Deployment looks successful '
Pods eventually healthy '

but users see occasional
502 / 503 / connection reset x
```

The solution is graceful termination.

1. When does a Pod get terminated?

Many normal Kubernetes operations can terminate a Pod:

```
Deployment rolling update
Scale down
kubectl delete pod
Node drain
HPA scale-down
Eviction
Job completion
Pod replacement
```

For example:

```
employee-service v1
3 Pods

      ↓ rollout

employee-service v2
3 new Pods

      ↓
```

```
old v1 Pods must terminate
```

Termination is not necessarily an error. It is a normal part of Kubernetes.

2. What would happen without graceful shutdown?

Imagine this request:

```
Client
  ↓
employee-service Pod
  ↓
processing payment...
```

The request takes:

```
8 seconds
```

At second 4 Kubernetes kills the process immediately.

Result:

```
Request starts
  ↓
business logic running
  ↓
Pod killed abruptly
  ↓
connection closed
  ↓
user gets error
```

Potential consequences are worse than just an HTTP error:

```
half-completed work
abandoned transactions
messages not acknowledged
connections interrupted
temporary 502/503
```

So Kubernetes tries to give the application time to shut down properly.

The big picture

A simplified graceful termination looks like:

```
Pod termination requested
  ↓
Pod marked Terminating
  ↓
Grace-period timer starts
```

```

↓
Service endpoint begins terminating
↓
preStop hook, if configured
↓
SIGTERM / configured stop signal
↓
Application begins graceful shutdown
↓
finish existing work
↓
process exits

```

If the application does not exit before the grace period expires:

```

Grace period expires
↓
SIGKILL
↓
process forcibly killed

```

Kubernetes' default Pod termination grace period is 30 seconds unless you configure another value.

Let's understand every step

Suppose you run:

```
kubectl delete pod employee-abc
```

or a Deployment decides that an old Pod must disappear.

The API server marks the Pod for deletion.

You may see:

NAME	STATUS
employee-abc	Terminating

The Pod is not immediately destroyed.

Instead Kubernetes records:

```

deletionTimestamp
+
termination grace period

```

Then the kubelet on that Node starts the shutdown process.

terminationGracePeriodSeconds

You configure it in the Pod template:

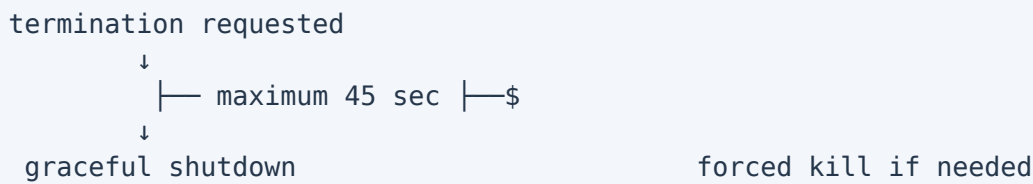
```
spec:
  terminationGracePeriodSeconds: 45

  containers:
    - name: employee-service
      image: employee-service:2.0
```

Meaning approximately:

Give this Pod up to 45 seconds to terminate gracefully before remaining processes are forcibly killed.

So:



6. What happens when the grace period expires?

Suppose:

```
terminationGracePeriodSeconds = 30
```

but your Java process takes:

```
45 seconds
```

to stop.

Kubernetes won't wait forever.

Conceptually:

```

0 sec
↓
termination starts

20 sec
↓
Java still shutting down

30 sec
↓
grace period exhausted

SIGKILL
↓
process forcibly terminated
```

So:

terminationGracePeriodSeconds must be long enough for your application's legitimate shutdown work.

SIGTERM vs SIGKILL

These two Linux signals are very important.

SIGTERM Think:

Please terminate gracefully.

The application has an opportunity to:

```
stop accepting requests
finish current requests
close DB connections
flush buffers
stop background workers
close Spring context
```

SIGKILL Think:

Terminate immediately. No cleanup opportunity.

Conceptually:

```
SIGTERM
→ request to stop
```

```
SIGKILL
→ force stop
```

After the grace period expires, Kubernetes/container runtime sends SIGKILL to processes still running.

8. Who receives SIGTERM?

Normally the container runtime sends the stop signal to:

```
PID 1
```

inside the container.

For your typical Spring Boot container:

```
Container
├─ java -jar employee.jar
   │
   └─ PID 1
```

Then:

```

SIGTERM
  ↓
Java process
  ↓
Spring Boot shutdown

```

With CRI-O and containerd, the default stop signal is SIGTERM when another stop signal hasn't been configured. Container-image STOPSIGNAL and newer Kubernetes stop-signal options can alter this behavior.

For your learning, the normal mental model remains:

```

Kubernetes
  ↓
SIGTERM
  ↓
Spring Boot

```

Spring Boot graceful shutdown

This is where Kubernetes and Spring Boot meet.

On receiving proper shutdown, Spring Boot can stop accepting new requests and give active requests time to finish before completing application-context shutdown.

Current Spring Boot documentation describes graceful shutdown as the default for its main embedded server implementations; it also provides `spring.lifecycle.timeout-per-shutdown-phase` to control how long a shutdown phase may take.

A deliberate configuration could look like:

```

server:
  shutdown: graceful

spring:
  lifecycle:
    timeout-per-shutdown-phase: 20s

```

Now conceptually:

```

SIGTERM
  ↓
Spring Boot
  ↓
stop accepting new requests
  ↓
allow existing requests to finish
  ↓
close application context
  ↓
Java exits

```

Kubernetes timeout and Spring timeout must align

This is extremely important.

Suppose Kubernetes:

```
terminationGracePeriodSeconds: 30
```

Spring:

```
spring:
  lifecycle:
    timeout-per-shutdown-phase: 40s
```

That is a bad mismatch.

Spring may think:

```
"I have up to 40 seconds."
```

while Kubernetes says:

```
"You have only 30."
```

So:

```
Spring still shutting down
      ↓
30 sec reached
      ↓
SIGKILL
```

A better design leaves Kubernetes enough time for all shutdown activities.

For example:

preStop budget	= 5 sec
Spring shutdown	= up to 20 sec
extra safety margin	= 10 sec
termination grace	= 35+ sec

Don't memorize those numbers; size them for the application.

11. What happens to Service traffic?

This is another very important part.

Suppose Service endpoints are:

```
employee-service
-
├─ Pod A
├─ Pod B
└─ Pod C
```


Pod C starts terminating.

Kubernetes does not want ordinary new Service traffic continually sent to Pod C.

Current EndpointSlice behavior represents a terminating endpoint like:

```
terminating = true
ready       = false
```

and regular load balancing should stop using it as a normal ready endpoint. Kubernetes can retain the endpoint temporarily with `serving=true` for consumers that intentionally support connection draining.

Conceptually:

Before:

Service

```
|— A '  
|— B '  
|— C '
```

Termination starts:

Service

```
|— A '  
|— B '  
|— C terminating
```

New normal traffic should shift toward:

```
A  
B
```

while C finishes existing work.

| Endpoint removal and process shutdown happen concurrently

One subtle point:

Don't imagine the process as strictly:

1. Remove Pod completely from every network system
2. Wait
3. Then start application shutdown

Kubernetes starts the Pod's local graceful shutdown while the control plane also updates its Service/EndpointSlice state.

This matters because real networking involves:

```
EndpointSlice
kube-proxy / service dataplane
Ingress/router
external load balancer
existing TCP connections
```

These systems may not all converge at precisely the same nanosecond.

That is why graceful application behavior matters.

| preStop lifecycle hook

Now we reach lifecycle hooks.

Kubernetes allows:

```
lifecycle:
  preStop:
    ...
```

A preStop hook runs immediately before normal container termination.

Importantly:

The termination grace-period countdown has already started before preStop runs, and the stop signal isn't sent until the hook completes.

So:

```
Termination starts
  ↓
grace timer starts
  ↓
preStop
  ↓
SIGTERM
  ↓
application shutdown
```

| Example preStop

```
spec:
  terminationGracePeriodSeconds: 40

  containers:
    - name: employee-service
      image: employee-service:2.0

      lifecycle:
        preStop:
          exec:
```

```
command:
  - /bin/sh
  - -c
  - sleep 5
```

Conceptually:

```
Termination requested
↓
Pod becomes terminating
↓
Service endpoint transitions away from normal traffic
↓
sleep 5
↓
SIGTERM
↓
Spring graceful shutdown
```

15. Why would anyone deliberately sleep?

At first this looks strange:

```
Pod terminating
↓
sleep 5
```

Why waste five seconds?

Because in some architectures, teams deliberately leave a small period for routing/load-balancer changes to propagate

before asking the application process to stop.

Conceptually:

```
Pod marked terminating
↓
EndpointSlice updated
↓
Service / proxy / ingress / external LB catches up
↓
few seconds
↓
application starts stopping
```

That can reduce a race where new traffic reaches a process that has already closed its listener.

But:

Don't blindly add sleep 10 to every application.

Whether it's useful depends on your networking/load-balancing setup. Kubernetes

now explicitly exposes terminating/serving EndpointSlice conditions for more sophisticated connection draining.

| preStop consumes the same grace period

Very important.

Suppose:

```
terminationGracePeriodSeconds = 30
```

and:

```
preStop = 20 seconds
```

Then your application doesn't suddenly receive another 30 seconds.

Roughly:

```
Total budget = 30 sec
preStop consumes 20 sec
      ↓
roughly 10 sec remains
      ↓
SIGTERM
      ↓
Java has only remaining budget
```

The Kubernetes docs explicitly note that the grace period covers both the preStop hook and normal container shutdown.

So this is wrong thinking:

```
30 sec preStop
+
30 sec shutdown
=
60 sec
```

No.

They share the same budget.

17. What if preStop takes too long?

Suppose:

```
grace = 30 sec
preStop = 28 sec
Java shutdown needs = 10 sec
```

There isn't enough time.

Kubernetes documents a small one-off extension when `preStop` is still executing as the grace period expires, but you should not design your application around that emergency allowance. Configure a sufficient grace period instead.

18. Do we always need `preStop` ?

No.

This is important.

If:

```
Service endpoint behavior works correctly
+
Spring Boot handles SIGTERM gracefully
+
load balancers drain connections properly
```

you may not need a custom `preStop` .

Use it when you need explicit application-specific behavior such as:

```
notify another system
stop consuming jobs
deregister from something external
wait for external LB propagation
perform an application-specific shutdown step
```

Don't add hooks without understanding why.

19. What should NOT go in `preStop` ?

Avoid huge business operations such as:

```
backup entire database
process 5000 pending jobs
run 15-minute data migration
```

Why?

Pod termination must eventually complete.

`preStop` is for termination coordination, not unlimited batch processing.

Example: message consumer

HTTP services aren't the only case.

Suppose your Spring application consumes Kafka messages.

It receives:

```
message A
```

and starts processing it.

Then Kubernetes terminates the Pod.

A graceful application may need to:

```
stop fetching new messages
finish/abort current work safely
commit offsets appropriately
close consumer
exit
```

rather than:

```
SIGKILL
↓
message processing disappears halfway through
```

The exact semantics depend on the messaging system and application design.

Deployment rolling update

Now connect this to Deployments.

Suppose:

```
v1:
Pod A
Pod B
Pod C
```

You're releasing v2.

Kubernetes creates:

```
Pod D v2
```

D becomes:

```
Ready '
```

Then an old Pod can terminate:

```
Pod A v1
↓
Terminating
↓
stop normal new traffic
↓
finish existing traffic
↓
```

```
exit
```

Meanwhile:

```
B  
C  
D
```

continue serving.

This is how:

```
RollingUpdate  
+  
readiness  
+  
graceful shutdown
```

work together to reduce downtime.

| **maxUnavailable also matters**

Imagine:

```
replicas = 3
```

and your Deployment allows too many unavailable Pods while shutdown/startup are slow.

Availability could drop.

So production rollout safety is really a combination of:

```
RollingUpdate strategy  
+  
maxUnavailable / maxSurge  
+  
readinessProbe  
+  
startupProbe  
+  
termination behavior  
+  
application graceful shutdown
```

No single field guarantees zero downtime.

| **Node drain connection**

Remember:

```
kubectl drain worker-3
```

The Node's appropriate Pods are evicted.

For each Pod:

```
Eviction
  ↓
graceful Pod termination
  ↓
controllers create/reposition replacement
```

Therefore graceful shutdown also matters during:

```
cluster maintenance
node upgrades
OpenShift upgrades
```

not just application deployments.

HPA scale-down connection

We just learned HPA.

Suppose:

```
8 Pods
```

traffic decreases.

HPA changes:

```
8 → 4
```

Four Pods now need to terminate.

If termination is bad:

```
HPA scale-down
  ↓
active requests dropped
```

So autoscaling also depends on graceful termination.

Everything is connected.

PDB does not replace graceful termination

Remember PDB from chapter 37.

PDB answers:

How many Pods may be voluntarily disrupted simultaneously?

Graceful shutdown answers:

Once this particular Pod is being terminated, how should it exit safely?

So:

PDB

→ controls disruption quantity

Graceful shutdown

→ controls termination quality

Different problems.

| Readiness vs graceful shutdown

Another important distinction:

Readiness

Should I receive normal traffic?

Graceful shutdown

I am leaving. How do I safely finish what I'm already doing?

They complement each other:

Pod healthy

↓

Ready=true

↓

traffic

Termination

↓

stop being normal endpoint

↓

finish in-flight work

↓

exit

| Lifecycle hooks

Kubernetes has container lifecycle hooks such as:

PostStart

PreStop

and current Kubernetes also has stop-signal configuration capabilities.

For this chapter, PreStop is the most operationally important.

| PostStart

A PostStart hook executes around container startup.

Conceptually:

```
container starts
  ↓
PostStart hook
```

It can be useful for special initialization actions.

But don't confuse it with:

```
initContainer
```

| Init container vs PostStart

Init container

```
init container
  ↓
must complete
  ↓
main application containers start
```

PostStart Runs in association with the container's startup and is not the same ordered precondition mechanism as an init container.

So if something absolutely must happen before your Spring Boot container starts:

```
initContainer
```

is generally the clearer mental model.

| The PID 1 problem

This is particularly relevant for container images.

Good Dockerfile:

```
DOCKERFILE
```

```
ENTRYPOINT ["java", "-jar", "employee.jar"]
```

Then:

```
java process
=
PID 1
```

Signals reach Java naturally.

Potentially problematic wrapper:

DOCKERFILE

ENTRYPOINT ["sh", "-c", "java -jar employee.jar"]

Now:

```
PID 1
=
=
child process
```

Depending on how the shell/script handles signals, SIGTERM propagation can become problematic.

So container entrypoint design matters for graceful shutdown.

A common good principle:

Let your actual application process receive the container's termination signal correctly, or make sure the wrapper uses proper signal forwarding / `exec`.

Better shell entrypoint

If you need a script:

SH

```
#!/bin/sh
```

```
exec java -jar employee.jar
```

`exec` replaces the shell process with Java.

Conceptually:

```
↓ exec
Java becomes PID 1
```

Then termination signals reach the intended process.

Force deletion is different

You may see:

```
kubectl delete pod employee-abc --grace-period=0 --force
```

Conceptually:

```
"Don't give this workload the normal graceful shutdown opportunity."
```

This is much more aggressive.

It should not be your normal application rollout strategy.

Force deletion is an operational tool for specific failure/recovery situations, not a substitute for proper lifecycle handling.

Example Spring Boot production configuration

Conceptually:

```
apiVersion: apps/v1
kind: Deployment

spec:
  replicas: 3

  template:
    spec:
      terminationGracePeriodSeconds: 40

      containers:
        - name: employee-service
          image: employee-service:2.7

          lifecycle:
            preStop:
              exec:
                command:
                  - /bin/sh
                  - -c
                  - sleep 5

          readinessProbe:
            httpGet:
              path: /actuator/health/readiness
              port: 8080
```

Spring:

```
server:
  shutdown: graceful

spring:
  lifecycle:
    timeout-per-shutdown-phase: 25s
```

Conceptually:

40 sec Kubernetes total budget

- └─ maybe ~5 sec routing/preStop
- └─ up to ~25 sec application shutdown
- └─ remaining safety margin

Again, these numbers are examples, not universal recommendations.

Practical timeline

Let's say a user request takes 8 seconds.

At:

T=0

request reaches Pod A.

At:

T=2

Deployment decides to terminate Pod A.

Correct behavior can look like:

T=2

Pod becomes terminating

T=2

Endpoint becomes non-ready for normal new traffic

T=2-5

preStop / routing propagation

T=5

SIGTERM

Spring starts graceful shutdown

existing request continues

T=8

request completes successfully

Spring closes resources

T=10

process exits

User never notices that the Pod was replaced.

That is the goal.

Bad termination timeline

Now badly configured:

grace period = 5 sec

but request needs 10 sec.

```
T=0 request starts
T=1 termination starts
T=5/6 grace exhausted
SIGKILL
```

Request:

```
× interrupted
```

Potential user symptom:

```
connection reset
502
503
timeout
```

depending on where the connection is terminated and what proxies are involved.

36. Why would 502/503 happen only during deployments?

This is a classic production symptom:

```
Normal operation:
everything fine
```

```
During rollout:
occasional 502/503
```

```
After rollout:
everything fine again
```

Strong places to investigate:

```
readiness behavior
termination grace period
application graceful shutdown
Ingress/Route connection draining
preStop behavior
rolling-update parameters
```

This is much more useful than:

```
"Restart the pods."
```

| OpenShift Route connection

For OpenShift:

```
Client
↓
OpenShift Router
↓
```

```
Route
↓
Service
↓
Pod
```

When Pod A terminates, the goal is still:

```
stop routing new normal traffic
      ↓
let Pod A finish suitable existing traffic
      ↓
terminate
```

OpenShift doesn't eliminate the need for correct application shutdown.

Spring Boot still needs to handle termination correctly.

38. What about long-running requests?

Suppose API:

```
GET /generate-report
```

takes:

```
2 minutes
```

but:

```
terminationGracePeriodSeconds = 30
```

Now you have an architectural question.

Do you really want HTTP requests taking 2 minutes?

Maybe this operation should instead be:

```
request report generation
      ↓
submit asynchronous job
      ↓
return job ID
      ↓
worker processes report
      ↓
client retrieves result
```

Graceful shutdown isn't a replacement for better application architecture.

| Background threads

Your Spring Boot application might have:

```
@Scheduled jobs
ExecutorService
Kafka consumers
async tasks
batch workers
```

Graceful shutdown should account for them too.

It's not enough to think only about HTTP.

Question:

```
When SIGTERM arrives,
what happens to every piece of active work?
```

That is an application-design responsibility.

| Database transactions

Suppose:

```
Begin DB transaction
      ↓
update employee
      ↓
Pod receives SIGTERM
```

A well-designed application/framework should finish or safely roll back active transactional work according to its lifecycle and timeout behavior.

You don't want arbitrary:

```
SIGKILL
```

halfway through everything.

Databases themselves provide transactional safety, but your application still needs sensible shutdown behavior.

| Kubernetes does not know your business transaction

This distinction is essential.

Kubernetes knows:

```
Pod
Container
Process
Grace period
```


It does not know:

```
"Employee payroll transaction #123 is 70| complete."
```

So Kubernetes can only provide:

```
time
+
signals
+
lifecycle hooks
+
traffic-state coordination
```

Your application decides how to use that opportunity.

Troubleshooting graceful shutdown

If rollout causes errors:

Check Pod termination events

```
kubectl describe pod <pod>
```

Check application logs Look for Spring logs such as graceful-shutdown activity.

Current Spring Boot logs can show messages indicating that graceful shutdown has begun and completed.

Check configured grace period

```
kubectl get pod <pod> -o yaml
```

Inspect Deployment

```
kubectl get deployment employee-service -o yaml
```

Check:

```
terminationGracePeriodSeconds
lifecycle.preStop
readinessProbe
strategy
```

Check how long shutdown actually takes

Don't guess.

Measure:

```
SIGTERM received
      ↓
application stopped
```

Suppose:

```
P95 shutdown time = 12 sec
P99 shutdown time = 18 sec
```

Then a:

```
5-second grace period
```

is obviously inappropriate.

Like resource sizing, termination settings should be informed by real behavior.

| Common mistake #1

Setting:

```
terminationGracePeriodSeconds: 1
```

because:

“I want deployments to be faster.”

This can simply convert:

```
slow graceful rollout
```

into:

```
fast broken rollout
```

Deployment speed should not come from killing active work prematurely.

| Common mistake #2

Setting:

```
terminationGracePeriodSeconds: 600
```

without understanding why.

Now broken Pods can take an unnecessarily long time to disappear during:

```
rollouts
node drains
scale-down
```

Too high can also hurt operations.

So choose a realistic budget.

| Common mistake #3

Very long preStop :

```
preStop sleep 30
grace = 30
```

Then practically no time remains for Spring graceful shutdown.

Remember:

```
preStop
+
application shutdown
```

share the grace-period budget.

| Common mistake #4

Assuming readiness alone guarantees graceful termination.

Readiness answers:

```
Should new normal traffic be sent here?
```

It does not automatically guarantee:

```
existing requests finish
background jobs stop safely
resources close correctly
```

You need application termination support too.

| Common mistake #5

Application ignores/doesn't correctly receive SIGTERM.

Then Kubernetes does:

```
SIGTERM
↓
nothing useful happens
↓
wait grace period
↓
SIGKILL
```

So correct signal propagation is fundamental.

49. Interview question — What happens when a Pod is deleted?

A strong answer:

Kubernetes marks the Pod terminating and starts its configured termination grace period. The kubelet executes a preStop hook if configured, then asks the container runtime to stop the container, normally with SIGTERM. At the same time, Service endpoint state transitions so the terminating Pod is no longer a regular ready destination. The application gets time to shut down gracefully. If processes remain after the grace period, they are forcibly terminated with SIGKILL.

50. Interview question — preStop vs SIGTERM?

```
preStop
→ Kubernetes lifecycle hook
→ executes before stop signal

SIGTERM
→ signal sent to application process
→ asks process to terminate
```

The hook must complete before the normal stop signal is sent, and both consume the same termination grace period.

51. Interview question — Does preStop get extra time?

No.

```
terminationGracePeriodSeconds
```

is the overall termination budget.

Conceptually:

```
Grace period
├─ preStop
├─ application shutdown
```

not:

```
Grace period
+
preStop duration
```

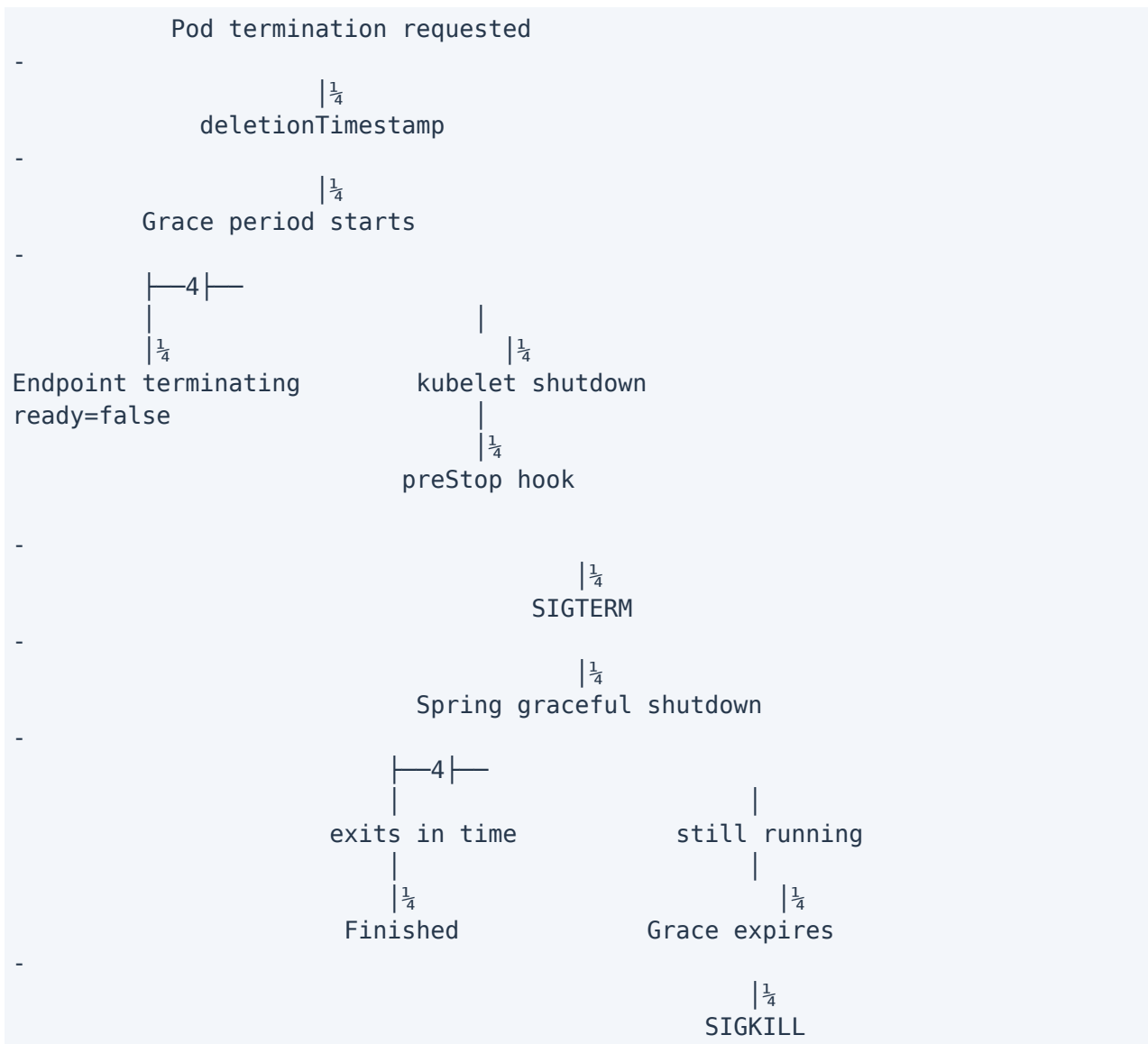
52. Interview question — Why is graceful shutdown important?

Good answer:

It allows a terminating Pod to stop receiving normal new traffic while giving active requests and application resources a chance to finish or close cleanly, reducing dropped requests and inconsistent behavior during rollouts, scaling and Node maintenance.

Full lifecycle mental model

Keep this diagram:



That is chapter 23 in one picture.

You have now filled:

21 HPA / Autoscaling '
23 Graceful Shutdown '

The remaining chapter you specifically asked to revisit is:

Chapter 16 — CPU, Memory, Requests, Limits, and QoS

CHAPTER FOCUS

Understand **CPU, Memory, Requests, Limits, and QoS** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

This chapter connects several things you’ve already learned:

```
Scheduler
+
LimitRange
+
ResourceQuota
+
Java/JVM
+
Troubleshooting
```

The central question is:

How does Kubernetes decide how much CPU and memory a Pod gets, and what happens when the application tries to use more?

Start with the real-world problem

Imagine one worker Node:

```
Node
CPU    = 8 cores
Memory = 16 GiB
```

Three Spring Boot applications run there:

```
employee-service
payment-service
report-service
```

Without resource controls, imagine report-service suddenly does a huge calculation:

```
report-service
  ↓
CPU → 100|
Memory → keeps increasing
```

It could hurt:

```
employee-service  
payment-service
```

This is called the noisy-neighbor problem.

Kubernetes therefore lets us define:

```
requests  
limits
```

| Basic YAML

A typical Spring Boot container might have:

```
resources:  
  requests:  
    cpu: "500m"  
    memory: "512Mi"  
  
  limits:  
    cpu: "1"  
    memory: "1Gi"
```

At first glance this looks simple.

But CPU and memory behave quite differently.

3. What is a Request?

Think:

Request = how much resource Kubernetes uses when deciding where the Pod can be scheduled.

Our example:

```
requests:  
  cpu: "500m"  
  memory: "512Mi"
```

means approximately:

```
CPU request    = 0.5 CPU  
Memory request = 512 MiB
```

The scheduler asks:

“Which Node has enough unallocated requested capacity to accommodate this Pod?”

Requests are mainly about scheduling

Suppose:

```
Node capacity:  
CPU = 4
```

```
Existing Pod requests:  
Pod A = 1 CPU  
Pod B = 2 CPU
```

Total requested:

```
3 CPU
```

Remaining from the scheduler's perspective:

```
4 - 3 = 1 CPU
```

Now another Pod requests:

```
2 CPU
```

Scheduler:

```
Required = 2 CPU  
Available for requests = 1 CPU  
  
× cannot schedule
```

The Pod may remain:

```
Pending
```

even if the existing Pods happen to be using very little CPU at that particular moment.

That is important.

Request is NOT exactly “reserved physical CPU”

Don't imagine:

```
500m request
```

means Kubernetes permanently locks half a CPU exclusively for your container.

That is too simplistic.

Instead:

```
Request  
→ scheduler capacity planning
```


- CPU weighting under contention
- contributes to QoS/resource behavior

Several Pods can use CPU dynamically when capacity is available.

CPU units

Remember:

1 CPU = 1000m

So:

100m = 0.1 CPU
250m = 0.25 CPU
500m = 0.5 CPU
1000m = 1 CPU
2000m = 2 CPU

For example:

cpu: "750m"

means:

0.75 CPU

7. What exactly does 1 CPU mean?

Approximately one CPU worth of compute capacity.

Depending on the infrastructure, that could correspond to one:

vCPU
hardware thread
logical CPU

Kubernetes expresses CPU as a standardized compute-resource unit.

For application sizing, think:

1 CPU
"H one logical CPU's processing capacity"

Memory units

You✓I commonly see:

128Mi
256Mi
512Mi

```
1Gi  
2Gi
```

For example:

```
memory: "512Mi"
```

means about:

```
512 MiB
```

And:

```
memory: "1Gi"
```

means:

```
1 GiB
```

9. Now what is a Limit?

Think:

Limit = maximum resource usage Kubernetes allows the container to consume.

Example:

```
limits:  
  cpu: "1"  
  memory: "1Gi"
```

means:

```
CPU ceiling    = 1 CPU  
Memory ceiling = 1 GiB
```

But this is where things become interesting.

CPU limit behavior and memory limit behavior are NOT the same.

CPU is a compressible resource

Suppose your container has:

```
limits:  
  cpu: "1"
```

And your Spring Boot application tries to consume:

```
2 CPU
```

Kubernetes/Linux doesn't normally say:

```
"You used too much CPU.  
Kill the Java process."
```

Instead, CPU use is generally:

throttled

Conceptually:

```
Application wants 2 CPU  
    ↓  
CPU limit = 1 CPU  
    ↓  
kernel constrains CPU time  
    ↓  
application runs slower
```

CPU throttling

Imagine your API normally processes:

```
1000 requests/sec
```

but it's constantly hitting its CPU limit.

It might fall to:

```
600 requests/sec
```

Latency increases:

```
100 ms  
    ↓  
300 ms
```

Yet:

```
Pod = Running  
Container = Running
```

There may be no obvious crash.

This is why CPU-limit problems can sometimes appear simply as:

"Application is slow."

CPU example

Resources:

```
requests:  
cpu: "500m"
```

```
limits:  
  cpu: "1"
```

Normal application load:

```
300m
```

Fine.

Load increases:

```
700m
```

Also fine because it's below the 1 CPU limit.

Load jumps:

```
1500m = 1.5 CPU requested by application
```

But limit:

```
1 CPU
```

Result:

```
application wants 1.5 CPU  
  ↓  
allowed roughly "d 1 CPU  
  ↓  
CPU throttling  
  ↓  
higher latency / lower throughput
```

| Memory is different

Memory is considered a non-compressible resource.

Suppose:

```
limits:  
  memory: "1Gi"
```

Your Java application uses:

```
700Mi
```

Fine.

Then:

```
900Mi
```

Still fine.

Then it exceeds its effective container memory limit.

Kubernetes/Linux can't simply make memory execution "slower" in the same way it throttles CPU.

The container process may be killed because of:

OOM OOM = Out Of Memory

OOMKilled

You may see:

```
Reason: OOMKilled
```

Conceptually:

```
Memory limit = 1Gi
  ↓
Application memory grows
  ↓
container exceeds limit
  ↓
kernel OOM handling
  ↓
Java process killed
  ↓
container terminates
```

Depending on restart policy/workload:

```
Kubelet
  ↓
restarts container
```

This can eventually look like CrashLoopBackOff

Suppose:

```
Spring Boot starts
  ↓
uses too much memory
  ↓
OOMKilled
  ↓
restart
  ↓
uses too much memory
```

```
    ↓  
OOMKilled  
    ↓  
restart
```

After repeated failures:

```
CrashLoopBackOff
```

So:

```
CrashLoopBackOff
```

is the symptom.

But:

```
OOMKilled
```

may be the real cause.

16. How do you identify this?

Run:

```
kubectl describe pod <pod-name>
```

You may see:

```
Last State:  
  Terminated:  
    Reason: OOMKilled
```

Then check:

```
kubectl logs <pod-name> --previous
```

You may also see JVM/application errors if it had time to log them.

Java makes memory especially interesting

Suppose container limit:

```
1Gi
```

You might think:

“Fine. I’ll set Java -Xmx1g .”

That is usually a bad idea.

Why?

Because JVM memory isn’t only the heap.

Conceptually:

```
Container memory
-
|— Java Heap
-
|— Metaspace
-
|— Thread stacks
-
|— Direct buffers
-
|— JVM native memory
-
|— JIT/code cache
|— Other native allocations
```

So if:

```
Container limit = 1Gi
```

and:

```
-Xmx = 1Gi
```

you've left essentially no space for all the other memory consumers.

OOM risk increases.

Better mental model

Suppose:

```
Container limit = 1Gi
```

You might budget something conceptually like:

Heap	650-750Mi
Metaspace	...
Thread stacks	...
Direct/native memory	...
Operating overhead	...

The exact value depends heavily on your application.

Don't memorize 700Mi as a rule.

The lesson is:

Leave memory headroom between JVM heap and container memory limit.

| Modern JVMs understand containers

Modern Java versions are container-aware.

The JVM can detect container CPU/memory restrictions and use them when calculating defaults.

But you still need to understand:

```
Container memory limit  
vs  
JVM heap  
vs  
non-heap/native memory
```

because automatic defaults don't guarantee optimal production sizing.

| Request vs limit — simplest comparison

Memorize this:

Request Limit

Main purpose Scheduling/resource baseline Maximum usage

Scheduler uses it? Yes Not as its primary placement reservation

CPU effect baseline/weight throttling ceiling

Memory effect scheduling baseline OOM boundary

Request Limit

Exceed it? Can often use more if available Constrained

So:

```
Request = "Plan for at least this much."
```

```
Limit = "Don't let me exceed this ceiling."
```

| Example: request 500m, limit 1 CPU

```
requests:  
  cpu: "500m"  
  
limits:  
  cpu: "1"
```

Think:

Scheduler:

"Need capacity for 0.5 CPU."

Runtime:

"Container can potentially use up to 1 CPU."

If Node has spare CPU:

0.5 → 0.7 → 0.9 → 1.0

possible.

Beyond configured CPU limit:

→ throttling

Example: memory request 512Mi, limit 1Gi

```
requests:  
  memory: "512Mi"
```

```
limits:  
  memory: "1Gi"
```

Scheduler:

"Need room for 512Mi requested memory."

Application could use:

600Mi
700Mi
900Mi

assuming resources exist.

If it crosses its effective memory limit:

OOM risk

23. What if there is no limit?

Suppose:

```
resources:  
  requests:  
    memory: "512Mi"
```

but no:

limits:

Then the container doesn't have that Kubernetes memory limit applied from this configuration.

It can potentially consume significantly more memory, subject to:

```
Node availability
QoS
node pressure
other platform controls
```

This could negatively affect other workloads.

24. What if there is no request?

Suppose:

```
resources:
  limits:
    memory: "1Gi"
```

but no request.

An important Kubernetes behavior is that, absent other admission-time defaults, when a CPU/memory limit exists but its request isn't specified, Kubernetes can default the corresponding request to the limit.

So conceptually this may become:

```
memory request = 1Gi
memory limit   = 1Gi
```

There are also LimitRange admission defaults that can change what gets filled in when fields are absent.

This leads directly to your earlier question.

Your earlier question — LimitRange vs Pod resources

You previously asked:

If I define resource requests/limits in my Deployment and there is also a LimitRange, which takes precedence? The correct answer is:

Your explicit Pod/container settings generally remain your settings. LimitRange does not normally overwrite valid explicit values just because it has defaults.

Instead, it can do two things:

1. Apply defaults when values are missing
2. Validate/enforce min/max/ratio constraints

Example: LimitRange defaults

Suppose namespace has:

```
default:
  cpu: "1"
  memory: "1Gi"

defaultRequest:
  cpu: "500m"
  memory: "512Mi"
```

Your Deployment explicitly says:

```
resources:
  requests:
    cpu: "250m"

    memory: "256Mi"

  limits:
    cpu: "500m"
    memory: "512Mi"
```

Assuming those values satisfy the LimitRange's enforcement constraints:

YOUR values remain.

The defaults don't replace them.

27. What if you don't specify resources?

Pod:

```
containers:
  - name: employee
    image: employee:v1
```

LimitRange:

```
default request:
CPU = 500m
Memory = 512Mi

default limit:
CPU = 1
Memory = 1Gi
```

Kubernetes admission can fill in:

```
request:
500m / 512Mi

limit:
```

1 CPU / 1Gi

So:

Missing value

↓

LimitRange default

28. What if your explicit value violates LimitRange?

Suppose LimitRange says:

maximum memory = 2Gi

Your Pod says:

```
limits:
  memory: "4Gi"
```

Does Kubernetes change:

4Gi → 2Gi

automatically?

No. The request is generally:

× rejected

because your configuration violates the policy.

That is extremely important.

Mental model for LimitRange

Think:

Your Pod specifies value?

-

├ NO

↓

| LimitRange may provide default

-

├ YES

↓

Is it valid against LimitRange?

-

```

├──4──┤
YES   NO
|     |
accept reject
```

Not:

```
LimitRange silently replaces everything.
```

30. And what about ResourceQuota?

ResourceQuota operates at the namespace level.

Suppose:

```
Namespace quota:
```

```
requests.cpu = 10
```

Existing workload requests total:

```
9 CPU
```

You deploy a Pod requesting:

```
2 CPU
```

Total would become:

```
11 CPU
```

But quota allows:

```
10 CPU
```

So the API request can be:

```
× rejected
```

This happens before the scheduler gets a chance to place the Pod.

Compare all three

Pod resources

```
"What does THIS container request/limit?"
```

LimitRange

```
"What defaults/rules apply to individual containers/Pods in this namespace?"
```

ResourceQuota

```
"How much may the whole namespace consume?"
```

Excellent interview distinction.

32. What if request > limit?

Suppose:

```
requests:
  memory: "2Gi"

limits:
  memory: "1Gi"
```

This configuration is invalid.

Why?

You're saying:

```
Please plan 2Gi for me

but

Never let me exceed 1Gi
```

That doesn't make sense.

Kubernetes rejects resource configurations where the request exceeds the corresponding limit.

So:

```
request <= limit
```

is the normal rule when both exist.

| Now let's discuss QoS

QoS =

Quality of Service Kubernetes classifies Pods into resource QoS classes.

The major classes are:

```
Guaranteed
Burstable
BestEffort
```

These become especially important during memory pressure and eviction decisions.

| BestEffort

Suppose your containers have no CPU/memory requests or limits:

```
containers:
- name: employee
  image: employee:v1
```

No resources at all.

Then the Pod can fall into:

BestEffort Conceptually:

```
"I didn't declare resource requirements."
```

This is generally the least protected QoS class under resource pressure.

| Burstable

Example:

```
requests:
  cpu: "500m"
  memory: "512Mi"

limits:
  memory: "1Gi"
```

The Pod has resource configuration but does not meet the strict requirements for Guaranteed.

This is typically:

Burstable Meaning:

The Pod has some declared resource expectations and may use additional resources when available.

A large number of real application Pods fall into this category.

| Guaranteed

For a Pod to qualify as Guaranteed, the CPU and memory requests and limits must satisfy strict conditions for its containers, essentially:

```
CPU request = CPU limit
```

```
Memory request = Memory limit
```

for all relevant containers/resources required by the QoS rules.

Example:

```
resources:
  requests:
    cpu: "1"
    memory: "1Gi"

  limits:
    cpu: "1"
```

```
memory: "1Gi"
```

Conceptually:

```
Request = Limit
```

This Pod gets:

Guaranteed QoS

QoS comparison

BestEffort

↓

No CPU/memory requests or limits

Burstable

↓

Some resource configuration,
but not Guaranteed

Guaranteed

↓

CPU request = CPU limit
Memory request = Memory limit
for all required containers

38. Why does QoS matter?

Imagine Node memory:

```
16Gi
```

Applications collectively consume too much.

The Node experiences:

```
MemoryPressure
```

Kubernetes may need to evict Pods.

QoS contributes to determining which workloads are more vulnerable.

Very simplified:

BestEffort

→ most vulnerable

Burstable

→ intermediate

Guaranteed

→ more protected

But don't memorize this as:

"Kubernetes always kills every BestEffort Pod first, then exactly every Burstable Pod."

Real eviction decisions also consider things such as:

```
actual usage
requests
priority
resource pressure
```

The QoS class is part of the decision.

| Container OOM vs Node eviction

These are different problems.

Situation A — Container exceeds its own limit

```
Container memory limit = 1Gi
Application uses > limit
      ↓
OOMKilled
```

That is a container memory-limit issue.

Situation B — Node itself runs out of memory

```
Node memory
  ↓
critically low
  ↓
MemoryPressure
  ↓
Kubelet eviction decisions
  ↓
Pods can be evicted
```

That is a Node resource pressure issue.

Don't confuse:

OOMKilled

with:

Evicted

Example

Node:

```
16 GiB
```

Pods:

```
Pod A → 6 GiB  
Pod B → 5 GiB  
Pod C → 4 GiB  
Pod D → growing
```

Eventually the Node is critically short on memory.

Kubelet may begin eviction behavior to protect Node stability.

You could see:

```
STATUS: Evicted
```

rather than simply:

```
OOMKilled
```

CPU doesn't normally cause eviction in the same way

If CPU is overloaded:

```
CPU = 100|
```

applications generally compete for CPU time.

They become:

```
slower
```

CPU is compressible.

Memory is different because:

```
You can't make 20Gi of live memory fit into 16Gi merely by slowing execution.
```

That is why memory pressure tends to be more destructive.

Java practical incident

Suppose your Deployment:

```
resources:  
  requests:  
    memory: "512Mi"
```

```
limits:  
  memory: "1Gi"
```

Your Java app normally consumes:

```
700Mi
```

Then a large operation occurs:

```
Loads huge report into memory  
  ↓  
Heap grows  
  ↓  
Native/direct memory also exists  
  ↓  
Container exceeds 1Gi  
  ↓  
OOMKilled
```

Kubernetes restarts it.

You see:

```
RESTARTS = 6
```

and maybe:

```
CrashLoopBackOff
```

43. How would you investigate?

Step 1:

```
kubectl get pods
```

See:

```
RESTARTS = high
```

Step 2:

```
kubectl describe pod <pod>
```

Find:

```
Reason: OOMKilled
```

Step 3:

```
kubectl logs <pod> --previous
```

Step 4:

Check resources:

```
kubectl get pod <pod> -o yaml
```

or inspect the Deployment.

Step 5:

Check actual memory:

```
kubectl top pod <pod>
```

if the metrics pipeline is available.

Step 6:

Investigate JVM/application memory.

Don't immediately increase memory limit

This is an important production lesson.

Suppose:

```
OOMKilled
```

Bad troubleshooting:

```
1Gi → 8Gi  
Done.
```

Maybe the application actually has:

```
memory leak  
unbounded cache  
huge response buffering  
bad query  
too many threads  
direct-buffer leak
```

Increasing the limit may only delay the failure.

Correct process:

```
OOMKilled  
↓  
Check resource configuration  
↓  
Check JVM sizing  
↓  
Observe memory behavior  
↓  
Heap/native analysis if needed  
↓  
Determine whether:
```

```
workload legitimately needs more memory
OR
application has a memory problem
```

CPU troubleshooting scenario

Users say:

API isn't crashing, but response time suddenly increased.

Pod:

```
Running
Ready
0 restarts
```

So no obvious problem.

Run:

```
kubectl top pod
```

Suppose:

```
CPU "H 1000m
```

while limit:

```
1000m
```

constantly.

Possible hypothesis:

```
CPU throttling
```

The Java application may want additional CPU but cannot get it.

So:

```
No Crash
+
High latency
+
CPU at limit
=
Investigate CPU throttling/capacity
```

Requests affect HPA too

This will become very important in our next chapter.

Suppose:

```
requests:
  cpu: "500m"
```

And Pod currently uses:

```
400m
```

CPU utilization relative to request is roughly:

```
400 / 500 × 100
=
80|
```

Horizontal Pod Autoscaler can use this idea.

So requests aren't just about scheduling.

They're also important for:

```
autoscaling metrics
```

We✓I cover that next.

47. What values should I choose?

There's no universal answer like:

```
Every Spring Boot app:
request = 500Mi
limit = 1Gi
```

Don't do that.

Instead use measurements.

Conceptually:

```
Observe actual workload
    ↓
Measure:
CPU
Memory
Peak usage
Normal usage
    ↓
Choose requests
    ↓
Choose limits
    ↓
Load test
    ↓
Observe again
```

↓
Tune

| Request sizing philosophy

Suppose production measurements show:

Normal CPU:
300-400m

Peak CPU:
700m

You might choose a request somewhere based on your application's expected sustained need and SLO/capacity strategy.

Not simply:

request = peak

or:

request = minimum ever observed

Requests affect:

scheduling density
cost
performance under contention
autoscaling

So resource sizing is an engineering/capacity-planning decision.

| Memory requests are especially important

Suppose actual Spring Boot memory almost always stays around:

900Mi

but request:

128Mi

The scheduler thinks:

"This application needs roughly 128Mi."

and may pack many similar Pods onto the same Node.

Actual workload:

```
900Mi each
```

could then produce Node pressure.

So extremely unrealistic requests can create bad scheduling decisions.

Requests vs actual usage

Imagine Node:

```
Memory = 8Gi
```

Each Pod says:

```
request = 256Mi
```

Scheduler might conceptually fit many of them.

But if each actually consumes:

```
1Gi
```

the Node may run into memory pressure much earlier than expected.

That is why requests should reasonably reflect expected needs.

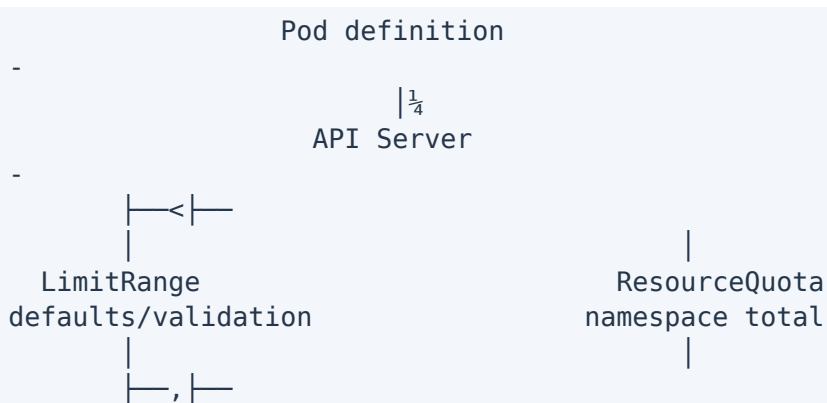
Complete resource pipeline

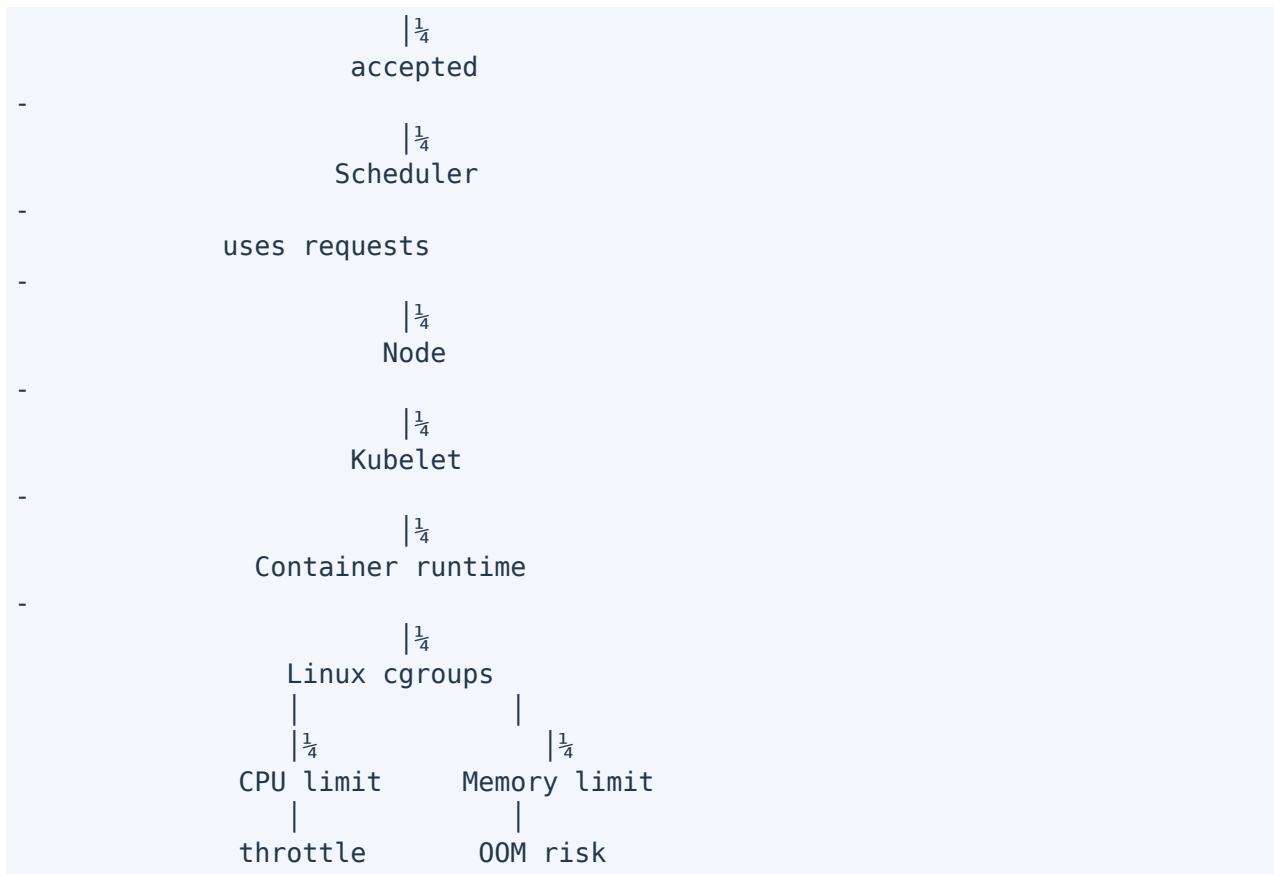
When you submit:

```
resources:
  requests:
    cpu: 500m
    memory: 512Mi

  limits:
    cpu: "1"
    memory: 1Gi
```

think:





That is the whole story.

Let's make your earlier precedence question crystal clear

Suppose:

LimitRange

```

Default request = 500m
Default limit   = 1 CPU
Maximum limit   = 2 CPU
  
```

Case 1 — Pod specifies nothing

LimitRange defaults may be applied.

Case 2 — Pod explicitly says:

```

request = 750m
limit = 1.5 CPU
  
```

Both satisfy the policy.

Result:

```
750m / 1.5 CPU stay.
```

Case 3 — Pod says:

```
limit = 4 CPU
```

but LimitRange maximum:

```
2 CPU
```

Result:

```
× request rejected
```

not:

```
4 CPU automatically changed to 2 CPU
```

That is the important answer.

| Interview question

What is the difference between request and limit?

Good answer:

A request represents the resource amount Kubernetes uses primarily for scheduling and resource guarantees/weighting, while a limit sets the maximum resource consumption allowed for the container. CPU exceeding a limit generally results in throttling, while memory exceeding its limit can result in OOM termination.

| Interview question

What happens if CPU reaches its limit? Answer:

The container is generally throttled rather than killed, so application performance may degrade.

| Interview question

What happens if memory exceeds the container limit? Answer:

The process may be killed by the kernel's OOM handling, and Kubernetes can report the container termination as OOMKilled .

| Interview question

Difference between OOMKilled and Evicted? Answer:

OOMKilled

→ container crossed memory constraints / OOM condition

Evicted

→ kubelet removed the Pod, commonly due to Node resource pressure or another
↪ eviction condition

Different layers.

| Interview question

What are Kubernetes QoS classes? Answer:

Guaranteed

Burstable

BestEffort

They are determined from the Pod's CPU/memory requests and limits and influence resource-pressure behavior.

| Interview question

Does LimitRange override values explicitly defined in Deployment YAML? Answer:

Normally no. LimitRange can default missing values and validate/enforce constraints. Explicit valid values remain; values violating the LimitRange can cause the Pod/API request to be rejected.

Excellent production/interview answer.

| What you should remember from this chapter

Requests

Scheduler planning baseline

CPU limit

Too much CPU

↓

throttling

↓

slower

Memory limit

Too much memory

↓

OOM

↓
container may be killed

LimitRange

default + validate individual resource settings

ResourceQuota

limit namespace totals

QoS

Guaranteed
Burstable
BestEffort

Container memory
" "
Java heap only

And this fills an important gap in our roadmap rather than repeating material we've already covered.

Next — chapter 21: Horizontal Pod Autoscaler (HPA) Now we can answer a very practical problem:

Normal traffic:
3 Pods

Huge traffic spike:
3 Pods aren't enough

What should Kubernetes do?

We'll learn:

3 Pods
↓ traffic !'
5 Pods
↓ traffic !'
8 Pods

then traffic falls

8 Pods
↓
5 Pods
↓
3 Pods

Most importantly, we'll understand why this:

```
requests:  
  cpu: 500m
```

is directly connected to something like:

```
HPA target CPU = 70|
```

We✓I also distinguish HPA vs VPA vs Cluster Autoscaler, which is a very common Kubernetes interview and production architecture question.

Sure. We'll park Modules 21-30 for later and jump directly to:

Chapter 17 — Namespaces, LimitRange, and ResourceQuota

CHAPTER FOCUS

Understand **Namespaces**, **LimitRange**, and **ResourceQuota** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

Now we're moving into an area you'll encounter constantly in real Kubernetes/OpenShift environments:

How do multiple teams and applications safely share one Kubernetes cluster?

Imagine your company has:

```
Kubernetes Cluster
-
├─ Payments team
├─ Employee team
├─ Analytics team
├─ AI team
└─ Platform team
```

You don't want everyone throwing Pods into one giant bucket.

That is where Namespaces come in.

1. What is a Namespace?

A Namespace is a logical boundary inside a Kubernetes cluster.

Think of it as a separate workspace.

```
Cluster
-
├─ namespace: payments
-
├─ namespace: employee
-
└─ namespace: analytics
```

Each namespace can contain its own:

```
Pods
Deployments
Services
ConfigMaps
```

```
Secrets
Jobs
etc.
```

2. Why do we need Namespaces?

Without namespaces:

```
Cluster
-
├─ payment-pod
├─ employee-pod
├─ analytics-pod
├─ frontend-pod
├─ backend-pod
├─ database-pod
└─ ...
```

As the cluster grows, this becomes difficult to manage.

With namespaces:

```
Cluster
-
├─ payments
│   ├── payment-api
│   ├── payment-worker
│   └─ payment-db
-
├─ employee
│   ├── employee-api
│   └─ employee-worker
-
├─ analytics
│   ├── analytics-api
│   └─ analytics-worker
```

Much easier.

Namespace is NOT a separate cluster

This distinction is important.

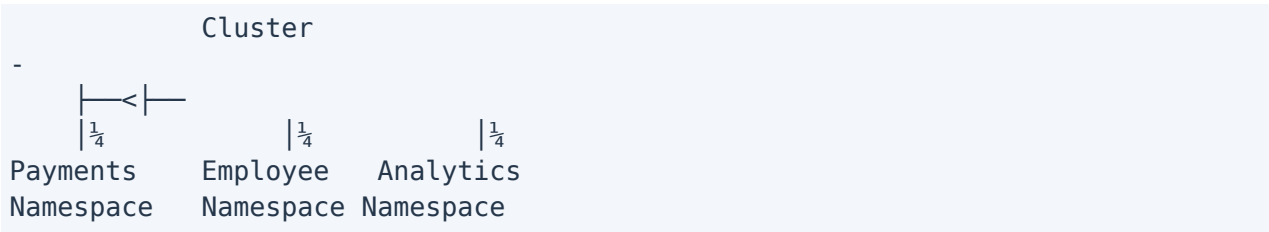
Suppose:

```
Cluster A
```

contains:

```
payments namespace
employee namespace
analytics namespace
```

They are still using the same Kubernetes cluster.



Namespaces provide logical organization/isolation, not the same level of isolation as separate physical clusters.

Creating a Namespace

You can create one with:

```
kubectl create namespace payments
```

Then:

```
kubectl get namespaces
```

You might see:

```
NAME
default
kube-system
kube-public
kube-node-lease
payments
employee
analytics
```

YAML way

You can also define:

```
apiVersion: v1
kind: Namespace

metadata:
  name: payments
```

Then:

```
kubectl apply -f namespace.yaml
```

This fits the declarative Kubernetes approach we've already learned.

| The default Namespace

If you don't specify a namespace:

```
kubectl get pods
```

you're generally looking in:

```
default
```

For example:

```
kubectl create deployment employee \  
  --image=employee:v1
```

without specifying a namespace typically creates the Deployment in default .

| Why production environments avoid default

You may see:

```
default
```

in tutorials.

But in a real organization, teams commonly use dedicated namespaces such as:

```
payments-prod  
payments-dev  
employee-prod  
employee-dev
```

or organizational equivalents.

This gives better:

```
Organization  
Access control  
Resource management  
Monitoring  
Billing/cost attribution  
Security
```

| Namespace-specific commands

Suppose:

```
payments
```

contains:

```
payment-api
```

You can run:

```
kubectl get pods -n payments
```

The -n means:

```
--namespace
```

You can also:

```
kubectl get deployments -n payments
```

or:

```
kubectl get services -n payments
```

A very common beginner mistake

You run:

```
kubectl get pods
```

and see:

```
No resources found.
```

But you know the application exists.

Maybe it's simply in another namespace.

Try:

```
kubectl get pods -A
```

-A means:

```
--all-namespaces
```

You✓I see Pods across the cluster.

Namespaces and Services

Remember our networking discussion.

Suppose:

```
payments
├─ payment-service
employee
```

```
└─ employee-service
```

Service names are namespace-aware.

A Pod in the payments namespace might refer to:

```
payment-service
```

within its own namespace.

For cross-namespace communication, you can use the namespace-qualified DNS name.

Conceptually:

```
payment-service.payments
```

or the fully qualified Service DNS form:

```
payment-service.payments.svc.cluster.local
```

So namespace becomes part of Kubernetes service discovery.

Namespace isolation isn't absolute

This is important.

Don't think:

"Different namespaces cannot communicate."

That is not true.

By default, Pods in different namespaces can generally communicate if network policies don't restrict them.

For example:

```
payments
-
  | network
  | 1/4
employee
```

can potentially communicate.

NetworkPolicy We✓I cover that later.

Namespaces and RBAC

Namespaces become especially powerful when combined with:

RBAC Role-Based Access Control.

Imagine:

```
Alice → Payments team
Bob   → Employee team
```

You can configure:

```
Alice
→ access payments namespace

Bob
→ access employee namespace
```

This becomes a major part of Kubernetes security.

We✓I cover RBAC in a dedicated security chapter.

Now the next problem

Namespaces solve:

Who/what belongs where?

But we have another problem.

Imagine the payments team creates:

```
100 Pods
```

and consumes:

```
90| of the cluster
```

Then:

```
employee
analytics
```

might struggle to get resources.

We need resource governance.

That is where:

ResourceQuota comes in.

14. What is ResourceQuota?

A ResourceQuota limits how much of certain resources a namespace can consume.

For example:

```
payments namespace
```

could have:

```
Maximum CPU requests = 20 CPU
Maximum memory requests = 40Gi
Maximum Pods = 50
```

Conceptually:

Cluster

```
-
├─ payments
│   └─ Quota:
│       CPU "d 20
│       Memory "d 40Gi
│       Pods "d 50
├─ employee
│   └─ Quota:
│       CPU "d 10
│       Memory "d 20Gi
├─ analytics
```

Why ResourceQuota matters

Without quotas:

```
Team A
  ↓
creates 200 Pods
  ↓
consumes cluster resources
```

Team B:

```
creates Pod
  ↓
Pending
  ↓
not enough resources
```

ResourceQuota provides a governance boundary.

ResourceQuota example

```
apiVersion: v1
kind: ResourceQuota

metadata:
  name: payments-quota
  namespace: payments
```

```
spec:
  hard:
    requests.cpu: "10"
    requests.memory: 20Gi
    limits.cpu: "20"
    limits.memory: 40Gi
    pods: "50"
```

Meaning approximately:

Payments namespace may consume at most:

CPU requests:
10 CPU

Memory requests:
20 GiB

CPU limits:
20 CPU

Memory limits:
40 GiB

Pods:
50

17. What happens when quota is exceeded?

Suppose:

```
Quota:
pods = 50
```

and there are already:

```
50 Pods
```

You try to create:

```
Pod #51
```

Kubernetes can reject the request because the namespace quota would be exceeded.

This is different from:

```
Pod Pending
```

due to scheduler resource availability.

That is an important distinction.

Quota vs actual cluster capacity

Suppose your cluster has:

```
100 CPU
```

but namespace quota says:

```
payments:  
10 CPU
```

Payments cannot simply consume all 100 CPUs.

Its namespace-level quota limits its allowed resource usage according to the configured quota.

So:

```
Cluster capacity  
    ↓  
100 CPU  
  
Payments quota  
    ↓  
10 CPU
```

The namespace is governed independently.

ResourceQuota can also limit object counts

It isn't only CPU and memory.

You can restrict counts such as:

```
Pods  
Services  
ConfigMaps  
Secrets  
PersistentVolumeClaims  
Jobs
```

For example:

```
hard:  
  pods: "50"  
  services: "10"  
  secrets: "100"
```

This can prevent accidental or malicious resource explosions.

But another problem appears

Suppose your namespace allows:

```
CPU requests = 20
```

A developer creates:

```
resources:
  requests:
    cpu: "10"
```

That is allowed.

But what if they create a Pod with no resource requests or limits at all?

```
containers:
- name: employee
  image: employee:v1
```

Now Kubernetes doesn't have explicit resource expectations for that container.

We need another mechanism.

Enter:

LimitRange

21. What is LimitRange?

A LimitRange defines default and/or minimum/maximum resource constraints for individual containers or Pods within a namespace.

Think:

```
ResourceQuota
→ controls the namespace as a whole
```

```
LimitRange
→ controls individual resource usage rules/defaults
```

This distinction is extremely important.

ResourceQuota vs LimitRange

Memorize this:

```
ResourceQuota
=
"How much can this namespace consume?"
```

```
LimitRange
=
"What resource rules/defaults apply to each Pod/container?"
```


Example LimitRange

```

apiVersion: v1
kind: LimitRange

metadata:
  name: employee-limits
  namespace: employee

spec:
  limits:
    - type: Container

      default:
        cpu: "500m"
        memory: "512Mi"

      defaultRequest:
        cpu: "250m"
        memory: "256Mi"

      max:
        cpu: "2"
        memory: "2Gi"

      min:
        cpu: "100m"
        memory: "128Mi"

```

Let's understand it.

default

Suppose developer creates:

```

containers:
  - name: employee
    image: employee:v1

```

No resources specified.

LimitRange can apply defaults such as:

```

CPU limit:
500m

```

```

Memory limit:
512Mi

```

defaultRequest

If the developer doesn't specify requests, Kubernetes can apply:

```
CPU request:
250m
```

```
Memory request:
256Mi
```

Now the scheduler has resource information to work with.

max

You can say:

```
A single container cannot request/use more than:
CPU = 2
Memory = 2Gi
```

This prevents one container from asking for an unreasonable amount.

min

You can also establish minimum resource constraints.

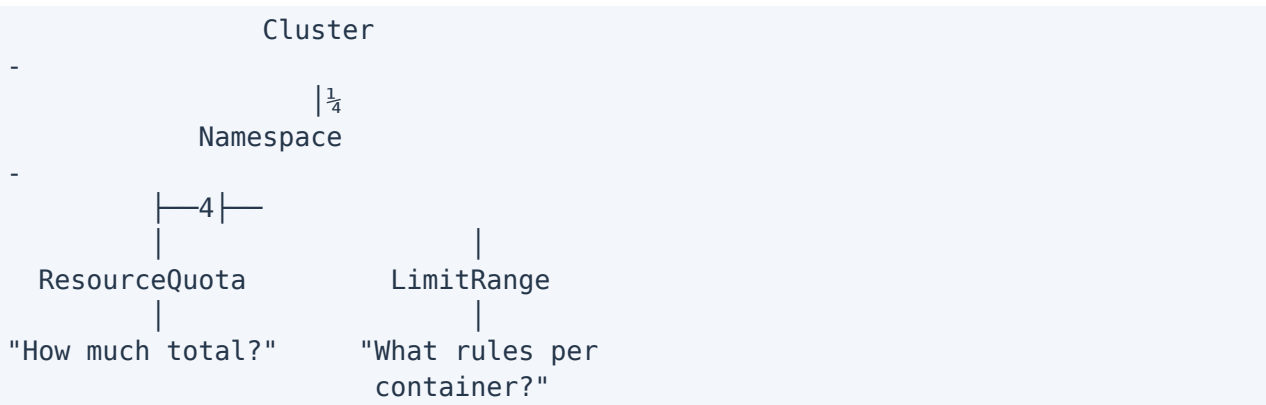
For example:

```
CPU "e 100m
Memory "e 128Mi
```

This can prevent workloads from being configured with extremely tiny resource values.

Think of the three layers

This is the easiest way to remember everything:



Then:

```
Pod
↓
Container
↓
Resources
```

Real company example

Imagine:

```
Kubernetes Cluster
```

with:

```
payments namespace
employee namespace
analytics namespace
```

For employee :

```
ResourceQuota:
CPU requests = 20
Memory requests = 40Gi
Pods = 100
```

LimitRange:

```
Default CPU request = 250m
Default memory request = 256Mi

Default CPU limit = 500m
Default memory limit = 512Mi
```

Now developers don't have to manually configure everything correctly every time.

30. What happens when a developer creates a Pod?

Developer writes:

```
containers:
- name: employee
  image: employee:v1
```

No resources.

LimitRange can inject defaults.

Conceptually:

```
Developer YAML
↓
LimitRange
```

```

      ↓
Default resources applied
      ↓
ResourceQuota checked
      ↓
API request accepted/rejected
      ↓
Scheduler

```

This is a useful mental model.

ResourceQuota doesn't schedule Pods

Another common confusion.

ResourceQuota says:

"This namespace has consumed/allowed this much."

It doesn't decide:

"Put this Pod on Node 3."

That is still the Scheduler.

So:

```

ResourceQuota
→ admission/resource governance

Scheduler
→ Node placement

```

Different responsibilities.

Namespace + Quota + LimitRange

Let's build a realistic environment.

```

Cluster
-
├─ employee-dev
│   ├── ResourceQuota
│   │   ├── CPU = 10
│   │   └── Memory = 20Gi
│   └── LimitRange
│       ├── default CPU = 250m
│       └── default memory = 256Mi
-
└─ employee-prod

```

```
- | ResourceQuota
  |   CPU = 40
  |   Memory = 80Gi
- | LimitRange
  |   default CPU = 500m
  |   default memory = 512Mi
```

This is a very common organizational pattern.

33. Why different quotas for dev and prod?

Production usually needs:

More resources
Higher availability
More replicas

Development might need:

Fewer resources
Lower cost
Smaller workloads

Namespaces give you a natural boundary for this.

| OpenShift perspective

Since you're learning Kubernetes + OpenShift, here's an important translation.

In OpenShift, the Kubernetes concept of a:

Namespace

is closely represented by:

Project When using OpenShift, you'll commonly hear:

Project

instead of:

Namespace

An OpenShift Project is essentially a Kubernetes Namespace with additional OpenShift-specific management and access behavior around it.

So when someone says:

"Deploy this into the payments project"

think:

```
Kubernetes:  
namespace = payments  
  
OpenShift:  
project = payments
```

Why OpenShift Projects are useful

Imagine:

```
OpenShift Cluster  
-  
├─ payments  
├─ employee  
└─ analytics
```

Each project can have its own:

```
Applications  
Pods  
Services  
Routes  
ConfigMaps  
Secrets  
RBAC  
ResourceQuota  
LimitRange
```

This is why OpenShift feels very natural for enterprise multi-team environments.

Namespace doesn't mean security boundary by itself

This is extremely important.

Don't say:

"Namespaces provide complete security isolation."

That is too strong.

Namespaces provide logical organization and can be combined with:

```
RBAC  
NetworkPolicy  
ResourceQuota  
Security controls
```

for stronger isolation.

Think:

```

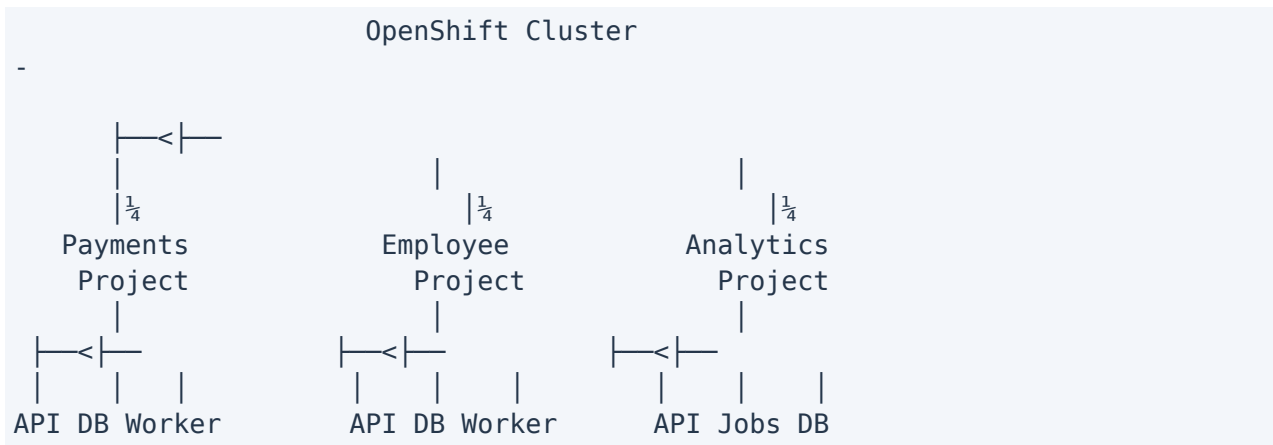
Namespace
+
RBAC
+
NetworkPolicy
+
ResourceQuota
+
Security policies

```

= much stronger multi-tenancy.

A real enterprise example

Imagine your company:



Then:

RBAC
→ Who can manage each project?

ResourceQuota
→ How much can each project consume?

LimitRange
→ What resource defaults/rules apply?

NetworkPolicy
→ Who can communicate with whom?

Now we're starting to get a real enterprise Kubernetes platform.

A very useful mental model

When you see:

Namespace

ask:

Who/what belongs here?

When you see:

ResourceQuota

ask:

How much can this namespace consume?

When you see:

LimitRange

ask:

What resource rules/defaults apply to individual workloads?

When you see:

RBAC

ask:

Who is allowed to do what?

When you see:

NetworkPolicy

ask:

Who is allowed to talk to whom?

These five concepts fit together extremely well.

Troubleshooting commands

List namespaces:

```
kubectl get namespaces
```

List everything in a namespace:

```
kubectl get all -n payments
```

See quota:

```
kubectl get resourcequota -n payments
```

More details:


```
kubectl describe resourcequota -n payments
```

See LimitRanges:

```
kubectl get limitrange -n payments
```

Details:

```
kubectl describe limitrange -n payments
```

A common production problem

You deploy:

```
employee-api
```

and get an error saying the resource quota has been exceeded.

You might think:

“The Node doesn’t have enough CPU.”

Not necessarily.

There are two different situations.

Situation A — Namespace quota exceeded

```
ResourceQuota exceeded
```

Meaning:

```
Namespace policy problem
```

Situation B — Node capacity unavailable

```
Insufficient cpu
```

Meaning:

```
Scheduling/capacity problem
```

They are different layers.

Let’s connect this to our previous chapter

Previously:

```
Scheduler
```

asks:

Which Node can run this Pod?

Now we add:

ResourceQuota

before that:

Is this namespace even allowed to consume the requested resources?

So conceptually:

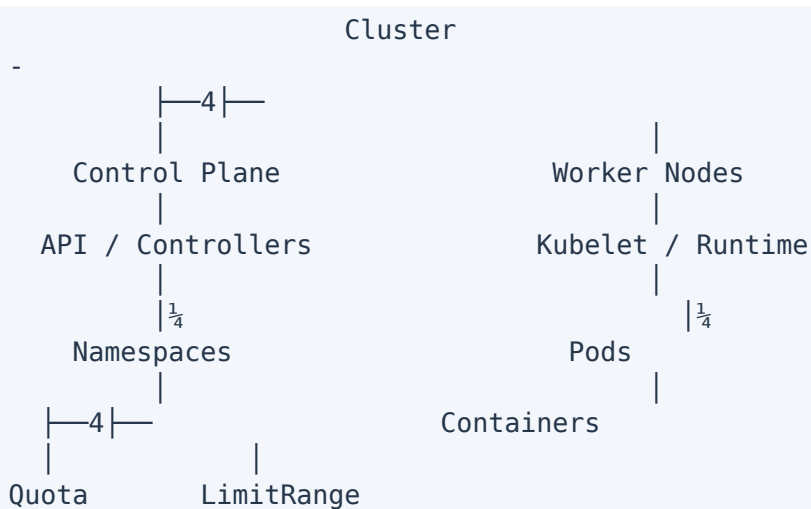
```

Developer
  ↓
Namespace policies
  ↓
ResourceQuota / LimitRange
  ↓
Pod accepted
  ↓
Scheduler
  ↓
Node
  
```

This is a more complete picture.

Your Kubernetes architecture is growing

You now understand:



And workloads:

```

Deployment
  ↓
ReplicaSet
  ↓
Pods
  ↓
Scheduler
  ↓
  
```

Nodes

And networking:

Service

↓

Ready Pods

And health:

Startup

Readiness

Liveness

One thing I want you to remember

Kubernetes is not just a container orchestrator.

You're gradually seeing that it's a collection of systems that solve different problems:

Deployment

→ application lifecycle

Scheduler

→ placement

Kubelet

→ execution

Service

→ networking/discovery

Probes

→ health

Namespace

→ organization/isolation

ResourceQuota

→ resource governance

LimitRange

→ resource defaults/constraints

Once you see these as separate responsibilities, Kubernetes becomes much easier to understand.

Mini exercise

Before moving ahead, try to answer these without looking back.

Q1 You have:

```
payments namespace
```

You want to limit it to:

```
50 Pods
```

Which object?

Answer: ResourceQuota

Q2 You want every container in a namespace to receive a default:

```
CPU request = 250m
```

Which object?

Answer: LimitRange

Q3 You want Alice to be allowed to deploy only into:

```
payments namespace
```

Which concept?

Answer: RBAC

Q4 You want:

```
payments → payment-db
```

but prevent:

```
employee → payment-db
```

Which concept?

Answer: NetworkPolicy

Q5 You want three Pods distributed across different Nodes.

Which scheduling concepts could help?

Answer: Pod anti-affinity and/or topology spread constraints.

chapter 14 Summary

The hierarchy to remember:

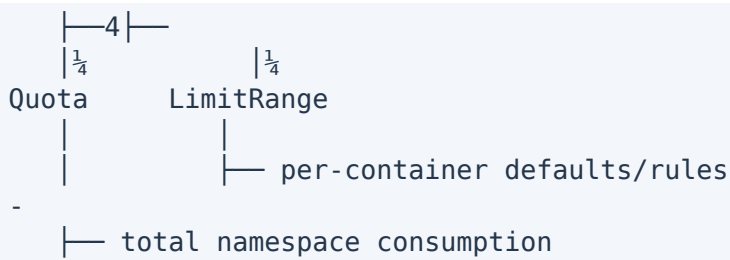
```
Kubernetes Cluster
```

```
-
```

```
  |  $\frac{1}{4}$ 
```

```
  Namespace
```

```
-
```



And:

Namespace
→ logical boundary

ResourceQuota
→ total resource/object limits

LimitRange
→ individual resource defaults/min/max

OpenShift Project
→ OpenShift's enhanced namespace/project concept

Next chapter — chapter 15: Kubernetes Security — RBAC This is the next logical step.

We✓I build a real company scenario:



We✓I learn:

User
ServiceAccount
Role
ClusterRole
RoleBinding
ClusterRoleBinding

And most importantly:

Why can I run `kubectl get pods` but not `kubectl delete pods`?

We✓I also connect this to OpenShift's security model, which is particularly important in enterprise environments.

Chapter 18 — Horizontal Pod Autoscaling

CHAPTER FOCUS

Understand **Horizontal Pod Autoscaling** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

We already know how to scale manually:

```
kubectl scale deployment employee-service --replicas=5
```

But production traffic doesn't stay constant.

Imagine:

```
10:00 AM → 500 requests/min
12:00 PM → 5,000 requests/min
06:00 PM → 700 requests/min
```

Keeping 10 Pods running all day wastes resources, while keeping only 2 Pods during peak traffic may overload the application.

We want Kubernetes to do this automatically:

```
Low traffic
  ↓
  2 Pods

Traffic increases
  ↓
  4 Pods
  ↓
  7 Pods

Traffic decreases
  ↓
  4 Pods
  ↓
  2 Pods
```

That is the purpose of the Horizontal Pod Autoscaler (HPA).

What does “horizontal” mean? Horizontal scaling means:

```
More load
  ↓
Add more Pods
```

Less load
↓
Remove Pods

For example:

3 Pods → 6 Pods

Vertical scaling is different:

One Pod
500m CPU / 512Mi
↓
One Pod
1 CPU / 1Gi

So:

Horizontal scaling
→ change NUMBER of Pods

Vertical scaling
→ change RESOURCES of Pods

Kubernetes' HPA automatically changes the replica count of scalable workloads such as Deployments and StatefulSets; it does not apply to workloads such as DaemonSets that do not expose normal replica scaling.

Where does HPA sit? Remember:

Deployment
↓
ReplicaSet
↓
Pods

Add HPA:

```

    Metrics
-
    | 1/4
    HPA
-
changes replicas
-
    | 1/4
    Deployment
-
    | 1/4
    ReplicaSet
-
    | 1/4

```

Pods

HPA doesn't directly create Pods.

It effectively changes:

```
spec:
  replicas: ...
```

on the target workload.

Then the normal Deployment/ReplicaSet controllers do what you already learned.

A simple HPA example Suppose:

employee-service

normally runs:

3 Pods

We want:

```
Minimum = 3
Maximum = 10
Target CPU = 70|
```

An HPA might look like:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler

metadata:
  name: employee-service

spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: employee-service

  minReplicas: 3
  maxReplicas: 10

  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

The stable HPA API is autoscaling/v2 , which supports resource, custom, external and multiple metrics.

What does 70% CPU mean? This is where chapter 20 becomes important.

Suppose every employee Pod has:

```
resources:
  requests:
    cpu: "500m"
```

HPA target:

70 |

Then approximately:

70 | of 500m
=
350m

So HPA tries to keep average CPU usage around:

350m per Pod

for this configuration.

This is why CPU requests matter for HPA. For utilization-based resource scaling, Kubernetes calculates utilization relative to the Pods' corresponding resource requests. If the necessary CPU request is missing from a relevant container, utilization for that Pod may be undefined and HPA cannot use that metric normally.

Practical example Assume:

3 Pods

CPU request per Pod = 500m

HPA CPU target = 70 |

Current usage:

Pod A → 450m

Pod B → 400m

Pod C → 500m

Average:

$(450 + 400 + 500) / 3$
=
450m

Relative to request:

$450 / 500$
=
90 |

But target is:

```
70|
```

So HPA says:

Current utilization is significantly above target. We need more capacity.

How does HPA calculate replicas? At a simplified level Kubernetes uses:

```
desired replicas
=
current replicas
×
(current metric / target metric)
```

and rounds upward.

Using our example:

```
Current replicas = 3
Current CPU = 90|
Target CPU = 70|
```

Calculation:

```
3 × (90 / 70)
```

```
"H 3.86
```

Round upward:

```
4 Pods
```

So:

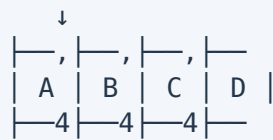
```
3 Pods
  ↓
HPA
  ↓
4 Pods
```

Why adding Pods helps Before:

```
Traffic
  ↓
|—, |—, |—
|Pod A |Pod B |Pod C |
|—4|—4|—
CPU "H 90|
```

After:

Traffic



CPU per Pod decreases

Because the Service distributes traffic across more ready endpoints.

Eventually average utilization might become:

65–70|

and HPA stops scaling.

What happens when traffic drops? Suppose now:

```

Current Pods = 8
CPU average = 25|
Target = 70|
  
```

The HPA calculation points toward fewer replicas.

Conceptually:

```

8 Pods
  ↓
6
  ↓
4
  ↓
3
  
```

but never below:

minReplicas: 3

And never above:

maxReplicas: 10

So:

```

minReplicas
  ↓
lower safety boundary

maxReplicas
  ↓
upper scaling boundary
  
```

Where does HPA get CPU usage? Remember chapter 34:

```

kublet
  ↓
Metrics Server
  ↓
Metrics API

```

HPA can retrieve Pod resource usage through the Kubernetes Resource Metrics API, which is commonly supplied by Metrics Server. The same lightweight metrics pipeline also powers commands such as `kubectl top`.

Conceptually:

```

Container Runtime
  ↓
Kubelet
  ↓
Metrics Server
  ↓
metrics.k8s.io
  ↓
HPA Controller
  ↓
Scaling decision

```

HPA is a reconciliation loop just like everything else in Kubernetes:

```

Observe
  ↓
Compare
  ↓
Act
  ↓
Observe again

```

HPA doesn't look once and disappear.

Conceptually:

```

Metrics
  ↓
CPU too high?
  ↓ yes
Increase replicas
  ↓
wait / observe
  ↓
Measure again

```

Kubernetes implements HPA as a periodic control loop rather than a truly continuous calculation; the default controller sync interval documented by Kubernetes is 15 seconds.

Why doesn't it instantly scale $3 \rightarrow 10 \rightarrow 3$? Imagine CPU briefly changes:

```
65 |  
72 |  
68 |  
71 |  
69 |
```

If Kubernetes reacted aggressively to every tiny fluctuation:

```
3 → 4 → 3 → 4 → 3
```

you'd get flapping.

HPA therefore supports tolerance, stabilization windows and configurable scale-up/scale-down policies to make scaling behavior less unstable.

Scale-up and scale-down are intentionally different When traffic suddenly explodes:

```
100 req/s  
  ↓  
5000 req/s
```

you usually want relatively quick scaling.

But when traffic falls briefly, immediately deleting many Pods can be dangerous.

So conceptually:

```
Scale UP  
→ respond reasonably quickly  
  
Scale DOWN  
→ more conservative/stabilized
```

You can tune this with the HPA behavior configuration.

Java startup creates an interesting HPA problem Imagine Spring Boot Pod startup:

```
Container starts  
  ↓  
JVM startup  
  ↓  
Class loading  
  ↓  
Spring context  
  ↓  
cache initialization  
  ↓  
CPU spike  
  ↓  
Ready
```

Suppose startup CPU hits:

95 |

HPA might otherwise misinterpret:

```
"Application overloaded!"
```

when it's actually:

```
"New JVM is warming up."
```

Kubernetes has HPA logic around not overreacting to initializing/unready Pods, and its documentation specifically

recommends correctly designed startup/readiness probes for workloads with startup CPU spikes such as Java applications.

This connects directly to the probe chapter.

HPA can use memory too Example:

```
metrics:
  - type: Resource
    resource:
      name: memory
      target:
        type: Utilization
        averageUtilization: 70
```

Conceptually:

```
Memory rises
  ↓
HPA
  ↓
More Pods
```

But memory can be trickier than CPU.

Imagine a Java memory leak:

```
Pod A leaks memory
  ↓
HPA adds Pod B
  ↓
Pod B leaks too
  ↓
HPA adds Pod C
```

You haven't fixed the leak.

You've only created more leaking processes.

So:

Autoscaling cannot fix inefficient or broken applications.

HPA can use more than CPU/memory With autoscaling/v2 , HPA can use resource, per-Pod custom, object and external metrics, and can consider multiple metrics. When multiple metrics are configured, the controller calculates the replica recommendation for each and uses the largest recommendation, subject to the configured maximum.

For example, you might eventually scale based on:

```
HTTP requests/sec
Kafka consumer lag
Queue depth
messages waiting
transactions/sec
```

Imagine a Kafka consumer:

```
Lag = 100
→ 2 consumers

Lag = 100,000
→ 10 consumers
```

Scaling based on queue lag may make much more business sense than CPU for that workload.

HPA vs VPA vs Cluster Autoscaler This is a very important distinction.

Autoscaler Changes

HPA Number of Pods

VPA CPU/memory sizing of Pods

Cluster Autoscaler Number of Nodes

Think:

```
HPA
3 Pods → 8 Pods

VPA
512Mi → 1Gi per Pod

Cluster Autoscaler
3 Nodes → 5 Nodes
```

They can work together Imagine traffic increases.

HPA:

```
3 Pods
↓
10 Pods
```

But scheduler says:

Pods 8–10

Pending

Reason:

Insufficient CPU

Why?

The cluster doesn't have enough Node capacity.

Then a Node autoscaler can conceptually do:

3 worker Nodes

↓

5 worker Nodes

Then:

Pending Pods

↓

Scheduler

↓

new Nodes

So:

Traffic !'

↓

HPA adds Pods

↓

Cluster lacks capacity

↓

Node autoscaler adds Nodes

Different layers solving different problems.

HPA doesn't mean unlimited scaling Suppose:

maxReplicas: 10

Traffic suddenly requires:

30 Pods

HPA stops at:

10

So:

HPA configured

"`

infinite capacity

You still need:

```
capacity planning
load testing
appropriate max replicas
database capacity
downstream capacity
```

Another important problem: the database Imagine:

```
3 application Pods
  ↓
DB connections:
10 each

Total = 30
```

HPA scales to:

```
10 Pods
```

Now:

```
10 × 10
=
100 DB connections
```

But database max:

```
60
```

Result:

```
Application autoscaling
  ↓
Database overload
```

So autoscaling one tier can overload another tier.

Very important production lesson:

Scale the system, not just the Pods.

HPA and manually setting replicas Suppose your Deployment YAML says:

```
replicas: 3
```

HPA changes live replicas to:

```
7
```

Then your GitOps system repeatedly enforces:

```
replicas: 3
```

while HPA wants:

```
7
```

You can create conflicting controllers/configuration ownership.

In HPA-managed deployments, teams typically design GitOps/manifest ownership so the autoscaler is allowed to manage replica count rather than continuously fighting with another controller.

The mental model is:

Git

→ defines HPA min/max/targets

HPA

→ dynamically manages current replicas

HPA troubleshooting Suppose you expect scaling but nothing happens.

Check:

```
kubectl get hpa
```

You might see:

NAME	REFERENCE	TARGETS	MINPODS
↪ MAXPODS REPLICAS employee-service	Deployment/employee-service	92 /70	3
↪ 10 3			

Then:

```
kubectl describe hpa employee-service
```

Look at:

```
current metrics
desired replicas
conditions
events
```

A common failure: You might see:

```
TARGETS
<unknown>/70|
```

Possible reasons include:

```
Metrics API unavailable
Metrics Server issue
```

CPU requests missing
target Pods not providing usable metrics

Remember the dependency:

```
HPA
↓
Metrics API
↓
resource metrics
```

Without metrics:

```
No measurement
↓
No sensible scaling decision
```

HPA + requests troubleshooting Suppose:

```
containers:
- name: employee
  resources:
    limits:
      cpu: "1"
```

but no CPU request is ultimately defined.

Then utilization-based CPU HPA may not have the resource request baseline it needs.

Better:

```
resources:
  requests:
    cpu: "500m"
  limits:
    cpu: "1"
```

Now:

```
usage = 400m
request = 500m

utilization "H 80|"
```

The HPA has meaningful utilization data.

OpenShift The same Kubernetes HPA resource and concepts apply in OpenShift because HPA is part of Kubernetes itself.

Conceptually:

```
OpenShift
  ↓
Deployment
  ↓
HPA
  ↓
replica count changes
  ↓
Pods
```

You can inspect HPA with familiar commands such as:

```
oc get hpa
```

and:

```
oc describe hpa employee-service
```

So this knowledge transfers directly.

Production example Suppose your employee API has:

```
minReplicas = 3
maxReplicas = 12
CPU target = 70|

CPU request = 500m
```

Morning:

```
3 Pods
CPU "H 35|
```

Lunch traffic:

```
CPU "H 95|
```

HPA:

```
3
↓
5
↓
7
```

Traffic distributes:

```
CPU "H 68|
```

HPA stabilizes around:

7 Pods

Evening:

```
traffic ↓  
CPU "H 20|
```

HPA gradually scales:

```
7  
↓  
5  
↓  
3
```

and stops because:

```
minReplicas = 3
```

That is the fundamental autoscaling lifecycle.

Chapter 19 — Kubernetes Scheduling

CHAPTER FOCUS

Understand **Kubernetes Scheduling** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

This chapter answers a very important question:

When I create a Pod, how does Kubernetes decide which Node should run it?

You already know:

```
Cluster
-
├─ Node 1
├─ Node 2
└─ Node 3
```

And:

```
Deployment
  ↓
  Pod
```

But who decides whether the Pod goes to Node 1, 2, or 3?

That is the Kubernetes Scheduler.

The Kubernetes Scheduler

Suppose you create:

```
replicas: 3
```

Kubernetes creates three Pods:

```
employee-1
employee-2
employee-3
```

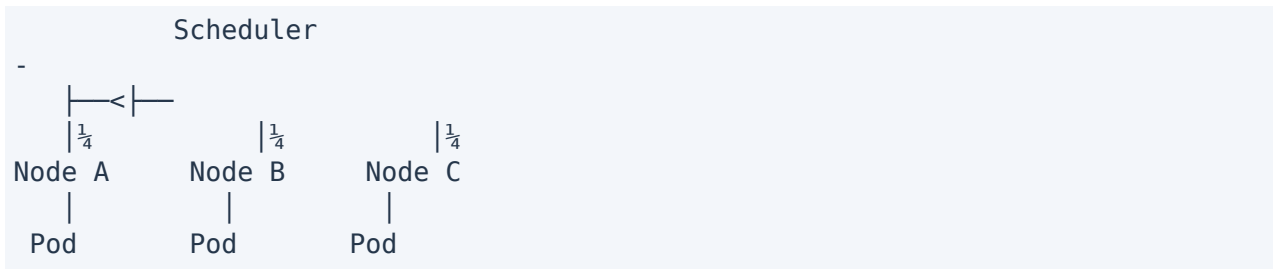
But they don't know where to run.

The scheduler looks at the available Nodes:

```
Node A
Node B
Node C
```

and decides where each Pod should be placed.

Conceptually:



Why scheduling matters

Imagine you have:

Node A → 2 CPUs
 Node B → 8 CPUs
 Node C → 16 CPUs

And your Pod needs:

CPU = 8
 Memory = 16 GB

The scheduler cannot put it on:

Node A ×
 Node B maybe
 Node C ' (not possible)

So scheduling isn't simply:

"Put the Pod on the first available Node."

Kubernetes evaluates several constraints.

The basic scheduling process

A simplified version is:

```

Pod created
↓
Scheduler sees unscheduled Pod
↓
Find Nodes that can run it
↓
Filter unsuitable Nodes
↓
Rank suitable Nodes
↓
Choose best Node
  
```

↓

Pod gets assigned to Node

The actual scheduler is more sophisticated, but this mental model is enough for now.

4. Why would a Node be unsuitable?

Several reasons.

For example:

```
Node doesn't have enough CPU
Node doesn't have enough memory
Node has incompatible taint
Pod requires a specific Node
Node doesn't satisfy affinity rules
Node is unavailable
```

We'll learn these one by one.

CPU and Memory Requests

This is one of the most important scheduling concepts.

Suppose your Pod says:

```
resources:
  requests:
    cpu: "500m"
    memory: "512Mi"
```

You're telling Kubernetes:

"Please schedule me on a Node that can reserve at least this much CPU and memory."

6. What is 500m CPU?

Kubernetes CPU is expressed in CPU units.

1 CPU = 1000m

Therefore:

500m = 0.5 CPU

Examples:

```
100m = 0.1 CPU
250m = 0.25 CPU
500m = 0.5 CPU
1000m = 1 CPU
```



```
2000m = 2 CPU
```

The m means millicpu, not megabytes.

| Memory units

You might see:

```
512Mi
1Gi
2Gi
4Gi
```

For example:

```
512Mi "H 512 MiB
1Gi   "H 1 GiB
```

So:

```
resources:
  requests:
    cpu: "500m"
    memory: "512Mi"
```

means approximately:

```
CPU:
0.5 CPU

Memory:
512 MiB
```

| Request vs Limit

You✓I often see:

```
resources:
  requests:
    cpu: "500m"
    memory: "512Mi"

  limits:
    cpu: "1"
    memory: "1Gi"
```

These are not the same thing.

Request

How much resource should Kubernetes reserve/consider for scheduling?

Limit

How much resource is the container allowed to consume?

Think:

```
Request = scheduling guarantee  
Limit   = consumption ceiling
```

We✓I go much deeper into this later.

| Scheduling example

Suppose:

```
Node A  
CPU capacity = 4 CPU  
Already requested = 3.5 CPU
```

```
Node B  
CPU capacity = 4 CPU
```

```
Already requested = 1 CPU
```

New Pod requests:

```
CPU = 1 CPU
```

Scheduler evaluates:

```
Node A  
3.5 + 1 = 4.5  
× insufficient
```

```
Node B  
1 + 1 = 2  
,
```

Therefore:

```
Pod → Node B
```

| Important: Scheduler uses requests

Beginners often think:

“Node has 4 CPUs and currently the applications are only using 2 CPUs, so a Pod requesting 3 CPUs should fit.” Not necessarily.

Kubernetes scheduling primarily considers resource requests, not simply instantaneous CPU usage.

For example:

```
Node capacity = 4 CPU
```

```
Existing requests:
```

```
Pod A = 2 CPU
```

```
Pod B = 1 CPU
```

```
Available allocatable based on requests:
```

```
"H 1 CPU
```

A new Pod requesting:

```
2 CPU
```

may remain Pending even if actual CPU usage happens to be low.

11. Why?

Because Kubernetes wants predictable resource allocation.

Imagine:

```
Pod A requested 2 CPU
```

```
but currently uses only 0.2 CPU.
```

If Kubernetes ignored requests:

```
new Pod
```

```
could consume the "unused" 1.8 CPU
```

Then Pod A suddenly needs its requested capacity.

You could get resource contention.

Requests establish the scheduler's planning model.

Node Labels

Now suppose you have different types of Nodes.

```
Node A → normal
```

```
Node B → normal
```

```
Node C → GPU
```

```
Node D → GPU
```

You can label Nodes:

```
Node A
```

```
type=general
```

```
Node B
```

```
type=general
```

```
Node C
```

```
type=gpu
```

```
Node D  
type=gpu
```

Labels are simple key-value metadata:

```
key=value
```

Example:

```
environment=production
```

or:

```
disk=ssd
```

or:

```
workload=gpu
```

| Why labels matter

Suppose you have an AI application requiring a GPU.

You don't want:

```
AI Pod → normal Node ×
```

You want:

```
AI Pod → GPU Node ✓
```

Labels and scheduling rules allow you to express this.

| nodeSelector

The simplest mechanism is:

```
nodeSelector:  
  workload: gpu
```

This means:

Schedule this Pod only on Nodes having workload=gpu .

Example:

```
Node A  
workload=general
```

```
Node B
```

```
workload=gpu
```

Node C

```
workload=gpu
```

Pod:

```
nodeSelector:  
  workload: gpu
```

Possible result:

```
Pod → Node B or Node C
```

It won't go to Node A.

| Real-world example

Imagine an organization has:

General Nodes

```
├─ node-1  
├─ node-2  
└─ node-3
```

High-memory Nodes

```
├─ node-4  
└─ node-5
```

A large data-processing application requires high memory.

Label:

```
node-4 → memory=high  
node-5 → memory=high
```

Pod:

```
nodeSelector:  
  memory: high
```

Now Kubernetes knows:

```
Data Processing Pod  
    ↓  
High-memory Nodes only
```

| Node Affinity

nodeSelector is simple but limited.

For more expressive rules, Kubernetes provides:

Node Affinity For example:

Prefer SSD Nodes.

or:

Must run on Nodes in this region.

or:

Run on either GPU or high-memory Nodes.

Conceptually:

```
nodeSelector
→ simple requirement

nodeAffinity
→ advanced scheduling preference/requirement
```

| Required vs Preferred

Affinity can express:

Required

This condition **MUST** be satisfied.

```
GPU required
```

If no GPU Node exists:

```
Pod → Pending
```

Preferred

I'd **LIKE** this condition to be satisfied, but it's not mandatory.

For example:

```
Prefer SSD Nodes
```

If no SSD Node is available:

```
Pod can still run elsewhere
```

This distinction is extremely useful.

| Think of affinity like this

```
Required
=
"Don't break this rule."
```

```
Preferred
=  
"Try to follow this preference."
```

| Pod Affinity

So far we said:

Put this Pod near a particular type of Node.

But Kubernetes can also express:

Put this Pod near another Pod.

That is:

Pod Affinity

| Real-world example

Suppose:

```
Frontend Pod  
Backend Pod
```

You may want:

```
Frontend  
  ↓  
close to  
  ↓  
Backend
```

Why?

Potentially:

```
lower network latency
```

You could use Pod affinity to prefer placing them in the same topology domain.

Conceptually:

```
Node A  
├─ Frontend  
├─ Backend
```

instead of:

```
Node A  
├─ Frontend
```

```
Node Z
├─ Backend
```

| Pod Anti-Affinity

Now the opposite.

Suppose you have:

```
Payment Service
replicas = 3
```

You don't want:

```
Node A
├─ payment-1
├─ payment-2
├─ payment-3
```

Why?

If Node A crashes:

```
ALL 3 replicas disappear
```

That is terrible for availability.

Instead, you'd prefer:

```
Node A → payment-1
Node B → payment-2
Node C → payment-3
```

This is a classic use case for:

Pod Anti-Affinity

| Anti-affinity mental model

```
Affinity
=
"Put these things together."
```

```
Anti-affinity
=
"Keep these things apart."
```

| Why this matters for high availability

Imagine:


```
3 replicas
3 Nodes
```

Without anti-affinity:

```
Node A
├── Pod 1
├── Pod 2
└── Pod 3
```

Node A failure:

```
× all replicas
```

With anti-affinity:

```
Node A → Pod 1
Node B → Pod 2
Node C → Pod 3
```

Node A failure:

```
Pod 1 ×
Pod 2 '
Pod 3 '
```

Much better.

But there is another problem

What if:

```
3 replicas
2 Nodes
```

and strict anti-affinity says:

Never place two replicas on the same Node.

Then Kubernetes has a problem:

```
Pod 1 → Node A
Pod 2 → Node B
Pod 3 → ????
```

There is nowhere valid to place Pod 3.

Result:

```
Pod 3 = Pending
```

This is why hard scheduling requirements need to be used carefully.

Taints and Tolerations

Now we come to one of the most important scheduling concepts.

Labels answer:

“Which Pods should prefer/use this Node?”

Taints answer:

“Which Pods should NOT run on this Node?”

This is a powerful distinction.

Imagine a special Node

Suppose:

```
Node A
```

is reserved for:

```
critical workloads
```

You don't want random application Pods landing there.

You can put a taint on Node A:

```
dedicated=critical:NoSchedule
```

Now Kubernetes says:

Don't schedule Pods here unless they explicitly tolerate this taint.

Toleration

A Pod can declare:

```
tolerations:
- key: "dedicated"
  operator: "Equal"
  value: "critical"
  effect: "NoSchedule"
```

This tells Kubernetes:

This Pod tolerates the dedicated=critical taint.

Now it can potentially run there.

Taint vs Toleration

Memorize this:

```
Taint
=
Node says:
"Keep away unless allowed."

Toleration
=
Pod says:
"I am allowed to tolerate that restriction."
```

| Important: Toleration doesn't force scheduling

This is a common interview question.

Suppose:

```
Node A
taint = dedicated=critical
```

and Pod has matching toleration.

Does Kubernetes automatically schedule the Pod onto Node A?

No.

The toleration merely means:

The Pod is not excluded by that taint.

Other scheduling rules still apply.

So:

```
Taint/Toleration
"\`
"Send Pod here"
```

Instead:

```
Taint
→ keeps Pods away

Toleration
→ removes that particular barrier
```

```
nodeSelector
```

or:

```
nodeAffinity
```

| Real-world example: GPU Nodes

Suppose you have:

```
Node 1 → normal
Node 2 → normal
Node 3 → GPU
```

GPU Nodes are expensive.

You don't want normal workloads accidentally consuming them.

You can use:

```
GPU Node
taint:
gpu=true:NoSchedule
```

Now:

```
Normal Pod
→ cannot schedule on GPU Node

GPU workload
→ has matching toleration
→ can schedule there
```

You might combine:

```
Taint + Toleration + Node Affinity
```

to ensure GPU workloads go to GPU Nodes and ordinary workloads stay away.

| NoSchedule

One common taint effect is:

```
NoSchedule
```

Meaning:

Don't schedule new Pods onto this Node unless they tolerate the taint.

Existing Pods aren't necessarily immediately removed merely because you added this taint.

| NoExecute

Another effect:

```
NoExecute
```

This is stronger.

It can affect Pods already running on the Node.

Conceptually:

```
Node gets NoExecute taint
↓
Pods without appropriate toleration
↓
may be evicted
```

We'll revisit eviction when we study node failures and resource pressure.

PreferNoSchedule

This expresses more of a preference:

Try not to schedule here if possible.

It's softer than NoSchedule .

Think:

```
NoSchedule
→ strong restriction

PreferNoSchedule
→ soft avoidance

NoExecute
→ can also evict affected existing Pods
```

Scheduling decision example

Let's put everything together.

You have:

```
Node A
CPU = 4
label: workload=general

Node B
CPU = 8
label: workload=gpu
taint: gpu=true:NoSchedule

Node C
CPU = 8
label: workload=general
```

Your GPU Pod requires:

```
CPU = 4
nodeSelector:
  workload=gpu

toleration:
  gpu=true:NoSchedule
```

Scheduler evaluates:

```
Node A
label mismatch
x

Node B
label matches
CPU sufficient
taint tolerated
'

Node C
label mismatch
x
```

Result:

```
GPU Pod → Node B
```

That is real Kubernetes scheduling logic.

35. Why does my Pod say Pending ?

This is one of the most common Kubernetes troubleshooting scenarios.

You run:

```
kubectl get pods
```

and see:

NAME	READY	STATUS
employee-abc	0/1	Pending

What does that mean?

The container may not even have started yet.

The scheduler hasn't successfully placed the Pod.

First troubleshooting command

Run:

```
kubectl describe pod employee-abc
```

Look at:

```
Events
```

You might see something like:

```
0/3 nodes are available:
2 insufficient cpu,
1 node had intolerated taint
```

Now you know exactly why scheduling failed.

Example: insufficient CPU

Suppose:

```
Pod request:
CPU = 4
```

Nodes:

```
Node A → only 2 CPU available
Node B → only 1 CPU available
Node C → only 3 CPU available
```

Scheduler says:

```
No suitable Node
```

Pod stays:

```
Pending
```

The solution isn't:

Restart the Pod.

You need to solve the scheduling/resource problem.

Example: taint problem

Events:

```
0/3 nodes are available:
3 node(s) had intolerated taint
```

This means:

```
Node restrictions
  ↓
Pod doesn't tolerate them
  ↓
No valid Node
  ↓
Pending
```

You need to understand the Node taints and decide whether the Pod should tolerate them.

Example: node selector problem

Pod:

```
nodeSelector:
  workload: gpu
```

But Nodes have:

```
workload=general
```

No Node matches.

Result:

```
Pending
```

This is a very common mistake when labels are misspelled.

For example:

```
Pod expects:
workload=gpu
```

```
Node actually has:
workloads=gpu
```

That tiny difference means:

```
No match
```

How to inspect Node labels

Run:

```
kubectl get nodes --show-labels
```

Or:


```
kubectl describe node <node-name>
```

You✓I see labels such as:

```
kubernetes.io/hostname=worker-1  
node-role.kubernetes.io/worker=  
workload=gpu
```

| How to add a label

For example:

```
kubectl label node worker-1 workload=gpu
```

Now:

```
worker-1  
workload=gpu
```

Your Pod with:

```
nodeSelector:  
  workload: gpu
```

can consider that Node.

| Scheduler doesn't just look at CPU

A simplified list of things that can affect scheduling:

```
CPU requests  
Memory requests  
Node selectors  
Node affinity  
Pod affinity  
Pod anti-affinity  
Taints  
Tolerations  
Topology constraints  
Node availability  
Other scheduling constraints
```

This is why the scheduler is more sophisticated than:

```
"find an empty Node"
```

| Topology

Now one more concept.

Nodes belong to physical/logical topology.

For example:

```
Region
├── Zone A
│   ├── Node 1
│   └── Node 2
└── -
    ├── Zone B
    │   ├── Node 3
    │   └── Node 4
```

You might want your replicas distributed across zones.

Why?

If:

```
Zone A
```

has an outage:

```
Node 1 x
Node 2 x
```

you still have:

```
Zone B
Node 3 '
Node 4 '
```

This is another aspect of high availability.

We✓I cover topology spread constraints in more detail later.

Pod topology spread

Modern Kubernetes provides:

```
topologySpreadConstraints
```

which can help distribute Pods evenly across topology domains.

For example:

```
3 replicas
2 zones
```

You might want:

```
Zone A → 2 Pods
Zone B → 1 Pod
```

rather than:

```
Zone A → 3 Pods  
Zone B → 0 Pods
```

This can be more flexible than relying only on pod anti-affinity.

| Why scheduling matters for your Spring Boot applications

Imagine:

```
Employee Service  
replicas = 3
```

If Kubernetes places all three on one Node:

```
Node A  
├─ Employee 1  
├─ Employee 2  
└─ Employee 3
```

then:

```
Node A crashes  
  ↓  
all replicas disappear
```

So production architecture often combines:

```
Deployment  
+  
Pod anti-affinity / topology spread  
+  
multiple Nodes
```

to improve availability.

| Scheduling and StatefulSets

Remember our StatefulSet:

```
database-0  
database-1  
database-2
```

Scheduling also decides where these Pods run.

You might want:

```
database-0 → Zone A  
database-1 → Zone B  
database-2 → Zone C
```

so one infrastructure failure doesn't take down the entire database cluster.

However, the database's own replication/HA design still matters.

Kubernetes scheduling alone doesn't create database HA.

Scheduling and DaemonSets

DaemonSets are different.

You don't normally tell a DaemonSet:

```
replicas: 5
```

Instead:

eligible Node

DaemonSet Pod

But even DaemonSet Pods can have:

tolerations

```
node selectors
```

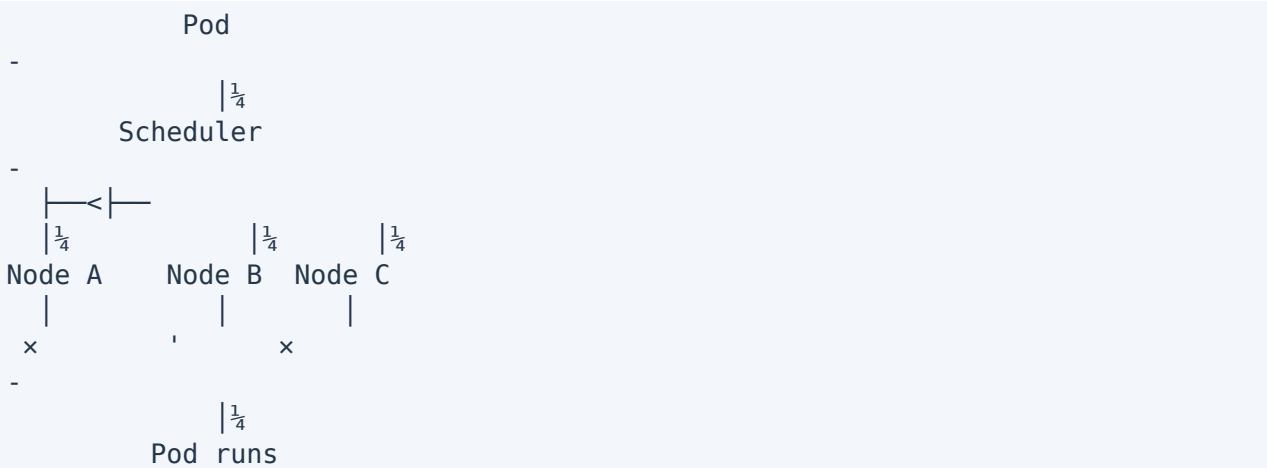
affinity

```
resource requests
```

So scheduling rules still matter.

The scheduler mental model

Whenever you create a Pod, think:



Why were A and C rejected?

Maybe:

CPU

Memory

Labels

Affinity
Taints
Topology

| A very useful distinction

Node Selector

"Run only on Nodes with this label."

Node Affinity

"Use more sophisticated Node selection rules."

Pod Affinity

"Put this Pod near other Pods."

Pod Anti-Affinity

"Keep this Pod away from other Pods."

Taint

"Keep Pods away from this Node."

Toleration

"This Pod is allowed to be on that tainted Node."

Memorize these six concepts.

| Real production example

Imagine your company has:

Cluster

```
-  
├─ General Nodes  
│   ├── Node 1  
│   └── Node 2  
│  
│   └── Node 3  
-  
├─ High-memory Nodes  
│   ├── Node 4  
│   └── Node 5  
-  
├─ GPU Nodes  
│   ├── Node 6  
│   └── Node 7
```

Applications:

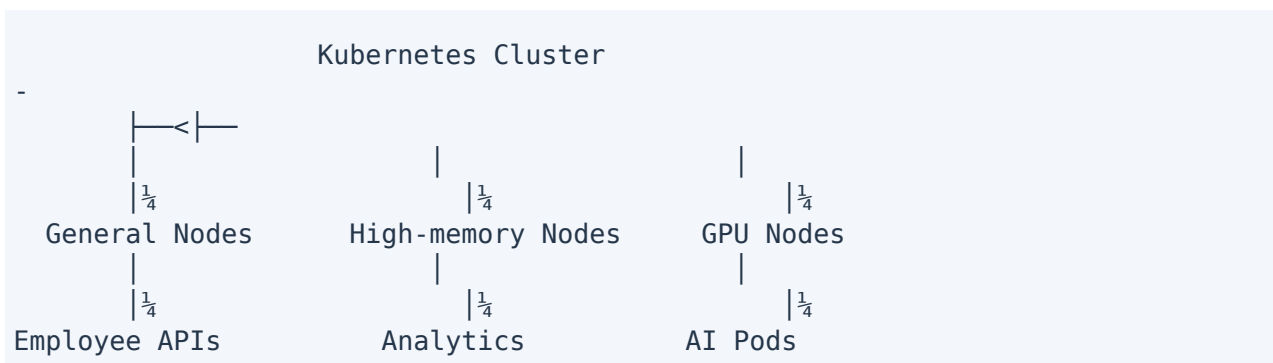
Employee API
→ General Nodes

Analytics
→ High-memory Nodes

AI inference
→ GPU Nodes

Scheduling policies can enforce this architecture.

Example architecture



And then:

Pod anti-affinity
+
topology spread

can distribute replicas for resilience.

Important interview question

What happens if a Pod cannot be scheduled? Answer:

The Pod remains in Pending state until the scheduler finds a suitable Node or the scheduling constraints/resources change.

Then investigate:

```
kubectl describe pod <pod>
```

especially the:

Events

section.

| Important interview question

Difference between taint and toleration? Answer:

A taint is applied to a Node to repel Pods, while a toleration is applied to a Pod to allow it to be scheduled onto a Node with a matching taint.

And remember:

A toleration does not force the Pod onto that Node.

| Important interview question

Difference between nodeSelector and nodeAffinity? Simple answer:

nodeSelector provides simple label-based Node selection, while node affinity provides more expressive required or preferred scheduling rules.

| Important interview question

Why use Pod anti-affinity? Answer:

To prevent replicas of the same workload from being concentrated on the same failure domain, improving availability if a Node or other topology domain fails.

| Let's connect this with what you've already learned

You previously learned:

```
Deployment
```

which manages:

```
ReplicaSet
```

```
↓
```

```
Pods
```

Now add scheduling:

```
Deployment
```

```
↓
```

```
ReplicaSet
```

```
↓
```

```
Pods
```

```
↓
```

```
Scheduler
```

```
↓
```

```
Node
```

```
↓
```

```
Container
```

And networking:

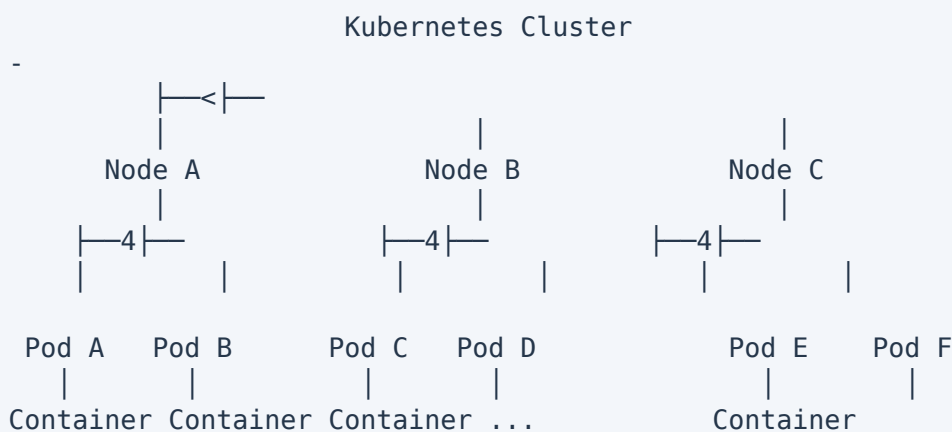
```
Service
  ↓
ready Pods
  ↓
running on Nodes
```

And health:

```
Pod
├─ Startup Probe
├─ Readiness Probe
└─ Liveness Probe
```

Now Kubernetes is starting to form one coherent picture.

The complete picture so far



Controllers decide what should exist.

Scheduler decides where Pods should run.

Kubelet on each Node makes sure the assigned Pods actually run.

Services decide how workloads are reached.

Probes tell Kubernetes whether applications are ready/alive.

That separation is fundamental.

One more important component: Kubelet

You haven't explicitly studied this yet.

Once scheduler decides:

```
Pod → Node B
```

who actually starts the Pod?

The:

Kubelet runs on each Node.

Conceptually:

Control Plane

-

Scheduler

-

| "Pod should run on Node B"
| $\frac{1}{4}$

Node B

-

Kubelet

-

| $\frac{1}{4}$

Container Runtime

-

| $\frac{1}{4}$

Container

We'll study this in the next chapter because it ties together:

Scheduler

Kubelet

Container Runtime

Control Plane

and will finally give you a strong understanding of how Kubernetes actually works internally, rather than just knowing YAML objects.

Security and Multi-Tenancy

Chapter 20 — RBAC and Service Accounts

CHAPTER FOCUS

Understand **RBAC and Service Accounts** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

Now we're entering one of the most important Kubernetes topics for real production environments.

So far, we learned:

```
Namespace
ResourceQuota
LimitRange
Deployment
Pod
Service
Scheduler
Kubelet
```

But we haven't answered a critical question:

Who is allowed to do what inside the cluster?

That is what RBAC solves.

1. What is RBAC?

RBAC = Role-Based Access Control

The basic idea is:

```
WHO
↓
CAN DO WHAT
↓
WHERE
```

For example:

```
Alice
↓
can get/list Pods
↓
in employee-dev namespace
```

But:

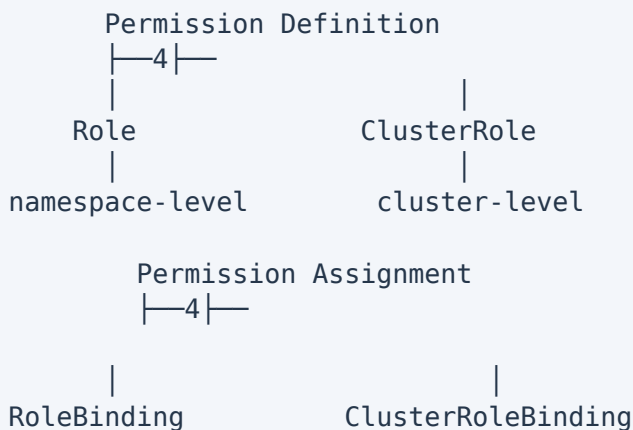
```
Alice
↓
cannot delete Pods
↓
in employee-prod
```

The four core RBAC objects

You should know these four very well:

```
Role
ClusterRole
RoleBinding
ClusterRoleBinding
```

Think of them as two pairs:



Don't worry if that looks confusing yet.

We'll build it step by step.

First: Permission

Suppose I tell you:

Alice can read Pods.

That is a permission.

But Kubernetes doesn't usually say:

```
Alice = read Pods
```

Instead, Kubernetes separates:

Permission definition

Role

from:

Who receives that permission

RoleBinding

This separation is extremely powerful.

Role

A Role defines permissions within a namespace.

Example:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role

metadata:
  name: pod-reader
  namespace: employee-dev

rules:
  - apiGroups: [""]
    resources: ["pods"]

    verbs: ["get", "list", "watch"]
```

Let's understand this carefully.

5. What does this Role say?

The important part:

```
resources:
  - pods
```

means:

We're defining permissions for Pods.

And:

```
verbs:
  - get
  - list
  - watch
```

means:

The user can read/watch Pods.

So:

```
get      → get one Pod
list     → list Pods
watch    → watch changes
```

6. What can this user NOT do?

They haven't been given:

```
create
update
patch
delete
```

Therefore they can't use those actions through this Role.

This is called:

Least privilege Give someone only the permissions they actually need.

Kubernetes verbs

You✓I see these frequently:

```
get
list
watch
create
update
patch
delete
deletecollection
```

A rough interpretation:

Verb Meaning

get Read one object

list List objects

watch Watch changes

create Create

update Replace/update

patch Partially modify

delete Delete

Example: Developer

Imagine a developer needs to inspect Pods.

They need:

```
get
list
watch
```

They don't necessarily need:

```
delete
```

So their Role might be:

```
Pod Reader
-
  ├── get
  ├── list
  └── watch
```

This is much safer than giving them full admin access.

| Now we have a problem

We've created:

```
Role:
pod-reader
```

But...

Who gets this Role?

A Role doesn't automatically give anyone permission.

We need:

RoleBinding

| RoleBinding

A RoleBinding connects:

```
WHO
↓
ROLE
↓
NAMESPACE
```

For example:

```
Alice
↓
pod-reader
↓
```

employee-dev

Example RoleBinding

```

apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding

metadata:
  name: alice-pod-reader
  namespace: employee-dev

subjects:
  - kind: User
    name: alice

roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io

```

Now the relationship is:

```

Alice
-
  | ¼
RoleBinding
-
  | ¼
pod-reader Role
-
  | ¼
employee-dev namespace

```

Alice can now read Pods there.

Important distinction

Remember:

Role

doesn't mean:

Alice has permission.

It means:

Here is a set of permissions.

The:


```
RoleBinding
```

says:

Give those permissions to Alice.

| Real example

Imagine:

```
employee-dev
```

contains:

```
employee-api  
employee-worker  
employee-service
```

Developer Alice needs to troubleshoot the application.

She should be able to:

```
get Pods  
list Pods  
watch Pods  
get logs
```

but shouldn't:

```
delete Deployment  
delete Secret  
modify production
```

So we create narrowly scoped permissions.

This is how enterprise Kubernetes security is designed.

14. What is a ClusterRole?

Now let's say you need a permission that applies across the cluster.

For example:

A monitoring system needs to read Pods across all namespaces.

A normal Role is namespace-scoped.

Instead, we can use:

ClusterRole Example:

```
apiVersion: rbac.authorization.k8s.io/v1  
kind: ClusterRole
```

```

metadata:
  name: pod-reader

rules:
  - apiGroups: [""]
    resources: ["pods"]
    verbs: ["get", "list", "watch"]

```

This defines a cluster-level permission set.

Role vs ClusterRole

Memorize this:

```

Role
→ permissions within a namespace

ClusterRole
→ cluster-scoped permission definition

```

But there's an important nuance:

A ClusterRole can also be bound into a specific namespace using a RoleBinding .

We✓I see that shortly.

ClusterRoleBinding

ClusterRoleBinding For example:

```

Alice
↓
ClusterRole
↓
entire cluster

```

Example:

```

apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding

metadata:
  name: alice-cluster-reader

subjects:
  - kind: User
    name: alice

roleRef:
  kind: ClusterRole
  name: pod-reader

```

```
apiGroup: rbac.authorization.k8s.io
```

Now Alice gets that ClusterRole's permissions cluster-wide.

| The four objects

Here's the simplest mental model:

Object Purpose Scope

Role Defines permissions Namespace

RoleBinding Assigns Role Namespace

ClusterRole Defines cluster-level permission set Cluster

ClusterRoleBinding Assigns ClusterRole cluster-wide Cluster

| Very important nuance

A ClusterRole doesn't necessarily mean:

"This permission must be used cluster-wide."

For example:

```
ClusterRole
  ↓
RoleBinding
  ↓
employee-dev
```

The ClusterRole's permissions can be granted only within employee-dev via a RoleBinding.

This is useful because ClusterRoles are reusable permission definitions.

| A real company structure

Imagine:

```
Kubernetes Cluster
-
|— employee-dev
|— employee-prod
|— payments-dev
|— payments-prod
```

Users:

```
Alice → Employee Developer
Bob   → Payments Developer
Carol → Platform Admin
```

Desired access:

```
Alice:
employee-dev → read/write
employee-prod → read-only
```

```
Bob:
payments-dev → read/write
payments-prod → read-only
```

```
Carol:
entire cluster → admin
```

This can be modeled using RBAC.

20. Why not give everyone cluster-admin ?

You might see:

```
cluster-admin
```

This is extremely powerful.

It effectively gives broad administrative access to the cluster.

Giving it to every developer is dangerous.

Imagine:

```
Developer
  ↓
kubectl delete namespace production
```

That is potentially catastrophic.

Instead:

```
Developer
  ↓
specific namespace
  ↓
specific resources
  ↓
specific actions
```

This is the principle of:

Least privilege

| kubectl auth can-i

This is one of the most useful troubleshooting commands.

You can ask:

```
kubectl auth can-i get pods
```

Kubernetes might respond:

```
yes
```

Or:

```
no
```

You can specify namespace:

```
kubectl auth can-i get pods -n employee-prod
```

Or:

```
kubectl auth can-i delete pods -n employee-prod
```

This is extremely useful when debugging RBAC.

| Asking on behalf of a user

An administrator can test:

```
kubectl auth can-i delete pods \  
-n employee-prod \  
--as=alice
```

You might get:

```
no
```

Then:

```
kubectl auth can-i get pods \  
-n employee-dev \  
--as=alice
```

might return:

```
yes
```

Now you can clearly see what Alice can and cannot do.

| Users vs ServiceAccounts

This is another major concept.

RBAC subjects can include:

```
Users
Groups
ServiceAccounts
```

A human developer might be:

```
Alice
```

A workload might use:

```
ServiceAccount
```

This is important because Pods often need to interact with the Kubernetes API.

24. What is a ServiceAccount?

A ServiceAccount is an identity intended for workloads running inside Kubernetes.

For example:

```
employee-api Pod
  ↓
ServiceAccount
  ↓
Kubernetes API
```

The application might need to:

```
read ConfigMaps
watch Pods
read certain Secrets
```

Instead of giving the Pod a human user's credentials, you give it a ServiceAccount with specific permissions.

Real example

Suppose you have:

```
monitoring-agent
```

running inside Kubernetes.

It needs to discover Pods.

You could create:

```
ServiceAccount:
monitoring-agent
```

Then:

```
ServiceAccount
  ↓
Role/ClusterRole
  ↓
get/list/watch Pods
```

Now the application has only the permissions it needs.

ServiceAccount flow

Think:

```
Pod
  ↓
ServiceAccount identity
  ↓
RBAC
  ↓
API Server
```

If the ServiceAccount has permission:

```
API request → allowed
```

Otherwise:

```
API request → forbidden
```

Why this matters for your Java applications

Suppose your Spring Boot application needs to dynamically inspect Kubernetes Pods.

You might write Java code using the Kubernetes API.

The application makes:

```
GET /api/v1/namespaces/.../pods
```

But Kubernetes asks:

Who are you?

The application's ServiceAccount provides its identity.

Then RBAC decides:

Is this ServiceAccount allowed to read Pods?

This is much safer than embedding an administrator's credentials into your Java application.

OpenShift connection

OpenShift uses Kubernetes RBAC too.

So the concepts:

```
Role
ClusterRole
RoleBinding
ClusterRoleBinding
ServiceAccount
```

still apply.

OpenShift adds additional security mechanisms around them, particularly its Security Context Constraints (SCCs), which we'll cover separately.

RBAC does NOT control everything

Another important distinction.

RBAC controls:

What Kubernetes API actions an identity is allowed to perform.

It doesn't directly mean:

"This Pod can or cannot talk to that Pod."

That is:

```
NetworkPolicy
```

RBAC:

```
WHO → CAN DO WHAT
```

NetworkPolicy:

```
WHO → CAN TALK TO WHOM
```

ResourceQuota:

```
WHO/namespace → CAN CONSUME HOW MUCH
```

Put them together

Now we have:



RBAC	ResourceQuota	NetworkPolicy
Who can do what?	How much?	Who can communicate?

And:

```
LimitRange
→ resource defaults and min/max rules
```

These are complementary security/governance mechanisms.

A production scenario

Let's build one.

Your company has:

```
employee-dev
employee-prod
```

Alice is a developer.

Requirements:

Alice can:

```
employee-dev:
  get Pods
  list Pods
  view logs
  restart her application
```

Alice cannot:

```
employee-prod:
  modify Deployments
  delete Pods
  modify Secrets
```

Monitoring application:

```
read Pods
read Nodes
read Services
```

across the cluster.

This could involve:

```
Alice
↓
Role/RoleBindings
```

```
Monitoring
↓
ServiceAccount
↓
ClusterRole
↓
ClusterRoleBinding
```

This is a very realistic Kubernetes architecture.

| The permission matrix

A useful way to design RBAC is to create a matrix:

Identity Namespace Resource Actions

Alice employee-dev Pods get/list/watch

Alice employee-dev Deployments get/update

Alice employee-prod Pods get/list

Monitoring all Pods get/list/watch

Monitoring all Nodes get/list

Platform Admin all Everything admin

Then turn that matrix into RBAC objects.

This is how you should think about RBAC rather than randomly creating Roles.

| One subtle concept: permissions are additive

Kubernetes RBAC is generally additive.

There isn't a normal "deny rule" in RBAC that overrides an allowed rule.

Suppose Alice has:

```
Role A:
get Pods
```

and:

```
Role B:
delete Pods
```

Then Alice effectively has:

```
get
delete
```

because both bindings contribute permissions.

Therefore, be careful about accidentally giving someone a powerful second Role.

| Common mistake

A developer says:

“I only gave Alice read access.”

But somewhere else:

```
ClusterRoleBinding
```

has given Alice a much broader permission.

So when debugging:

```
kubectl auth can-i ...
```

is extremely useful.

Also inspect:

```
kubectl get rolebindings -A  
kubectl get clusterrolebindings
```

| Another important concept: Groups

Instead of binding permissions individually:

```
Alice  
Bob  
Charlie
```

you can use a group:

```
employee-developers
```

Then:

```
employee-developers  
  ↓  
RoleBinding  
  ↓  
employee-dev
```

Now when a new developer joins:

```
David  
  ↓  
employee-developers
```

he automatically gets the group's permissions.

This is much easier to manage at enterprise scale.

| Real enterprise identity flow

In many organizations, Kubernetes/OpenShift integrates with an external identity provider.

Conceptually:

```
Employee
  ↓
Corporate Identity Provider
  ↓
User / Groups
  ↓
OpenShift/Kubernetes
  ↓
RBAC
  ↓
Namespace permissions
```

So you don't necessarily manually create a Kubernetes user for every employee.

Instead, enterprise identity and groups can be integrated with the platform.

| OpenShift example

Imagine:

```
AD/LDAP group:
employee-developers
```

OpenShift can map that identity/group into RBAC.

Then:

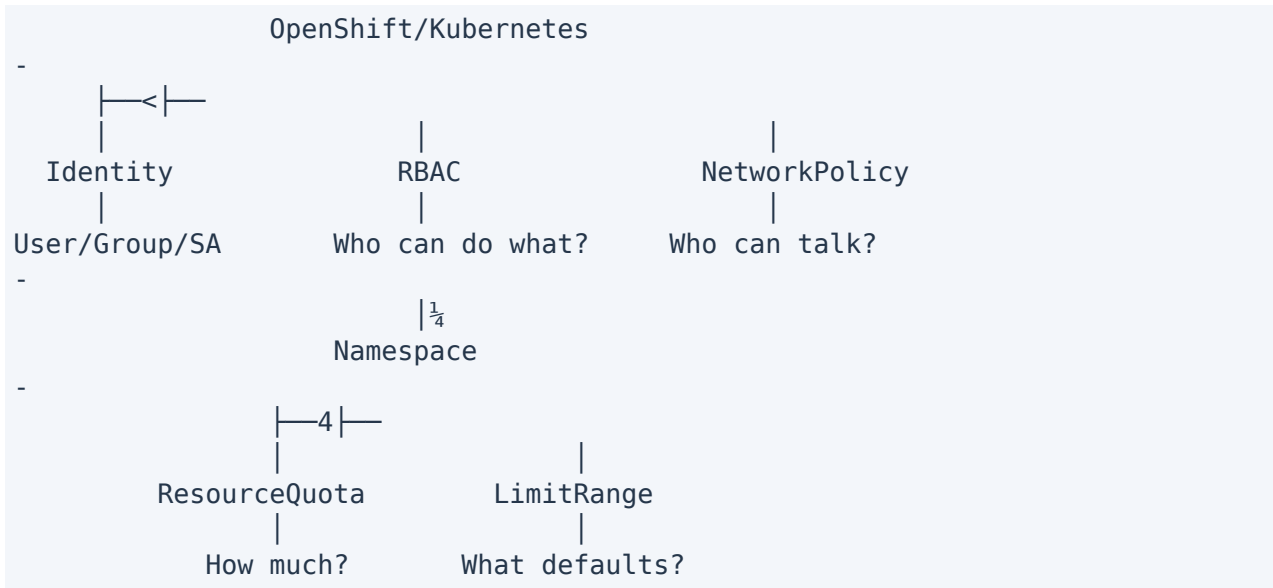
```
employee-developers
  ↓
RoleBinding
  ↓
employee-dev project
```

Every member gets the required permissions.

This is one reason OpenShift is popular in enterprise environments.

| The complete security picture

You're now ready to understand this model:



This is the beginning of enterprise Kubernetes security.

What you should memorize

Don't memorize YAML yet.

Memorize these relationships:

```

Role
↓
defines permissions

RoleBinding
↓
assigns permissions to a subject

ClusterRole
↓
defines reusable cluster-level permission set

ClusterRoleBinding
↓
assigns ClusterRole permissions cluster-wide

ServiceAccount
↓
identity for workloads
  
```

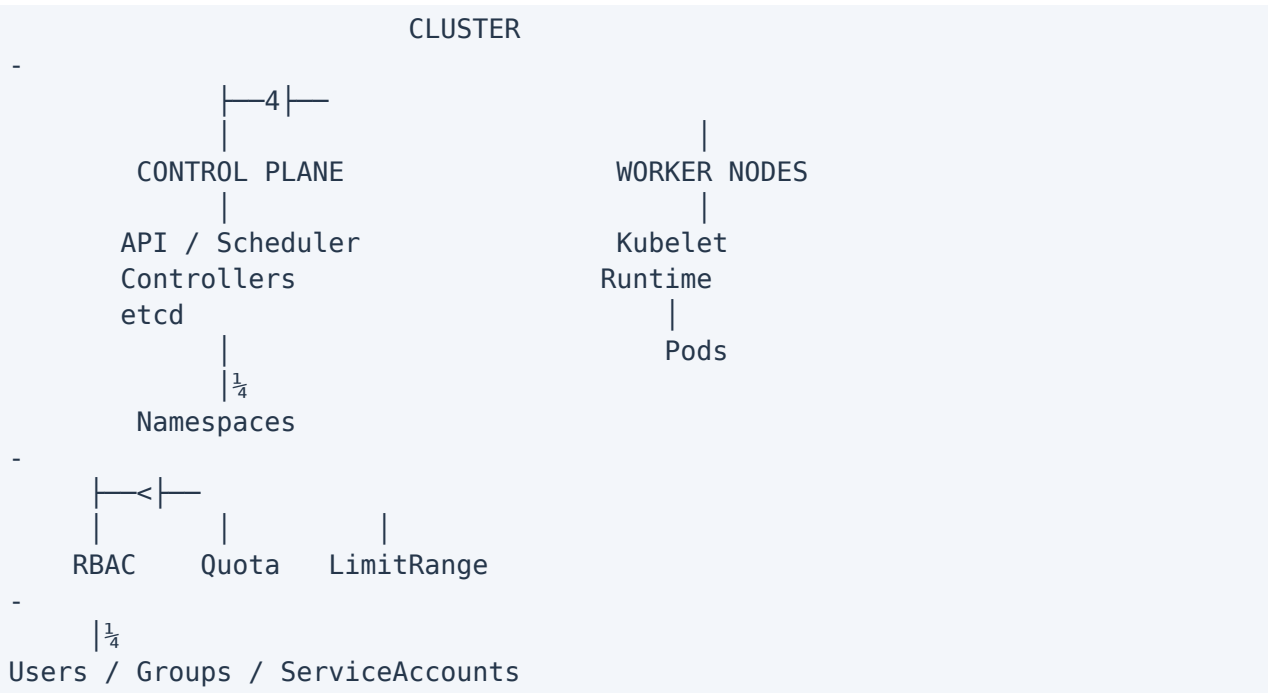
And:

```

RBAC
=
Who can do what?
  
```

Your mental model so far

We have now built a fairly complete Kubernetes picture:



And workload:

```

Deployment
  ↓
ReplicaSet
  ↓
Pod
  ↓
Container
  
```

Networking:

```

Pod
  ↓
Service
  ↓
DNS / Network
  
```

Health:

```

Startup
Readiness
Liveness
  
```

You're now moving beyond beginner Kubernetes concepts into real platform administration.

Next chapter — chapter 16: Kubernetes Storage The next logical topic is Storage, because we've already discussed the basics of:

- Volumes
- Mounting
- PV
- PVC
- Static provisioning
- Dynamic provisioning
- StatefulSets

We✓I now connect those concepts into one complete real-world architecture.

We✓I take a Spring Boot application backed by PostgreSQL and walk through:

Spring Boot

-

| $\frac{1}{4}$

Deployment

-

| $\frac{1}{4}$

Pod

-

| $\frac{1}{4}$

PVC

-

| $\frac{1}{4}$

PV

-

| $\frac{1}{4}$

StorageClass

-

| $\frac{1}{4}$

Actual storage

Then we✓I answer the practical questions you raised earlier:

Why can't a Deployment simply use a PVC like a StatefulSet?

What exactly happens when a Pod dies?

What happens when a Node dies?

Why does StatefulSet give stable storage and identity?

That will tie your earlier storage and StatefulSet learning together rather than teaching the concepts in isolation.

Chapter 21 — SecurityContext and Pod Security

CHAPTER FOCUS

Understand **SecurityContext** and **Pod Security** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

This chapter answers:

A container is isolated, so why do we still care whether it runs as root, what Linux capabilities it has, or whether it can access the host?

Because containers are process isolation, not a separate physical security boundary.

A badly privileged container can potentially gain much more access to its Node than an ordinary application should have.

The security chain is:

```
User/CI
  ↓
creates Deployment
  ↓
Pod specification
  ↓
SecurityContext
  ↓
Admission security policy
  ↓
Container runtime
  ↓
Linux kernel security controls
```

Start with Linux users

Before Kubernetes, understand this:

Linux processes run as a user identified by a numeric:

UID

and groups identified by:

GID

The special Linux root user is:


```
UID = 0
```

A normal user might be:

```
UID = 1000  
GID = 1000
```

So when your Java process runs inside a container:

```
java -jar employee.jar
```

it is still a Linux process with:

```
UID  
GID  
permissions  
capabilities
```

| Root inside container

Suppose:

```
Container  
├─ java process  
   UID = 0
```

That means the process runs as:

```
root
```

inside the container.

You might think:

But the container is isolated, so what's the issue?

Container root is not automatically identical to unrestricted root on the host, because namespaces, capabilities, seccomp and other isolation mechanisms still matter.

But running as root increases the damage possible if:

```
application compromised  
+  
container misconfigured  
+  
host access/capabilities available
```

So least privilege says:

Ordinary Spring Boot applications generally should not need root.

3. What is securityContext ?

Kubernetes defines:

securityContext:

to configure security and access-control settings for Pods and containers.

It can control things such as:

```
UID/GID
non-root requirements
Linux capabilities
privileged mode
privilege escalation
read-only root filesystem
seccomp
SELinux
filesystem groups
```

Kubernetes explicitly defines Security Context as the place for these privilege/access-control settings.

| There are two levels

You can have:

Pod-level

```
spec:
  securityContext:
    runAsUser: 1000
```

and:

Container-level

```
containers:
- name: employee-service
  securityContext:
    runAsUser: 2000
```

If an overlapping setting exists at both levels, the container-level value applies to that particular container. Kubernetes

documents this override behavior directly.

Think:

```
Pod securityContext
→ common/default Pod security settings
```

```
Container securityContext
→ container-specific security settings
```

| runAsUser

Example:

```
spec:
  containers:
    - name: employee
      image: employee-service:2.7

      securityContext:
        runAsUser: 10001
```

Now the application process runs as:

```
UID 10001
```

instead of root.

Conceptually:

```
Before:
java → UID 0

After:
java → UID 10001
```

| runAsGroup

Likewise:

```
securityContext:
  runAsUser: 10001
  runAsGroup: 10001
```

Now:

```
process UID = 10001
primary GID = 10001
```

Linux filesystem permissions can then be based on:

```
owner
group
other
```

just like normal Linux.

| runAsNonRoot

Now an important difference.

```
securityContext:  
  runAsNonRoot: true
```

doesn't mean:

```
"Run specifically as UID 10001."
```

It means:

Do not allow this container to run as UID 0.

So:

```
runAsUser: 10001
```

says:

Run as this specific user.

while:

```
runAsNonRoot: true
```

says:

Whatever UID is used, it must not be root.

The Kubernetes Restricted Pod Security Standard requires workloads to run as non-root.

| Common combination

You might see:

```
securityContext:  
  runAsNonRoot: true  
  runAsUser: 10001
```

Meaning:

```
require non-root  
+  
explicitly choose UID 10001
```

But there's an important OpenShift consideration we'll discuss later: OpenShift commonly assigns allowed UIDs dynamically, so hard-coding a particular UID isn't always the best container-image design.

| Dockerfile also matters

Your container image can say:

DOCKERFILE

USER 10001

Then the image already intends to run as that user.

So:

```
Dockerfile USER
→ image-level default identity

Kubernetes securityContext
→ runtime security configuration/enforcement
```

SecurityContext can override or further restrict the image's defaults.

| fsGroup

Now imagine your application mounts a PVC:

```
/data
```

Process:

```
UID = 10001
```

but the volume files aren't writable by that user.

Kubernetes provides Pod-level filesystem group handling:

```
spec:
  securityContext:
    fsGroup: 2000
```

Conceptually:

```
Container process
UID 10001

also associated with filesystem group
GID 2000
  ↓
mounted volume writable according
to group permissions
```

Kubernetes documents fsGroup as a Pod-level mechanism affecting supported mounted volumes and supplementary process group membership.

This is particularly relevant with:

```
PVCs
shared files
non-root containers
```

| Now Linux capabilities

Historically Linux root had:

```
all-powerful privilege
```

Linux capabilities split root-like privileges into smaller pieces.

For example, capabilities can govern operations related to:

```
network administration
changing ownership
binding certain ports
system administration
```

So instead of:

```
give application root
```

you can sometimes give:

```
one specific capability
```

Kubernetes supports adding and dropping capabilities through container securityContext .

| Example

```
securityContext:
  capabilities:
    drop:
      - ALL
```

This means:

Drop all optional Linux capabilities from the container.

Then, if absolutely necessary, add a specific one.

Example:

```
capabilities:
  drop:
    - ALL
  add:
    - NET_BIND_SERVICE
```

Conceptually:

Not:

```
"Give me root"
```

Instead:

```
"I only need this one privilege."
```

This is the least-privilege principle.

13. Why drop: ALL ?

Suppose your Spring Boot service just needs:

```
listen on 8080
call Oracle
read configuration
write application data
```

Why should it have Linux capabilities related to administration of the host/network?

It shouldn't.

So a strong container-security posture often begins with:

```
capabilities:
  drop:
    - ALL
```

The Kubernetes Restricted Pod Security Standard requires dropping ALL capabilities and only permits a tightly limited capability addition such as NET_BIND_SERVICE .

| privileged: true

This is very different.

```
securityContext:
  privileged: true
```

essentially asks for a highly privileged container with much of the normal container isolation relaxed.

Think:

```
Normal container
→ limited access

Privileged container
→ very broad host/kernel privileges
```

Ordinary Spring Boot applications should almost never require this.

Privileged workloads are generally special infrastructure components such as some:

```
low-level node agents
storage/network components
system administration tools
```

The Kubernetes Baseline and Restricted Pod Security Standards disallow privileged containers.

15. Why is privileged dangerous?

Imagine an attacker exploits your Spring application.

With a normal hardened container:

```
Attacker
↓
limited application process
↓
restricted container environment
```

With unnecessary privileged access:

```
Attacker
↓
privileged container
↓
much greater host/kernel access
```

The potential blast radius becomes dramatically larger.

That is why:

Application compromise should not automatically become Node compromise.

allowPrivilegeEscalation

Now a more subtle setting:

```
securityContext:
  allowPrivilegeEscalation: false
```

This means:

A process should not be allowed to gain more privileges than its parent process.

For example, Linux has mechanisms such as set-user-ID executables that can otherwise result in increased privileges.

Kubernetes maps this control to Linux's `no_new_privs` behavior; it defaults to allowing escalation when not otherwise constrained, with privileged containers and `CAP_SYS_ADMIN` being important exceptions to disabling it.

| Don't confuse these three

```
runAsNonRoot
→ don't run as UID 0

allowPrivilegeEscalation: false
→ don't gain additional privilege later

privileged: false
→ don't run as a privileged container
```

They're related but solve different problems.

A hardened application commonly wants all three ideas.

| readOnlyRootFilesystem

Normally a container has a writable container layer.

You learned this in the storage chapter:

```
Container image
+
writable container layer
```

With:

```
securityContext:
  readOnlyRootFilesystem: true
```

the container's root filesystem is mounted read-only.

Meaning the application can't casually write:

```
/usr
/etc
/app
/random-path
```

19. But what if the application needs temporary files?

You deliberately provide writable storage.

For example:

```
volumeMounts:
- name: temp
  mountPath: /tmp

volumes:
- name: temp
  emptyDir: {}
```

Architecture:

```
Container root filesystem  
→ read-only
```

```
/tmp  
→ writable emptyDir
```

This is better security design because:

Only paths that genuinely need writes are writable.

| Spring Boot example

Suppose Java needs:

```
/tmp
```

for temporary data.

Instead of allowing the entire container filesystem to be writable:

```
/
├─ etc          writable
├─ app          writable
├─ usr          writable
└─ tmp          writable
```

you can aim for:

```
/
├─ etc          read-only
├─ app          read-only
├─ usr          read-only
└─ tmp          writable volume
```

Smaller writable attack surface.

| Seccomp

Now we go deeper into Linux.

Your Java process talks to the Linux kernel using:

System calls Examples conceptually:

```
open file
create socket
read
write
allocate memory
create process/thread
```

A compromised application might attempt dangerous system calls.

Linux:

seccomp can filter which system calls a process is allowed to use.

Kubernetes supports:

```
securityContext:
  seccompProfile:
    type: RuntimeDefault
```

which asks the runtime to apply its default seccomp profile. Kubernetes supports RuntimeDefault , Unconfined , and node-local profiles.

| Think of seccomp as

```
Application
  ↓
system call request
  ↓
Seccomp filter
  ↓
allowed?
  /   \
yes    no
  ↓     ↓
kernel blocked
```

You normally do not need to design syscall lists yourself for a Spring Boot service.

The important idea is:

```
RuntimeDefault
```

gives another isolation layer around the container.

| SELinux

Another Linux security layer, particularly relevant in Red Hat/OpenShift environments:

SELinux Traditional Unix permissions ask:

```
UID/GID
→ Is this user allowed to access this file?
```

SELinux adds security labels and mandatory access-control policy.

Conceptually:

```
Process label
+
File/resource label
+
SELinux policy
  ↓
Access allowed?
```

This gives another barrier even if ordinary filesystem permissions might otherwise allow access.

OpenShift uses SELinux as part of its workload-security architecture, which we'll connect to SCC later.

| Host namespaces

Remember containers get isolated namespaces:

```
network
PID
IPC
```

Kubernetes can allow Pods to share host namespaces.

Examples:

```
hostNetwork: true
hostPID: true
hostIPC: true
```

These reduce isolation.

For example:

```
hostNetwork: true
```

means the Pod shares the Node's network namespace rather than receiving normal Pod-network isolation.

The Baseline Pod Security Standard disallows sharing these host namespaces.

Ordinary microservices should generally not need them.

| hostPath is also sensitive

You learned volumes such as:

```
emptyDir
PVC
ConfigMap
```

Secret

hostPath allows a Pod to mount a directory from the actual Node filesystem.

Conceptually:

```
Node:
  /var/something
    ↓
Pod mounts it
```

That can be dangerous because you're breaking through an important container boundary.

For example, broad host filesystem access can potentially expose:

```
node configuration
container runtime data
credentials
system files
```

This is why privileged infrastructure workloads require much more trust than ordinary applications.

Put the controls together

A relatively hardened application container might conceptually contain:

```
spec:
  securityContext:
    runAsNonRoot: true
    seccompProfile:
      type: RuntimeDefault

  containers:
    - name: employee-service
      image: employee-service:2.7

      securityContext:
        allowPrivilegeEscalation: false
        capabilities:
          drop:
            - ALL
        readOnlyRootFilesystem: true
```

Then writable application paths would be explicitly mounted if required.

This closely matches the direction of Kubernetes' Restricted Pod Security Standard.

27. But who forces developers to use these settings?

This is the next important question.

Suppose Developer A writes:

```
securityContext:  
  runAsNonRoot: true
```

Good.

Developer B writes:

```
securityContext:  
  privileged: true
```

Can Kubernetes enforce organizational policy?

Yes.

That is where:

Pod Security Standards and:

Pod Security Admission come in.

| Pod Security Standards — PSS

Kubernetes defines three broad security profiles:

Profile Meaning

Privileged Essentially unrestricted

Baseline Prevent known privilege escalations while remaining broadly compatible

Restricted Strong Pod-hardening best practices

These are official Kubernetes Pod Security Standards.

Think:

```
Privileged  
  ↓  
Baseline  
  ↓  
Restricted  
  
increasing security restrictions
```

| Privileged profile

Think:

```
"I trust these workloads deeply."
```

This profile places effectively no meaningful Pod-security restriction.

It may be appropriate for certain trusted:

```
cluster infrastructure  
node-level components
```

but not normal application teams.

Baseline profile

Think:

Allow common containerized workloads but block obvious privilege-escalation configurations.

For example, Baseline prevents things such as:

```
privileged containers  
host PID/IPC access  
many dangerous capabilities
```

while being easier for existing applications to adopt.

Restricted profile

Think:

Strong hardening appropriate for ordinary applications where possible.

Among other requirements, Restricted expects things such as:

```
run as non-root  
allowPrivilegeEscalation=false  
seccomp profile  
drop ALL capabilities
```

with narrowly defined exceptions.

For your typical Spring Boot API, this is much closer to the security posture you should aim to support.

32. What is Pod Security Admission?

Pod Security Standards are:

```
the policy definitions
```

Pod Security Admission is:

```
the built-in Kubernetes admission mechanism  
that can apply them
```

It has been stable since Kubernetes 1.25.

Remember our API flow:

```
kubectl apply
  ↓
API Server
  ↓
Admission
  ↓
accepted / rejected
  ↓
stored
```

Pod Security Admission sits in that admission stage.

| Namespace-level enforcement

Suppose namespace:

```
employee-prod
```

has:

```
metadata:
  labels:
    pod-security.kubernetes.io/enforce: restricted
```

Developer submits a Pod:

```
securityContext:
  privileged: true
```

Flow:

```
Developer
  ↓
API Server
  ↓
Pod Security Admission
  ↓
Restricted standard
  ↓
Pod violates policy
  ↓
REJECT
```

The built-in admission controller applies these policies at namespace level.

| Three admission modes

Pod Security Admission supports:

```
enforce
audit
```


warn

enforce

violation
→ reject Pod

warn

violation
→ Pod may proceed
→ user receives warning

audit

violation
→ Pod may proceed
→ violation recorded in audit information

These three modes are defined by Kubernetes and can even use different security levels simultaneously.

Why warn and audit are useful

Suppose a company has:

300 old applications

and immediately turns on:

restricted + enforce

Maybe 150 applications fail to deploy.

A safer migration can be:

```
restricted + warn
restricted + audit
  ↓
find incompatible workloads
  ↓
fix them
  ↓
restricted + enforce
```

This is a standard policy-rollout pattern.

Very important distinction: SecurityContext vs security policy

SecurityContext says:

```
"This Pod wants to run this way."
```

Pod Security Admission/SCC says:

```
"Is the Pod ALLOWED to run this way?"
```

For example:

```
securityContext:  
  privileged: true
```

is merely a request.

An admission policy can reject it.

So:

```
SecurityContext  
→ requested/configured runtime security
```

```
Admission security policy  
→ organizational guardrail
```

This distinction matters tremendously.

RBAC is still different

You've already learned RBAC.

Suppose Hari has:

```
create pods
```

permission.

RBAC says:

```
Can Hari submit a Pod?  
→ Yes
```

But Hari submits:

```
privileged: true
```

Pod security policy may say:

```
Can this Pod configuration run?  
→ No
```

So:

```
RBAC  
→ WHO can perform API actions?
```

Pod Security
→ HOW may Pods run?

| ServiceAccount is also different

Your Pod may run using:

```
serviceAccountName: employee-service
```

That controls its identity when talking to the Kubernetes API.

SecurityContext controls its Linux/container privileges.

Think:

```
ServiceAccount  
→ Kubernetes API identity  
  
runAsUser  
→ Linux process identity
```

Completely different layers.

| NetworkPolicy is different too

Now connect three security layers:

```
RBAC  
→ API authorization  
  
SecurityContext / Pod Security  
→ workload/container privilege  
  
NetworkPolicy  
→ network communication
```

Example:

```
Can employee-service list Secrets?  
→ RBAC  
  
Can employee-service run privileged?  
→ Pod security  
  
Can employee-service connect to payroll?  
→ NetworkPolicy
```

This is a very strong interview distinction.

| SecurityContext does not replace application security

Even if:

```
runAsNonRoot = true
capabilities dropped
seccomp enabled
read-only filesystem
```

your Spring Boot application can still contain:

```
SQL injection
authentication bug
authorization bug
vulnerable dependency
```

Container security is one defensive layer.

Think:

```
Application security
+
Container security
+
Kubernetes security
+
Network security
+
Identity/RBAC
```

Defence in depth.

| A practical Spring Boot example

Suppose your application only needs:

```
HTTP :8080
Oracle connectivity
read config
write /tmp
```

A good conceptual design:

```
Run non-root '
No privileged mode '
No hostNetwork '
No hostPID '
Drop Linux capabilities '
No privilege escalation '
RuntimeDefault seccomp '
Read-only root filesystem '
Writable /tmp using emptyDir '
```

Architecture:

```
Spring Boot container
-
├─ UID non-root
├─ root FS read-only
├─ /tmp → emptyDir
├─ capabilities → none
├─ seccomp → RuntimeDefault
├─ network → normal Pod network
```

That is dramatically safer than:

```
root
+
privileged
+
hostNetwork
+
hostPath
```

| Why some containers break when hardened

You enable:

```
readOnlyRootFilesystem: true
```

and application crashes:

```
Permission denied: /app/logs
```

This doesn't automatically mean:

Disable security.

Instead ask:

Why does the application need to write there?

Maybe:

```
logs
```

should go to:

```
stdout
```

which you learned in chapter 35.

Or:

```
temporary data
```

should go to an emptyDir .

Security controls often expose poor container-image assumptions.

| Another example: root requirement

Application fails when:

```
runAsNonRoot: true
```

because startup script tries:

```
apt install ...
```

inside the running container.

That is usually a container design smell.

Package installation belongs during:

```
image build
```

not application startup.

Better:

```
Dockerfile
↓
install dependencies
↓
produce immutable image
↓
runtime runs non-root
```

| Now connect this to OpenShift SCC

This sets up our later OpenShift chapter perfectly.

OpenShift provides:

Security Context Constraints — SCC SCC can control things such as:

```
privileged containers
privilege escalation
capabilities
UID
SELinux
FSGroup
host networking/namespaces
host volumes
```

```
seccomp
read-only filesystem requirements
```

Red Hat explicitly documents these as SCC-controlled dimensions.

So all the controls we've just learned are exactly the kinds of things OpenShift SCC evaluates.

| OpenShift example

Your Pod asks:

```
securityContext:
  runAsUser: 0
  privileged: true
```

Your OpenShift project/service account only qualifies for a restrictive SCC.

Conceptually:

```
Pod request
  ↓
OpenShift admission
  ↓
SCC evaluation
  ↓
root/privileged not allowed
  ↓
Pod rejected
```

This is why learning SecurityContext before SCC was important.

| Pod Security Admission vs SCC in OpenShift

Modern OpenShift can involve both mechanisms.

Red Hat explicitly states that Pod Security Admission and SCC operate independently, so a workload may need to comply with both. SCC can also mutate/default some fields before validating them.

Conceptually:

```
Pod submitted
  ↓
OpenShift SCC processing
  ↓
Pod Security Admission
  ↓
must satisfy applicable security requirements
  ↓
accepted
```

The exact admission sequence has implementation details, but the learning point is: Don't think SCC and Kubernetes Pod Security Admission are the same mechanism.

| OpenShift restrictive SCCs

Current OpenShift includes restrictive SCC variants that can require behavior such as:

```
non-root / allocated UID
drop capabilities
RuntimeDefault seccomp
no privilege escalation
```

Red Hat's current documentation describes variants such as restricted-v2 and newer restrictive profiles with these characteristics.

This is one reason OpenShift can reject container images that worked elsewhere with permissive settings.

| The security hierarchy

Here's the mental model I want you to retain:

```
Application
-
  | 1/4
  Container image
-
  | 1/4
  SecurityContext
  "how I want to run"
-
  | 1/4
  Admission policies
Pod Security / OpenShift SCC
  "may I run this way?"
-
  | 1/4
  Container runtime
-
  | 1/4
  Linux kernel
  UID / capabilities
  seccomp / SELinux etc.
```

| Common production settings

For an ordinary microservice, this is an excellent conceptual baseline:


```
spec:
  securityContext:
    runAsNonRoot: true
    seccompProfile:
      type: RuntimeDefault

  containers:
    - name: employee-service
      image: employee-service:2.7

      securityContext:
        allowPrivilegeEscalation: false

        capabilities:

          drop:
            - ALL

      readOnlyRootFilesystem: true
```

But don't blindly copy it.

You must understand whether the application requires:

```
writable paths
specific UID
additional capabilities
shared storage permissions
```

Security should be strict and compatible, not cargo-cult YAML.

Troubleshooting security failures

Suppose Pod fails during creation.

Look at:

```
kubectl describe pod <pod>
```

or, if the Pod itself wasn't created, inspect:

```
Deployment events
ReplicaSet events
API error returned during apply
```

Potential messages might point toward:

```
runAsNonRoot violation
privileged container disallowed
forbidden capability
seccomp requirement
SCC unable to validate Pod
```

Don't debug the Java logs if the container never started.
That is an admission/security problem.

| Runtime permission failure is different

Pod successfully starts:

```
Running
```

then application logs:

```
java.nio.file.AccessDeniedException:  
/app/cache
```

Now admission succeeded.

Problem may instead be:

```
UID/file ownership  
fsGroup  
readOnlyRootFilesystem  
volume permissions
```

So distinguish:

```
Pod rejected before start  
→ policy/admission issue
```

```
Pod starts but gets Permission denied  
→ runtime filesystem/UID issue
```

| Important interview distinctions

Concept Question it answers

runAsUser Which UID runs the process?

runAsNonRoot Must UID 0 be prohibited?

runAsGroup Which primary GID?

fsGroup Which group can help access supported volumes?

privileged Does container get broad elevated host privileges?

allowPrivilegeEscalation May the process gain more privilege?

capabilities Which fine-grained Linux privileges exist?

readOnlyRootFilesystem Can container write to its root filesystem?

seccomp Which system calls may be made?

Pod Security Admission Does namespace policy allow this Pod configuration?

SCC Does OpenShift allow/default this Pod security configuration?

| chapter 26 mental model

Remember this progression:

```
Linux Process
  ↓
UID / GID
  ↓
Capabilities
  ↓
Filesystem permissions
  ↓
Seccomp / SELinux
  ↓
Container isolation
  ↓
Kubernetes SecurityContext
  ↓
Pod Security Admission
  ↓
OpenShift SCC
```

And the most important sentence:

securityContext describes how a workload should run; admission mechanisms such as Pod Security Admission or OpenShift SCC determine whether that workload is permitted to run that way.

You have now filled all three modules you specifically asked to revisit:

```
21 – HPA / Autoscaling           '
23 – Graceful Shutdown / Lifecycle Hooks      '
26 – SecurityContext / Pod Security           '

```

That leaves the earlier roadmap much cleaner. The most natural remaining gaps are now 27 — OpenShift SCC in depth, 28 — ServiceAccounts/workload identity in depth, and the remaining parts of 29 — OpenShift Builds & ImageStreams before returning to chapter 39 Operators/CRDs/OLM.

Yes. We'll do 27 → 29 next. One correction: chapter 32 (Kustomize) is already complete, so after 29 we can either return to 28 (ServiceAccounts/workload identity) or continue to 39 (Operators/CRDs/OLM).

Packaging, Customization, and GitOps

Chapter 22 — Helm

CHAPTER FOCUS

Understand **Helm** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

The main question Helm answers is:

How do I manage large, reusable Kubernetes YAML configurations without copying and editing dozens of files manually?

The problem Helm solves

Imagine your Spring Boot application needs:

```
Deployment
Service
ConfigMap
Secret
Ingress
ServiceAccount
HPA
PVC
```

Without Helm, you might have:

```
deployment.yaml
service.yaml
configmap.yaml
secret.yaml
ingress.yaml
serviceaccount.yaml
hpa.yaml
pvc.yaml
```

Now imagine you have:

```
DEV
QA
UAT
PROD
```

and each environment needs slightly different values.

For example:

```
DEV:
replicas = 1
CPU = 250m
image tag = dev

PROD:
replicas = 5
CPU = 1
image tag = 2.4.1
```

A bad approach would be:

```
dev-deployment.yaml
qa-deployment.yaml
uat-deployment.yaml
prod-deployment.yaml

dev-service.yaml
qa-service.yaml
...
```

You end up duplicating Kubernetes YAML everywhere.

Helm solves this.

2. What is Helm?

Helm is commonly described as:

Package manager for Kubernetes A useful analogy for you:

```
↓
Maven / Gradle

Kubernetes
↓
Helm
```

But that analogy is only partly correct.

Helm also acts as a:

```
Templating engine
+
Packaging mechanism
+
Release manager
```

So think:

```
Helm
=
Reusable Kubernetes templates
+
```

```
configuration values
+
installation/upgrade management
```

| Without Helm vs with Helm

Without Helm:

```
deployment.yaml
service.yaml
configmap.yaml
ingress.yaml
```

Hardcoded:

```
replicas: 3
image: employee-service:1.0
```

With Helm:

```
replicas: {{ .Values.replicaCount }}

image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
```

Then values live separately:

```
replicaCount: 3

image:
  repository: employee-service
  tag: "1.0"
```

This separation is the core Helm idea.

| First mental model

Remember:

```
Kubernetes YAML
↓
Make values dynamic
↓
Helm Templates
+
values.yaml
↓
Helm renders YAML
↓
Kubernetes API
```

In other words:

Template + Values = Final Kubernetes YAML

That is Helm.

5. What is a Helm Chart?

A:

Chart is a packaged collection of Kubernetes templates and Helm metadata.

Think:

```
employee-service Helm Chart
-
├─ Deployment template
├─ Service template
├─ ConfigMap template
├─ Ingress template
├─ values.yaml
└─ Chart metadata
```

So:

Chart = reusable Kubernetes application package

Typical Helm chart structure

If you create:

```
helm create employee-service
```

you✓I get something roughly like:

```
employee-service/
-
├─ Chart.yaml
├─ values.yaml
├─ charts/
├─ templates/
│   ├─ deployment.yaml
│   ├─ service.yaml
│   ├─ ingress.yaml
│   ├─ serviceaccount.yaml
│   ├─ _helpers.tpl
│   └─ tests/
└─ .helmignore
```

Let's understand each piece.

Chart.yaml

This contains metadata about the chart.

Example:

```
apiVersion: v2
name: employee-service
description: Helm chart for employee-service
type: application
version: 0.1.0
appVersion: "1.0.0"
```

Important distinction:

version

and:

appVersion

are not the same.

version vs appVersion

Suppose:

```
version: 0.4.2
appVersion: "2.7.1"
```

version means:

Version of the Helm chart itself.

Maybe you changed:

```
Ingress template
Service annotations
Resource defaults
```

so the chart version changes.

appVersion means:

Version of the application the chart represents.

For example:

```
employee-service 2.7.1
```

So:

```
Chart version
"\"
Application version
```

Very important.

| values.yaml

This is one of the most important files.

Example:

```
replicaCount: 3

image:
  repository: employee-service
  tag: "1.0.0"

service:

  type: ClusterIP
  port: 8080

resources:
  requests:
    cpu: 500m
    memory: 512Mi
  limits:
    cpu: "1"
    memory: 1Gi
```

This is the chart's default configuration.

Think:

```
values.yaml
=
default input values for templates
```

| Templates

Inside:

```
templates/
```

you put Kubernetes resource templates.

Example deployment.yaml :

```
apiVersion: apps/v1
kind: Deployment

metadata:
  name: {{ .Release.Name }}
```

```
spec:
  replicas: {{ .Values.replicaCount }}

  selector:
    matchLabels:
      app: {{ .Release.Name }}

  template:
    metadata:
      labels:
        app: {{ .Release.Name }}

    spec:
      containers:
        - name: employee-service
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"

      resources:
        {{- toYaml .Values.resources | nindent 12 }}
```

This looks unusual because of:

```
{{ ... }}
```

Those are Helm template expressions.

.Values

One of the most common Helm expressions is:

```
.Values
```

For example:

```
replicas: {{ .Values.replicaCount }}
```

Helm looks inside:

```
values.yaml
```

and finds:

```
replicaCount: 3
```

Then renders:

```
replicas: 3
```

Simple.

Another example

Template:

```
image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
```

Values:

```
image:
  repository: employee-service
  tag: "2.1.0"
```

Rendered result:

```
image: "employee-service:2.1.0"
```

So:

```
.Values.image.repository
```

means:

```
values.yaml
↓
image
↓
repository
```

Think of Helm like method parameters

As a Java developer, imagine:

```
deployApplication(
    replicas,
    image,
    cpu,
    memory
)
```

Instead of hardcoding:

```
replicas = 3;
```

Helm does the same conceptually:

```
Template
+
parameters
```

where:

```
values.yaml
=
arguments
```

This is a useful analogy.

14. What is a Release?

This is another major Helm concept.

A:

Release is a specific installed instance of a Helm Chart.

Suppose you have one Chart:

```
employee-service
```

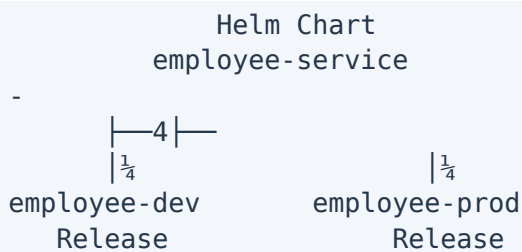
You can install it as:

```
employee-dev
```

and separately as:

```
employee-prod
```

Conceptually:



Same Chart.

Different Releases.

Different values.

Chart vs Release

Memorize this.

Chart

```
Reusable package/template
```

Release

Installed instance of that chart

Analogy:

```
Java class
  ↓
Object instances
```

Similar mental model:

```
Helm Chart
  ↓
Helm Releases
```

Not perfect, but useful.

Installing a Chart

Suppose you're inside the directory containing:

```
employee-service/
```

You can run:

```
helm install employee-dev ./employee-service
```

Here:

```
employee-dev
```

is the release name.

And:

```
./employee-service
```

is the Chart.

Helm renders the templates and sends the generated Kubernetes resources to the cluster.

17. What happens internally?

Conceptually:

```
helm install
  ↓
Read Chart
  ↓
Read values.yaml
  ↓
Render templates
```

```
↓  
Generate normal Kubernetes YAML  
↓  
Send resources to Kubernetes API  
↓  
Deployment / Service / etc created
```

Important:

Kubernetes itself doesn't understand Helm templates.

Kubernetes only receives the final rendered Kubernetes manifests.

Helm is not a Kubernetes controller

This distinction is useful.

Helm doesn't replace:

```
Deployment Controller  
StatefulSet Controller  
Scheduler  
Kubelet
```

It generates/submits Kubernetes resources.

After that:

```
Deployment  
↓  
ReplicaSet  
↓  
Pods
```

are still managed by normal Kubernetes controllers.

So:

```
Helm  
→ packaging/deployment layer  
  
Kubernetes controllers  
→ runtime reconciliation layer
```

Environment-specific values

This is probably the biggest reason companies use Helm.

You could maintain:

```
values.yaml  
values-dev.yaml
```

```
values-qa.yaml
values-prod.yaml
```

For example:

values-dev.yaml

```
replicaCount: 1

image:
  tag: "latest"

resources:
  requests:
    cpu: 100m
    memory: 256Mi
```

values-prod.yaml

```
replicaCount: 5

image:
  tag: "2.5.1"

resources:
  requests:
    cpu: 500m
    memory: 1Gi
```

Same templates.

Different values.

| Installing with a specific values file

You can run:

```
helm install employee-prod \
  ./employee-service \
  -f values-prod.yaml
```

Conceptually:

```
Chart templates
  +
values.yaml defaults
  +
values-prod.yaml overrides
  ↓
Final rendered manifests
```


| Value precedence

This is important.

Suppose default:

```
replicaCount: 2
```

values-prod.yaml :

```
replicaCount: 5
```

Then:

```
helm install employee-prod ./chart -f values-prod.yaml
```

results in:

```
replicaCount = 5
```

because the supplied values override chart defaults.

| -set

You can also override individual values from the command line.

For example:

```
helm install employee-prod ./employee-service \
  --set replicaCount=6
```

Now:

```
replicaCount = 6
```

This is useful for small overrides.

| Typical precedence idea

Think broadly:

Chart defaults

↓

values.yaml

then overridden by:

-f custom-values.yaml

then potentially overridden by:

--set

Higher-precedence supplied values win for the same key.

24. Why not just use `--set` for everything?

Technically you could do:

```
helm install app ./chart \
  --set replicaCount=5 \
  --set image.tag=2.3 \
  --set resources.requests.cpu=500m \
  ...
```

But this quickly becomes painful.

For production:

```
values-prod.yaml
```

or GitOps-managed values are usually much easier to review and maintain.

| **helm template**

This is one of the best Helm learning/troubleshooting commands.

Run:

```
helm template employee-prod ./employee-service
```

Helm renders the chart without installing it.

You see normal Kubernetes YAML.

Conceptually:

```
Helm Templates
  ↓
helm template
  ↓
Rendered Deployment
Rendered Service
Rendered ConfigMap
...
```

This is extremely useful for debugging.

| **Why helm template is important**

Suppose template:

```
replicas: {{ .Values.replicaCount }}
```

but you accidentally set:

replicaCount:

You may produce invalid output.

Before touching the cluster:

```
helm template ...
```

lets you inspect what Helm generated.

As a developer, think of it almost like:

```
compile/check generated output
```

before deploying.

| helm lint

Another very useful command:

```
helm lint ./employee-service
```

It checks the Chart for common problems.

Think:

```
helm lint
→ basic chart validation
```

A useful workflow is:

```
helm lint
  ↓
helm template
  ↓
helm upgrade/install
```

| Upgrading a Release

Suppose production currently uses:

```
employee-service:2.4.0
```

Now you release:

```
2.5.0
```

Update values:

```
image:
  tag: "2.5.0"
```

Then:

```
helm upgrade employee-prod \  
  ./employee-service \  
  -f values-prod.yaml
```

Helm calculates the updated manifests and applies them.

Then Kubernetes handles the Deployment rollout.

So:

```
Helm upgrade  
  ↓  
Deployment updated  
  ↓  
Deployment Controller  
  ↓  
RollingUpdate  
  ↓  
new Pods
```

Helm doesn't personally perform the Pod rolling algorithm.

The Deployment controller does.

helm upgrade -install

You✓I see this constantly in CI/CD:

```
helm upgrade --install employee-prod \  
  ./employee-service \  
  -f values-prod.yaml
```

Meaning:

```
If release exists  
→ upgrade
```

```
If release doesn't exist  
→ install
```

This is convenient for pipelines.

Viewing Releases

```
helm list
```

You might see:

NAME	NAMESPACE
employee-dev	employee-dev
employee-prod	employee-prod
payments-prod	payments-prod

To specify namespace:

```
helm list -n employee-prod
```

| Release history

Run:

```
helm history employee-prod
```

You might conceptually see:

```
Revision 1 → app 2.3  
Revision 2 → app 2.4  
Revision 3 → app 2.5
```

Helm tracks revisions of the release.

This enables rollback.

| Helm rollback

Suppose:

```
Revision 3
```

introduced a bad configuration.

You can roll back to:

```
Revision 2
```

using:

```
helm rollback employee-prod 2
```

Then Helm restores the previous release configuration/manifests.

This is a powerful operational feature.

| Helm revision vs Deployment revision

Be careful.

Helm release revision:

```
Revision 1  
Revision 2  
Revision 3
```

is Helm's release history.

Deployment rollout history:

```
Deployment revision
```

is Kubernetes Deployment's own rollout history.

They're related operationally but are not the same thing.

| Uninstalling

To remove a Release:

```
helm uninstall employee-dev
```

Helm removes the Kubernetes resources belonging to that release, subject to chart/resource behavior.

Think:

```
Chart  
  ↓ install  
Release  
  ↓ uninstall  
Resources removed
```

| .Release.Name

Let's look at built-in Helm objects.

You saw:

```
name: {{ .Release.Name }}
```

If you run:

```
helm install employee-prod ./employee-service
```

then:

```
.Release.Name
```

becomes:

```
employee-prod
```

So rendered YAML might contain:

```
metadata:  
  name: employee-prod
```

.Release.Namespace

You can also use:

```
.Release.Namespace
```

If installed with:

```
helm install employee-prod \  
  ./employee-service \  
  -n employee-prod
```

then templates can refer to:

```
employee-prod
```

through:

```
.Release.Namespace
```

.Chart

Another built-in object:

```
.Chart
```

contains metadata from Chart.yaml .

For example:

```
.Chart.Name  
.Chart.Version  
.Chart.AppVersion
```

These are often used in labels.

Helm functions

Templates support functions.

For example:

```
{{ .Values.appName | quote }}
```

If:

```
appName: employee-service
```

you might render:

```
"employee-service"
```

The:

```
|
```

is a pipeline operator in Go templating.

You✓I encounter functions like:

```
quote
default
toYaml
indent
nindent
include
required
```

These are important in real Charts.

| default

Suppose:

```
replicas: {{ .Values.replicaCount | default 1 }}
```

If replicaCount isn't supplied, Helm can use:

```
1
```

This provides template-level defaults.

Usually the primary defaults still live in values.yaml .

| toYaml

Suppose values:

```
resources:
  requests:
    cpu: 500m
    memory: 512Mi
```

Instead of manually writing every nested field, a template can use:

```
resources:
  {{- toYaml .Values.resources | nindent 10 }}
```


This converts the structure back into properly indented YAML.

You✓I see this pattern very often.

| if

Helm supports conditions.

Suppose Ingress should be optional.

Values:

```
ingress:
  enabled: true
```

Template:

```
{{- if .Values.ingress.enabled }}

apiVersion: networking.k8s.io/v1
kind: Ingress
...

{{- end }}
```

If:

```
enabled = false
```

Helm doesn't render the Ingress resource.

| Why conditional templates matter

DEV:

```
ingress:
  enabled: false
```

PROD:

```
ingress:
  enabled: true
```

Same Chart.

No need for:

```
dev-ingress.yaml
prod-ingress.yaml
```

Helm controls whether the resource exists.

| range

Helm also supports loops.

Suppose:

```
env:
  - name: LOG_LEVEL
    value: INFO

  - name: FEATURE_X
    value: "true"
```

You can loop through values and render repeated YAML blocks.

This is useful but can make Charts harder to read if overused.

A good Helm Chart should stay understandable.

| _helpers.tpl

You✓I see:

```
templates/_helpers.tpl
```

This stores reusable template fragments.

For example:

```
application name
common labels
fullname
chart labels
```

Instead of duplicating label generation everywhere.

Think:

```
_helpers.tpl
=
reusable template methods/functions
```

As a Java analogy:

```
utility/helper methods
```

| Why helpers matter

Imagine every resource needs:

```
labels:
  app.kubernetes.io/name: employee-service
  app.kubernetes.io/instance: employee-prod
```

You don't want to copy this into:

```
Deployment
Service
Ingress
ConfigMap
```

A helper can generate consistent labels everywhere.

Helm dependencies

Charts can depend on other Charts.

Suppose your application requires:

```
PostgreSQL
```

You could theoretically include a PostgreSQL Chart as a dependency.

Conceptually:

```
employee-service Chart
-
  └─ dependency
      ↓
  PostgreSQL Chart
```

Dependencies are declared in Chart.yaml .

Then:

```
helm dependency update
```

can fetch/update them.

But be careful with dependencies

Just because Helm lets you package:

```
Application + Database
```

together doesn't automatically mean that's the best production architecture.

Production databases may be:

```
Managed services
Operator-managed
Separate platform services
```

So use dependencies based on architecture, not merely convenience.

| Helm repositories

Charts can be published to repositories.

Conceptually:

```
Chart developer
  ↓
Chart repository
  ↓
Other teams
  ↓
helm install
```

Similar to:

```
Maven artifact
→ Maven repository
```

This is another reason the package-manager analogy works.

| Example third-party installation

Instead of manually building YAML for a platform component, organizations often use published Charts.

The pattern is:

```
Add repository
  ↓
Update repository index
  ↓
Install chart
```

Commands commonly look like:

```
helm repo add <name> <repository>
helm repo update
helm install ...
```

We'll focus more on application Charts first.

| Helm and OpenShift

Helm works on OpenShift too.

Remember:

```
OpenShift
=
Kubernetes
+
```

additional platform capabilities

So Helm can create standard resources such as:

```
Deployment
Service
ConfigMap
Secret
PVC
```

and it can also template OpenShift-specific resources such as:

```
Route
```

Example:

```
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: {{ .Release.Name }}
```

So your organization may maintain Helm Charts specifically designed for OpenShift deployments.

Real Spring Boot Chart

Let's build a conceptual Chart:

```
employee-service/
-
|— Chart.yaml
-
|— values.yaml
-
|— templates/
    |— deployment.yaml
    |— service.yaml
    |— configmap.yaml
    |— route.yaml
    |— _helpers.tpl
```

Values:

```
replicaCount: 3

image:
  repository: company/employee-service
  tag: "2.5.0"

service:
  port: 8080
```

```
route:
  enabled: true
  host: employee.company.com

config:
  logLevel: INFO
```

Same Chart — DEV

values-dev.yaml :

```
replicaCount: 1

image:
  tag: "2.5.0-SNAPSHOT"

route:
  host: employee-dev.company.com

config:
  logLevel: DEBUG
```

Same Chart — PROD

values-prod.yaml :

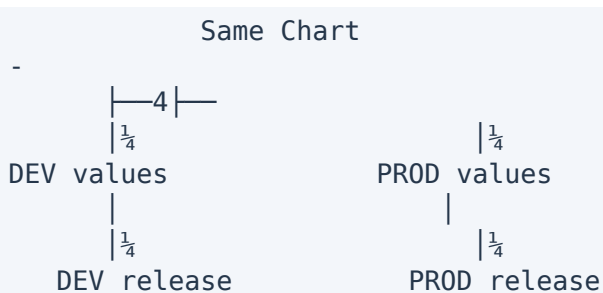
```
replicaCount: 5

image:
  tag: "2.5.0"

route:
  host: employee.company.com

config:
  logLevel: INFO
```

Now:



This is exactly why Helm is valuable.

| Helm vs plain YAML

Plain YAML Good when:

```
small application
few resources
little environment variation
```

Helm Useful when:

```
many resources
multiple environments
reusable deployment pattern
different configuration
shared platform standards
```

| Helm vs Kustomize

We✓I formally learn Kustomize later, but a quick distinction helps.

Helm

```
Templates + values
```

Kustomize

```
Base YAML + patches/overlays
```

Very simplified:

```
Helm
→ generate manifests through templates
```

```
Kustomize
→ customize existing manifests
```

Both are widely used.

| Helm does not eliminate Kubernetes knowledge

This is important.

Bad approach:

“I know Helm, so I don’t need to know Deployment, Service, PVC, probes...”

No.

Helm is just producing those resources.

When something fails:

```
Helm
↓
generated Deployment
↓
Pod
↓
CrashLoopBackOff
```

you still need Kubernetes troubleshooting knowledge.

That is why learning Helm after Kubernetes fundamentals, as you're doing now, makes much more sense.

Debugging Helm

Useful commands:

```
helm lint ./employee-service
```

Check Chart structure/templates.

Then:

```
helm template employee-prod ./employee-service \
  -f values-prod.yaml
```

Inspect generated Kubernetes YAML.

You can also use:

```
helm install ... --dry-run --debug
```

or equivalent upgrade dry-run workflows to inspect rendering and release behavior before a real change.

The key troubleshooting principle:

```
Helm issue?
↓
Look at rendered YAML
↓
Is rendered YAML correct?
-
  └─ NO → Chart/template problem
-
  └─ YES → Kubernetes/runtime problem
```

Excellent mental model.

Common Helm failure example

Suppose values:


```
service:
  targetPort: 8081
```

but Spring Boot listens on:

```
8080
```

Helm successfully installs.

Deployment runs.

Service exists.

But application is unreachable.

Helm itself isn't necessarily broken.

It rendered exactly what you asked for.

The actual problem is:

```
Incorrect configuration
↓
Service targetPort mismatch
```

So:

```
helm install succeeded
"\`
application is correctly configured
```

| Another common failure

Template:

```
replicas: {{ .Values.replicas }}
```

But values use:

```
replicaCount: 3
```

Notice:

```
replicas
vs
replicaCount
```

The template is looking for a different key.

This can produce missing/invalid output.

Use:

```
helm template
```

to catch these errors.

| Chart design principle

A common mistake is to make everything configurable.

For example:

```
values.yaml
```

with 500 parameters.

That is hard to maintain.

A good Chart usually exposes values that genuinely need customization:

```
image  
replicas  
resources  
environment configuration  
ingress/route  
autoscaling  
storage
```

while keeping stable implementation details in templates.

Think:

Expose useful flexibility, not unlimited complexity.

| Important security point

Don't treat plain Helm values files as a secure vault.

Bad:

```
dbPassword: MyProductionPassword
```

committed into:

```
values-prod.yaml
```

in Git.

Helm templates don't magically make that password secure.

For production secret management you may integrate:

```
Kubernetes Secrets  
Vault  
External Secrets
```

Sealed Secrets
other organization-approved mechanisms

depending on your platform.

| Helm in CI/CD

A real pipeline might look like:

```
Developer pushes code
  ↓
Build Spring Boot JAR
  ↓
Build container image
  ↓
Push image to registry
  ↓
Update image tag/value
  ↓
helm upgrade --install
  ↓
Kubernetes Deployment rollout
```

So Helm commonly sits in the deployment stage of CI/CD.

| Helm with GitOps

Later when we study Argo CD, you✓I see something like:

```
Git Repository
-
|— Helm Chart
|— values-prod.yaml
  ↓
  Argo CD
  ↓
Render Helm
  ↓
Kubernetes / OpenShift
```

So learning Helm now will make GitOps much easier.

| Most important Helm objects

Memorize these first:

Chart
→ reusable package

Release

→ installed instance

Chart.yaml

→ chart metadata

values.yaml

→ default configuration

templates/

→ Kubernetes manifest templates

.Values

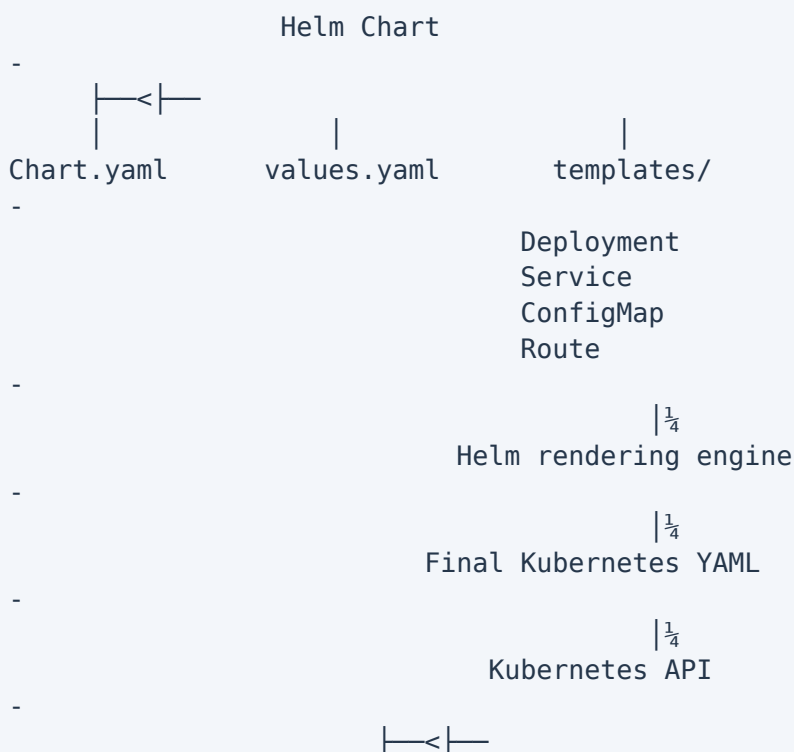
→ access provided values

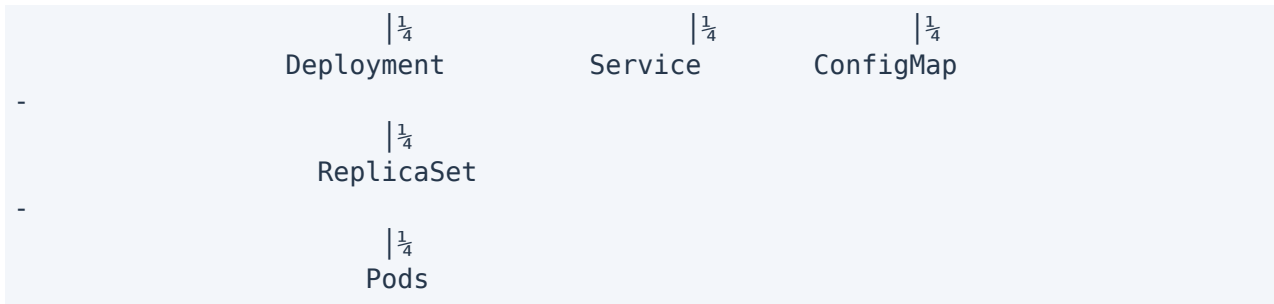
Then commands:

```
helm install
helm upgrade
helm uninstall
helm list
helm history
helm rollback
helm template
helm lint
```

That is your Helm foundation.

Complete mental model





Helm ends around:

render + submit/manage release

Kubernetes takes over the runtime lifecycle.

Chapter 23 — Kustomize

CHAPTER FOCUS

Understand **Kustomize** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

Helm and Kustomize solve a similar problem:

How do I manage Kubernetes manifests across different environments without duplicating everything?

But they solve it in very different ways.

The most important distinction is:

```
Helm
=
Templates + Values

Kustomize
=
Base YAML + Overlays/Patches
```

Why Kustomize exists

Suppose you have a normal Kubernetes Deployment:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: employee-service
spec:
  replicas: 2
  template:
    spec:
      containers:
        - name: employee-service
          image: company/employee-service:1.0
```

For DEV you want:

```
replicas = 1
```

For PROD:

```
replicas = 5
```

You could create:

```
deployment-dev.yaml
deployment-prod.yaml
```

But most of the content would be duplicated.

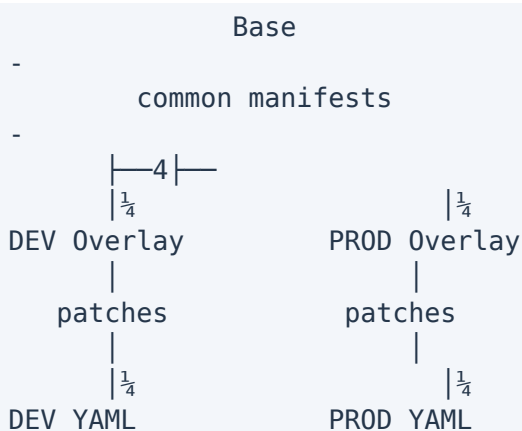
Kustomize lets you keep:

```
common configuration
```

once and apply only the differences for each environment.

Core mental model

Imagine:



So instead of templating variables like Helm:

```
replicas: {{ .Values.replicaCount }}
```

Kustomize keeps valid Kubernetes YAML and modifies it.

Base

The base contains common resources.

Example directory:

```
employee-service/
-
├─ base/
|   ├── deployment.yaml
|   ├── service.yaml
|   └── kustomization.yaml
-
└─ overlays/
```

base/deployment.yaml might contain:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: employee-service
spec:
  replicas: 2
  selector:
    matchLabels:
      app: employee-service
  template:
    metadata:
      labels:
        app: employee-service
    spec:
      containers:
        - name: employee-service
          image: company/employee-service:1.0
```

This is plain Kubernetes YAML.

No template syntax.

| customization.yaml

Kustomize needs a file called:

```
kustomization.yaml
```

Example:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
- deployment.yaml
- service.yaml
```

This tells Kustomize:

These manifests belong to this customization.

Think:

```
kustomization.yaml
=
instructions for composing Kubernetes YAML
```

| Overlay

Now create:


```
overlays/dev/
```

with:

```
overlays/  
├─ dev/  
│   └─ kustomization.yaml  
-  
├─ prod/  
│   └─ kustomization.yaml
```

The DEV overlay references the base:

```
resources:  
- ../../base
```

Meaning:

Start with everything from base .

Then modify only what DEV needs.

DEV patch

Suppose DEV needs:

```
replicas = 1
```

A patch might conceptually say:

```
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: employee-service  
spec:  
  replicas: 1
```

Kustomize applies it to the base.

Result:

```
Base:  
replicas = 2
```

```
DEV patch:  
replicas = 1
```

```
Final DEV:  
replicas = 1
```

| PROD patch

For PROD:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: employee-service
spec:
  replicas: 5
```

Result:

```
Base:
replicas = 2
```

```
PROD patch:
replicas = 5
```

```
Final PROD:
replicas = 5
```

So:

```
same Base
+
different Overlay
=
different environment manifests
```

| Helm vs Kustomize philosophy

This is the biggest conceptual difference.

Helm You start with a template:

```
replicas: {{ .Values.replicaCount }}
```

and inject:

```
replicaCount: 5
```

Kustomize You start with normal Kubernetes YAML:

```
replicas: 2
```

and overlay:

```
replicas: 5
```

So:

```
Helm:  
generate YAML  
  
Kustomize:  
transform YAML
```

| Why some teams prefer Kustomize

One major advantage is:

Your base manifests remain valid Kubernetes YAML.

For example:

```
apiVersion: apps/v1  
kind: Deployment
```

can still be understood directly by someone who knows Kubernetes.

With Helm:

```
name: {{ include "employee.fullname" . }}
```

you need to understand Helm templating too.

So Kustomize can sometimes be simpler for straightforward environment differences.

| kubectl supports Kustomize

A very useful feature is that Kustomize functionality is built into kubectl .

You can run:

```
kubectl kustomize overlays/prod
```

to generate the manifests.

Or directly apply:

```
kubectl apply -k overlays/prod
```

Notice:

```
-f
```

vs:

```
-k
```

Normal YAML

```
kubectl apply -f deployment.yaml
```

Kustomize directory

```
kubectl apply -k overlays/prod
```

Easy distinction.

| Directory model

A common structure:

```
employee-service/  
-  
|— base/  
|   |— deployment.yaml  
|   |— service.yaml  
|   |— configmap.yaml  
|   |— kustomization.yaml  
-  
|— overlays/  
-  
|   |— dev/  
|   |   |— kustomization.yaml  
|   |   |— deployment-patch.yaml  
-  
|   |— qa/  
|   |   |— kustomization.yaml  
|   |   |— deployment-patch.yaml  
-  
|   |— prod/  
|   |   |— kustomization.yaml  
|   |   |— deployment-patch.yaml
```

Think:

```
base  
=  
everything common  
  
overlay  
=  
environment differences
```

12. What kinds of things can differ?

For DEV:

```
replicas = 1  
CPU request = 100m  
memory = 256Mi
```

```
log level = DEBUG
```

PROD:

```
replicas = 5
CPU request = 500m
memory = 1Gi
log level = INFO
```

Rather than maintaining complete copies, overlays modify the common base.

| Image changes

Kustomize can also change container images.

Base might use:

```
image: company/employee-service:1.0
```

Production overlay could specify a different image/tag.

Conceptually:

```
Base:
employee-service:1.0

PROD:
employee-service:2.5.0
```

Final output:

```
employee-service:2.5.0
```

This is very useful in GitOps.

| Name prefix/suffix

Suppose your base object is:

```
employee-service
```

Kustomize can add:

```
dev-
```

resulting in:

```
dev-employee-service
```

Or suffix:

```
employee-service-prod
```

This can help distinguish resources.

However, teams often prefer namespaces rather than stuffing environment identity into every resource name.

| Namespace transformation

You can configure:

```
namespace: employee-prod
```

and Kustomize can apply that namespace to namespace-scoped resources.

Conceptually:

```
Base
  ↓
employee-service

PROD overlay
  ↓
namespace: employee-prod
```

Final:

```
Deployment → employee-prod
Service → employee-prod
ConfigMap → employee-prod
```

Useful.

| Labels and annotations

You may want PROD resources to contain:

```
labels:
  environment: production
```

or:

```
annotations:
  owner: employee-team
```

Kustomize can apply common metadata without copying it into every resource.

Conceptually:

```
Deployment ||
Service  |<| environment=production
ConfigMap |
```

ConfigMap generation

Kustomize can generate ConfigMaps.

For example, instead of writing an entire ConfigMap manually:

```
LOG_LEVEL=INFO
APP_NAME=employee
```

Kustomize can generate the Kubernetes ConfigMap from literals/files.

One useful behavior is that generated ConfigMaps can have content-dependent names.

Conceptually:

```
employee-config-abc123
```

When configuration changes:

```
employee-config-def456
```

This helps cause workloads referencing the generated name to receive an updated configuration reference.

Secret generation

Kustomize also supports generating Kubernetes Secrets.

But remember from our Secrets chapter:

Kustomize is not a secure secret vault.

If you put a production password directly into Git:

```
password=Production123
```

Kustomize won't magically secure it.

Secret management still requires the correct external/platform approach.

Patches

The heart of Kustomize is:

Patching Conceptually:

```
Original resource
    +
Patch
    ↓
Modified resource
```

For example:

Base:

```
spec:
  replicas: 2
```

Patch:

```
spec:
  replicas: 6
```

Final:

```
spec:
  replicas: 6
```

| Why patches are powerful

Suppose the base Deployment is 80 lines.

PROD differs only in:

```
replicas
resources
```

You don't need another 80-line Deployment.

You might need only a small patch.

This drastically reduces duplication.

| Real example

Base:

```
employee-service
replicas = 2
```

```
CPU request = 250m
Memory request = 512Mi
```

DEV overlay:

```
replicas = 1
CPU request = 100m
```

PROD overlay:

```
replicas = 6
CPU request = 750m
Memory request = 1Gi
```

Architecture:



Kustomize doesn't deploy by itself

Important distinction.

Kustomize mainly:

```

takes manifests
+
applies transformations
+
produces manifests
  
```

Then:

```
kubectl apply -k
```

can send them to Kubernetes.

So Kustomize isn't:

```

Deployment Controller
Scheduler
Package manager/release manager
  
```

It's primarily a configuration customization tool.

23. Does Kustomize have Helm-style releases?

No.

This is a major difference.

Helm maintains the concept:

```

Release
Revision
History
Rollback
  
```

Kustomize doesn't inherently provide that same release-management model.

So:

```
Helm
→ templating + packaging + release management

Kustomize
→ manifest transformation/customization
```

This difference matters.

Helm rollback vs Kustomize

With Helm:

```
helm rollback employee-prod 2
```

Helm understands release revisions.

With Kustomize, rollback is commonly handled by something like:

```
Git history
+
GitOps
```

For example:

```
Git commit A
↓
PROD config

Git commit B
↓
bad config

git revert B
↓
Argo CD reconciles previous state
```

So Kustomize fits particularly well with GitOps.

Helm + Kustomize aren't always competitors

This is important.

People sometimes ask:

Should I use Helm OR Kustomize?

Sometimes yes.

But organizations can also combine them.

For example:

```

Third-party Helm Chart
    ↓
Rendered resources
    ↓
Kustomize modifications

```

Or GitOps tooling can understand both.

So don't treat them as mutually exclusive technologies.

Helm vs Kustomize comparison

Feature Helm Kustomize

Main approach Templates Base + overlays

Variable syntax Yes No template variables normally

Base files valid Kubernetes YAML Not always Yes

Environment config values files overlays

Packaging Strong Not primary purpose

Release history Yes No Helm-style history

Rollback Helm revisions Usually Git/GitOps

Built into kubectl No Yes

Third-party app distribution Very common Less focused on packaging

GitOps use Excellent Excellent

27. When would you use Helm?

Suppose your organization has:

25 Spring Boot microservices

and every one needs:

```

Deployment
Service
ConfigMap
HPA
Route
ServiceAccount

```

with a standardized structure.

A reusable Helm Chart can be very useful:

```
company-microservice-chart
+
service-specific values
```

28. When would you use Kustomize?

Suppose you already have clean Kubernetes manifests:

```
deployment.yaml
service.yaml
configmap.yaml
```

and differences between:

```
DEV
QA
PROD
```

are relatively small.

Kustomize is very natural.

```
Base manifests
+
small environment overlays
```

No template language required.

One simple decision rule

You can think:

```
Need reusable configurable package?
↓
Helm
```

while:

```
Have Kubernetes YAML and mainly
need environment-specific modifications?
↓
Kustomize
```

Not an absolute rule, but a good starting point.

Java analogy

A rough analogy:

Helm Feels more like:

```
TemplateFactory.create(
    replicas,
    image,
    memory
);
```

Generate the configuration.

Kustomize Feels more like:

```
Deployment base = loadBase();
```

```
base.setReplicas(5);
base.setImage("2.5");
```

Take something existing and modify selected fields.

GitOps connection

This is where Kustomize becomes especially powerful.

Repository:

```
git/
-
├─ base/
-
├─ overlays/
│   ├─ dev/
│   ├─ qa/
│   └─ prod/
```

Then Argo CD can watch:

```
overlays/prod
```

Conceptually:

```
Developer
  ↓
Git commit
  ↓
Kustomize overlay
  ↓
Argo CD
  ↓
Kubernetes
```

No developer needs to manually execute:

```
kubectl apply
```

in production.

We'll cover that in the next chapter.

| Customize troubleshooting

Suppose PROD doesn't contain the expected replicas.

First generate the output:

```
kubectl kustomize overlays/prod
```

Then inspect:

```
spec:
  replicas:
```

This lets you determine:

Wrong generated YAML?

```
-
  └─ YES
      ↓
      Kustomize/base/patch problem
-
  └─ NO
      ↓
      Kubernetes/runtime problem
```

Very similar to the helm template mental model.

| Common problem: patch doesn't match resource

Suppose base contains:

```
metadata:
  name: employee-service
```

but your patch targets:

```
metadata:
  name: employees-service
```

Notice the extra s .

Kustomize may not patch the resource you expected.

So always verify the rendered result.

| Don't make overlays enormous

A bad Kustomize architecture looks like:

```
Base
+
DEV changes 80|
+
PROD changes 90|
```

At that point, the environments aren't really sharing much of a common base.

Overlays work best when:

```
most configuration is shared
```

and:

```
only selected fields differ
```

| Base should represent common truth

For example:

```
Base:

Application name
container port
Service
health probes
common labels
volume mounts
```

Overlays:

```
DEV:
replicas
resources
image tag
environment values

PROD:
replicas
resources
image tag
environment values
```

That is a healthy structure.

| What you should remember

The most important formula:

```
Base
+
Overlay/Patch
=
Environment-specific Kubernetes YAML
```

And:

```
Helm
=
Templates + Values

Kustomize
=
Base + Patches
```

Also:

```
kubectl apply -f
→ normal manifest

kubectl apply -k
→ Kustomize directory
```

You don't need to memorize every Kustomize directive right now. The core goal is understanding why it exists and how it differs from Helm.

Next chapter — chapter 33: GitOps & Argo CD This is the logical next step because Helm and Kustomize are frequently used inside GitOps workflows.

We✓I answer:

Why should production configuration live in Git?

What does Argo CD actually do?

What does “Git is the source of truth” mean if Kubernetes state is stored in etcd?

That last question is especially important because you've raised that exact conceptual issue before.

We✓I build:

```
Developer
  ↓
Git
  ↓
Argo CD
  ↓
Kubernetes API
  ↓
etcd
  ↓
Cluster
```


and clearly distinguish:

Git

→ desired configuration source of truth for GitOps

etcd

→ Kubernetes cluster's operational state store

so the two concepts don't conflict.

Chapter 24 — GitOps and Argo CD

CHAPTER FOCUS

Understand **GitOps** and **Argo CD** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

This chapter answers a very practical question:

How do companies safely deploy Kubernetes/OpenShift changes without manually running `kubectl apply` in production?

The core idea is:

```
Git
=
desired configuration

Argo CD
=
continuously compares Git with cluster

Kubernetes
=
runs the actual workloads
```

The problem before GitOps

Imagine production deployment works like this:

```
Developer
  ↓
edits YAML
  ↓
kubectl apply -f deployment.yaml
  ↓
Production cluster
```

This works technically.

But now ask:

- Who changed production?
- What exact YAML was applied?

- When was it changed?
- Can we review it before deployment?
- Can we roll it back?
- What if someone manually edits the cluster later?
- What if DEV and PROD drift apart?

This becomes difficult at scale.

GitOps solves much of this.

2. What is GitOps?

GitOps means:

The desired state of your system is stored declaratively in Git, and automation continuously makes the actual environment match that Git state.

Think:

```
Git
↓
Desired state

Kubernetes cluster
↓
Actual state
```

Then:

```
Desired " ` Actual
      ↓
GitOps controller
      ↓
Reconcile
```

This should already sound familiar.

Why?

Because Kubernetes itself uses the same reconciliation philosophy.

Kubernetes reconciliation vs GitOps reconciliation

You've already learned:

```
Deployment says:
replicas = 3
```

```
Actual:
```

```
replicas = 2  
  
Kubernetes controller  
  ↓  
creates another Pod
```

GitOps applies the same idea at a higher level.

Example:

```
Git says:  
Deployment replicas = 5  
  
Cluster says:  
Deployment replicas = 3  
  
Argo CD  
  ↓  
detects difference  
  ↓  
syncs cluster toward Git
```

So:

```
Kubernetes controller  
→ reconciles Kubernetes resources  
  
Argo CD  
→ reconciles Git-defined configuration with Kubernetes resources
```

Excellent distinction.

4. What is Argo CD?

Argo CD is a Kubernetes-native continuous delivery / GitOps controller.

Its job is broadly:

```
Watch Git  
  ↓  
Understand Kubernetes configuration  
  ↓  
Compare with cluster  
  ↓  
Report differences  
  ↓  
Optionally apply changes
```

Conceptually:

```
Git Repository  
-  
  |  
  1/4
```

```

    Argo CD
-   | 1/4
Kubernetes API
-   | 1/4
    Cluster

```

Source of truth

You✓I hear this phrase constantly:

Git is the source of truth.

Suppose Git contains:

```
replicas: 5
```

But someone manually runs:

```
kubectl scale deployment employee-service --replicas=8
```

Now:

```
Git:
5 replicas
```

```
Cluster:
8 replicas
```

This is called:

Drift

6. What does Argo CD see?

Argo CD compares:

```
Git desired state
    vs
Cluster actual configuration
```

and reports:

```
OutOfSync
```

because they differ.

If automated synchronization is enabled, Argo CD can restore:

```
8 → 5
```

because Git says:

5

So manual production changes don't necessarily remain permanently.

A very important question you raised before

You previously asked something like:

If Git is the source of truth, what about etcd? Doesn't etcd store Kubernetes state?

Both statements are correct because they mean different things.

Git vs etcd

Git In GitOps:

```
Git
=
authoritative desired configuration
```

Example:

```
Deployment replicas = 5
image = employee-service:2.7
resources = ...
```

Git records what your organization intends the cluster to look like.

etcd

```
etcd
=
Kubernetes control-plane state store
```

It stores the state Kubernetes is currently managing.

That includes:

```
Deployments
Pods
Services
ConfigMaps
Secrets
Nodes

resource status
etc.
```

So:

Git

→ external desired-state source for GitOps

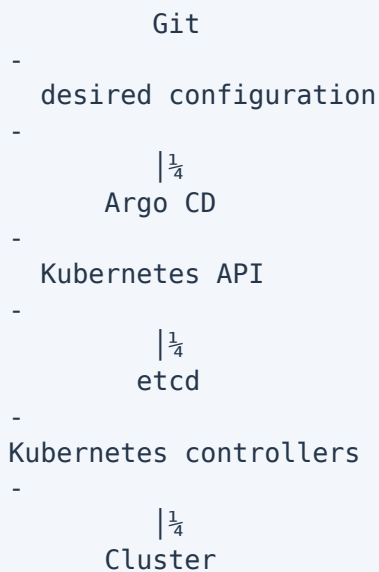
etcd

→ internal operational state store for Kubernetes

They are not competing databases.

Full relationship

Think:



Then status flows back:



Example with your Spring Boot application

Suppose your Git repository says:

```

image:
  tag: "2.5.0"

replicaCount: 3
  
```

Argo CD deploys it.

Production becomes:

```
employee-service:2.5.0  
replicas = 3
```

Now developer releases:

```
2.6.0
```

Instead of manually running:

```
helm upgrade ...
```

someone changes Git:

```
image:  
  tag: "2.6.0"
```

Then:

```
Git commit  
  ↓  
Argo CD detects change  
  ↓  
Sync  
  ↓  
Deployment updated  
  ↓  
Kubernetes rolling update  
  ↓  
2.6.0 Pods
```

That is GitOps.

GitOps and Helm

This connects directly to chapter 31.

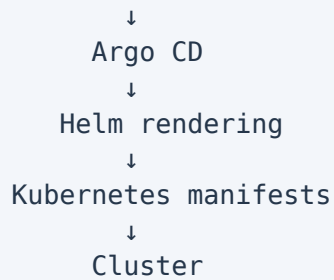
Your repository may contain:

```
employee-service/  
-  
├─ Chart.yaml  
  
├─ templates/  
└─ values-prod.yaml
```

Argo CD can use the Helm Chart.

Conceptually:

```
Git  
-  
├─ Helm Chart  
└─ values-prod.yaml
```

So developers don't necessarily run:

```
helm upgrade
```

against production manually.

Argo CD can do the rendering/synchronization.

GitOps and Kustomize

Same story.

Repository:

```
base/  
overlays/  
  dev/  
  prod/
```

Argo CD can point at:

```
overlays/prod
```

Then:

```
Git  
↓  
Kustomize  
↓  
Argo CD  
↓  
Kubernetes
```

So Argo CD works well with:

```
Plain Kubernetes YAML  
Helm  
Kustomize
```

13. What is an Argo CD Application?

An important Argo CD concept is:

Application An Argo CD Application defines things like:

```
Where is my desired state?
Which Git repository?
Which path/chart?
Which revision/branch?
Which cluster?
Which namespace?
```

Conceptually:

```
Argo CD Application
-
|— Source
|   |— Git repository
-
|— Destination
|   |— Kubernetes cluster
|   |— namespace
```

| Simplified Application example

Conceptually:

```
apiVersion: argoproj.io/v1alpha1
kind: Application

metadata:
  name: employee-service

spec:
  source:
    repoURL: <git repository>
    targetRevision: main
    path: apps/employee-service

  destination:
    namespace: employee-prod
```

The important idea is:

```
Source
→ Git

Destination
→ Kubernetes
```

15. What is Sync?

Argo CD uses the concept:

Sync Meaning:

Apply the desired state from Git to the cluster.

Suppose:

```
Git:
replicas = 5
```

```
Cluster:
replicas = 3
```

Sync means:

```
Argo CD
↓
apply desired configuration
↓
Cluster becomes 5
```

| Synced vs OutOfSync

You✓I commonly see:

Synced

meaning:

Cluster configuration matches the desired Git configuration.

And:

OutOfSync

meaning:

Git and cluster differ.

Example:

```
Git:
image = 2.5
```

```
Cluster:
image = 2.4
```

→ OutOfSync

After synchronization:

```
Cluster:
image = 2.5
```

→ Synced

| Synced does NOT mean healthy

This is extremely important.

Argo CD may report:

```
Sync Status:
Synced
```

but application:

```
Pod → CrashLoopBackOff
```

Why?

Because:

```
Synced
=
the desired manifests were applied
```

It does not mean:

```
application is functioning correctly
```

So Argo CD also tracks a separate concept:

Health

| Sync vs Health

Think:

Sync status

```
Does cluster configuration match Git?
```

Health status

```
Are the Kubernetes resources healthy?
```

Example:

```
Git matches cluster
  ↓
Synced '
But Deployment has
0/3 ready Pods
  ↓
Degraded ×
```

This distinction is very important.

Example

Suppose Git says:

```
replicas = 3  
image = employee-service:2.7
```

Cluster has exactly:

```
replicas = 3  
image = employee-service:2.7
```

Therefore:

Synced

But image contains a Java bug:

```
Spring Boot  
↓  
throws exception  
↓  
CrashLoopBackOff
```

Argo CD could show something like:

```
Sync:  
Synced  
  
Health:  
Degraded
```

The configuration matches Git, but the application is unhealthy.

Manual Sync

One approach is:

```
Git changes  
↓  
Argo CD detects OutOfSync  
↓  
Human clicks/executes Sync  
↓  
Deployment happens
```

This gives humans an approval point.

Automatic Sync

You can also configure automatic synchronization.

Conceptually:

```
Git changes
  ↓
Argo CD detects change
  ↓
automatically syncs
  ↓
Cluster updated
```

This makes Git commits directly drive environment changes.

Organizations choose different policies for:

```
DEV
QA
PROD
```

For example:

```
DEV
→ automatic

PROD
→ controlled promotion/approval
```

depending on governance requirements.

Self-healing

One powerful GitOps feature is:

Self-heal Suppose Git says:

```
replicas = 3
```

Someone manually changes production:

```
kubectl scale ... --replicas=10
```

Argo CD notices:

```
Git = 3
Cluster = 10
```

With self-healing enabled:

```
Argo CD
  ↓
reverts cluster
  ↓
3 replicas
```

This reduces configuration drift.

| Why manual kubectl becomes discouraged

In GitOps-managed environments, you may hear:

“Don’t manually modify production.”

Why?

Because if you do:

```
kubectl edit deployment
```

you’ve created configuration that exists only in:

```
cluster
```

but not in:

```
Git
```

Now:

```
Git " ` Cluster
```

and you’ve lost the reliable source-of-truth model.

Better:

```
Edit Git
↓
Pull Request
↓
Review
↓
Merge
↓
Argo CD
↓
Cluster
```

| Why GitOps is powerful for auditing

Imagine someone asks:

Why did production change at 2:30 PM?

With manual deployment, maybe nobody knows.

With GitOps:

```
Git commit
-
├─ author
├─ timestamp
├─ diff
├─ pull request
└─ review
```

You can trace:

```
Production change
  ↓
Git commit
  ↓
who/what/why
```

This is a major enterprise advantage.

Rollback with GitOps

Suppose:

```
Commit A
→ image 2.5

Commit B
→ image 2.6
```

2.6 is broken.

You can revert Commit B.

Then:

```
Git
↓
image returns to 2.5
↓
Argo CD
↓
syncs cluster
↓
2.5 restored
```

So Git history becomes an important rollback mechanism.

This is especially natural with Kustomize.

Helm rollback vs GitOps rollback

Remember:

Helm Can have:

```
helm rollback
```

using Helm release history.

GitOps Often prefers:

```
Git revert  
↓  
Argo CD reconciliation
```

Why?

Because Git should remain the desired-state record.

If you perform a Helm rollback manually but don't update Git:

```
Git says 2.6
```

```
Cluster says 2.5
```

Argo CD may simply see drift and restore 2.6.

So in GitOps environments:

The rollback should usually be represented in Git too.

CI vs CD

This is another important distinction.

CI — Continuous Integration Usually handles:

```
Source code  
↓  
Compile  
↓  
Tests  
↓  
Build JAR  
↓  
Build container image  
↓  
Push image
```

Example:

```
employee-service:2.7
```

CD — Continuous Delivery/Deployment Handles:

Which version should run in environment?

↓

Deployment configuration

↓

Kubernetes

Argo CD lives mostly on the:

CD

side.

| Complete CI/CD GitOps flow

For your Spring Boot application:

```
Developer
  ↓
Java commit
  ↓
CI pipeline
  ↓
mvn test/package
  ↓
Container build
  ↓
Registry
employee-service:2.7
  ↓
Update GitOps repo
image tag = 2.7
  ↓
Argo CD
  ↓
OpenShift/Kubernetes
```

This is a very common modern architecture.

| Two repositories model

Some organizations separate:

Application repository

```
employee-service/
-
|— src/
|— pom.xml
|— Dockerfile
```

from:

GitOps repository

```
environments/  
-  
├─ dev/  
├─ qa/  
└─ prod/
```

The application pipeline builds:

```
employee-service:2.7
```

Then GitOps configuration is updated to reference:

```
2.7
```

This provides a clean separation between:

```
Application source
```

and:

```
Deployment state
```

Example promotion flow

Suppose new image:

```
employee-service:2.7
```

First:

```
DEV values  
image = 2.7
```

After testing:

```
QA values  
image = 2.7
```

Later:

```
PROD values  
image = 2.7
```

Conceptually:

```
Build once  
  ↓  
2.7  
  ↓
```

```
DEV
  ↓
QA
  ↓
PROD
```

The image remains the same.

Only Git environment configuration changes.

This fits perfectly with the “build once, deploy many” principle we learned earlier.

31. What is drift?

Let’s define it precisely.

Drift means:

```
Desired configuration
    „`
Actual cluster configuration
```

Examples:

```
Git replicas = 3
Cluster replicas = 5
```

or:

```
Git CPU limit = 1
Cluster CPU limit = 2
```

or:

```
Git Service type = ClusterIP
Cluster type = NodePort
```

Argo CD identifies these differences.

32. What is pruning?

Suppose Git contains:

```
Deployment
Service
ConfigMap
```

You remove:

```
ConfigMap
```

from Git.

Should the existing Kubernetes ConfigMap remain forever?

GitOps systems can support:

Pruning Meaning:

```
Resource no longer exists in Git
      ↓
Remove resource from cluster
```

So Git can define not only:

What should exist?

but effectively:

What should no longer exist?

| Pruning must be used carefully

Imagine someone removes:

```
PVC
```

from Git accidentally.

You don't want careless automation deleting critical persistent data.

So resource deletion policies, sync settings, and retention behavior need careful design.

GitOps is powerful, but automation magnifies mistakes too.

| Argo CD doesn't replace Kubernetes

Very important.

Argo CD doesn't replace:

```
Deployment Controller
Scheduler
Kubelet
Service
Ingress
PVC
```

It sits above them.

Think:

```
GitOps layer
      ↓
Argo CD
      ↓
Kubernetes API
      ↓
```

```
Kubernetes controllers
```

↓

```
Nodes/Pods
```

Argo CD says:

“This Deployment should look like this.”

Then Kubernetes does:

```
ReplicaSet
```

```
Pods
```

```
Scheduling
```

```
Health
```

| **Argo CD doesn't build your Java application either**

Another common confusion.

Argo CD generally isn't responsible for:

```
mvn package
```

or:

```
docker build
```

That is CI.

Think:

```
Jenkins / GitHub Actions / Tekton / CI system
```

```
→ build/test/image
```

then:

```
Argo CD
```

```
→ deployment reconciliation
```

| **OpenShift connection**

Argo CD works very naturally with OpenShift.

A Git repository might define:

```
Deployment
```

```
Service
```

```
Route
```

```
ConfigMap
```

```
Secret references
```

```
ServiceAccount
```

Then:

```
Argo CD
↓
OpenShift API
↓
Project
↓
Resources
```

OpenShift also has a GitOps offering based on Argo CD.

Conceptually, though, the fundamental GitOps principles remain the same.

Real incident example

Suppose someone reports:

“I increased replicas from 3 to 6 using oc scale , but five minutes later it went back to 3.”

Why?

Check Git.

If Git still says:

```
replicas: 3
```

then:

```
Manual scale → 6
↓
Argo CD sees drift
↓
Self-heal
↓
back to 3
```

This is a classic GitOps behavior.

The correct change should be:

```
Git:
replicas: 6
```

not just manual scaling.

Another incident

Developer says:

“I updated the Helm values-prod.yaml to image 2.8, but production still runs 2.7.”

Investigate:

```
Git change merged?
  ↓
Argo CD detects commit?
  ↓
Application OutOfSync?
  ↓
Sync succeeded?
  ↓
Deployment updated?
  ↓
Pod image correct?
```

The troubleshooting chain becomes:

```
Git
  ↓
Argo CD
  ↓
Rendered manifests
  ↓
Kubernetes Deployment
  ↓
ReplicaSet
  ↓
Pod
```

GitOps troubleshooting mindset

If Git says the correct value but cluster is wrong:

```
Git correct
Cluster wrong
```

check:

```
Argo CD sync status
```

If Argo CD says Synced but application is broken:

```
Git = Cluster
```

move into normal Kubernetes troubleshooting:

```
Deployment
Pods
logs
probes
Service
etc.
```

This is how GitOps and Kubernetes troubleshooting connect.

Source of truth hierarchy

Here's a very useful mental model:

```
Organization intent
  ↓
  Git
  ↓
  Argo CD
  ↓
  Kubernetes API
  ↓
  etcd
  ↓
  Kubernetes Controllers
  ↓
  Actual Pods
```

Different layers have different responsibilities.

Don't confuse GitOps with "Git stores everything"

Git should not blindly contain all runtime state.

You generally don't store things such as:

```
Pod current CPU usage
Pod restart count
Node health
live request latency
```

in Git.

These are runtime state/observability concerns.

Git stores declarative desired configuration.

So:

```
Git
→ desired state

etcd / Kubernetes API
→ cluster state

Prometheus/logging
→ runtime observations
```

Excellent separation.

Secrets in GitOps

This is a major challenge.

You want configuration in Git.

But you don't want:

```
password: MyProdPassword
```

sitting in Git.

So GitOps architectures commonly integrate mechanisms such as:

```
External Secrets  
Vault  
Sealed Secrets  
cloud secret managers
```

depending on the organization.

Conceptually:

```
Git  
↓  
Secret reference/configuration  
↓  
External secret system  
↓  
Kubernetes Secret  
↓  
Application
```

rather than storing raw credentials directly in Git.

GitOps does not mean every developer can deploy prod

GitOps actually works well with strong governance.

For example:

```
Developer  
↓  
Pull Request  
  
↓  
Senior review  
↓  
Approval  
↓  
Merge  
↓  
Argo CD  
↓
```

Production

So production changes become:

reviewed
auditable
reproducible

| Why GitOps is better than “pipeline pushes to cluster”

Traditional CI/CD might be:

```
Jenkins
↓
kubectl apply
↓
Production
```

This is called a push model.

GitOps commonly uses a pull/reconciliation model:

```
Git
!
Argo CD checks
↓
Cluster
```

Argo CD running in/near the platform pulls desired state and reconciles it.

| Push vs Pull

Traditional push

```
CI system
↓
credentials to production cluster
↓
kubectl/helm
↓
production
```

GitOps pull/reconcile

```
CI
↓
updates Git

Argo CD
↓
reads Git
```

↓
updates cluster

This can reduce the need for CI pipelines themselves to possess broad production cluster credentials.

| Important interview question

What is GitOps? Good answer:

GitOps is an operational model where declarative infrastructure and application configuration are stored in Git as the desired state, and an automated controller continuously reconciles the runtime environment toward that state.

| Interview question

Why is Git called the source of truth if Kubernetes uses etcd? Answer:

In GitOps, Git is the authoritative source for the desired declarative configuration, while etcd is Kubernetes' internal persistent store for the current cluster state and resource objects. They serve different purposes. That is an excellent answer.

| Interview question

What is Argo CD? Answer:

Argo CD is a Kubernetes-native GitOps continuous delivery controller that compares Kubernetes resources with declarative configuration in Git and synchronizes the cluster toward that desired state.

| Interview question

What does OutOfSync mean? Answer:

The live Kubernetes configuration differs from the desired configuration Argo CD rendered from Git.

| Interview question

Can an application be Synced but unhealthy?

Synced
→ Git matches cluster

Healthy
→ application resources are functioning correctly

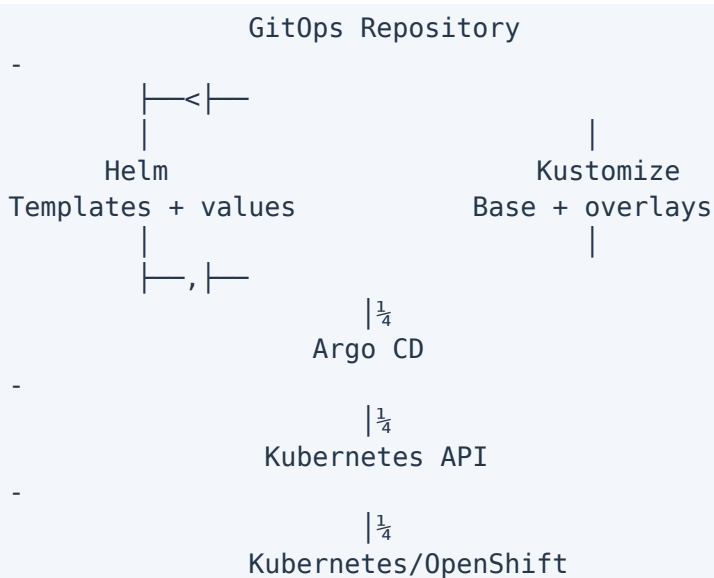
Therefore:

Synced + Degraded

is possible.

Helm + Kustomize + Argo CD

You can now connect your last three modules:



Helm/Kustomize answer:

How should manifests be constructed/customized?

Argo CD answers:

How does the cluster continuously receive and reconcile them from Git?

chapter 33 Summary

Remember these five ideas:

Git
=
desired state

Argo CD
=
Git !" Cluster reconciler

etcd
=
Kubernetes internal state store

OutOfSync

```
=  
Git and cluster differ  
  
Synced  
"\  
Healthy
```

And the end-to-end flow:

```
Java code  
  ↓  
CI  
  ↓  
Container image  
  ↓  
Registry  
  ↓  
GitOps configuration update  
  ↓  
Argo CD  
  ↓  
Kubernetes/OpenShift  
  ↓  
Deployment  
  ↓  
Pods
```

Next chapter — chapter 34: Monitoring, Prometheus & Grafana The next topic moves from:

“Is Git synchronized?”

to:

“Is my production application actually performing well?”

We✓I learn the difference between:

```
Metrics  
Logs  
Traces
```

then go deeper into:

```
Prometheus  
Grafana  
Alertmanager  
Metrics Server  
Application metrics  
Spring Boot Actuator / Micrometer
```

and answer practical questions like:

Why does kubectl top work?

Where does Prometheus get metrics from?

Does Prometheus push or pull metrics?

What's the difference between Kubernetes metrics and application metrics?

How would I detect that my Spring Boot API has 5% errors or 2-second latency even though all Pods are Running ?

OpenShift

Chapter 25 — OpenShift Architecture

CHAPTER FOCUS

Understand **OpenShift Architecture** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

Now we're moving from:

"I understand Kubernetes"

to:

"I understand what OpenShift actually adds to Kubernetes."

I'll use OpenShift Container Platform 4.20 terminology for the current architecture, but the core ideas apply broadly across OpenShift 4.x. Red Hat describes OpenShift as a Kubernetes-based platform with additional enterprise

platform-management capabilities, including integrated lifecycle management, networking/routing, authentication, monitoring, Operators, and RHCOS.

The most important idea

Don't think:

Kubernetes vs OpenShift

as two completely different orchestration systems.

Think:

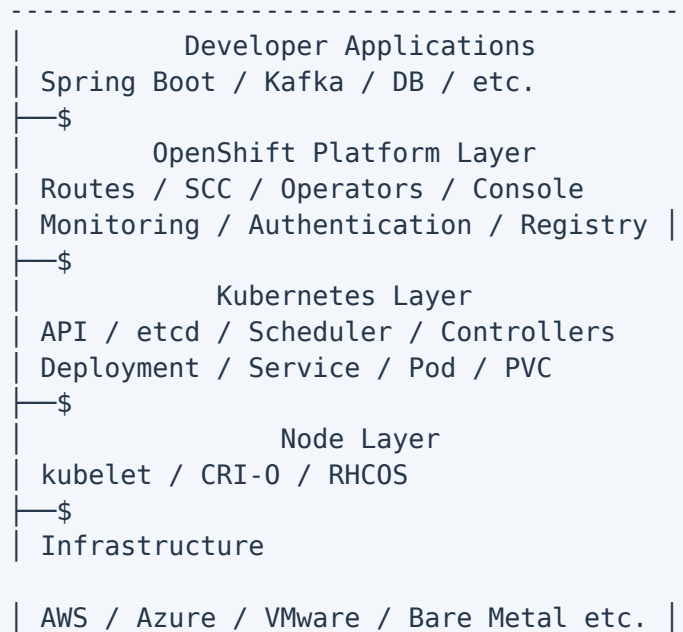


So:

OpenShift is Kubernetes plus an opinionated enterprise platform around Kubernetes.
Your Kubernetes knowledge transfers directly.

| OpenShift's architecture as layers

A useful mental model is:



This explains OpenShift much better than memorizing individual features.

| Kubernetes still sits at the core

The control plane concepts you've already learned still exist:

```
API Server  
etcd  
Scheduler  
Controller Manager
```

Worker Nodes still have:

```
kubelet  
CRI-O  
networking components  
Pods
```

And your application still follows:

```
Deployment  
↓  
ReplicaSet  
↓
```

```
Pod
  ↓
Container
```

OpenShift doesn't replace that machinery.

It adds management and platform services around it.

| OpenShift uses RHCOS

One important OpenShift architectural difference is its Node operating-system strategy.

OpenShift commonly uses:

Red Hat Enterprise Linux CoreOS — RHCOS It's designed specifically for OpenShift Nodes and integrates tightly with cluster lifecycle management. Current OpenShift documentation describes RHCOS as the container-oriented OS used by OpenShift, with CRI-O as its Kubernetes-native container runtime.

Conceptually:

```
Worker Node
-
|— RHCOS
-
|— CRI-O
-
|— kubelet
-
|— Application Pods
```

5. Why is that useful?

In ordinary server administration, you might separately maintain:

```
Linux OS
Kubernetes
Container runtime
Security patches
Configuration
```

OpenShift tries to manage more of this as one platform lifecycle.

That is where another major component comes in:

Machine Config Operator — MCO

Machine Config Operator

The MCO manages Node operating-system configuration and coordinates RHCOS updates/config changes across the cluster.

Think:

```

Desired Node configuration
  ↓
Machine Config Operator
  ↓
Machine Config Pool
  ↓
Worker Nodes
  ↓
RHCOS updated/configured consistently

```

This is a very OpenShift-style philosophy:

Manage infrastructure using Kubernetes-like declarative APIs.

Machine Config Pools

Nodes are grouped into:

```
MachineConfigPool
```

Typically you will encounter pools such as:

```

master/control-plane
worker

```

Conceptually:

```

Cluster
-
├─ master MachineConfigPool
│   ├── master-0
│   ├── master-1
│   └── master-2
-
├─ worker MachineConfigPool
│   ├── worker-0
│   ├── worker-1
│   └── worker-2

```

The MCO can progressively update these Nodes rather than you manually SSHing into every machine.

You can inspect them with:

```
oc get mcp
```

| Operators are fundamental to OpenShift

This is probably the biggest architectural idea in modern OpenShift.

OpenShift itself is heavily operated using:

Operators Red Hat describes Operators as a fundamental part of the OpenShift platform; they encode operational knowledge and manage components using Kubernetes APIs and reconciliation.

You've already learned:

```
Deployment controller:  
desired replicas = 3  
actual replicas = 2  
→ reconcile
```

Operator uses the same pattern for more complicated software.

| Example: Ingress Operator

Instead of an administrator manually maintaining:

```
router pods  
certificates  
configuration  
upgrades  
availability
```

OpenShift has an:

```
Ingress Operator
```

which manages the Ingress Controller infrastructure.

Conceptually:

```
Ingress configuration  
    ↓  
Ingress Operator  
    ↓  
Ingress Controller / Router  
    ↓  
Route traffic
```

The OpenShift Ingress Operator manages HAProxy-based Ingress Controllers that serve Route and Kubernetes Ingress traffic.

| Two kinds of Operators

This distinction is extremely important.

Cluster Operators These operate OpenShift itself.

Examples conceptually include:

```
authentication
ingress
DNS
networking
monitoring
etcd
API server
machine config
image registry
```

They are managed as part of the OpenShift platform by the:

Cluster Version Operator — CVO

Add-on Operators These manage optional software you install.

Examples might include:

```
database Operator
Kafka Operator
storage Operator
GitOps Operator
```

These are typically managed through:

```
Operator Lifecycle Manager – OLM
```

and the software catalog/OperatorHub ecosystem.

So remember:

```
Cluster Operators
→ operate OpenShift itself

OLM Operators
→ optional application/platform extensions
```

Cluster Version Operator

The:

CVO — Cluster Version Operator coordinates the lifecycle of core OpenShift platform components.

Very simplified:

```
OpenShift desired version
    ↓
Cluster Version Operator
    ↓
Cluster Operators
    ↓
```

API
Ingress
Networking
Monitoring
etc.

This is one reason OpenShift upgrades feel different from building a Kubernetes cluster manually.

OpenShift is designed so that much of the platform is managed as a coordinated release.

| oc get clusteroperators

As an OpenShift administrator, this becomes a critical command:

```
oc get clusteroperators
```

or shorter:

```
oc get co
```

You✓I see statuses such as:

AVAILABLE
PROGRESSING
DEGRADED

For a healthy steady-state cluster, Red Hat’s documentation commonly expects:

AVAILABLE = True
PROGRESSING = False
DEGRADED = False

for cluster Operators.

This is almost like a high-level health dashboard for major OpenShift subsystems.

| Example

You might see conceptually:

NAME	AVAILABLE	PROGRESSING	DEGRADED
authentication	True	False	False
console	True	False	False
etcd	True	False	False
ingress	True	False	False
network	True	False	False
monitoring	True	False	False

If:

```
ingress
AVAILABLE=False
DEGRADED=True
```

and all externally exposed applications stop working:

that's a very strong clue

because the problem may be at the OpenShift Ingress platform layer rather than in every individual application.

| OpenShift Project

You've already learned Kubernetes:

```
Namespace
```

OpenShift commonly exposes a richer user-facing abstraction called:

Project A Project is built on a Kubernetes Namespace and provides OpenShift-specific management/provisioning behavior around it. Red Hat describes Projects as the mechanism for groups to organize and manage resources with isolation and access controls.

Think:

```
Kubernetes:
namespace employee-prod
```

```
OpenShift:
project employee-prod
```

| Example

OpenShift Cluster

```
-
|— employee-dev
|   |— Deployment
|   |— Service
|   |— Route
|   |— ConfigMap
|
|— employee-prod
|   |— Deployment
|   |— Service
|   |— Route
|   |— Secret
|
|— payments-prod
```


Projects become a natural organizational boundary for:

```
resources
RBAC

quotas
applications
```

| Creating a Project

Instead of:

```
kubectl create namespace employee-dev
```

OpenShift gives you:

```
oc new-project employee-dev
```

and switches your current CLI context to that Project.

You can then:

```
oc project employee-prod
```

to change Projects.

| oc vs kubectl

OpenShift provides:

oc Think:

```
kubectl capabilities
+
OpenShift-specific capabilities
=
oc
```

Most familiar commands remain intuitive:

```
oc get pods
oc get deployment
oc describe pod
oc logs
oc exec
oc get svc
```

But oc also understands OpenShift-specific workflows/resources:

```
oc new-project
oc get routes
oc expose service
oc get clusteroperators
oc adm ...
```

So as an OpenShift engineer, you✓I normally use:

```
oc
```

even though Kubernetes fundamentals remain the same.

| Routes

You already learned Kubernetes:

```
Ingress
```

OpenShift provides its own widely used external exposure object:

Route Architecture:

```
Internet
  ↓
External DNS / Load Balancer
  ↓
OpenShift Ingress Controller / Router
  ↓
Route
  ↓
Service
  ↓
Ready Pods
```

Routes expose Services with externally reachable hostnames.

| Route vs Router

Never confuse these.

Route

```
Configuration
```

For example:

```
employee.apps.company.com
  ↓
employee-service
```

Router / Ingress Controller

actual software handling traffic

So:

Route

→ says WHERE traffic should go

Router

→ actually processes the traffic

Exactly like the Ingress/Ingress Controller distinction you already learned.

| oc expose

Suppose you already have:

```
employee-service
```

You can create a Route with:

```
oc expose service employee-service
```

Then:

```
oc get route
```

OpenShift provides an externally reachable hostname for that Service according to the cluster's routing configuration.

| Complete Spring Boot request flow in OpenShift

For your Java application:

```
Browser
↓
employee.apps.company.com
↓
External DNS
↓
Load Balancer
↓
OpenShift Router / Ingress Controller
↓
Route
↓
employee-service
↓
EndpointSlice
↓
employee Pod
```

```
↓  
Spring Boot :8080  
↓  
Oracle / external services
```

Almost everything after the Route is standard Kubernetes knowledge you already have.

| Security Context Constraints — SCC

Now one of the most important OpenShift-specific security features:

SCC — Security Context Constraints You've already learned RBAC:

WHO can perform WHAT API action?

SCC answers a different question:

What security characteristics is this Pod allowed to run with?

For example:

```
Can container run privileged?  
Can container run as root/arbitrary UID?  
Can it access host networking?  
Can it mount host directories?  
Which Linux capabilities are allowed?  
Which SELinux settings are permitted?
```

OpenShift SCCs control these Pod security characteristics during admission.

| RBAC vs SCC

This distinction is critical.

RBAC

```
User Hari:  
Can create Pods?  
↓  
YES / NO
```

SCC

```
That Pod:  
Can run privileged?  
Can run with this UID?  
Can mount hostPath?  
↓  
YES / NO
```

So:

RBAC

→ permission to request the operation

SCC

→ security constraints on the workload being admitted

| Practical example

Developer creates:

```
securityContext:  
  privileged: true
```

RBAC might say:

Developer can create Pods ' '

But SCC evaluation might say:

Privileged workload not allowed ×

Pod creation can be rejected.

This is one reason an application might work on a loosely configured Kubernetes environment but fail when moved into OpenShift.

| Why OpenShift apps often shouldn't assume a fixed UID

OpenShift traditionally emphasizes running applications without unnecessary root privileges and can assign workload identities according to its security constraints.

So an application image that assumes:

```
"I must run as root"
```

or:

```
"I must always run as UID 1000"
```

can encounter problems.

Good container images should avoid unnecessary assumptions about privileged execution.

We✓I study SCC much more deeply in its own chapter.

| Integrated authentication

OpenShift also provides a more complete platform authentication experience.

At the architecture level:

```
User
  ↓
Identity Provider
  ↓
OpenShift authentication
  ↓
User / Group
  ↓
RBAC
  ↓
Projects/resources
```

Organizations can integrate corporate identity systems and then use Kubernetes/OpenShift RBAC to decide what authenticated identities may access.

This makes OpenShift feel more like an enterprise application platform than a raw Kubernetes installation.

| Web Console

Plain Kubernetes primarily exposes APIs/CLI primitives.

OpenShift includes a comprehensive Web Console.

Conceptually:

```
Developer Console
-
  ├── Applications
  ├── Pods
  ├── Routes
  ├── Builds/deployments
  └── Logs/topology

Administrator Console
-
  ├── Nodes
  ├── Operators
  ├── Storage
  ├── Networking
  └── Cluster configuration
```

The web console and oc CLI are part of the management experience OpenShift adds around Kubernetes.

| Image capabilities

OpenShift also provides integrated container image capabilities.

Conceptually:

```
Source/application
  ↓
Container image
  ↓
Registry/image management
  ↓
Deployment
```

OpenShift can work with external registries and also provides platform image-registry functionality; image management is part of the broader developer/platform experience.

For your Java team, however, your company might still use something external such as:

```
Artifactory
Quay
enterprise registry
cloud registry
```

and OpenShift simply pulls images from there.

| Monitoring

Remember:

```
Prometheus
Grafana-style dashboards
alerts
cluster metrics
```

OpenShift provides integrated platform monitoring capabilities rather than requiring administrators to assemble every monitoring component manually from scratch. Red Hat's architecture documentation explicitly includes monitoring, cluster health/performance, networking, routing, logging integrations, and upgrade management among OpenShift platform capabilities.

Again, this is part of the theme:

OpenShift packages a Kubernetes operational platform.

| Operators + Monitoring + Upgrades

This is where OpenShift architecture becomes very elegant.

Instead of:

```
Administrator manually manages:
DNS
Ingress
API server
```

Monitoring
Image registry
Node config
...

OpenShift moves toward:



Each Operator continually reconciles its subsystem.

This is Kubernetes' reconciliation philosophy applied to the platform itself.

This is perhaps the best way to understand OpenShift

Kubernetes says:

"Controllers operate your applications."

OpenShift extends that philosophy:

"Operators operate the Kubernetes platform itself."

That is a powerful mental model.

Plain Kubernetes vs OpenShift

Area Kubernetes OpenShift

Core orchestration Kubernetes Kubernetes

Pod Pod Pod

Deployment Deployment Deployment

Namespace Namespace Project abstraction around Namespace

CLI kubectl oc + Kubernetes commands

External HTTP routing Ingress Ingress + OpenShift Route

Router management depends on installed controller Ingress Operator manages OpenShift ingress/router

Security RBAC + Pod security mechanisms RBAC + SCC and platform security defaults

OS management implementation dependent RHCOS + Machine Config Operator

Platform lifecycle assembled by cluster operator/admin Cluster Operators + CVO

Add-on lifecycle varies OLM / software catalog

UI implementation dependent integrated OpenShift console

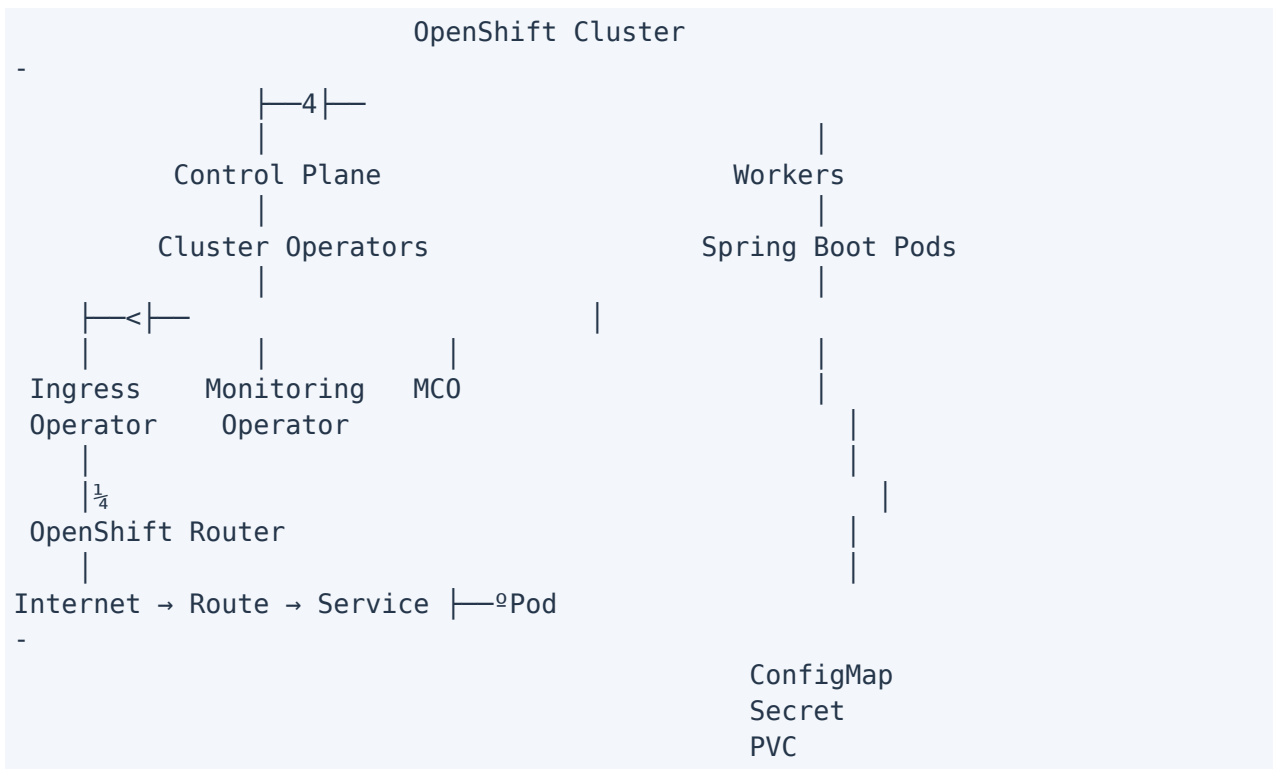
Monitoring must be assembled/integrated integrated platform monitoring

Enterprise support depends on distribution Red Hat product/support ecosystem

This is the distinction I want you to retain.

Real company architecture

Imagine your organization runs:



Security surrounds it:

```

graph TD
    Identity --> RBAC
    RBAC --> Project
    Project --> Pod_admission[Pod admission]
    Pod_admission --> SCC

```

The diagram shows the security layers surrounding the OpenShift cluster. It starts with Identity, which leads to RBAC (Role-Based Access Control). RBAC then leads to Project-level security. Project-level security leads to Pod admission, which finally leads to SCC (Security Context Constraints).

That is a very realistic mental model.

| Developer perspective

As a Java developer, your day may look surprisingly similar to Kubernetes:

```
oc project employee-dev

oc get pods

oc get deploy

oc get svc

oc get route

oc logs employee-service-xyz

oc describe pod employee-service-xyz
```

You're still working with:

```
Deployment
Service
ConfigMap
Secret
PVC
Pod
```

Most of your Kubernetes knowledge transfers unchanged.

| Administrator perspective

The OpenShift admin deals more with:

```
Cluster Operators
MachineConfigPools
SCC
authentication
IngressControllers
storage classes
cluster networking

    cluster updates
    certificates
    Nodes
    Operators
```

Commands might start with:

```
oc get clusteroperators
oc get nodes
oc get mcp
oc get ingresscontroller -n openshift-ingress-operator
oc get scc
```

So developer and platform-admin perspectives are different layers of the same platform.

Troubleshooting OpenShift

Suppose every application Route suddenly fails.

Bad debugging approach:

```
Check 200 Spring Boot Pods individually
```

Better:

```
All applications affected
      ↓
Large blast radius
      ↓
Check shared infrastructure
      ↓
Ingress Operator / Router
```

For example:

```
oc get clusteroperators
```

If:

```
ingress:
AVAILABLE=False
DEGRADED=True
```

that's a major clue.

Your chapter 36 blast-radius reasoning applies directly.

Another example

Only:

```
employee-service
```

fails.

But:

```
payments
payroll
gateway
```

work normally.

Then the OpenShift platform itself is less likely to be the root cause.

Investigate:

```
employee Deployment
Route
Service
Pods
Config
Secret
```

Same layered troubleshooting mindset.

Why enterprises use OpenShift

The value isn't simply:

"It can run containers."

Kubernetes already does that.

The attraction is the integrated operational model:

```
Kubernetes
+
secure defaults/controls
+
node OS lifecycle
+
cluster upgrades
+
Operators
+
routing
+
authentication
+
monitoring
+
developer tooling
+
support/certification
```

OpenShift essentially gives an organization a more opinionated, integrated Kubernetes platform instead of requiring the platform team to select, integrate, operate, and support every layer independently.

Don't make one common mistake

Avoid thinking:

OpenShift Route replaces Kubernetes Service

It doesn't.

The path is still:

```
Route
  ↓
Service
  ↓
Pods
```

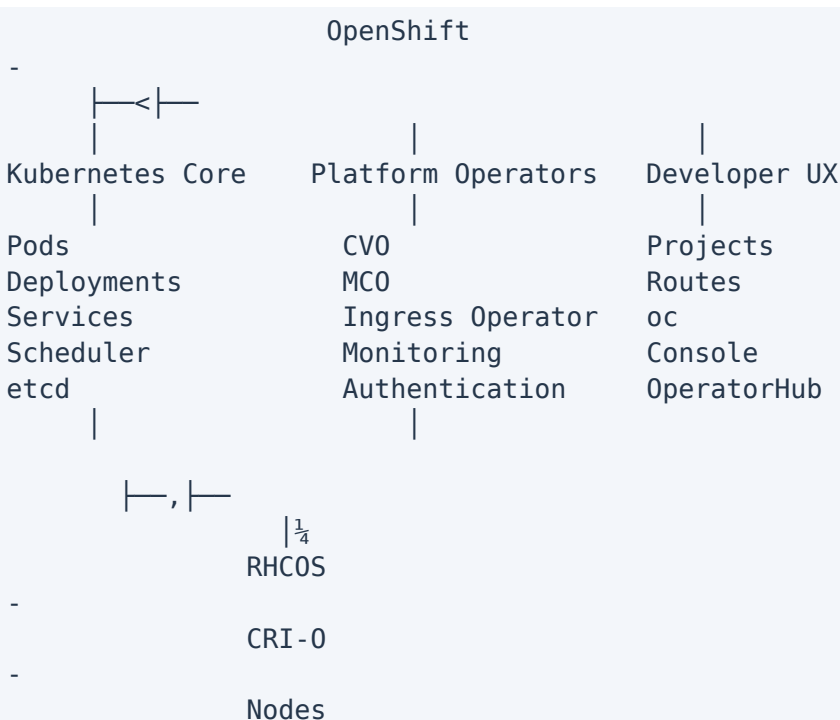
Likewise:

```
Project doesn't replace Pods
SCC doesn't replace RBAC
Operator doesn't replace Deployment
OpenShift doesn't replace Kubernetes
```

Each operates at a different layer.

The OpenShift mental model to keep

If you remember only this diagram:



you understand OpenShift architecture at a strong conceptual level.

The most important translations are:

```
Kubernetes Namespace
  ↓
OpenShift Project
```

```
kubectl
  ↓
oc

Kubernetes Ingress concept
  ↓
OpenShift Ingress Controller + Route option

RBAC
→ Who can do what?

SCC
→ What security privileges can the Pod run with?

Linux Node management
  ↓
RHCOS + Machine Config Operator

Platform subsystem management
  ↓
Cluster Operators
  ↓
Cluster Version Operator
```

And the single most important OpenShift idea:

OpenShift takes Kubernetes' reconciliation model and applies it not only to your applications, but also extensively to the platform itself through Operators.

Next chapter — chapter 39: OpenShift Operators, CRDs & OLM This is the natural next topic because Operators are central to the architecture we just studied.

We✓I answer:

```
What exactly is an Operator?
What is a CRD?
What is a Custom Resource?
How can Kubernetes understand something like KafkaCluster?
What does the Operator watch?
How does its reconciliation loop work?
What is OperatorHub?
What is OLM?
What is a Subscription?
What is a CSV?
Cluster Operator vs application Operator?
```

We✓I use an example such as:

```
Kafka custom resource
  ↓
Kafka Operator
  ↓
```

StatefulSets
Services
PVCs
ConfigMaps

so you can see how an Operator turns a high-level API into ordinary Kubernetes resources.

You're now well past the Kubernetes fundamentals and into the OpenShift/platform engineering part of the roadmap.

Here's the clean status as of now.

Modul Topic Status e

1-20 Kubernetes fundamentals through resources/QoS ' Covered

21 HPA / autoscaling #ó Skipped for later

22 PodDisruptionBudget / HA basics Ø=ßá Partly covered later in chapter 37

23 Graceful shutdown / lifecycle hooks #ó Mostly pending

24 Advanced scheduling / topology Ø=ßá Basics covered

25 NetworkPolicy in production Ø=ßá Basics covered

Modul Topic Status e

26 SecurityContext / Pod Security #ó Pending

27 OpenShift SCC in depth Ø=ßá Introduced, deep dive pending

28 ServiceAccounts/workload identity in depth Ø=ßá Basics covered

29 OpenShift Projects / Routes / Builds / ImageStreams Ø=ßá Projects & Routes covered; Builds/ImageStreams pending

30 Operators introduction Ø=ßá Concept introduced; dedicated chapter next

31 Helm ' Covered

32 Kustomize ' Covered

33 GitOps / Argo CD ' Covered

34 Prometheus / Grafana / monitoring ' Covered

35 Centralized logging / tracing / OpenTelemetry ' Covered

36 Advanced production troubleshooting ' Covered

37 HA / upgrades / DR / etcd quorum / cordon-drain / ' Covered PDB

38 OpenShift architecture in depth ' Covered

39 Operators, CRDs & OLM %¶p Next

So your current position is:

```
Kubernetes Foundations '
  ↓
Kubernetes Production Operations '
  ↓
Helm / Kustomize / GitOps '
  ↓
Observability & Troubleshooting '
  ↓
HA / DR '
  ↓
OpenShift Architecture '
  ↓
YOU ARE HERE
  ↓
Operators / CRDs / OLM
```

The biggest topics you intentionally left for later are HPA/autoscaling, graceful shut-down, SecurityContext/Pod Security, deeper SCC, and a few OpenShift developer-platform features such as Builds/ImageStreams.

The next chapter should therefore be:

Chapter 26 — Projects, Routes, Builds, and ImageStreams

CHAPTER FOCUS

Understand **Projects, Routes, Builds, and ImageStreams** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

You already know most Kubernetes runtime concepts. This chapter explains the OpenShift-specific developer platform around them.

The end-to-end picture is:

```
Source Code
  ↓
OpenShift Build
  ↓
Container Image
  ↓
ImageStream
  ↓
Deployment
  ↓
Pods
  ↓
Service
  ↓
Route
  ↓
External User
```

OpenShift Project

A Project is essentially OpenShift's user-facing abstraction around a Kubernetes Namespace, with OpenShift management, permissions, quotas, and developer workflows layered around it. Red Hat describes Projects as a way for groups of users to organize and manage their resources separately from other groups.

For example:

```
OpenShift Cluster
├─ employee-dev
│  └─ Deployment
```

```
|   |— Service
|   |— ConfigMap
|   |— ImageStream
|   |— Route
-
|— employee-qa
-
|— employee-prod
```

You switch projects using:

```
oc project employee-dev
```

or create one using:

```
oc new-project employee-dev
```

For your mental model:

```
Kubernetes Namespace
      "H
OpenShift Project
```

| Route recap

A normal Service such as:

```
employee-service
ClusterIP
```

is primarily reachable inside the cluster.

To expose the application externally, OpenShift commonly uses a:

Route

```
Internet
  ↓
OpenShift Router / Ingress Controller
  ↓
Route
  ↓
Service
  ↓
Ready Pods
```

A Route is an OpenShift resource that exposes a Service through an HTTP/HTTPS-aware routing layer and can include TLS configuration.

Example:

```
apiVersion: route.openshift.io/v1
kind: Route

metadata:
  name: employee-service

spec:
  host: employee.apps.company.com

  to:
    kind: Service
    name: employee-service

  port:
    targetPort: 8080
```

You can also create one quickly:

```
oc expose service employee-service
```

Then:

```
oc get route
```

Now the missing part: OpenShift Builds

Normally your Java delivery flow might be:

```
Java source
  ↓
Maven
  ↓
JAR
  ↓
Dockerfile
  ↓
docker/buildah build
  ↓
Registry
  ↓
OpenShift
```

But OpenShift can also perform container builds inside the platform.

That is what OpenShift Builds are for.

OpenShift's current BuildConfig mechanism provides declarative build definitions with source, strategy, output and triggers. Dockerfile and Source-to-Image builds produce runnable container images.

| BuildConfig vs Build

This distinction is similar to several Kubernetes concepts you've already learned.

BuildConfig Defines:

How should builds be performed?

Think:

```
BuildConfig
=
build template / desired build configuration
```

Build Represents:

One actual execution of that BuildConfig.

Think:

```
BuildConfig
  ↓
Build #1
Build #2
Build #3
```

Java analogy:

```
BuildConfig
"H class

Build
"H object/execution
```

| Example BuildConfig

Simplified:

```
apiVersion: build.openshift.io/v1
kind: BuildConfig

metadata:
  name: employee-service

spec:
  source:
    git:
      uri: https://git.example.com/employee-service.git

  strategy:
    dockerStrategy: {}

  output:
    to:
```

```
kind: ImageStreamTag
name: employee-service:latest
```

Conceptually:

```
Git repository
  ↓
BuildConfig
  ↓
Build
  ↓
Dockerfile/build process
  ↓
Container image
  ↓
employee-service:latest
```

Build strategies

Current OpenShift BuildConfig documentation describes three primary build strategies:

```
Dockerfile build
Source-to-Image (S2I)
Custom build
```

Dockerfile and S2I builds are supported normally; custom builds are much more privileged and therefore security-sensitive.

For your Java work, the two important ones are:

```
Dockerfile/Buildah build

or

S2I build
```

Dockerfile build

Suppose your Git repository contains:

```
employee-service/
├─ pom.xml
├─ src/
└─ Dockerfile
```

Dockerfile:

```
DOCKERFILE
```

```
FROM eclipse-temurin:21-jre
```

```
COPY target/employee-service.jar /app/app.jar
```

```
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

OpenShift can take that build context and use Buildah to build the container image.

Conceptually:

```
Git
↓
Dockerfile
↓
OpenShift Build
↓
Buildah
↓
Image
```

| Source-to-Image — S2I

S2I is more OpenShift-specific and worth understanding.

Instead of requiring you to completely define a Dockerfile:

```
Source code
+
Builder image
=
Runnable application image
```

Example:

```
Java source
+
Java/Maven builder image
↓
S2I
↓
Runnable application image
```

S2I uses builder-provided scripts such as:

```
assemble
run
save-artifacts
```

to build and run the application.

| S2I mental model

Imagine builder image:

Java Builder Image

```
-  
├─ JDK  
├─ Maven  
├─ build tooling  
└─ S2I scripts
```

Then:

Your source code

```
↓  
Builder image  
↓  
assemble  
↓  
mvn package  
↓  
generated image  
↓  
run  
↓  
java -jar application
```

So developers can focus more on source code and less on container-build details.

I Build input

A build can consume sources such as:

```
Git source  
Dockerfile content  
other images/files  
binary input
```

For example:

```
source:  
  git:  
    uri: https://git.example.com/employee-service.git  
    ref: main
```

OpenShift clones the repository into the build context.

I Build output

After building:

```
Build  
↓  
new container image
```

Where should it go?

BuildConfig specifies:

```
output:
  to:
    kind: ImageStreamTag
    name: employee-service:latest
```

When output is an ImageStreamTag , OpenShift can push the resulting image into the integrated registry and associate it with that ImageStream tag. A build can also output directly to an external Docker-compatible registry reference.

12. Now: what is an ImageStream?

This is the most important missing concept.

An ImageStream is not the image itself.

Think:

```
Container Registry
=
stores actual image blobs/layers

ImageStream
=
OpenShift object that tracks/references images and tags
```

Red Hat defines an ImageStream as an object that maintains mappings between tags and images, along with image metadata/history and optional registry references.

| Registry vs ImageStream

Suppose registry contains:

```
registry.company.com/employee-service:1.0
registry.company.com/employee-service:1.1
registry.company.com/employee-service:2.0
```

OpenShift might have:

```
ImageStream:
employee-service
```

with:

```
employee-service:dev
employee-service:qa
employee-service:prod
employee-service:latest
```

ImageStream tags can point to particular images.

So:

```
Registry  
→ physically stores images
```

```
ImageStream  
→ logical OpenShift image tracking/tag abstraction
```

| ImageStreamTag

Suppose:

```
ImageStream:  
employee-service
```

You can have:

```
employee-service:latest  
employee-service:2.7  
employee-service:prod
```

Each is an:

ImageStreamTag Red Hat describes ImageStreamTag as the human-readable tag reference for an image inside an ImageStream.

Syntax:

```
employee-service:latest
```

| ImageStreamImage

There's another reference:

```
employee-service@sha256:abc123...
```

That is conceptually:

ImageStreamImage It references an immutable image digest rather than a movable tag.

So:

```
ImageStreamTag  
employee-service:latest  
→ human-readable/movable
```

while:

```
ImageStreamImage  
employee-service@sha256:...  
→ immutable image identity
```

Red Hat distinguishes `ImageStreamTag` , `ImageStreamImage` , and direct external `DockerImage` references this way.

16. Why not just use registry tags directly?

Good question.

You absolutely can use:

```
registry.company.com/employee-service:2.7
```

directly.

`ImageStreams` add OpenShift-native capabilities around images:

```
image metadata/history
tag management
importing external images
build integration
image-change triggers
OpenShift RBAC around image streams
```

So they act as a platform abstraction around container images.

| Image import

You can reference an external image:

```
registry.redhat.io/ubi9/ubi:latest
```

and map it into an `ImageStream`.

Conceptually:

```
External Registry
  ↓
registry.redhat.io/ubi9/ubi
  ↓
OpenShift ImageStream
ubi9
  ↓
ubi9:latest
```

This allows builds and workloads to use an OpenShift-native image reference.

| ImageStream history

Imagine:

```
employee-service:latest
```

first points to:

```
sha256:AAA
```

later:

```
sha256:BBB
```

later:

```
sha256:CCC
```

ImageStream status can retain tag history until image pruning policies remove old references.

This can help you see how a tag changed over time.

Build triggers

Now this gets interesting.

A BuildConfig can have triggers.

For example:

```
Git/webhook event
```

↓

```
Build starts
```

or:

```
Builder/base image changes
```

↓

```
ImageStream detects change
```

↓

```
Build starts again
```

OpenShift supports image-change triggers in BuildConfigs, including triggers linked to an ImageStreamTag .

Example: base image changes

Suppose your S2I application uses:

```
java-builder:21
```

Your BuildConfig watches that ImageStreamTag.

New secure Java builder released:

```
java-builder:21
```

```
old digest AAA
```

↓

```
new digest BBB
```

ImageStream detects the tag update.

Then:

```

Image change
  ↓
BuildConfig trigger
  ↓
new employee-service Build
  ↓
new application image

```

This is useful for rebuilding applications when base images change.

Build pipeline for Spring Boot

Let's connect everything:

```

Git Repository
-
  | 1/4
BuildConfig
-
  | 1/4
Build Pod
-
  |— Maven / S2I / Dockerfile
-
  | 1/4
Container Image
-
  | 1/4
ImageStreamTag
employee-service:latest
-
  | 1/4
Deployment workflow
-
  | 1/4
Pods

```

Then:

```

Pods
  ↓
Service
  ↓
Route
  ↓
User

```

That is the full chapter 29 picture.

| OpenShift Build vs Jenkins/GitHub Actions/Tekton

An important modern distinction:

OpenShift BuildConfig is one way to build container images.

Your company could instead use:

```
Jenkins
GitHub Actions
GitLab CI
Tekton / OpenShift Pipelines
Azure DevOps
```

to do:

```
mvn package
↓
build image
↓
push registry
```

OpenShift's current CI/CD docs also position OpenShift Pipelines/Tekton as the Kubernetes-native framework for richer CI/CD workflows, while BuildConfig remains a declarative in-cluster image-build mechanism.

So:

```
BuildConfig
" `
mandatory for using OpenShift
```

Very important.

| Modern enterprise flow

Your company may use:

```
Java developer
↓
Git
↓
Jenkins / Tekton
↓
mvn package
↓
container build
↓
Quay / Artifactory / registry
↓
GitOps repo update
↓
Argo CD
```

↓
OpenShift

In this architecture:

BuildConfig

may not be used at all.

That is perfectly valid.

24. Why learn BuildConfig then?

Because OpenShift supports it as a native build primitive and you✓I encounter:

Build
BuildConfig
ImageStream
ImageStreamTag
S2I

in many existing enterprise clusters.

Understanding it helps you troubleshoot and understand OpenShift-native application workflows.

| Security note about Builds

OpenShift builds execute build processes in containers, and Red Hat warns that build strategies have security implications; custom builds are particularly privileged and are disabled by default unless explicitly authorized.

That connects directly to:

chapter 26 → container privileges
chapter 27 → SCC

Build systems must be treated as trusted code-execution infrastructure.

| chapter 29 mental model

Remember:

Project
→ Where my application resources live

BuildConfig
→ How an image should be built

Build
→ One execution of that build

ImageStream
→ OpenShift image/tag tracking abstraction

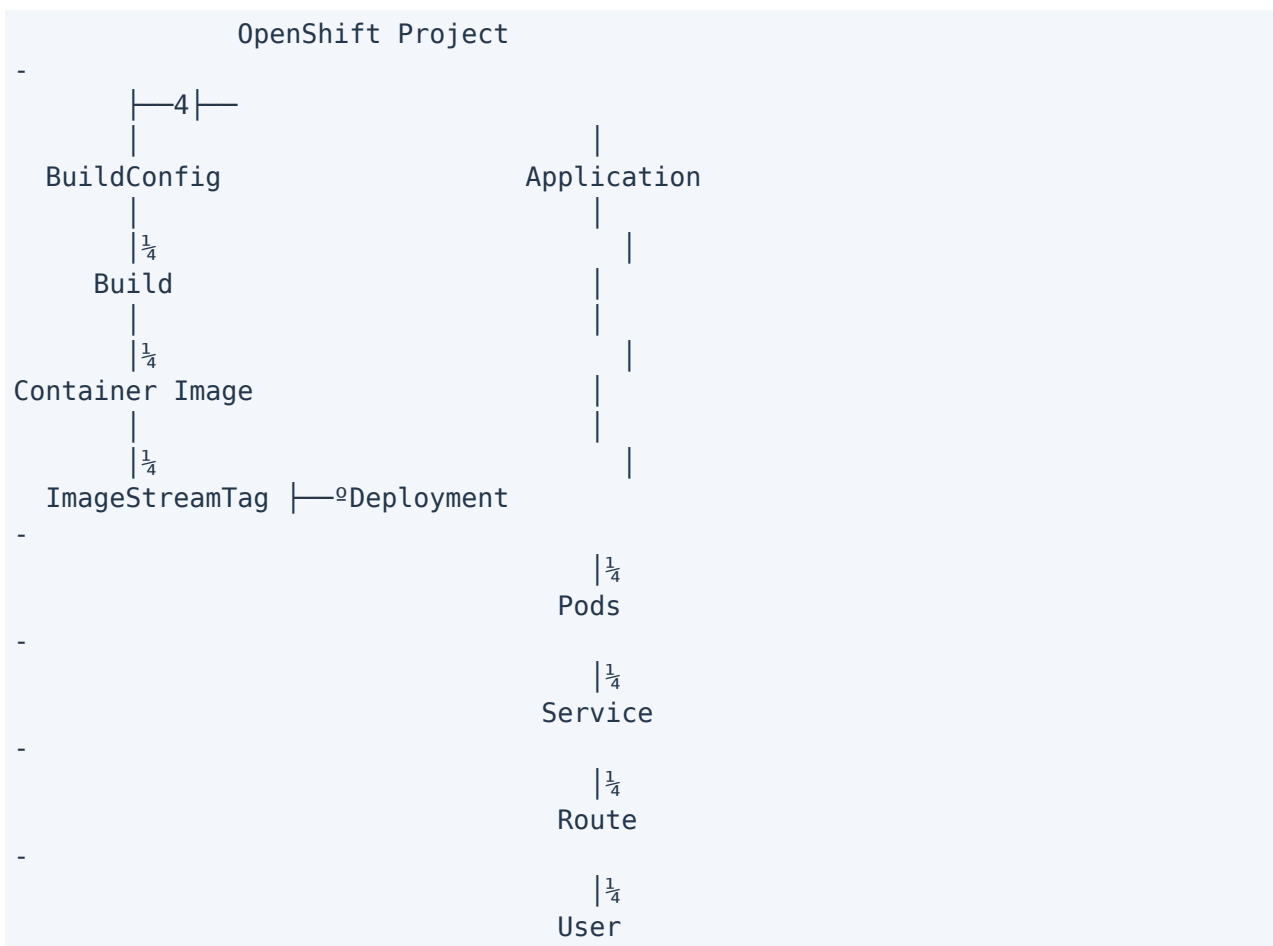
ImageStreamTag
→ named version/tag within an ImageStream

Deployment
→ runs that image

Service
→ stable internal access

Route
→ external HTTP/HTTPS access

Complete flow:



chapter 29 ' Complete

Chapter 27 — OpenShift Security Context Constraints

CHAPTER FOCUS

Understand **OpenShift Security Context Constraints** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

Now that you understand SecurityContext , SCC becomes much easier.

The key idea is:

```
Pod securityContext
=
"What security settings does this workload request?"

SCC
=
"Is this workload allowed to run with those settings?"
```

In OpenShift, Security Context Constraints (SCCs) are admission controls that define which security characteristics a Pod may use, including UID strategy, privileged mode, capabilities, host namespaces, SELinux, volume types, seccomp, and privilege escalation.

Where SCC sits in the Pod creation flow

Suppose your Deployment creates a Pod.



Node

So SCC acts before the container starts.

That means if SCC rejects the Pod, debugging Spring Boot logs is pointless because:
Java never started.

2. Why does OpenShift need SCC?

Imagine a developer submits:

```
securityContext:
  privileged: true
  runAsUser: 0
```

The developer is effectively requesting:

```
Run this workload:
- as root
- with privileged access
```

OpenShift should not blindly trust every workload.

SCC can say:

root?	×
privileged?	×
hostPath?	×
hostPID?	×

and reject it.

That gives platform administrators a centralized security boundary.

| SCC is not the same as RBAC

This is one of the most important distinctions.

Suppose you have permission to:

```
create Pods
```

RBAC says:

```
Can Hari create a Pod?
→ Yes
```

But you request:

```
privileged: true
```

SCC says:

Can this Pod run privileged?
→ No

So:

RBAC
→ WHO may submit the request?

SCC
→ HOW may the Pod run?

Both must succeed.

| SCC is also different from Pod Security Admission

Modern OpenShift has both:

SCC
+
Kubernetes Pod Security Admission

They are independent mechanisms, and workloads may need to comply with both. OpenShift's SCC controller can also default/mutate certain security settings before validation.

Mental model:

```
Pod
↓
SCC processing
↓
Pod Security Admission
↓
Accepted / Rejected
```

Don't treat:

SCC = PSS

They're related security layers, but not the same implementation.

| Default SCCs you should know

OpenShift includes several default SCCs. Three names are especially useful to understand:

SCC Meaning

restricted-v2 Normal restrictive default for ordinary workloads

anyuid Similar restrictions but allows arbitrary UID/GID

privileged Very broad access; highly trusted workloads only

On new OpenShift installations, restricted-v2 is the default restrictive SCC generally available to authenticated users; it drops all capabilities by default, uses runtime/default seccomp, and disallows privilege escalation.

| restricted-v2

This is the one you should think of for normal Spring Boot applications.

Conceptually:

```
restricted-v2
-
|— no privileged container
|— restricted UID strategy
|— no host filesystem access
|— no host namespaces
|— drop ALL capabilities
|— allowPrivilegeEscalation = false
|— seccomp = RuntimeDefault
```

This is why an ordinary OpenShift workload often runs very differently from a Docker container you started locally with:

```
docker run ...
```

Your local environment may allow much more.

| The UID behavior is especially important

This is one of the most OpenShift-specific concepts.

Suppose your Pod does not specify:

```
runAsUser:
```

With a restrictive SCC, OpenShift can assign a UID from the UID range allocated to that Project/Namespaces.

Conceptually:

```
employee-dev
UID range:
1000800000–1000809999
```

Your Pod might run as:

```
UID 1000800000
```

Another Project might receive:

```
1000900000–1000909999
```

Therefore:

You should not assume the runtime UID will always be 1000 , 10001 , etc.

Red Hat documents that restricted-v2 uses a MustRunAsRange strategy and that OpenShift derives the allowed range from Project annotations, so the effective UID can differ between Projects.

Why hardcoded UIDs can cause trouble

Imagine your Dockerfile:

```
DOCKERFILE
```

```
USER 1000
```

and OpenShift Project allows:

```
1000800000–1000809999
```

Then requesting:

```
UID 1000
```

may not satisfy the SCC.

Possible result:

```
Pod rejected
```

unless the workload has access to an SCC that permits that UID.

This is one reason OpenShift-friendly container images are designed to tolerate arbitrary non-root UIDs.

Container-image design for arbitrary UID

Suppose your application directory is:

```
/app
```

and owned only by:

```
UID 1000
```

with:

```
chmod 700
```

Then OpenShift runs the container as:

```
UID 1000800000
```

Result:

```
Permission denied
```

So an OpenShift-compatible image should avoid requiring one exact UID.

A common design principle is:

```
files readable/executable by arbitrary non-root UID
writable directories deliberately group-writable
runtime data stored in mounted writable locations
```

rather than:

```
"Everything only works as UID 1000."
```

| **anyuid**

Suppose a legacy application absolutely requires:

```
UID 1000
```

or even:

```
UID 0
```

A more permissive SCC such as:

```
anyuid
```

allows workloads to run with arbitrary UID/GID values while retaining many of the other restrictions of a restricted SCC.

Conceptually:

```
restricted-v2
→ OpenShift-controlled UID range

anyuid
→ workload may select its UID
```

But:

Don't grant anyuid just because an image is badly designed.

First ask whether the image can be fixed.

| **privileged**

At the opposite extreme:

```
privileged
```

permits broad host/container privileges.

This may be required by certain trusted platform workloads, storage/networking agents, or administrative components.

It can allow things such as:

```
privileged containers  
host namespaces  
host filesystem mounts  
arbitrary users/groups  
broad capabilities
```

Red Hat explicitly warns that privileged is the most relaxed default SCC and should be granted with great caution.

For a normal Spring Boot API:

```
privileged SCC
```

should immediately make you ask:

Why does this application need that?

| SCC does more than allow/deny

This is a subtle but very important point.

SCC can:

```
validate
```

and in some cases:

```
default/generate security values
```

For example, your Pod may omit:

```
runAsUser:
```

and SCC strategy determines an allowed UID.

Or you omit:

```
seccompProfile:
```

and a restrictive SCC may default it appropriately.

OpenShift's SCC settings use boolean controls, allowable sets, and strategy-based fields that can generate/default permitted values.

So SCC isn't only:

```
"yes/no firewall"
```

It can also help construct a safe runtime security context.

| SCC strategies

This is the conceptual model.

For some fields SCC may use strategies such as:

```
MustRunAs  
MustRunAsRange  
RunAsAny
```

Example:

```
runAsUser:  
MustRunAsRange
```

means:

The container must run using a UID inside the permitted range.

Whereas:

```
RunAsAny
```

means:

SCC doesn't constrain it to a specific range.

You don't need to memorize every strategy yet. Understand that OpenShift evaluates values against a policy strategy.

| ServiceAccount connection

This is extremely important.

When a Deployment creates Pods, those Pods typically run as a:

```
ServiceAccount
```

For example:

```
spec:  
  serviceAccountName: employee-sa
```

SCC authorization can be granted to that ServiceAccount.

Conceptually:

```
employee Pod
  ↓
employee-sa
  ↓
Which SCCs may this SA use?
  ↓
restricted-v2
```

or perhaps:

```
employee-sa
  ↓
restricted-v2 + custom SCC
```

Then OpenShift finds an SCC that both:

the ServiceAccount is allowed to use

and:

can validate/default the Pod

This is why SCC and ServiceAccounts are tightly connected.

15. Why not give SCC directly to every developer?

Suppose a special database application needs anyuid .

Bad idea:

give anyuid to all developers

because then many workloads could potentially use it.

Better:

```
specific ServiceAccount
  ↓
specific namespace
  ↓
specific SCC permission
```

Example conceptually:

```
legacy-db-sa
  ↓
may use anyuid
```

while:

```
default SA
employee-sa
payment-sa
```


continue using restrictive policies.

Least privilege again.

| Granting an SCC

You may encounter:

```
oc adm policy add-scc-to-user anyuid -z legacy-sa -n legacy-project
```

Conceptually:

```
legacy-sa
  ↓
gets permission to use
  ↓
anyuid SCC
```

This does not mean every Pod using that ServiceAccount automatically runs under anyuid .

It means:

anyuid becomes one of the SCCs OpenShift is allowed to consider for that ServiceAccount.

SCC selection still depends on which constraint can admit the Pod and SCC prioritization/evaluation.

17. Which SCC actually admitted my Pod?

Very useful troubleshooting question.

You can inspect the running Pod:

```
oc get pod <pod> -o yaml
```

and look for its SCC annotation.

Typically you✓I see something like:

```
openshift.io/scc: restricted-v2
```

Meaning:

```
This Pod was admitted using restricted-v2
```

That is extremely useful when troubleshooting security differences between Pods.

| Example: normal Spring Boot service

Suppose your Deployment says:

```
spec:
  template:
    spec:
      serviceAccountName: employee-sa

      containers:
      - name: employee
        image: employee-service:2.7
```

No explicit UID.

Flow:

```
Deployment
  ↓
Pod
  ↓
employee-sa
  ↓
restricted-v2 available
  ↓
SCC chooses allowed UID
  ↓
seccomp/runtime settings applied
  ↓
Pod admitted
```

Container might run:

```
UID 1000810000
```

Your application should simply work.

Example: application hardcodes root

Now container image expects:

```
root
```

Pod requests:

```
securityContext:
  runAsUser: 0
```

With only restricted-v2 :

```
UID 0 outside permitted range
  ↓
SCC validation fails
  ↓
Pod rejected
```

Developer says:

“But it works in Docker!”

That doesn’t matter.

The OpenShift security policy intentionally rejects it.

Correct engineering question:

Why does the application need root?

Example: filesystem permission issue

Suppose the Pod is admitted.

OpenShift assigned:

```
UID 1000810000
```

Then Java starts and logs:

```
Permission denied:  
/app/logs/application.log
```

SCC worked correctly.

The problem is your container filesystem.

Likely:

```
/app/logs owned by UID 1000  
and not writable by assigned UID
```

Possible better design:

```
log to stdout
```

or mount a properly permissioned writable volume.

Remember chapter 35:

```
container logs  
→ stdout/stderr
```

So SCC often exposes container-design problems.

Example: privileged infrastructure component

Suppose a networking agent requires:

```
hostNetwork: true
```

and special capabilities.

restricted-v2 will reject it.

But that doesn't necessarily mean the Pod is malicious.

A trusted platform component might legitimately require a more permissive SCC.

The principle is:

```
normal application
→ restrictive SCC

special infrastructure workload
→ narrowly approved elevated SCC
```

Not:

```
everything → privileged
```

22. Create custom SCC or modify built-in SCC?

Important OpenShift operational rule:

Do not modify the default SCCs.

Red Hat recommends creating custom SCCs instead; built-in SCCs can be reset during upgrades, and modifying them can create platform problems.

So if your organization needs something between:

```
restricted-v2
```

and:

```
anyuid
```

you may define:

```
company-special-scc
```

with exactly the required permissions.

Example custom security need

Suppose application needs:

```
specific host port
```

but does not need:

```
privileged
hostPID
hostPath
root
```

Bad solution:

```
grant privileged SCC
```

Better:

```
custom SCC
  ↓
allow only needed host port/security feature
  ↓
keep other restrictions
```

Least privilege at the policy level.

| SCC and capabilities

Suppose application requests:

```
capabilities:
  add:
    - NET_ADMIN
```

restricted-v2 won't normally allow arbitrary capabilities.

It drops all capabilities and only allows a narrow explicit addition such as NET_BIND_SERVICE.

So:

```
requested capability
  ↓
SCC allowable capabilities
  ↓
match?
```

If not:

```
Pod rejected
```

Again, securityContext is only the request.

| SCC and allowPrivilegeEscalation

With restricted-v2 :

```
allowPrivilegeEscalation
```

must be unset or false.

So:

```
securityContext:  
  allowPrivilegeEscalation: true
```

can cause admission failure.

For normal Java microservices:

```
false
```

is the sensible default anyway.

| SCC and seccomp

For restricted-v2 , OpenShift defaults to:

```
runtime/default
```

seccomp behavior for newly created Pods.

This connects directly to chapter 26:

```
Application  
↓  
system calls  
↓  
seccomp profile  
↓  
kernel
```

Again SCC is enforcing/setting runtime hardening.

| SCC and SELinux

SCC can also define permitted SELinux strategies/contexts.

You don't normally need to manually understand every MCS label for everyday Java development.

The important idea:

```
Pod  
↓  
OpenShift assigns/enforces SELinux context  
↓  
Node filesystem/resources
```

This helps isolate workloads even when they share the same Node.

So OpenShift security isn't just:

```
UID
```

It's defense in depth:

```
UID/GID
+
capabilities
+
seccomp
+
SELinux
+
namespaces
+
SCC
```

A useful troubleshooting sequence

Suppose Deployment never gets a Pod.

Check:

```
oc get deploy
oc get rs
oc describe rs <replicaset>
```

Why ReplicaSet?

Because:

```
Deployment
  ↓
ReplicaSet tries creating Pod
  ↓
API rejects Pod because SCC
```

If the Pod was never created, you may see the failure on the ReplicaSet events rather than having a Pod to inspect.

Typical clue:

```
unable to validate against any security context constraint
```

Now think:

```
SCC/admission issue
```

not:

```
scheduler
Spring Boot
Service
```

| Another troubleshooting case

Pod exists and is Running, but:

```
Permission denied
```

Then SCC admission succeeded.

Check:

```
oc get pod <pod> -o yaml
```

Find:

```
openshift.io/scc  
runAsUser  
fsGroup  
securityContext
```

Then inspect:

```
file ownership  
volume permissions  
read-only filesystem  
image assumptions
```

This is now a runtime permission problem.

| SCC and Pod Security Admission together

Imagine SCC accepts the Pod.

Could Pod Security Admission still complain?

Yes.

OpenShift documentation explicitly notes these mechanisms operate independently and workloads should comply with both.

Think:

```
Passed SCC  
↓  
does not automatically mean  
↓  
Passed Pod Security Admission
```

and vice versa.

| The most important SCC mental model

Keep this:


```
Pod YAML
  ↓
securityContext requests
  ↓
ServiceAccount identity
  ↓
Which SCCs can this identity use?
  ↓
Can one of those SCCs admit/default the Pod?
  ↓
Yes
  ↓
Pod admitted
  ↓
container runs with resulting Linux security settings
```

That is SCC in one flow.

| Spring Boot/OpenShift recommendation

For a normal backend application, your goal should be:

```
No root requirement
No privileged requirement
No hostPath
No hostNetwork
No special capabilities
No fixed UID dependency
Logs to stdout
Writable paths explicit
Works under arbitrary non-root UID
```

Then:

```
restricted-v2
```

should be a natural fit.

That is a much healthier container than:

```
"Give me anyuid/privileged or I don't start."
```

| Interview-level distinction

What is an SCC? A strong answer:

An OpenShift Security Context Constraint is an admission policy that controls the security characteristics a Pod can use, such as UID strategies, privileged mode, capabilities, host namespaces, volume access, SELinux, seccomp, and privilege escalation.

SCC vs SecurityContext?

SecurityContext
→ workload asks/configures

SCC
→ OpenShift validates/defaults/permits

SCC vs RBAC?

RBAC
→ may this identity perform API action?

SCC
→ may this Pod run with these privileges?

restricted-v2 ?

The default restrictive SCC for ordinary newly created workloads on modern OpenShift installations; it enforces non-root-style UID allocation, drops capabilities, uses runtime-default seccomp, and disallows privilege escalation.

Your gap status is now:

```

21 – HPA / Autoscaling           '
23 – Graceful Shutdown           '
26 – SecurityContext / Pod Security '
27 – OpenShift SCC               '

29 – OpenShift Projects / Routes / Builds / ImageStreams      Ø=ßá NEXT

32 – Kustomize              ' already completed

```

For chapter 29, we won't repeat Projects and Routes because you've already learned them in chapter 38. We'll focus specifically on the parts still missing:

```

Image
↓
ImageStream
↓
ImageStreamTag
↓
BuildConfig
↓
Build
↓
new image
↓
Deployment

```

and explain what OpenShift Builds and ImageStreams actually add over simply using Docker/CI + an external registry.

Yes. We'll do chapter 29 first, then chapter 32 as a clean, consolidated Kustomize

lesson. For chapter 29 I'll focus on the OpenShift-specific parts you haven't fully covered yet: Projects, Routes, Builds, ImageStreams, BuildConfig, and how they connect to deployments.

Chapter 28 — Operators, CRDs, and OLM

Supplementary explanation. The source introduced this chapter but did not contain the same depth as the surrounding modules. This section connects the controller and reconciliation ideas already established in the book.

The problem Operators solve

A Deployment can keep a fixed number of database Pods running, but it does not understand database-specific operations such as creating a replica, electing a leader, performing a schema-aware backup, or safely upgrading a clustered database. Those actions require domain knowledge.

An **Operator** packages that operational knowledge into software. It normally combines:

- a **CustomResourceDefinition (CRD)**, which adds a new API type;
- a **Custom Resource (CR)**, which describes the desired instance;
- a controller, which watches that resource and reconciles the real system toward the desired state.

```
DatabaseCluster custom resource
      ↓
Operator controller
      ↓
StatefulSets + Services + PVCs
      ↓
Backup, restore, upgrade, failover
```

CRD versus Custom Resource

The CRD defines the schema and API endpoint. A Custom Resource is one object created from that definition. The relationship is similar to a Java class and one object created from that class, although the Kubernetes API and reconciliation behavior add important semantics.

Operator Lifecycle Manager

In OpenShift, Operator Lifecycle Manager (OLM) helps discover, install, update, and track compatible Operators. OperatorHub is the catalog-oriented user experi-

ence. OLM manages the Operator itself; the installed Operator manages its domain resources.

| Cluster Operators versus add-on Operators

OpenShift Cluster Operators manage core platform capabilities such as networking or authentication. Add-on Operators manage optional products or application platforms. Both use the Operator pattern, but their ownership and operational impact differ.

| Key takeaways

- CRDs extend the Kubernetes API with new resource types.
- Custom Resources express desired state using those types.
- Operators reconcile domain-specific systems, not merely Pods.
- OLM manages Operator installation and lifecycle.
- An Operator is useful when software requires repeatable operational knowledge.

| Common mistakes

- Treating a CRD as if it automatically implements behavior; the controller provides the behavior.
- Assuming every application needs an Operator.
- Upgrading an Operator without checking compatibility with its managed Custom Resources.

| Self-test

1. What is the difference between a CRD and a Custom Resource?
2. Why is a Deployment alone often insufficient for a database cluster?
3. What does an Operator reconcile?
4. What problem does OLM solve?

| Answers and explanations

1. A CRD defines a new API type; a Custom Resource is an instance of it.
2. A Deployment understands replica lifecycle but not database-specific backup, failover, membership, or upgrade rules.
3. It compares the desired state in Custom Resources with the managed system and performs domain-specific corrective actions.
4. OLM supports discovery, installation, upgrades, and dependency/compatibility management for Operators.

Observability and Production Troubleshooting

Chapter 28 — Monitoring with Prometheus and Grafana

CHAPTER FOCUS

Understand **Monitoring with Prometheus and Grafana** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

Now we're moving from:

"Is the application deployed?"

to:

"Is the application actually healthy, fast, and behaving correctly?"

A Pod can be:

Running
Ready

and still have:

HTTP 500 errors
High latency
Slow database queries
Too many requests
CPU saturation
Memory growth

So Kubernetes status alone is not enough.

Observability has 3 major pillars

You'll hear this constantly:

Observability
-
├─ Metrics
├─ Logs
└─ Traces

They answer different questions.

Metrics

What is happening numerically?

Examples:

```
CPU = 80|  
Memory = 900Mi  
Requests/sec = 1200  
Error rate = 5|  
Average latency = 450ms
```

Logs

What exactly happened?

Example:

```
ERROR PaymentService:  
Database connection timeout
```

Traces

Where did one request spend its time?

Example:

```
Client  
  ↓ 20ms  
Gateway  
  ↓ 40ms  
Employee Service  
  ↓ 700ms  
Database
```

Tracing tells us:

```
Database = bottleneck
```

Simple mental model

Remember:

```
Metrics  
→ HOW MUCH?
```

```
Logs  
→ WHAT HAPPENED?
```

```
Traces  
→ WHERE DID THE REQUEST GO?
```

For example:

```
Metric:  
5| requests are failing
```

Then:


```
Logs:
SQLException: connection timeout
```

Then:

```
Trace:
Database call took 4.2 seconds
```

Together, they give you a much stronger diagnosis.

| Why Kubernetes monitoring is needed

Imagine your Spring Boot application:

```
employee-service
replicas = 3
```

Kubernetes shows:

employee-1	Running	Ready
employee-2	Running	Ready
employee-3	Running	Ready

You might think:

Everything is fine.

But users report:

Employee search takes 5 seconds.

Kubernetes doesn't automatically know your business/API latency.

You need application metrics.

| Two broad types of metrics

This distinction is important.

Infrastructure metrics Examples:

```
Node CPU
Node memory

Pod CPU
Pod memory
Disk
Network
```

These tell you about the platform.

Application metrics Examples:

```

HTTP request rate
HTTP error rate
API latency
DB connection pool usage
JVM heap
GC pauses
Thread count
Business events

```

These tell you about the application.

So:

```

Infrastructure healthy
"\`
Application healthy

```

5. What is Prometheus?

Prometheus is a metrics collection, storage, and querying system widely used with Kubernetes.

Its basic model is:

```

Application / Kubernetes component
    ↓
  exposes metrics
    ↓
    Prometheus
    ↓
  stores time series

```

For example:

```
employee_service_http_requests_total      153920
```

or:

```
jvm_memory_used_bytes      734003200
```

Prometheus records these values over time.

Time-series data

Prometheus doesn't just store:

```
CPU = 80|
```

It stores something closer to:

```
10:00 → 30 |  
10:01 → 35 |  
10:02 → 50 |  
10:03 → 80 |  
10:04 → 95 |
```

That is called:

Time-series data Meaning:

A metric value observed over time.

Why time-series matters

Suppose CPU is currently:

```
35 |
```

That looks fine.

But graphing the last 6 hours shows:

```
09:00 → 20 |  
10:00 → 30 |  
11:00 → 60 |  
12:00 → 90 |  
13:00 → 95 |  
14:00 → 35 |
```

Now you know there was a significant spike.

Current state alone wouldn't tell you that.

8. Does Prometheus push or pull metrics?

Very important question.

Prometheus primarily follows a:

Pull model Meaning:

```
Prometheus  
  ↓  
HTTP request  
  ↓  
Application metrics endpoint
```

Prometheus periodically scrapes the endpoint.

For example:

```
GET /actuator/prometheus
```

from your Spring Boot application.

9. What is scraping?

Suppose:

```
employee-service
```

exposes:

```
/actuator/prometheus
```

Prometheus might call it every:

```
15 seconds
```

Conceptually:

```
12:00:00 → scrape  
12:00:15 → scrape  
12:00:30 → scrape  
12:00:45 → scrape
```

Each scrape collects the latest metric values.

This process is called:

Scraping

| Spring Boot connection

This is particularly relevant for you.

A Spring Boot application can use:

```
Spring Boot Actuator  
+  
Micrometer
```

to expose metrics.

Conceptually:

```
Spring Boot  
↓  
Micrometer  
↓  
Actuator  
↓  
/actuator/prometheus  
↓  
Prometheus
```

Micrometer acts as an instrumentation facade for metrics.

| Example application metrics

Your Spring Boot service might expose metrics related to:

```
JVM heap
GC
Threads
HTTP requests
Database connection pools
Tomcat
CPU
```

For example:

```
http_server_requests_seconds_count
```

could represent request counts.

And:

```
jvm_memory_used_bytes
```

represents JVM memory usage.

| Prometheus metric structure

Prometheus metrics usually have:

```
Metric name
+
Labels
+
Value
```

For example:

```
http_requests_total{
  method="GET",
  status="200",
  uri="/employees"
} 15230
```

Meaning:

```
Metric:
http_requests_total

method:
GET

status:
200
```

```
endpoint:  
/employees
```

```
value:  
15230
```

| Why labels are powerful

Suppose:

```
http_requests_total
```

has labels:

```
method  
status  
service  
namespace  
pod
```

Then you can ask:

```
How many 500 errors?
```

or:

```
How many requests to employee-service?
```

or:

```
Which Pod is getting the most traffic?
```

Labels let one metric be sliced in many dimensions.

| But too many labels can be dangerous

Imagine adding:

```
userId  
requestId  
email
```

as metric labels.

You could generate millions of unique time series.

This is called:

High cardinality For example:

```

userId=1
userId=2
userId=3
...
userId=10,000,000

```

Prometheus has to store enormous numbers of distinct series.

So labels should usually contain bounded values like:

```

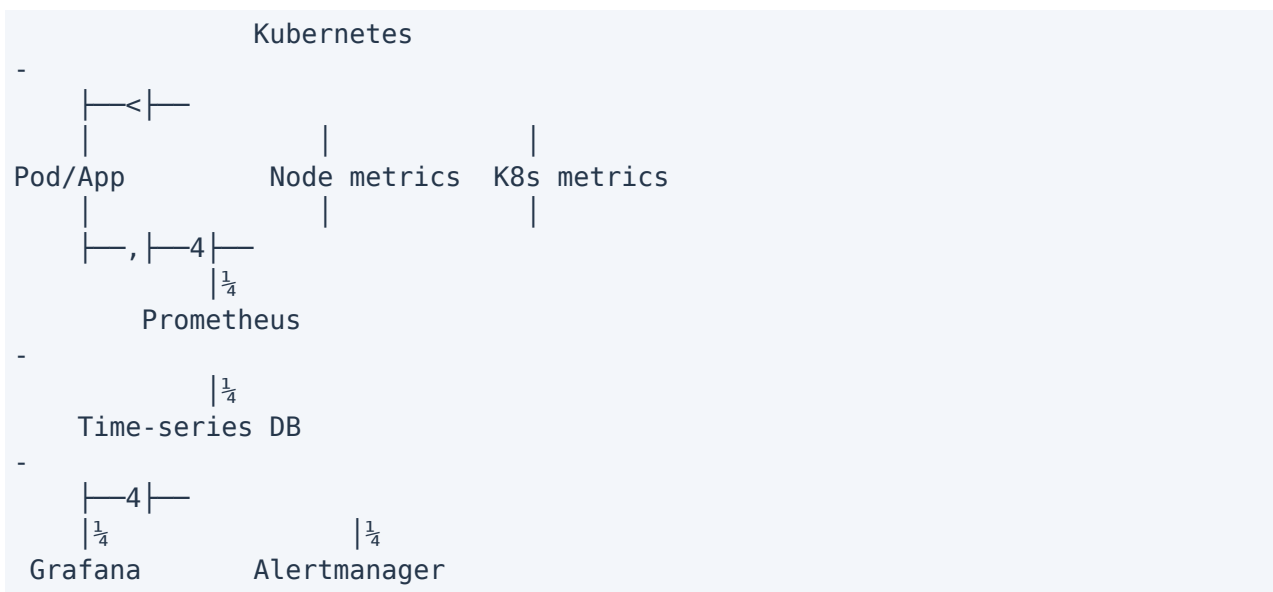
status
method
service
region

```

not unique identifiers.

Prometheus architecture

Simplified:



We✓I understand Grafana and Alertmanager shortly.

16. What is Grafana?

Prometheus stores and queries metrics.

Grafana is commonly used to visualize metrics.

Think:

```

Prometheus
=
metrics/database/query engine

```

Grafana

```
=
dashboards/visualization
```

For example Grafana can show:

Employee API

Request Rate	^ ^ ^ ^ ^ ^ ^
Error Rate	^ ^
Latency	^ ^ ^ ^ ^
CPU	^ ^ ^
Memory	^ ^ ^ ^

| Prometheus vs Grafana

A common beginner confusion:

Does Grafana collect the metrics?

Usually, not in this architecture.

Think:

```
Application
  ↓
Prometheus
  ↓
Grafana
```

Prometheus collects/stores.

Grafana queries and displays.

| Example production dashboard

For employee-service , a useful dashboard could include:

```
Requests/sec
HTTP 2xx
HTTP 4xx
HTTP 5xx
P50 latency
P95 latency
P99 latency
CPU
Memory
JVM heap
GC pauses
DB connection pool
Pod replicas
Pod restarts
```


This gives both infrastructure and application visibility.

19. What are P50, P95 and P99?

Very important for backend developers.

Suppose request latencies are:

```
100ms
120ms
150ms
180ms
200ms
...
2 seconds
5 seconds
```

Average latency can hide slow requests.

Percentiles are often more useful.

P50 50% of requests completed within this latency.

Example:

```
P50 = 120ms
```

P95 95% completed within:

```
P95 = 800ms
```

P99 99% completed within:

```
P99 = 2.5s
```

So P99 shows the experience of the slower tail.

Why average latency can mislead

Imagine:

```
99 requests = 100ms
1 request   = 10 seconds
```

Average is roughly:

```
199ms
```

which doesn't sound terrible.

But one user waited:

10 seconds

Percentile metrics make these tail-latency problems easier to spot.

The four golden signals

A very useful monitoring model is:

Latency
Traffic
Errors
Saturation

For your Spring Boot API:

Latency

How long are requests taking?

Traffic

How many requests/sec?

Errors

How many requests are failing?

Saturation

How close are CPU/memory/thread pools/DB connections to capacity?

This is an excellent starting point for production monitoring.

Practical example

Users say:

Employee API is slow.

Dashboard:

Traffic:
normal

Errors:
normal

CPU:
95|

Latency:
P95 increased from 300ms → 2.1s

Hypothesis:

```
CPU saturation
  ↓
possible throttling/contention
  ↓
higher latency
```

Now connect this to chapter 20.

Monitoring helps us detect what resource behavior is doing to the application.

| Another example

Dashboard:

```
CPU = 30|
Memory = 50|
```

Looks fine.

But:

```
HTTP 500 = 18|
```

Then infrastructure isn't the problem.

Check application metrics/logs.

Maybe:

```
Database errors
```

or:

```
External API failures
```

Again:

```
Pod Running
"\"
Business/application healthy
```

| Metrics Server

Now an important distinction.

You have used:

```
kubectl top pods
```

Where does that information come from?

Usually from:

Metrics Server Conceptually:

```
Kubelet
  ↓
resource metrics
  ↓
Metrics Server
  ↓
Kubernetes Metrics API
  ↓
kubectl top
```

| Metrics Server vs Prometheus

These are often confused.

Metrics Server Designed mainly for lightweight Kubernetes resource metrics such as:

```
Pod CPU
Pod memory
Node CPU
Node memory
```

and supports components such as HPA.

Prometheus More complete monitoring system:

```
Infrastructure metrics
Application metrics
Historical time series
Queries
Dashboards/alerts integration
```

So:

```
Metrics Server
" `
Prometheus
```

| kubectl top

When you run:

```
kubectl top pods
```

you might see:

NAME	CPU	MEMORY
employee-abc	420m	720Mi
employee-def	350m	690Mi
employee-ghi	390m	710Mi

Useful for quick troubleshooting.

But it isn't a replacement for long-term monitoring.

Why?

Because you usually need questions like:

What was CPU at 2 AM yesterday?

That is where a time-series monitoring system such as Prometheus becomes valuable.

Kubernetes metrics sources

Different components can expose metrics.

For example:

```
API Server
Scheduler
Controller Manager
Kubelet
Applications
```

Prometheus can collect relevant metrics from them based on the platform's monitoring configuration.

This lets platform teams monitor both:

```
Kubernetes itself
```

and:

```
applications running on Kubernetes
```

Service discovery

You might wonder:

If Pods keep changing, how does Prometheus know where to scrape?

This is where Kubernetes service discovery becomes useful.

Prometheus can discover targets based on Kubernetes metadata.

Conceptually:

```
Pod A created
↓
Kubernetes API
↓
Prometheus discovers target
↓
```

```
starts scraping  
  
Pod A deleted  
↓  
Prometheus stops scraping
```

So monitoring adapts to Kubernetes' dynamic nature.

| OpenShift connection

OpenShift includes integrated monitoring capabilities built around technologies such as Prometheus.

Conceptually, you✓l often have:

```
OpenShift platform monitoring
```

for cluster/platform components, and potentially:

```
user workload monitoring
```

for application metrics.

The exact administration/configuration depends on the OpenShift environment, but the Prometheus concepts you're learning remain the same.

30. What is PromQL?

Prometheus has its own query language:

PromQL You use it to ask questions of metrics.

Conceptually:

```
Give me CPU usage
```

or:

```
Show HTTP 500 rate
```

or:

```
Show P95 latency
```

Grafana dashboards frequently use PromQL queries behind the scenes.

| Example rate calculation

Imagine:

```
http_requests_total
```

keeps increasing:

```
10:00 → 10,000  
10:01 → 10,600
```

Difference:

```
600 requests/minute
```

or:

```
10 requests/sec
```

Prometheus functions such as `rate()` help calculate how quickly counters are increasing.

Conceptually:

```
Counter  
  ↓  
rate()  
  ↓  
requests per second
```

Counter vs Gauge

These are common metric types.

Counter Only normally increases until reset.

Example:

```
total_requests  
  
1  
2  
3  
4  
5  
...
```

Useful for:

```
request count  
error count  
jobs completed
```

Gauge Can go up and down.

Example:

```
current_memory_usage
```

```
500Mi  
700Mi  
650Mi  
900Mi
```

Useful for:

```
memory  
active threads  
queue size  
DB connections
```

Histograms

Latency is often measured with histograms.

Conceptually, Prometheus tracks how many requests fall into buckets:

"d100ms	5000
"d250ms	8200
"d500ms	9400
"d1 second	9800
"d5 seconds	10000

Using this data you can calculate latency percentiles such as:

```
P95  
P99
```

You don't need to memorize histogram mathematics now.

Just understand:

Histograms are very useful for latency distributions.

34. What is Alertmanager?

Monitoring isn't useful if someone has to stare at Grafana all day.

You need alerts.

Conceptually:

```
Prometheus  
  ↓  
Alert condition becomes true  
  ↓  
Alertmanager  
  ↓  
Notification system
```


Possible destinations include organizational tools such as:

```
Email
Pager/incident system
Chat/on-call integration
```

| Example alert

Suppose:

```
employee-service HTTP 5xx > 5|
```

for:

```
10 minutes
```

Then:

```
Prometheus rule
  ↓
fires alert
  ↓
Alertmanager
  ↓
on-call engineer
```

This allows proactive incident response.

| Good alerts vs bad alerts

Bad alert:

```
CPU > 60| for 10 seconds
```

Why bad?

CPU reaching 60% briefly can be completely normal.

This generates noise.

Better alerts are tied to actual risk or user impact.

For example:

```
HTTP error rate > 5|
for 10 minutes
```

or:

```
P95 latency > 2s
for 10 minutes
```

or:

```
Pod unavailable  
and redundancy below safe level
```

The goal isn't:

Alert on everything.

It's:

Alert on conditions requiring action.

| Alert fatigue

If engineers receive:

```
100 alerts/day
```

and 95 aren't important, eventually they ignore alerts.

That is:

Alert fatigue A good monitoring system should prioritize:

```
actionable  
meaningful  
user-impacting
```

conditions.

| Logs connection

Suppose alert:

```
HTTP 500 rate = 12|
```

Monitoring tells you:

There is a problem.

Then logs tell you:

```
org.postgresql.util.PSQLException:  
too many connections
```

So:

```
Metrics  
↓  
detect problem  
  
Logs  
↓  
explain problem
```

This is why they complement each other.

| Tracing connection

Now suppose:

P95 latency = 3 seconds

Logs don't show errors.

Trace shows:

Gateway	20ms
Employee Service	40ms
Payment Service	80ms
Database	2800ms

Now:

Database

is the likely bottleneck.

That is where distributed tracing becomes valuable.

| OpenTelemetry

You✓I often hear:

OpenTelemetry OpenTelemetry provides standards and tooling for generating/collecting observability telemetry such as:

Traces
Metrics
Logs

For Java applications, it can help instrument requests across microservices.

Conceptually:

```
Client
↓
Gateway
↓
Employee
↓
Payroll
↓
Database
```

One trace can follow the request across these components.

| Trace ID

Each distributed request can carry something like:

```
traceId = abc123
```

Then:

```
Gateway log:  
traceId=abc123
```

```
Employee log:  
traceId=abc123
```

```
Payroll log:  
traceId=abc123
```

Now you can correlate the full request.

This is extremely useful in microservice systems.

| Monitoring a JVM

For your Spring Boot application, useful JVM metrics include:

```
Heap used  
Heap max  
Metaspace  
GC duration  
GC count  
Thread count  
CPU  
Class loading
```

Suppose memory gradually behaves like:

```
09:00 → 500Mi  
10:00 → 600Mi  
11:00 → 700Mi  
12:00 → 800Mi  
13:00 → 900Mi
```

and never comes down significantly.

That might suggest something worth investigating, such as:

```
memory leak  
unbounded cache  
retained objects
```

Monitoring helps detect this before repeated OOMKilled events.

| Database connection pool monitoring

Suppose HikariCP:

```
max connections = 20
```

Metrics show:

```
active = 20  
idle = 0  
pending = 50
```

Then API latency increases.

The problem may not be CPU or memory.

It could be:

```
DB connection pool saturation
```

This is why application metrics matter.

| A realistic production dashboard

For employee-service , I would mentally group metrics into:

Availability

```
Ready replicas  
Pod restarts  
HTTP success rate
```

Traffic

```
Requests/sec  
Endpoint traffic
```

Performance

```
P50/P95/P99 latency
```

Errors

```
HTTP 4xx  
HTTP 5xx  
Exceptions
```

Resources

```
CPU  
Memory  
CPU throttling
```

JVM

```
Heap
GC
Threads
```

Dependencies

```
DB connection pool
DB latency
External service calls
```

That is a much stronger view than merely checking:

```
kubectl get pods
```

| Monitoring vs health probes

We've already learned:

```
Readiness probe
Liveness probe
Startup probe
```

Don't confuse those with monitoring.

Probe Answers a narrow operational question:

```
Can Kubernetes send traffic?
Should Kubernetes restart me?
```

Monitoring Answers much broader questions:

```
How fast?
How many errors?
How much traffic?
How much memory?
What is the trend?
```

So:

```
Probes
" `
Monitoring
```

Both are needed.

| Example: probe says healthy, metrics say bad

Pod:

```
Readiness = SUCCESS
Liveness = SUCCESS
```

But metrics:

```
P95 latency = 4 seconds
HTTP 500 = 8|
```

Kubernetes may correctly keep the Pod running because the application is technically alive and capable of receiving traffic.

But users are having a poor experience.

Monitoring detects this.

Monitoring vs HPA

Monitoring and autoscaling also connect.

HPA may use:

```
CPU metrics
```

or other configured metrics to decide:

```
3 Pods → 6 Pods
```

Prometheus-style application metrics can also be integrated into more advanced autoscaling architectures.

But remember:

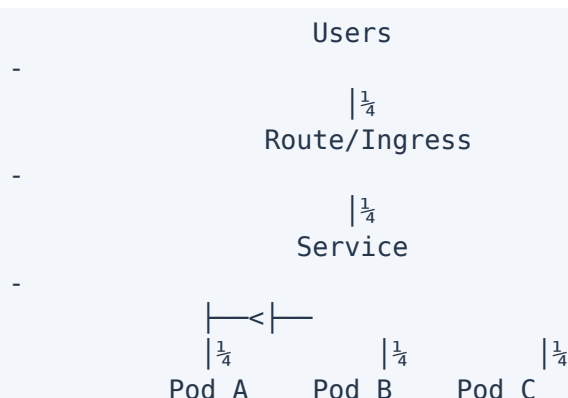
```
Monitoring
→ observe
```

```
Autoscaling
→ act on capacity
```

Different responsibilities.

Complete architecture

Let's connect everything:





Alongside:

```

Application
  ↓
Logs
  ↓
Central log platform
  
```

and:

```

Application
  ↓
OpenTelemetry
  ↓
Trace backend
  
```

Together:

```

Metrics + Logs + Traces
  ↓
Observability
  
```

Troubleshooting example

User says:

API became slow around 3 PM.

Start:

```

Grafana
  ↓
P95 latency spike at 15:00
  
```

Check traffic:

```

Requests/sec increased 4x
  
```


Check CPU:

```
CPU at limit
```

Check Pods:

```
All Running
```

Now your hypothesis:

```
Traffic spike
  ↓
CPU saturation/throttling
  ↓
Latency increase
```

Then you can investigate whether:

```
More replicas are needed
CPU requests/limits need tuning
application code is inefficient
```

This is how monitoring drives engineering decisions.

Another troubleshooting example

Alert:

```
HTTP 500 > 10|
```

Dashboard:

```
CPU normal
Memory normal
Traffic normal
```

Logs:

```
DB connection timeout
```

DB metrics:

```
connection pool exhausted
```

Trace:

```
DB call = 6 seconds
```

Root cause becomes much clearer.

```
Metrics
→ detected incident
```

Logs
→ showed error

Trace
→ showed slow dependency

That is observability in practice.

What you should remember

Metrics Server

Lightweight Kubernetes CPU/memory metrics
→ kubectl top / HPA-related usage

Prometheus

Collect + store + query time-series metrics

Grafana

Visualize metrics in dashboards

Alertmanager

Route/manage alerts

Micrometer + Spring Boot Actuator

Expose application/JVM metrics

OpenTelemetry

Observability instrumentation, especially distributed tracing

The biggest conceptual distinction

Remember this table:

Tool/concept Question

Probe Can Kubernetes use/restart this Pod?

Metrics How is the system behaving numerically?

Logs What happened?

Traces Where did the request spend time?

Prometheus Collect/store/query metrics

Grafana Visualize metrics

Alertmanager Who should be notified?

You now understand where monitoring fits:

Application



Next chapter — chapter 35: Centralized Logging & Distributed Tracing Next we'll focus specifically on:

Container logs
 stdout/stderr
 Node-level log collection
 Central logging
 Loki / OpenSearch-style architecture

Log aggregation
 Trace IDs
 OpenTelemetry
 Distributed tracing

And we'll answer an important production question:

If a Pod is deleted and replaced, where do its old logs go?

That question explains why relying only on `kubectl logs` is not enough in production.

Chapter 29 — Centralized Logging and Distributed Tracing

CHAPTER FOCUS

Understand **Centralized Logging and Distributed Tracing** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

Now we'll complete the other two major observability pillars:

```
Observability
├─ Metrics    ' chapter 34
├─ Logs      !• today
├─ Traces    !• today
```

The central production problem is:

Pods are temporary. If a Pod crashes, moves to another Node, or gets deleted, how do we retain and search its logs later?

And for microservices:

If one request passes through six services and becomes slow, how do we find which service caused the delay?

Start with Spring Boot logging

Your Spring Boot application might produce:

```
2026-09-22 10:31:20 INFO      EmployeeController - Fetch employee 123
2026-09-22 10:31:20 INFO      EmployeeService - Fetching employee
2026-09-22 10:31:21 ERROR EmployeeRepository - Database timeout
```

Inside a container, the recommended general pattern is to write application logs to:

```
stdout
stderr
```

rather than depending only on application-specific log files inside the container.

Conceptually:

```
Spring Boot
  ↓
stdout / stderr
```

```
↓  
Container runtime  
↓  
Node logging system
```

2. What are stdout and stderr?

They are standard process output streams.

Your Java application may effectively produce:

```
System.out / logging framework  
↓  
stdout
```

while errors may also go to:

```
stderr
```

Kubernetes/container runtimes capture these streams.

That is why this works:

```
kubectl logs employee-api-abc
```

kubectl isn't generally reading your application's arbitrary `/logs/app.log` file.

It is retrieving container logs exposed through the Kubernetes/node logging mechanism.

3. What does kubectl logs actually give you?

Suppose:

```
Pod  
├─ employee-container
```

and the application writes:

```
INFO Application started  
ERROR Database unavailable
```

Then:

```
kubectl logs employee-api-abc
```

can return those container logs.

For a previous crashed container instance:

```
kubectl logs employee-api-abc --previous
```

can often retrieve the previous container's logs.

You already used this during CrashLoopBackOff troubleshooting.

But kubectl logs has a major limitation

Imagine:

```
employee-api-abc
```

crashes.

Deployment creates:

```
employee-api-xyz
```

Later the old Pod disappears.

Now someone asks:

What happened in employee-api-abc yesterday at 2:17 AM?

If those logs only existed on the Node/container log files and they've already been rotated/deleted, you may no longer have them conveniently available.

That is why production systems use:

Centralized Logging

Centralized logging architecture

Instead of:

```
Engineer
  ↓
kubectl logs Pod A
kubectl logs Pod B
kubectl logs Pod C
```

we want:

```
Pod A ||
Pod B || <—° Log Collector ||° Central Log Store ||° Search/UI
Pod C ||
```

Now logs survive independently of the individual Pod lifecycle according to the logging platform's retention policy.

The key principle

Remember:

Pod lifecycle

"`

Log lifecycle

A Pod might live:

30 minutes

while operational policy may retain logs for:

days

weeks

months

depending on company requirements.

Centralized logging separates those lifecycles.

7. How are logs collected from every Node?

Imagine:

Cluster

-

```

├─ Node A
│   ├─ Pod 1
│   ├─ Pod 2
│   └─ Pod 3

```

-

```

├─ Node B
│   ├─ Pod 4
│   └─ Pod 5

```

-

```

├─ Node C
│   └─ Pod 6

```

A common architecture uses a log collector on each Node.

Remember the Kubernetes workload that naturally runs one Pod per Node?

DaemonSet Exactly.

Conceptually:

```

Node A
├─ Applications
└─ Log Collector

```

```

Node B
├─ Applications
└─ Log Collector

```

```

Node C

```

```
├─ Applications
├─ Log Collector
```

That log collector can collect container logs from its Node and send them to centralized storage.

This connects directly to your earlier DaemonSet chapter.

| Common logging architecture

A generic architecture looks like:

```
Application Pods
  ↓
stdout/stderr
  ↓
Node container logs
  ↓
DaemonSet Log Collector
  ↓
Central Logging Backend
  ↓
Search / Dashboard
```

Different organizations use different technologies.

You may hear combinations involving:

```
Fluent Bit / Fluentd
Vector

OpenSearch / Elasticsearch
Loki
Splunk
cloud logging platforms
```

Don't focus on memorizing products yet.

Understand the architecture.

| Loki-style architecture

One common modern pattern is:

```
Application
  ↓
Logs
  ↓
Collector
  ↓
Loki
  ↓
```


Grafana

You've already seen Grafana for metrics.

Grafana can also act as a user interface for querying logs from compatible log backends such as Loki.

So one UI might show:

```
Metrics
+
Logs
+
possibly traces
```

| OpenSearch / Elasticsearch-style architecture

Another common architecture:

```
Application Pods
  ↓
Log Collector
  ↓
OpenSearch / Elasticsearch
  ↓
Search/dashboard UI
```

The central idea is the same:

Collect logs from distributed workloads and store them in a searchable system.

| Why Kubernetes metadata matters

Imagine this log:

```
Database connection failed
```

Not very useful by itself.

You need context:

```
namespace=employee-prod
pod=employee-api-765fd
container=employee-service
node=worker-5
environment=prod
```

So a collector can enrich logs with Kubernetes metadata.

Conceptually:

```
Raw log:
Database connection failed

      ↓ enrich

namespace = employee-prod
pod        = employee-api-765fd
container  = employee
node       = worker-5
message    = Database connection failed
```

Now you can query:

Show all errors from employee-service in employee-prod during the last 30 minutes.

That is much more useful.

| Structured logging

Consider this traditional log:

```
ERROR Employee 123 payment failed amount 5000 request abc789
```

A computer has to parse the text.

A better approach is structured logging:

```
{
  "level": "ERROR",
  "service": "employee-service",
  "employeeId": "123",
  "message": "Payment failed",
  "amount": 5000,
  "traceId": "abc789"
}
```

Now the logging platform can query fields directly.

For example:

```
level = ERROR
service = employee-service
traceId = abc789
```

Structured logging is very useful in microservices.

| Be careful with sensitive data

Logs can accidentally become a security problem.

Don't casually log:

```
Passwords
Access tokens
Authorization headers
Full credit-card data
Private keys
Sensitive personal data
```

Once logs enter centralized systems, they may be retained and accessible to operational teams.

So application logging design must include:

```
redaction
masking
access control
retention policy
```

| Log levels

You're familiar with Java log levels:

```
TRACE
DEBUG
INFO
WARN
ERROR
```

In production, typically you don't want enormous DEBUG logging permanently enabled unless you have a specific reason.

Why?

```
More log volume
      ↓
More storage
More network traffic
Higher ingestion costs
Harder searching
Potential information exposure
```

So logging isn't free.

| Logs aren't metrics

Suppose you want:

What percentage of requests failed during the last hour?

You could potentially analyze logs.

But metrics are generally better suited for numerical questions like:

```
request rate
error percentage
latency
```

Logs are better for:

```
What exception happened?
Which customer operation failed?
What did the application report?
```

So:

```
Metrics → detect patterns
```

```
Logs → investigate details
```

Now distributed tracing

Suppose a request follows:

```
Client
  ↓
API Gateway
  ↓
Employee Service
  ↓
Payroll Service
  ↓
Tax Service
  ↓
Database
```

The user says:

This request took 4 seconds.

Which component caused the delay?

Logs alone can make this difficult.

That is the problem distributed tracing solves.

17. What is a trace?

A trace represents the journey of one request through a distributed system.

Example:

```
Trace: abc123
```

```
Gateway
  ↓
```

```
Employee Service
  ↓
Payroll Service
  ↓
Database
```

The entire request chain belongs to one trace.

18. What is a span?

A trace is made up of:

Spans Each span represents one unit of work.

For example:

```
Trace abc123
-
|— Gateway                20ms
-
|— Employee Service       40ms
-
|— Payroll Service        80ms
-
|— Database               2500ms
```

Each box is roughly a span.

So:

```
Trace
=
complete request journey

Span
=
one operation inside that journey
```

Trace ID

Every trace has an identifier.

For example:

```
traceId = abc123
```

When Gateway calls Employee Service, it propagates tracing context.

Conceptually:

```
Client
  ↓
traceId=abc123
  ↓
Gateway
  ↓
traceId=abc123
  ↓
Employee
  ↓
traceId=abc123
  ↓
Payroll
```

So all operations can be connected to the same distributed request.

| Span ID

Individual operations also have their own identifiers.

For example:

```
Trace ID: abc123
```

```
Gateway span:
spanId = 111
```

```
Employee span:
spanId = 222
```

```
Database span:
spanId = 333
```

The relationships let the tracing system reconstruct:

```
parent
  ↓
child
  ↓
child
```

and produce a request tree/timeline.

| Why this matters

Suppose:

```
Total request = 4 seconds
```

Trace:

Gateway	15ms
Employee	30ms
Payroll	50ms
Tax	40ms
Database	3800ms

You immediately know:

```
Database
"H bottleneck
```

Without tracing, you might inspect five different services separately.

| Parallel operations

Tracing becomes even more useful when services execute operations in parallel.

For example:

```
Employee Service
-
  |— Payroll API |— 500ms
-
  |— Benefits API |— 700ms
-
  |— Profile DB |— 100ms
```

A trace visualization can show which calls overlapped and which one determined the total latency.

| OpenTelemetry

Now we reach an important modern observability concept:

OpenTelemetry — OTel OpenTelemetry provides vendor-neutral APIs, SDKs, instrumentation and protocols for telemetry.

It can work with:

```
Traces
Metrics
Logs
```

Its biggest conceptual value is:

Instrument your application using an open standard instead of tightly coupling your application to one monitoring vendor.

| Java/Spring Boot example

Conceptually:

```

Spring Boot
  ↓
OpenTelemetry instrumentation
  ↓
Generate traces/metrics
  ↓
OTel Collector
  ↓
Observability backend

```

You may use automatic Java instrumentation or explicit application instrumentation depending on requirements.

OpenTelemetry Collector

An important component is:

OpenTelemetry Collector Think of it as a telemetry processing pipeline.

```

Applications
  ↓
OTel Collector
  ↓
process / enrich / batch
  ↓
Observability backend

```

It separates applications from the final storage vendor.

For example:

```

Application
  ↓
OpenTelemetry
  ↓
OTel Collector
  ├── Trace backend
  ├── Metrics backend
  └── Log backend

```

26. Why use a collector?

Without it:

```
Application → Vendor A
```

If you change vendor:

```
Application code/config may need changes
```

With a collector:


```
Application
  ↓
OTel Collector
  ↓
Vendor A
```

Later:

```
Application
  ↓
same Collector interface
  ↓
Vendor B
```

This provides architectural flexibility.

| Trace sampling

Here's another important concept.

Imagine your system processes:

```
50,000 requests/sec
```

Do you necessarily want to store a detailed trace for every request forever?

That could be extremely expensive.

So tracing systems use:

Sampling For example:

```
1000 requests
  ↓
record detailed traces for 100
```

= 10% sample.

More sophisticated policies can retain:

```
errors
slow requests
important transactions
```

at higher rates.

| Logs + trace IDs

This is where observability becomes powerful.

Suppose trace:

```
traceId = abc123
```

Your Java logs also contain:

```
2026-09-22 ERROR
traceId=abc123
Database timeout
```

Now:

```
Trace
↓
Find slow Database span
↓
traceId abc123
↓
Search centralized logs
↓
Find exact exception
```

This connects:

```
Traces
+
Logs
```

Metrics + traces + logs together

Imagine production alert:

```
P95 latency > 2 seconds
```

That is a:

```
Metric
```

You inspect traces:

```
Database spans are slow
```

Then search logs:

```
Connection pool timeout
```

The diagnosis becomes:

```
Metric
→ Something is wrong.
```

```
Trace
→ Database path is slow.
```

```
Log
→ Hikari connection pool exhausted.
```

This is exactly why observability has multiple pillars.

A complete incident example

Users report:

Employee API is slow.

Grafana:

```
P95 latency:
300ms → 4 seconds
```

CPU:

```
40|
```

Memory:

```
55|
```

So Kubernetes resources appear fine.

Trace:

Gateway	20ms
Employee	40ms
Payroll	3700ms

Now inspect Payroll logs:

```
traceId=xyz123
WARN Waiting for DB connection
```

DB pool metric:

```
active = 20
max    = 20
pending = 80
```

Root cause:

```
DB connection pool saturation
```

Notice how we used:

```
Metrics
→ detect
```

```
Trace
```

→ locate

Logs

→ explain

Why Pod logs alone aren't enough

Imagine:

```
employee-api  
replicas = 10
```

Request could hit:

```
Pod-7
```

Then Employee calls Payroll:

```
payroll Pod-3
```

then Tax:

```
tax Pod-11
```

Do you want to manually run:

```
kubectl logs pod-1  
kubectl logs pod-2  
kubectl logs pod-3  
...
```

until you find the correct request?

No.

Centralized logging + tracing allows:

```
traceId = abc123
```

to find events across the entire distributed request.

Kubernetes labels vs trace information

Kubernetes metadata tells you:

```
Which namespace?  
Which Pod?  
Which Node?  
Which container?
```

Trace context tells you:

Which user request?
Which distributed operation?
Which downstream call?

Together:

Kubernetes metadata
+
application trace context
=
very powerful troubleshooting

| Logging and DaemonSets

Let's connect one more earlier chapter.

Why are log collectors often DaemonSets?

Because:

Every Node
↓
runs containers
↓
produces logs

We want:

Every Node
↓
one collector

That is exactly the DaemonSet model:

Node A → Collector
Node B → Collector
Node C → Collector

Perfect practical example of why DaemonSets exist.

| Sidecar logging pattern

Another possible model is a logging sidecar:

Pod
├─ Application Container
└─ Log Sidecar

For example:

```
Application
↓
writes file
↓
Sidecar tails file
↓
logging backend
```

This can be useful in particular cases.

But it adds:

```
extra container
extra CPU/memory
more configuration
```

So Kubernetes-wide node collectors are often preferred for ordinary stdout/stderr logs.

35. Should your Spring Boot app write to a file?

Inside traditional VMs you might configure:

```
/opt/app/logs/app.log
```

Inside containers, a common approach is:

```
Spring Boot
↓
stdout
```

and let the platform collect it.

Why?

Because containers are disposable.

If you write only to:

```
/container/logs/app.log
```

and the container disappears, that local file can disappear with it unless you've separately designed persistent logging.

36. Should logs use persistent volumes?

Usually not as the primary centralized logging design for ordinary application logs.

You generally don't want:

```
Every application Pod
↓
PVC for logs
```

That creates unnecessary storage coupling.

Instead:

```
Application stdout
  ↓
Collector
  ↓
Central logging system
```

Persistent storage belongs in the centralized log backend according to its own architecture.

| Log retention

Centralized logging systems need retention policies.

For example:

DEBUG logs	→ 3 days
application logs	→ 30 days
audit logs	→ longer

Exact values depend on:

```
business requirements
security requirements
compliance
storage costs

operational needs
```

There is no universal retention duration.

| Logs and audit logs are different

Application log:

```
Payment failed because database timed out.
```

Audit log:

```
User Alice changed Deployment X at 14:30.
```

Kubernetes/OpenShift audit logs focus on API/security actions.

Application logs focus on application behavior.

Don't confuse them.

| OpenShift logging

In OpenShift environments, centralized cluster/application logging may be provided through platform logging components and organization-specific observability tooling.

The architecture still follows the same basic model:

```
Pods
↓
Collectors
↓
Central backend
↓
Query / dashboards
```

So even if your company uses a particular OpenShift logging stack, the concepts from this chapter remain valid.

| Practical troubleshooting workflow

Suppose someone says:

Employee API returned HTTP 500 at 17:42.

A good investigation flow might be:

```
1. Check metric dashboard
   ↓
   Was error rate elevated?

2. Find trace around 17:42
   ↓
   Which dependency failed?

3. Get traceId
   ↓
   Search centralized logs

4. Examine Kubernetes metadata
   ↓
   Which Pod handled the request?

5. If needed:
   kubectl describe/logs
   ↓
   Inspect infrastructure/container issue
```

You move from broad evidence to increasingly specific evidence.

| Don't log every request blindly

Suppose traffic:


```
10,000 requests/sec
```

and you log 10 lines per request.

That is:

```
100,000 log lines/sec
```

This can become extremely expensive.

Instead, logging design should be deliberate.

For example:

```
INFO → significant application events  
WARN → unusual but recoverable conditions  
ERROR → actual failures  
DEBUG → diagnostic detail when needed
```

while metrics handle high-volume numerical monitoring.

| Observability and correlation IDs

Even without full distributed tracing, many systems use:

```
correlationId
```

For example:

```
x-correlation-id: req-123
```

Each service propagates it.

Then logs contain:

```
correlationId=req-123
```

This provides basic cross-service correlation.

Distributed tracing takes this further using standardized trace/span context.

| Correlation ID vs Trace ID

A correlation ID is often application-defined:

```
request-abc123
```

A Trace ID belongs to the distributed tracing model:

```
traceId = ...
```

In some architectures, one may effectively serve both purposes.

But conceptually:

Correlation ID

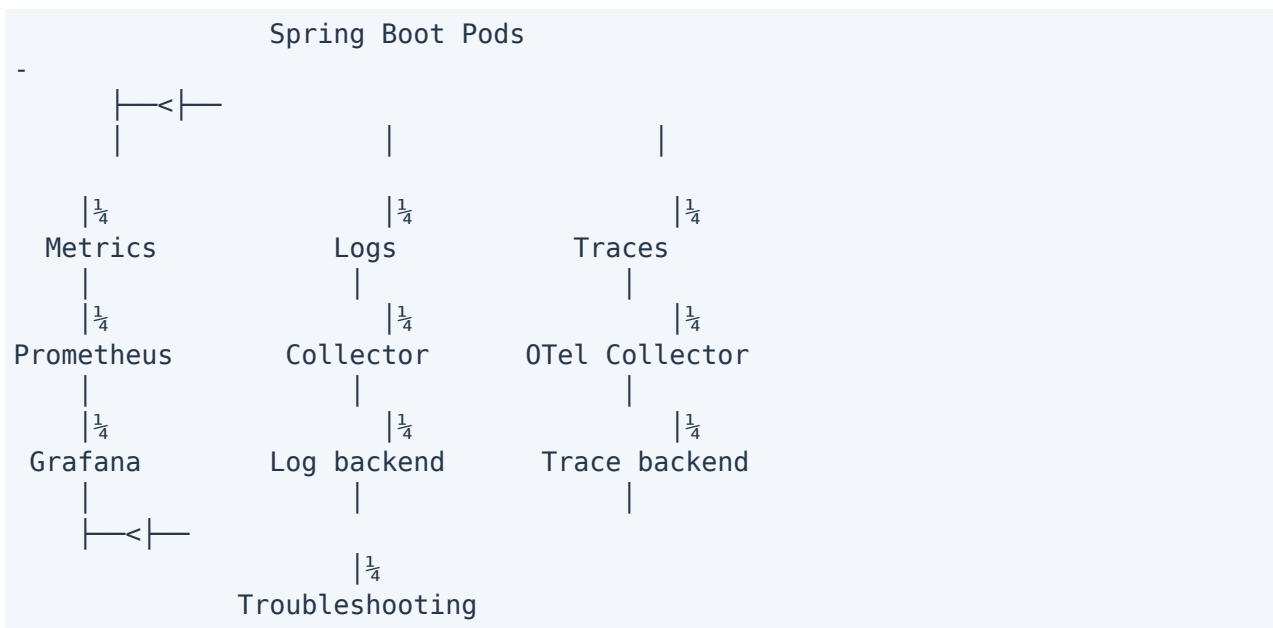
→ generic application request correlation

Trace ID

→ standardized tracing identity linking spans

Observability architecture so far

You can now draw:



One extremely useful mental model

When an incident happens:

Metrics

→ Is there a problem?

Traces

→ Where is the problem?

Logs

→ What exactly happened?

It's simplified, but it's an excellent operational mental model.

46. Interview question: Why centralized logging?

A good answer:

Kubernetes Pods and containers are ephemeral, so relying only on local container logs makes historical troubleshooting difficult. Centralized logging collects logs from

distributed workloads into durable searchable storage, usually enriched with Kubernetes metadata such as namespace, Pod and container.

47. Interview question: Why use a DaemonSet for logging?

A DaemonSet can run a log collector on every eligible Node, allowing it to collect container logs generated by workloads running on that Node and forward them to a centralized logging platform.

48. Interview question: Trace vs Span?

```
Trace
→ complete distributed request

Span
→ individual operation within that trace
```

Example:

```
Trace
├─ Gateway span
├─ Employee span
├─ Payroll span
└─ Database span
```

49. Interview question: What is OpenTelemetry?

OpenTelemetry is a vendor-neutral observability framework providing APIs, SDKs, instrumentation and protocols for generating and transporting telemetry such as traces, metrics and logs.

50. Interview question: Why isn't kubectl logs enough?

Because it is primarily useful for inspecting logs associated with Kubernetes container instances. In a dynamic cluster, Pods are replaced frequently, logs can be rotated, and applications may have many replicas across many Nodes.

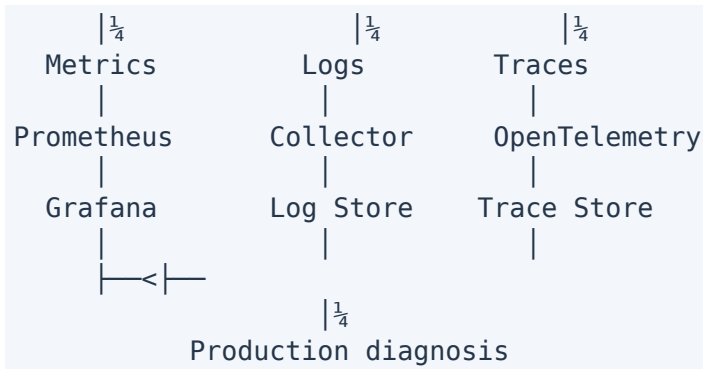
Centralized logging provides:

```
historical retention
cross-Pod search
cross-service search
Kubernetes metadata
aggregation
longer-term analysis
```

You've now covered the core observability picture:

```

      Observability
-
  └─<─┘
```



And the most important sentence to retain is:

Metrics tell you that something is wrong, traces help locate where it is wrong, and logs help explain what happened.

Next chapter — chapter 36: Advanced Production Troubleshooting This is a very good next step because we’ve now learned all the major evidence sources.

We✓I stop learning isolated objects and instead solve realistic incidents end-to-end, such as:

Application unavailable but Pods Running
 Intermittent 503s
 CrashLoopBackOff after deployment
 OOMKilled only during peak traffic
 CPU throttling without Pod failures
 Service has no endpoints
 DNS failures between services
 NetworkPolicy blocks traffic
 PVC/volume attachment failures
 Node failure
 Bad Helm release
 Argo CD says Synced but app is broken

The focus will be: given a production symptom, which layer should you inspect first and why?

Got it. Next up is chapter 36 — Advanced Production Troubleshooting whenever you want to continue.

Chapter 30 — Observability and Troubleshooting Foundations

CHAPTER FOCUS

Understand **Observability and Troubleshooting Foundations** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

This is one of the most important modules for you as a backend developer.

So far we've learned how to create Kubernetes resources.

Now we switch mindset:

"Something is broken. How do I systematically find the problem?"

A Kubernetes expert doesn't randomly run commands. They follow the application's path and eliminate possibilities one by one.

The Kubernetes troubleshooting mindset

Imagine your user reports:

"The employee API is not working."

Don't immediately look at Java logs.

First understand the request path:

```
User
-
  | ¼
Ingress / OpenShift Route
-
  | ¼
Service
-
  | ¼
Pod
-
  | ¼
Container
-
  | ¼
Spring Boot
-
  | ¼
```

Database

Your job is to identify:

At which layer does the request fail?

| The six commands you✓I use constantly

Memorize these:

```
kubectl get
kubectl describe
kubectl logs
kubectl exec
kubectl events
kubectl top
```

They answer different questions.

get

What exists and what is its current state?

describe

Why is this resource in this state?

logs

What is the application/container saying?

exec

Can I inspect the running container?

events

What did Kubernetes recently report?

top

How much CPU/memory is being consumed?

| Start with kubectl get

Suppose your application is:

```
employee-api
```

First:

```
kubectl get pods -n employee-prod
```

You might see:

NAME	READY	STATUS
employee-api-7d8c9f7c5b-x2k4p	1/1	Running
employee-api-7d8c9f7c5b-q8m2n	0/1	CrashLoopBackOff
employee-api-7d8c9f7c5b-p4s8k	1/1	Running

Immediately we know:

```
2 healthy
1 unhealthy
```

That is your first clue.

4. What does READY 1/1 mean?

This is important.

```
READY
1/1
```

means:

```
1 container ready
out of 1 container
```

It doesn't simply mean:

"The Pod process exists."

A Pod can be:

```
Running
```

but:

```
0/1 Ready
```

That distinction becomes extremely important with readiness probes.

Pod lifecycle states

You✓I commonly see:

```
Pending
Running
Succeeded
Failed
Unknown
```

But you✓I also encounter human-friendly statuses such as:

```
CrashLoopBackOff
ImagePullBackOff
ErrImagePull
```

```
ContainerCreating  
Terminating
```

These aren't all literal Pod phase values; some are container/runtime status indicators.

Don't memorize them as one flat list.

First troubleshooting example: Pending Pod

Suppose:

```
kubectl get pods
```

shows:

```
employee-api-abc123      0/1    Pending
```

Don't run:

```
kubectl logs
```

yet.

Why?

Because the container may not even have started.

Instead:

```
kubectl describe pod employee-api-abc123
```

Look at:

Events:

You might find:

```
0/3 nodes are available:  
Insufficient memory
```

Now you've found the problem.

7. Why can a Pod remain Pending?

Common causes:

```
Insufficient CPU  
Insufficient memory  
PVC not available  
Node selector mismatch  
Affinity rules  
Taints/tolerations
```


Scheduling constraints
ResourceQuota

Think:

Pending

↓

Pod hasn't successfully been scheduled/started

↓

Investigate scheduler + events

Example: insufficient memory

Suppose the Pod requests:

```
resources:
  requests:
    memory: 8Gi
```

but every suitable node has less available capacity.

Then Kubernetes may say:

Insufficient memory

The Pod stays:

Pending

This isn't an application bug.

It's a scheduling/resource problem.

Second example: CrashLoopBackOff

Now suppose:

```
kubectl get pods
```

shows:

employee-api-abc123	0/1	CrashLoopBackOff
---------------------	-----	------------------

This is a completely different problem.

The Pod was scheduled and the container started, but the process repeatedly exits.

Conceptually:

Container starts

↓

Application crashes

```
    ↓  
Container exits  
    ↓  
Kubernetes restarts it  
    ↓  
Application crashes again  
    ↓  
Repeat
```

Eventually Kubernetes backs off between restarts.

Hence:

```
CrashLoopBackOff
```

| First thing to do

Run:

```
kubectl logs employee-api-abc123
```

You might see:

```
Failed to configure DataSource  
Connection refused
```

Now you're getting somewhere.

| Previous container logs

Here's a very useful command:

```
kubectl logs employee-api-abc123 --previous
```

Why?

Suppose the container:

```
starts  
  ↓  
crashes  
  ↓  
restarts
```

The current container may have little or no useful output.

`--previous` lets you inspect logs from the previous terminated container instance.

This is one of the commands you should remember.

| CrashLoopBackOff isn't the root cause

This is a common beginner mistake.

People see:

```
CrashLoopBackOff
```

and think:

“Kubernetes is broken.”

No.

Usually:

```
CrashLoopBackOff
=
Kubernetes is repeatedly trying to restart a container that keeps failing.
```

The root cause could be:

```
Java exception
Database unavailable
Wrong environment variable
Missing Secret
Invalid configuration
Port conflict
Application bug
OutOfMemory
```

| Real Spring Boot example

Suppose logs show:

```
Failed to bind properties under 'server.port'
```

Maybe your ConfigMap contains:

```
SERVER_PORT=abc
```

Your Java application can't parse it.

Result:

```
Spring Boot starts
↓
Configuration parsing
↓
Exception
↓
Process exits
↓
CrashLoopBackOff
```

The Kubernetes symptom is:

```
CrashLoopBackOff
```

but the actual root cause is:

```
Invalid application configuration
```

| Third example: ImagePullBackOff

Suppose:

```
employee-api          0/1    ImagePullBackOff
```

This means Kubernetes can't successfully obtain/start the specified container image.

Run:

```
kubectl describe pod employee-api-abc123
```

Look at Events.

You may see:

```
Failed to pull image
```

Possible causes:

```
Image doesn't exist
Wrong image tag
Private registry authentication problem
Registry unavailable
Network issue
ImagePullSecret problem
```

| Example

Deployment says:

```
image: employee-api:1.7
```

But the registry only has:

```
employee-api:1.6
```

Kubernetes tries:

```
employee-api:1.7
```

and can't find it.

Result:

```
ErrImagePull
  ↓
ImagePullBackOff
```

Again:

The status tells you what Kubernetes is experiencing; events help explain why.

| **kubectl describe is your friend**

For a Pod:

```
kubectl describe pod <pod>
```

You✓I get information such as:

```
Name
Namespace
Node
Labels
Annotations
Status
IP
Containers
Images
Environment
Mounts

Conditions
Events
```

The Events section is often extremely valuable.

| **Kubernetes Events**

You can also inspect events directly:

```
kubectl get events -n employee-prod
```

Depending on your Kubernetes version/configuration, sorting/filtering can help.

For example:

```
kubectl events -n employee-prod
```

Events can tell you:

```
Scheduled
Pulled
Created
Started
Failed
```

```
Unhealthy  
Killing  
BackOff
```

Think of events as:

Kubernetes' recent operational breadcrumbs.

| Logs vs Events

This distinction is important.

Events Tell you:

What Kubernetes is doing/seeing.

Example:

```
FailedScheduling  
FailedMount  
Failed to pull image  
  
Unhealthy
```

Application logs Tell you:

What your application is doing/seeing.

Example:

```
SQLException  
NullPointerException  
Connection refused  
Invalid JWT
```

So:

```
Kubernetes problem  
→ Events
```

```
Application problem  
→ Logs
```

Often you need both.

| kubectl exec

Suppose the Pod is running:

```
employee-api-abc123
```

You can execute a command inside the container:

```
kubectl exec -it employee-api-abc123 -- sh
```

Now you're inside the container.

You can inspect things like:

```
env
```

or:

```
ls
```

or:

```
cat /etc/config/...
```

depending on what's available.

| Why exec is useful

Suppose you think the application isn't receiving:

```
DB_URL
```

You can inspect the environment:

```
kubectl exec -it employee-api-abc123 -- env
```

You may find:

```
LOG_LEVEL=INFO  
DB_URL=jdbc:postgresql://prod-db:5432/employee
```

Now you know the ConfigMap injection worked.

The problem might be somewhere else.

| Don't assume every container has Bash

This command:

```
kubectl exec -it pod -- bash
```

may fail.

Why?

Minimal container images may not contain Bash.

Try:

```
kubectl exec -it pod -- sh
```

But even sh isn't guaranteed.

This is one reason debugging minimal/distroless images can require different techniques.

Fourth example: Service not working

Suppose:

```
employee-api Pods = Running
```

but users can't access the application.

Don't immediately blame the Pods.

Check:

```
kubectl get service -n employee-prod
```

You might see:

NAME	TYPE	CLUSTER-IP
employee-api	ClusterIP	10.96.20.10

Now inspect:

```
kubectl describe service employee-api
```

Look for:

```
Selector
Endpoints / EndpointSlices
Ports
```

The critical Service question

A Service needs to find the right Pods.

For example:

```
selector:
  app: employee-api
```

Pod:

```
labels:
  app: employee-api
```

If the labels don't match:

Service

-

└ selector finds nothing

Then you can have:

Pods = Running

Service = Exists

Application = Unreachable

This is a classic Kubernetes troubleshooting problem.

Think about Endpoints

Conceptually:

Service

-

| $\frac{1}{4}$

EndpointSlice

-

└ Pod A

└ Pod B

└ Pod C

If the Service has no usable endpoints:

Service

-

| $\frac{1}{4}$

No endpoints

traffic has nowhere to go.

So when a Service isn't working:

Check whether it has endpoints.

Modern Kubernetes uses EndpointSlices rather than relying solely on the older Endpoints API.

Fifth example: Readiness probe failing

Suppose:

employee-api	0/1	Running
--------------	-----	---------

Notice:

```
STATUS = Running  
READY = 0/1
```

The container is running.

But Kubernetes doesn't consider it ready to receive normal Service traffic.

Why?

Potentially:

```
Readiness probe failing
```

| Readiness vs Liveness

This is extremely important.

Readiness Asks:

Can this container receive traffic right now?

If readiness fails:

```
Pod remains running  
but  
Pod is removed from normal Service endpoints
```

Liveness Asks:

Is this container still healthy enough to keep running?

If liveness fails repeatedly:

```
Kubernetes restarts the container
```

| Example

Spring Boot starts:

```
Application process = running
```

but database connection isn't ready yet.

If readiness says:

```
FAIL
```

then:

```
Pod  
Running  
but  
Not Ready
```

Service should avoid sending normal traffic to it.

This prevents users from hitting an application that isn't ready.

| Liveness failure

Suppose the application gets stuck:

```
Thread deadlock
```

The process is technically still alive:

```
Java process = running
```

but the application is unhealthy.

Liveness probe fails:

```
Liveness
  ↓
FAIL
  ↓
Container restart
```

This can recover certain classes of failures automatically.

| Startup probe

There's another important probe:

```
startupProbe
```

Useful for applications that take a long time to start.

For example:

```
Spring Boot
  ↓
loads large application
  ↓
initializes caches
  ↓
connects to systems
  ↓
takes 90 seconds
```

If liveness starts checking too aggressively, Kubernetes might restart the container before startup completes.

A startup probe can protect slow-starting applications.

Conceptually:

```
Startup probe
  ↓
"Has the app finished starting?"
  ↓
YES
  ↓
Begin normal liveness/readiness checks
```

Sixth example: PVC problem

Suppose:

postgres-0	0/1	Pending
------------	-----	---------

You inspect:

```
kubectl describe pod postgres-0
```

and discover:

```
FailedMount
```

Then:

```
kubectl get pvc
```

shows:

postgres-data	Pending
---------------	---------

Now follow the chain:

```
Pod
  ↓
PVC
  ↓
PV
  ↓
StorageClass
  ↓
Storage backend
```

This is the same resource-chain thinking we learned in storage.

Seventh example: Secret missing

Suppose your Spring Boot app crashes:

```
Could not resolve placeholder 'DB_PASSWORD'
```

Your Deployment references:

secretKeyRef:

name: employee-db
key: password

Check:

```
kubectl get secret employee-db
```

Maybe:

Error from server (NotFound)

Then you've found the issue.

The Pod can't receive the expected Secret value.

Eighth example: CPU/memory problem

Sometimes:

```
Pod
↓
Running
↓
then disappears/restarts
```

You might suspect:

OutOfMemory

Check:

```
kubectl describe pod <pod>
```

Look for termination information.

You may see:

Reason: OOMKilled

That means the container exceeded its memory limit and was killed.

kubectl top

If Metrics Server or the platform's metrics stack is available:

```
kubectl top pods -n employee-prod
```

You might see:

NAME	CPU	MEMORY
employee-api	250m	900Mi

For nodes:

```
kubectl top nodes
```

This gives you current resource usage.

| Requests vs limits revisited

Suppose:

```
resources:
  requests:
    memory: 512Mi

  limits:
    memory: 1Gi
```

Think:

```
Request
=
scheduler's planning number
```

and:

```
Limit
=
maximum allowed resource amount for the container
```

If the application exceeds:

```
1Gi
```

it can be killed for memory exhaustion.

This connects directly to:

```
OOMKilled
```

| A real incident

Imagine your users report:

“Employee API is intermittently unavailable.”

You run:

```
kubectl get pods
```

and see:

employee-api-1	1/1	Running
employee-api-2	0/1	Running
employee-api-3	1/1	Running

Pod 2 isn't ready.

You run:

```
kubectl describe pod employee-api-2
```

and find:

```
Readiness probe failed
```

Then:

```
kubectl logs employee-api-2
```

shows:

```
Database connection timeout
```

Now the investigation becomes:

```
Pod
↓
Readiness failing
↓
Database connection timeout
↓
Investigate database/network/configuration
```

That is how you troubleshoot systematically.

Another incident

Users say:

“API completely stopped.”

You run:

```
kubectl get pods
```

and see:

employee-api	0/3	CrashLoopBackOff
--------------	-----	------------------

Then:

```
kubectl logs <pod>
```

shows:

```
java.lang.OutOfMemoryError
```

Then:

```
kubectl describe pod <pod>
```

shows:

```
OOMKilled
```

Now the root cause is much clearer:

```
Application memory consumption
      ↓
Memory limit exceeded
      ↓
Container killed
      ↓
Restart
      ↓
CrashLoopBackOff
```

The troubleshooting ladder

I want you to memorize this workflow:

```
1. kubectl get
      ↓
2. kubectl describe
      ↓
3. kubectl logs
      ↓
4. kubectl logs --previous
      ↓
5. kubectl exec
      ↓
6. kubectl events
      ↓
7. kubectl top
```

You won't always use every step.

You use the next step based on what the previous one tells you.

A practical decision tree

Pod is Pending


```
Pending
↓
describe
↓
Events
↓
Scheduling / PVC / resource issue
```

Pod is CrashLoopBackOff

```
CrashLoopBackOff
↓
logs
↓
logs --previous
↓
Application/configuration/resource problem
```

ImagePullBackOff

```
ImagePullBackOff
↓
describe
↓
Events
↓
Image / registry / credentials
```

Running but 0/1 Ready

```
Running + Not Ready
↓
describe
↓
Readiness probe
↓
Application dependency / probe endpoint
```

Service unreachable

```
Service
↓
describe service
↓
selector
↓
EndpointSlices
↓
Pod readiness
↓
networking
```

| Don't forget namespaces

A very common mistake:

```
kubectl get pods
```

shows nothing.

You think:

“There are no Pods!”

Maybe you're simply in the wrong namespace.

Try:

```
kubectl get pods -n employee-prod
```

Or:

```
kubectl get pods --all-namespaces
```

In modern Kubernetes, `-A` is also commonly used:

```
kubectl get pods -A
```

Always know which namespace you're working in.

| Useful commands to build muscle memory

```
kubectl get pods -n employee-prod
```

```
kubectl get pods -o wide -n employee-prod
```

The `-o wide` output can show additional information such as:

```
Node
```

```
Pod IP
```

```
kubectl get deployment -n employee-prod
```

```
kubectl get svc -n employee-prod
```

```
kubectl get pvc -n employee-prod
```

```
kubectl get events -n employee-prod
```

| Watch resources in real time

Very useful:

```
kubectl get pods -w -n employee-prod
```

-w means:

```
watch
```

You might see:

```
Pending
↓
ContainerCreating
↓
Running
↓
Ready
```

This is useful during deployments and troubleshooting.

Logs from multiple Pods

If you have:

```
employee-api-1
employee-api-2
employee-api-3
```

you often want to understand the application collectively rather than looking at one Pod.

In real production environments, this is where centralized logging becomes important:

```
Pods
↓
Log collector
↓
Central logging platform
↓
Search / dashboards
```

Examples include:

```
Elasticsearch/OpenSearch
Loki
Splunk
Cloud logging platforms
```

We'll cover observability architecture later.

Kubernetes observability has three pillars

A common model is:

```
Logs
Metrics
Traces
```

Logs Tell you:

What happened?

Metrics Tell you:

How much/how often?

Traces Tell you:

Where did this request spend time across services?

For example:

```
User request
↓
API
↓
Employee service
↓
Database
```

Distributed tracing can show where latency occurred.

Kubernetes itself also exposes metrics

Metrics can help answer:

```
CPU usage
Memory usage
Pod restarts
Request rate
Latency
Error rate
Node health
```

For production observability you✓I often encounter:

```
Prometheus
Grafana
OpenTelemetry
```

These will become important later when we discuss production monitoring.

One important distinction

Kubernetes health:

Pod = Running

doesn't necessarily mean:

Application = healthy

Similarly:

Pod = Ready

doesn't mean:

Business logic = correct

A Pod could be:

Running
Ready

while the application returns:

HTTP 500

because the application itself has a bug.

Always separate:

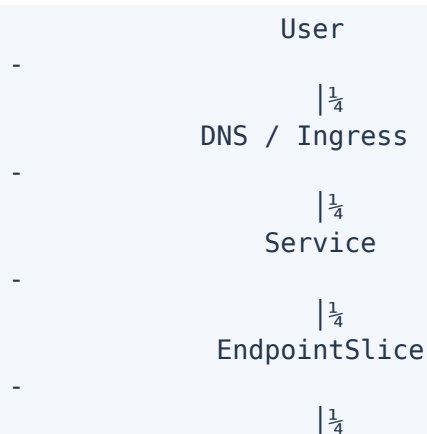
Infrastructure health

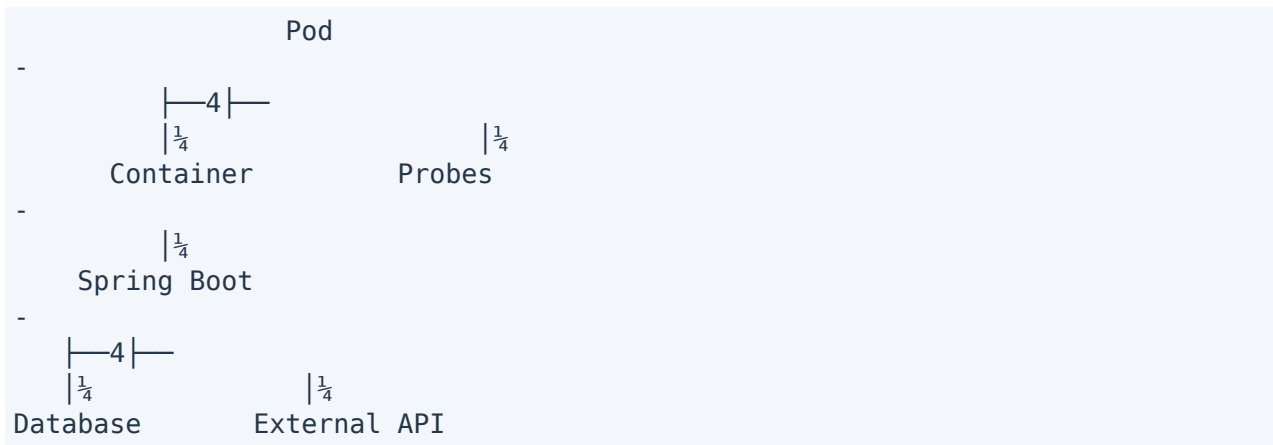
from:

Application health

The complete troubleshooting model

For a production incident, think in layers:





And alongside it:

Resources

- CPU
- Memory
- Storage

Configuration

- ConfigMap
- Secret

Security

- RBAC

This is the mental model I want you to develop.

Your first production-style troubleshooting exercise

Imagine this:

```
kubectl get pods
```

NAME	READY	STATUS
employee-api-abc	0/1	CrashLoopBackOff
employee-api-def	1/1	Running
employee-api-ghi	1/1	Running

What should you do?

Step 1

```
kubectl logs employee-api-abc
```

Suppose:

Failed to connect to PostgreSQL

Step 2 Check configuration:

```
kubectl describe pod employee-api-abc
```

Look at:

```
Environment
Mounts
Events
```

Step 3 Check Secret:

```
kubectl get secret employee-db
```

Step 4 Check database Service:

```
kubectl get svc
```

Step 5 Check whether database Pods are healthy.

You are now following the dependency chain instead of randomly changing YAML.

The golden rule of Kubernetes troubleshooting

When something fails:

Don't immediately change something. First collect evidence.

Bad approach:

```
Pod failed
↓
restart everything
```

Good approach:

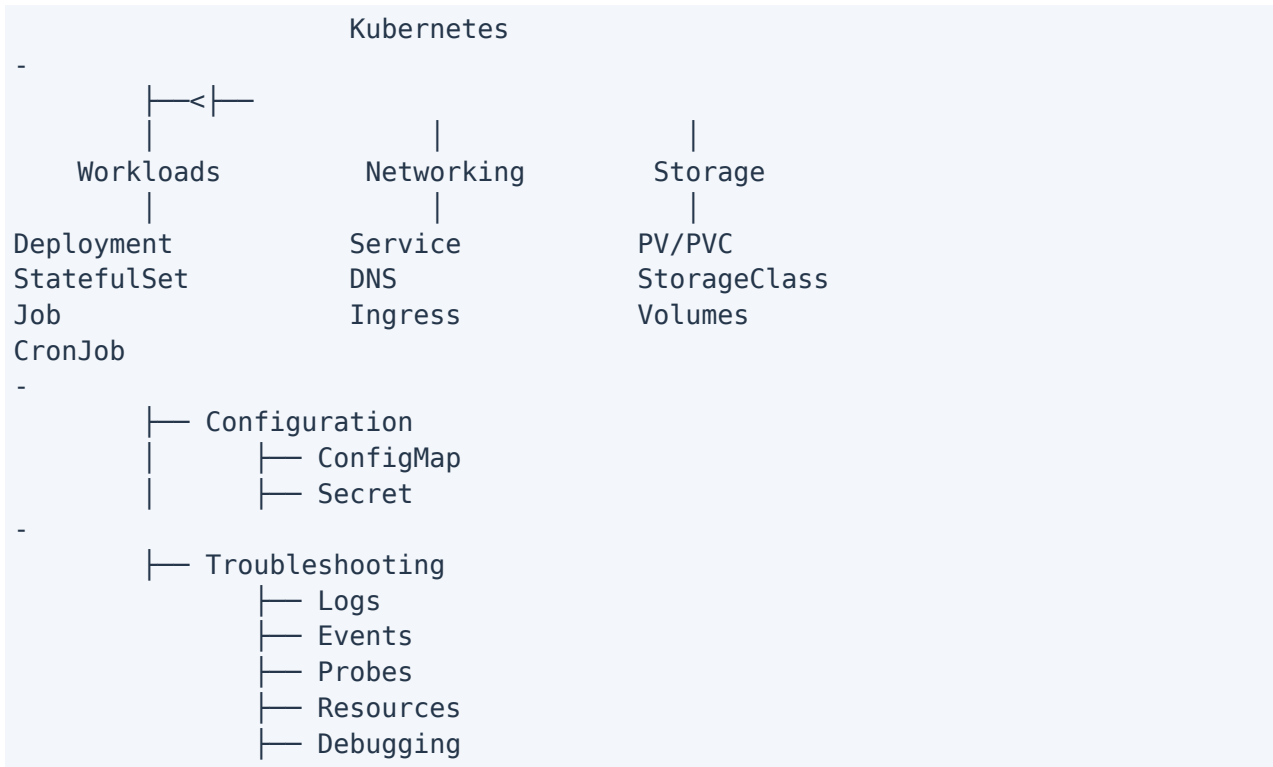
```
Observe
↓
Identify symptom
↓
Collect events
↓
Check logs
↓
Trace dependency
↓
Identify root cause
↓
Fix
↓
Verify
```

This mindset is far more valuable than memorizing hundreds of commands.

Where we are now

Our roadmap has reached an important milestone.

You've learned:



You're no longer just learning individual Kubernetes objects. You're starting to understand how the platform behaves as a system.

Next chapter — chapter 20: Health Probes in Depth We'll go deeper into one of the most important production concepts we just touched:

```

startupProbe
readinessProbe
livenessProbe
  
```

We'll use a real Spring Boot application and walk through:

```

Application starts
  ↓
Startup probe
  ↓
Readiness probe
  ↓
Service sends traffic
  ↓
Application becomes unhealthy
  ↓
Liveness probe
  
```


↓
Container restart

We✓I also answer a very common interview/production question:

Why can a Pod be Running but not Ready?

And we✓I design actual probe configurations for a Spring Boot API rather than just memorizing definitions.

Yes. Looking at our full learning history, you’ve covered most Kubernetes fundamentals already. A few topics were accidentally repeated later—especially networking, storage, ConfigMaps/Secrets, architecture, and probes—so from here I✓I avoid repeating them and continue from the actual gap.

Your Kubernetes / OpenShift Roadmap So Far

’ Phase 1 — Why Kubernetes Exists You learned the evolution:

```
Physical Server
  ↓
Virtual Machine
  ↓
Container
  ↓
Many Containers
  ↓
Need orchestration
  ↓
Kubernetes
```

Covered:

- Physical server vs VM vs container
- Container image vs running container
- Docker/container-runtime idea
- Registry
- Why Kubernetes exists
- Orchestration
- Desired state
- Reconciliation
- Cluster and Node basics

’ Phase 2 — Kubernetes Architecture You learned:

```
kubectl
  ↓
API Server
  ↓
etcd
Controllers
Scheduler
  ↓
Worker Node
  ↓
kubelet
  ↓
Container Runtime
```

Covered:

- Control Plane vs Worker Node
- API Server
- etcd
- Controller Manager
- Scheduler
- kubelet
- containerd / CRI-O
- CRI
- CoreDNS basics
- kube-proxy / Service dataplane concept
- End-to-end kubectl apply lifecycle

You already understand the important distinction:

```
Controller
→ WHAT should exist?

Scheduler
→ WHERE should it run?

Kubelet
→ MAKE it run on that Node
```

' Phase 3 — Pods Covered deeply:

```
Cluster
  ↓
Node
  ↓
Pod
  ↓
Container
```

You learned:

- Why Kubernetes needs Pods
- Pod vs container
- Single-container Pods
- Multi-container Pods
- Shared network namespace
- localhost between containers
- Shared volumes
- Pod IP
- Pod lifecycle
- Pods are disposable
- Container restart vs Pod replacement
- Init containers
- Sidecar concept
- Labels
- kubectl logs
- kubectl exec
- kubectl describe

' Phase 4 — ReplicaSets & Deployments You learned:

```
Deployment
  ↓
ReplicaSet
  ↓
Pods
```

Covered:

- Why raw Pods aren't normally used for production
- Desired replicas
- ReplicaSet reconciliation
- Labels/selectors
- Deployment
- Scaling
- Rolling updates
- Old vs new ReplicaSets
- Rollback
- Rollout history
- maxSurge
- maxUnavailable
- Recreate vs RollingUpdate

' Phase 5 — Kubernetes Services Covered:

```
Client
  ↓
Service
  ↓
Pod A
Pod B
Pod C
```

You learned:

- Why Pod IPs can't be depended upon
- Stable Service identity
- Service selectors
- Service DNS
- ClusterIP
- NodePort
- LoadBalancer
- port

- targetPort
- nodePort
- EndpointSlices
- Service load distribution
- Service vs Deployment
- Service troubleshooting

One important concept you've already mastered:

Deployment

→ makes application instances exist

Service

→ makes those instances reachable

' Phase 6 — Kubernetes Networking We covered this quite extensively already.

You learned:

Gateway → Employee

means simultaneously:

Gateway EGRESS

Employee INGRESS

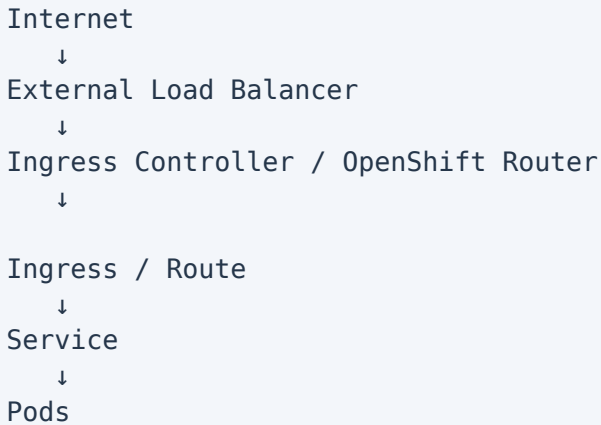
Covered:

- Pod-to-Pod networking
- Pod IP vs Node IP vs Service IP
- CNI
- Cross-Node Pod communication
- DNS
- Internal Service discovery
- NetworkPolicy
- Ingress policy vs egress policy
- Default allow behavior
- Default-deny concept
- podSelector

- namespaceSelector
- Gateway → Service example
- Network troubleshooting

So we should not repeat networking fundamentals again.

' Phase 7 — Ingress & OpenShift Routes You learned:



Covered:

- Ingress resource
- Ingress Controller
- Difference between the two
- Path-based routing
- Host-based routing
- LoadBalancer vs Ingress
- External DNS vs internal Kubernetes DNS
- TLS basics
- TLS termination
- OpenShift Route
- OpenShift Router
- Edge TLS
- Passthrough TLS
- Re-encrypt TLS

' Phase 8 — ConfigMaps & Secrets We actually covered this twice, so it is definitely complete.

Covered:

```
Container Image
  +
ConfigMap
  +
Secret
  ↓
Application
```

You learned:

- External configuration
- ConfigMap
- Secret
- Environment variable injection
- env
- envFrom
- Mounted configuration files
- Secret-mounted files
- Base64 “” encryption
- ConfigMap changes vs running containers
- Secret rotation basics
- Kubernetes Secrets vs HashiCorp Vault
- Why credentials shouldn’t be committed to Git

’ Phase 9 — Storage Also covered very deeply.

You understand:

```
Pod
↓
PVC
↓
PV
↓
Actual Storage
```

and:

```
PVC
↓
StorageClass
↓
CSI Provisioner
↓
PV
```

Covered:

- Container writable layer
- Volume
- emptyDir
- Mount / mounting
- PersistentVolume
- PersistentVolumeClaim
- StorageClass
- Static provisioning
- Dynamic provisioning
- CSI
- Access modes:
 - RWO – ROX – RWX – RWOP
- Reclaim policies
- Delete
- Retain
- PVC Pending / Bound
- Storage troubleshooting
- Persistent storage “” backup

’ Phase 10 — StatefulSets You learned this extensively:

```
StatefulSet
-
├─ database-0 → PVC-0
├─ database-1 → PVC-1
├─ database-2 → PVC-2
```


Covered:

- Deployment vs StatefulSet
- Stable names
- Ordinals
- Why database-0 , database-1 , etc.
- Stable storage
- volumeClaimTemplates
- Ordered creation
- Reverse deletion
- Why identity matters
- Headless Service
- Stable DNS
- Per-Pod PVCs
- Deployment can use PVC too
- Why a shared PVC isn't equivalent to StatefulSet
- StatefulSet doesn't automatically make DB highly available

' Phase 11 — DaemonSets You learned:

```
Node 1 → Agent  
Node 2 → Agent  
Node 3 → Agent
```

Covered:

- One Pod per eligible Node
- Difference from Deployment
- Monitoring agents
- Logging agents
- CNI/storage agents
- Adding/removing Nodes
- Node selection for DaemonSets

' Phase 12 — Health Probes This topic is already complete, so we don't need the previously suggested "Health Probes in Depth" chapter.

Covered:

Startup

→ Have I finished starting?

Readiness

→ Can I receive traffic?

Liveness

→ Am I alive or should I restart?

You learned:

- Startup probe
- Liveness probe
- Readiness probe
- HTTP probe
- TCP probe
- Exec probe
- gRPC probe
- initialDelaySeconds
- periodSeconds
- timeoutSeconds
- failureThreshold
- successThreshold
- Spring Boot Actuator
- Running "" Ready
- Readiness failure → remove from Service traffic
- Liveness failure → container restart
- Why DB failure generally shouldn't automatically trigger liveness restart
- Startup probe for slow Spring Boot applications

' Phase 13 — Scheduling Covered:

```
Pod
↓
Scheduler
↓
Suitable Node
```

You learned:

- CPU/memory requests
- Requests vs limits concept
- Node labels
- nodeSelector
- Node affinity
- Required vs preferred affinity
- Pod affinity
- Pod anti-affinity
- Taints
- Tolerations
- NoSchedule
- PreferNoSchedule
- NoExecute
- Topology
- Topology spread
- Scheduling Pending troubleshooting

' Phase 14 — Namespaces & Resource Governance Covered:

```
Cluster
↓
Namespace
├─ ResourceQuota
├─ LimitRange
```

You learned:

- Namespaces
- OpenShift Projects

- Namespace DNS
- Cross-namespace communication
- ResourceQuota
- LimitRange
- Default requests/limits
- Min/max resource constraints
- Namespace quotas
- kubectl get -A

' Phase 15 — RBAC Covered:

```
WHO
↓
CAN DO WHAT
↓
WHERE
```

You learned:

- User
- Group
- ServiceAccount
- Role
- RoleBinding
- ClusterRole
- ClusterRoleBinding
- Kubernetes verbs
- Least privilege
- Namespace-scoped vs cluster-scoped permissions
- Additive RBAC
- kubectl auth can-i
- Workload identity basics

' Phase 16 — Jobs & CronJobs Covered:

```
Deployment
→ continuously running

Job
→ run until successful

CronJob
→ run Jobs on schedule
```

You learned:

- Job
- Completion
- Failure/retry
- backoffLimit
- restartPolicy
- Job vs initContainer
- CronJob
- Cron syntax
- concurrencyPolicy
- Allow
- Forbid
- Replace
- Job history
- Time zones
- Parallel Jobs
- parallelism
- completions
- Migration Job use case

' Phase 17 — Basic Troubleshooting We completed a major troubleshooting chapter.

Covered:

```
kubectl get
kubectl describe
kubectl logs
kubectl logs --previous
kubectl exec
kubectl events
kubectl top
```

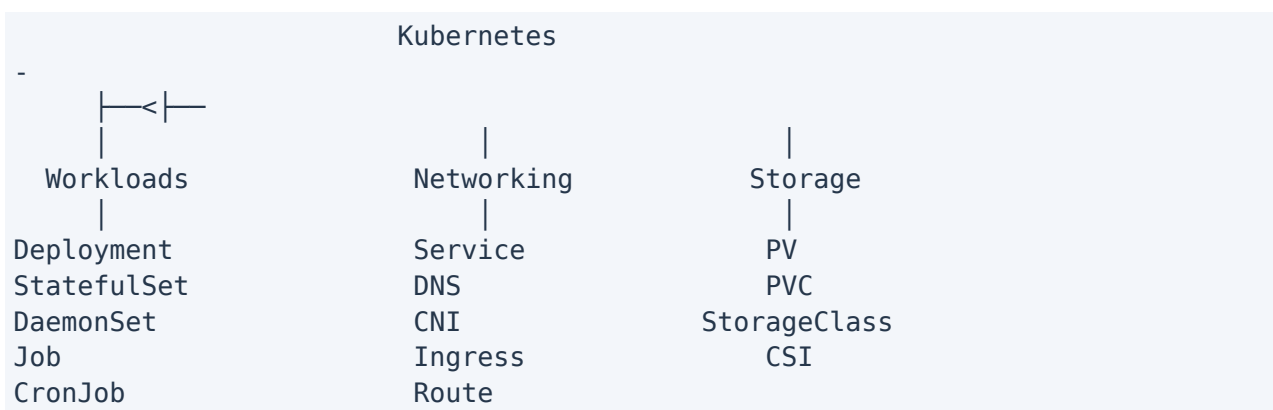
And common failures:

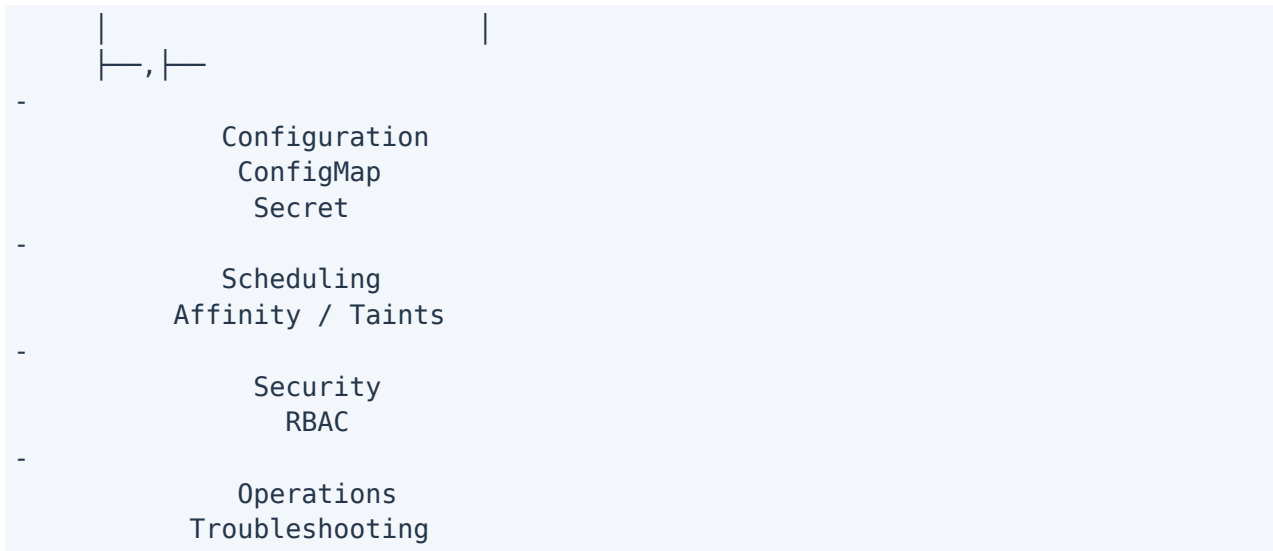
- Pending
- CrashLoopBackOff
- ImagePullBackOff
- ErrImagePull
- OOMKilled
- Running but 0/1 Ready
- Readiness failures
- Liveness failures
- PVC Pending
- Missing ConfigMap/Secret
- Service selector mismatch
- No Service endpoints
- Application/DB connectivity problems

Where You Are Now

You have completed roughly the core Kubernetes foundation.

Your current knowledge can be represented as:





Ø=βá What Is Still Pending? Now we should stop repeating fundamentals and move into intermediate/advanced Kubernetes + OpenShift.

I recommend the remaining roadmap in this order:

Next Topic Status

20 CPU/Memory Requests, Limits, QoS & OOM in depth #ó Pending

21 Horizontal Pod Autoscaler / Vertical Autoscaling #ó Pending

22 PodDisruptionBudget & High Availability #ó Pending

23 Graceful Shutdown, terminationGracePeriod & lifecycle hooks #ó Pending

24 Advanced Scheduling & Topology Ø=βá Basics done

25 NetworkPolicy in production Ø=βá Basics done

26 SecurityContext & Pod Security #ó Pending

27 OpenShift SCC — Security Context Constraints #ó Very important

28 ServiceAccounts & workload identity in depth Ø=βá Basics done

29 OpenShift Projects, Routes, Builds & ImageStreams #ó Pending

Next Topic Status

30 Operators & Operator Lifecycle Manager #ó Pending

31 Helm #ó Pending

32 Kustomize #ó Pending

33 GitOps / Argo CD #ó Pending

34 Prometheus, Grafana, metrics & alerting #ó Pending

35 Centralized logging & OpenTelemetry tracing #ó Pending

36 Advanced troubleshooting / production incidents #ó Pending

37 Kubernetes upgrades, HA & disaster recovery concepts #ó Pending

38 OpenShift architecture in depth #ó Pending

What Should Come Next? The best next logical chapter is:

Chapter 31 — Advanced Production Troubleshooting

CHAPTER FOCUS

Understand **Advanced Production Troubleshooting** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

Now we stop learning Kubernetes objects one by one and start thinking like a production engineer.

The goal of this chapter is:

Given a symptom, identify which layer is failing before changing anything.

The full request path is:

```
User
↓
DNS
↓
Ingress / OpenShift Route
↓
Service
↓
EndpointSlice
↓
Pod
↓
Container
↓
Spring Boot
↓
Database / External API
```

And around that path we also have:

```
CPU / Memory
Storage
NetworkPolicy
ConfigMap / Secret
Helm
Argo CD
Node health
```

The golden troubleshooting rule

Don't begin with:

```
"Restart the Pod"
```

Begin with:

```
Symptom
↓
Identify failing layer
↓
Collect evidence
↓
Form hypothesis
↓
Validate
↓
Fix root cause
↓
Verify
```

A restart can temporarily hide the problem.

Your first 5 commands

When an application is reported broken:

```
kubectl get pods -n employee-prod
```

then:

```
kubectl get deployment,svc -n employee-prod
```

then, depending on what you see:

```
kubectl describe pod <pod> -n employee-prod
```

```
kubectl logs <pod> -n employee-prod
```

```
kubectl get events -n employee-prod
```

Don't run every command blindly. Let each result tell you where to go next.

Scenario 1 — Users cannot access the app, but Pods are Running

Suppose:

employee-1	1/1	Running
employee-2	1/1	Running

employee-3	1/1	Running
------------	-----	---------

Yet:

“Employee API is unreachable.”

A beginner may say:

Pods are healthy, so Kubernetes is fine.

Not enough.

Follow the traffic chain:

```
User
↓
Route / Ingress
↓
Service
↓
Endpoints
↓
Pods
```

Check the Service

```
kubectl get svc -n employee-prod
```

Then:

```
kubectl describe svc employee-service -n employee-prod
```

Look at:

```
Selector
Port
TargetPort
Endpoints
```

Suppose:

```
Selector:
app=employee
```

but Pods have:

```
app=employee-api
```

Then:

```
Service selector
  ↓
matches zero Pods
  ↓
no endpoints
  ↓
traffic fails
```

Pods can be perfectly healthy while the Service is broken.

Check EndpointSlices

You can inspect:

```
kubectl get endpointslice -n employee-prod
```

Conceptually:

```
Service
  ↓
EndpointSlice
  ↓
10.10.1.5:8080
10.10.2.8:8080
10.10.3.4:8080
```

If there are no usable endpoints:

```
Service exists
+
Pods exist
+
traffic still fails
```

Possible causes:

```
selector mismatch
Pods not Ready
wrong labels
```

Scenario 2 — Intermittent 503 errors

Suppose users say:

Some requests work, some return 503.

Pods:

```
Pod A 1/1 Running
Pod B 1/1 Running
Pod C 0/1 Running
```

Notice:

```
Pod C
STATUS = Running
READY = 0/1
```

The first question should be:

Why isn't Pod C ready?

Run:

```
kubectl describe pod <pod-c>
```

You may see:

```
Readiness probe failed
```

Then inspect application logs.

Maybe:

```
Database connection timeout
```

The chain becomes:

```
DB dependency problem
  ↓
Readiness fails
  ↓
Pod removed from normal Service traffic
```

That is actually Kubernetes protecting you.

7. But why would there still be 503s?

Perhaps the remaining ready Pods don't have enough capacity.

Example:

```
Normally:
3 Pods

One becomes NotReady:
2 Pods

Traffic remains:
same
```

Then:

```
remaining Pods overloaded
  ↓
latency/error rate increases
```

The real incident may be:

```
dependency failure  
+  
reduced capacity
```

not simply “readiness probe issue.”

Scenario 3 — CrashLoopBackOff after a deployment

Suppose a new release goes out.

Before:

```
employee-service:2.6 '
```

After:

```
employee-service:2.7  
    ↓  
CrashLoopBackOff
```

First:

```
kubectl logs <pod> --previous
```

Suppose:

```
Could not resolve placeholder 'PAYROLL_URL'
```

Then ask:

Did the new version introduce a required configuration property?

Check:

```
ConfigMap  
Secret  
Helm values  
Deployment env
```

The root cause could be:

```
New application expects PAYROLL_URL  
    ↓  
Helm values don't provide it  
    ↓  
Spring Boot startup fails  
    ↓  
Container exits  
    ↓  
CrashLoopBackOff
```

| This is where Helm enters troubleshooting

Suppose Argo CD/Helm deployed successfully.

Do:

```
helm template ...
```

or inspect the live Deployment:

```
kubectl get deployment employee-service -o yaml
```

Ask:

Did Helm generate the expected env variable?

If not:

Helm values/template problem

If yes:

application/runtime problem

This distinction saves time.

| Scenario 4 — Argo CD says Synced, but application is broken

Very common.

Argo CD:

```
Sync:    Synced
Health:  Degraded
```

What does it mean?

```
Git configuration
=
cluster configuration
```

but:

application resources
are unhealthy

Don't waste time checking Git synchronization first.

Move down:

```
Deployment
↓
```

```
ReplicaSet
↓
Pods
↓
Logs / Events
```

Argo CD did what you asked.

Your desired configuration may itself be wrong.

| Synced “ Correct

Example:

Git:

```
image:
  tag: "2.7.0-broken"
```

Cluster:

```
2.7.0-broken
```

Argo CD:

```
Synced '
```

Application:

```
CrashLoopBackOff x
```

Argo CD is correct.

The desired state itself is bad.

Important production lesson:

Automation can faithfully deploy bad configuration.

| Scenario 5 — OOMKilled only during peak traffic

Normal time:

```
Memory = 650Mi
```

Peak:

```
Memory climbs to 1Gi
```

Container limit:


```
1Gi
```

Then:

```
OOMKilled
```

Pod restarts.

The first temptation is:

Increase memory.

But investigate first.

Possible causes:

```
legitimate higher traffic
unbounded cache
huge payloads
memory leak
too many concurrent requests
large DB result sets
JVM heap sizing
direct/native memory
```

Evidence for memory incidents

Check:

```
kubectl describe pod <pod>
```

Look for:

```
Reason: OOMKilled
```

Then:

```
kubectl top pod <pod>
```

and your monitoring graphs.

You want a pattern like:

```
Normal:
```

```
650Mi
```

```
Peak:
```

```
800Mi
```

```
900Mi
```

```
980Mi
```

```
1Gi
```

```
OOM
```

Then correlate with:

```
request rate
heap usage
GC
active threads
DB results
```

| Java-specific memory reasoning

Remember:

```
Container memory
-
├─ Java heap
├─ Metaspace
├─ Thread stacks
├─ Direct buffers
├─ JVM native memory
├─ Other allocations
```

If:

```
container limit = 1Gi
-Xmx = 1Gi
```

you have likely configured almost no headroom.

So production diagnosis should include JVM memory design, not just Kubernetes limits.

| Scenario 6 — API is slow but nothing crashes

Pods:

```
Running
Ready
0 restarts
```

Yet:

```
P95 latency = 3 seconds
```

Check:

```
CPU
Memory
application metrics
traces
DB pool
external dependencies
```

Suppose:

```
CPU limit = 1
Actual CPU constantly "H 1
```

Possible:

```
CPU throttling
```

The application isn't dead.

It's constrained.

This is why:

```
No crash
"\`
No resource problem
```

| Scenario 7 — CPU looks normal, but API is slow

Suppose:

```
CPU = 30|
Memory = 50|
```

but:

```
P95 = 5 seconds
```

Don't keep staring at Kubernetes resource metrics.

Use tracing.

Trace:

Gateway	20ms
Employee	40ms
Payroll	60ms
Database	4700ms

Now you know:

```
Infrastructure CPU
is not the bottleneck
```

```
Database latency
is
```

Move investigation to the dependency layer.

| Scenario 8 — Pod stuck Pending

Suppose:

```
employee-api          0/1    Pending
```

First:

```
kubectl describe pod employee-api
```

Events might say:

```
0/4 nodes are available:
2 Insufficient cpu
1 Insufficient memory
1 untolerated taint
```

This is not:

```
Spring Boot problem
```

The container hasn't even started.

It's:

```
Scheduling problem
```

Pending troubleshooting tree

```
Pod Pending
  ↓
describe Pod
  ↓
Events
-   ├── Insufficient CPU
  |       → requests/node capacity
-   ├── Insufficient memory
  |       → requests/node capacity
-   ├── Untolerated taint
  |       → toleration/placement
-   ├── node affinity mismatch
  |       → labels/affinity
-   └── PVC issue
        → storage chain
```

Very different causes, same high-level symptom.

Scenario 9 — PVC Pending

Suppose:

```
postgres-0  
Pending
```

and:

```
data-postgres-0  
PVC Pending
```

Follow:

```
Pod  
↓  
PVC  
↓  
StorageClass  
↓  
Provisioner  
↓  
PV/storage backend
```

Commands:

```
kubectl describe pvc data-postgres-0  
  
kubectl get storageclass
```

Look for:

```
No StorageClass  
Provisioning failed  
Insufficient storage capacity  
Topology mismatch  
Provisioner unavailable
```

Scenario 10 — Volume can't attach after Node failure

Suppose:

```
postgres-0
```

was on:

```
Node A
```

Node A fails.

Scheduler moves workload toward:

Node B

but you see:

```
FailedAttachVolume
```

Possible issue:

```
Volume still attached to Node A
```

Some block-storage backends need detach/attach transitions.

Think:

```
Pod scheduling succeeded  
but  
storage attachment failed
```

Different layer.

Scenario 11 — Service-to-service DNS failure

Spring Boot logs:

```
UnknownHostException:  
payroll-service
```

Ask:

```
Does Service exist?  
Correct namespace?  
Correct DNS name?  
Cluster DNS healthy?
```

Check:

```
kubectl get svc -n payroll
```

If Employee Pod is in:

```
employee
```

and Payroll Service in:

```
payroll
```

using only:

```
payroll-service
```

may look in the local namespace.

Use namespace-qualified naming when needed:

```
payroll-service.payroll
```

Conceptually:

```
UnknownHostException
```

↓

```
DNS/service discovery problem
```

not automatically:

```
NetworkPolicy problem
```

DNS failure vs connection refused vs timeout

These three errors give different clues.

Unknown host

```
UnknownHostException
```

Think:

```
DNS/service name
```

Connection refused

```
Connection refused
```

Think:

```
IP reachable  
but nothing listening on target port
```

Possibilities:

```
wrong targetPort  
app not listening  
wrong port
```

Timeout

```
Connection timed out
```

Think more broadly:

```
network path  
NetworkPolicy  
firewall  
dependency unavailable  
packet dropped
```

This isn't absolute, but it's a very useful diagnostic heuristic.

Scenario 12 — NetworkPolicy blocks traffic

Architecture:

```
Gateway
  ↓
Employee
```

Everything seems correct:

```
Service exists
Endpoints exist
Pods Ready
Ports correct
```

But request times out.

Now inspect NetworkPolicy.

Maybe Employee allows ingress only from:

```
app=trusted-gateway
```

but Gateway Pods have:

```
app=gateway
```

Result:

```
Policy selector mismatch
  ↓
traffic blocked
```

This can look like a mysterious networking failure.

Networking troubleshooting order

For service-to-service connectivity:

1. Service name correct?
2. Service exists?
3. DNS resolves?
4. EndpointSlice has ready endpoints?
5. targetPort correct?
6. App listening on that port?
7. NetworkPolicy allows traffic?
8. CNI/network infrastructure healthy?

Do not jump directly to CNI.

Most real issues are simpler.

| Scenario 13 — ImagePullBackOff

Pod:

```
ImagePullBackOff
```

Run:

```
kubectl describe pod <pod>
```

Possible events:

```
manifest unknown
unauthorized
pull access denied
registry timeout
```

Map them:

```
manifest unknown
→ wrong image/tag

unauthorized
→ registry credentials/imagePullSecret

timeout
→ registry/network
```

Again:

Status is only the symptom. Events give the reason.

| Scenario 14 — Readiness works locally, fails in Kubernetes

Developer says:

/actuator/health/readiness works on my laptop.

But Kubernetes says:

```
Readiness probe failed
```

Check:

```
probe port
probe path
HTTP vs HTTPS
context path
management port
startup timing
```

Example:

Spring Boot Actuator actually runs on:

```
management.server.port=9090
```

but readiness probe checks:

```
port: 8080
```

Then:

```
application is fine  
probe is wrong
```

Important distinction.

| Scenario 15 — Liveness causes restart storm

Suppose liveness checks:

```
/actuator/health
```

and that endpoint reports DOWN whenever the database is unavailable.

Database has a 5-minute outage.

Then:

```
DB down  
↓  
liveness fails  
↓  
container restarts  
↓  
DB still down  
↓  
liveness fails again  
↓  
restart storm
```

This makes the incident worse.

Better conceptual design:

```
Liveness  
→ Is JVM/application process irrecoverably stuck?
```

```
Readiness  
→ Can I serve traffic given dependencies?
```

A dependency outage often belongs in readiness, not necessarily liveness.

| Scenario 16 — Bad manual change keeps disappearing

Engineer runs:

```
kubectl edit deployment employee-service
```

changes:

```
replicas 3 → 6
```

A minute later:

```
back to 3
```

Why?

If GitOps is enabled:

```
Git = 3
Cluster = 6
  ↓
Argo CD detects drift
  ↓
self-heal
  ↓
Cluster = 3
```

Correct fix:

```
change Git
```

not fight the reconciler manually.

Scenario 17 — Helm deployment succeeded, but wrong environment

config Suppose:

```
helm upgrade
SUCCESS
```

But PROD points to DEV database.

Why?

values-prod.yaml accidentally contains:

```
DB_URL=dev-db
```

Helm succeeded because:

```
rendering and installation were valid
```

The business configuration was wrong.

Troubleshooting:

```
Helm release
```

```
↓
```

```
Rendered Deployment/ConfigMap
```

```
↓
```

```
Actual env values
```

```
↓
```

```
Application
```

Success of deployment tooling does not validate business correctness.

Scenario 18 — One Node fails

Suppose:

```
Node A
```

```
├─ employee-1
```

```
├─ employee-2
```

```
└─ payroll-1
```

Node A becomes unavailable.

Questions:

```
Are workloads managed by Deployment/StatefulSet?
```

```
Are other Nodes available?
```

```
Are resources sufficient?
```

```
Do scheduling rules permit placement?
```

```
Can storage move?
```

For stateless Deployment:

```
Node failure
```

```
↓
```

```
Pods unavailable
```

```
↓
```

```
controllers eventually replace
```

```
↓
```

```
scheduler chooses healthy Nodes
```

But replacement can fail due to:

```
resource shortage
```

```
strict affinity
```

```
taints
```

```
volume topology
```

Self-healing is not magic.

It requires valid infrastructure capacity.

Why anti-affinity/topology matters here

Suppose 3 replicas all run on Node A:

```
Node A
├─ employee-1
├─ employee-2
└─ employee-3
```

Node fails:

```
all replicas lost
```

If distributed:

```
Node A → employee-1
Node B → employee-2
Node C → employee-3
```

one Node failure leaves:

```
2 replicas still serving
```

This connects scheduling directly to production availability.

Scenario 19 — Everything looks healthy, but only some requests fail

Suppose:

```
3 Pods
```

and exactly one Pod has a bad configuration.

Example:

```
Pod A '
Pod B '
Pod C × talks to wrong DB
```

Load balancing gives users:

```
A → success
B → success
C → failure
```

Symptom:

```
intermittent failures
```

This should make you ask:

Is every replica really identical?

Compare:

```
image
ConfigMap
Secret
environment
mounted files
```

Normally a Deployment's replicas should be consistent, but rollout/configuration timing can create transitional differences.

Scenario 20 — Problems started immediately after release

This is one of the most useful incident clues.

Timeline:

```
14:00 release
14:02 errors begin
```

Strong hypothesis:

```
release-related regression/configuration change
```

Check:

```
Argo CD history
Helm revision
Deployment rollout history
image version
config diff
```

Not proof—but very strong evidence.

Production troubleshooting is often about correlating:

```
when symptom started
```

with:

```
what changed
```

The three most important questions in an incident

Always ask:

1. What changed?

Deployment?
Config?
Secret?
Infrastructure?
Traffic?
Dependency?

2. What is the blast radius?

One Pod?
One Service?
One Namespace?
One Node?
Whole cluster?

3. Which layer first shows failure?

Ingress?
Service?
Pod?
App?
Database?

These three questions quickly narrow the search space.

Blast-radius reasoning

Suppose:

Only employee-service broken

but:

payroll
payments
gateway

are healthy.

Less likely:

entire cluster networking failure

More likely:

employee-specific Deployment
config
Service
dependency

If:

many unrelated services fail

on the same Node:

Node-level issue

becomes more plausible.

If all namespaces experience DNS failures:

cluster DNS

becomes more plausible.

Use blast radius to identify the layer.

Troubleshooting from broad → narrow

A useful production sequence:

```
Is cluster reachable?
  ↓
Is namespace/application present?
  ↓
Is Deployment healthy?
  ↓
Are Pods scheduled?
  ↓
Are containers running?
  ↓
Are Pods Ready?
  ↓
Does Service have endpoints?
  ↓
Does DNS work?

  ↓
Does network path work?
  ↓
Is application healthy?
  ↓
Are dependencies healthy?
```

This avoids random debugging.

Your symptom-to-first-check table

Symptom First thing to inspect

Pod Pending describe pod / scheduling Events

CrashLoopBackOff logs + -previous

ImagePullBackOff Pod Events

OOMKilled resources + JVM memory

Running but 0/1 Ready readiness probe

Service unreachable selector + EndpointSlice

UnknownHostException Service DNS/name/namespace

Connection refused port/listener/targetPort

Connection timeout network path/policy/dependency

PVC Pending PVC/StorageClass/provisioner

Manual changes revert GitOps/Argo CD

Slow but no crash metrics/traces/resources

Intermittent failures compare replicas/dependencies

This is worth retaining.

Evidence hierarchy

When troubleshooting, different evidence answers different questions.

```
kubectl get  
→ current state  
  
kubectl describe  
→ Kubernetes explanation/events  
  
kubectl logs  
→ application explanation  
  
Prometheus/Grafana  
  
→ trend and timing  
  
Tracing  
→ failing/slow dependency  
  
Argo CD/Git  
→ configuration/change history
```

Use the evidence source that matches the question.

A complete incident walkthrough

Incident:

Employee API has 20% HTTP 500 errors.

Step 1 — Metrics

HTTP 500 increased at 14:05

Step 2 — Change history

New release deployed at 14:00

Step 3 — Pods

3/3 Running, Ready

So infrastructure health looks normal.

Step 4 — Tracing Failed requests show:

Employee → Payroll call returning 401

Step 5 — Logs

Invalid payroll API credential

Step 6 — Config New Helm values changed Secret reference.

Root cause:

```
release
↓
wrong Secret reference
↓
Payroll auth failure
↓
HTTP 500
```

Notice how kubectl get pods alone would have told you almost nothing.

Another complete incident

Incident:

Application occasionally becomes unavailable for 30 seconds.

Metrics:

```
Memory climbs repeatedly
↓
drops suddenly
```

Pods:

restart count increasing

Describe:

OOMKilled

JVM metrics:

heap continuously grows

Logs:

large report generation

Root cause hypothesis:

```
large report workload
↓
high memory consumption
↓
container memory limit exceeded

↓
OOMKilled
↓
temporary capacity loss
```

Now you can decide whether the correct fix is:

```
application optimization
memory sizing
workload redesign
streaming
separate batch processing
```

rather than blindly restarting.

Production troubleshooting mindset

Don't ask:

"Which Kubernetes command fixes this?"

Ask:

"What evidence proves which layer is failing?"

That is the difference between command memorization and engineering.

What you've now connected

You can now use almost everything we've learned:

```
Deployment
Service
Ingress/Route
```

```
DNS
NetworkPolicy
ConfigMap
Secret
PVC
StorageClass
Scheduler
Requests/Limits
Probes
Helm
Kustomize
Argo CD
Prometheus
Grafana
Logs
Tracing
```

as parts of one troubleshooting system.

chapter 36 Summary

For production incidents, remember:

```
Symptom
↓
Blast radius
↓
Recent changes
↓
Identify layer
↓
Collect evidence
↓
Find root cause
```

And this layered model:

```
Git / Helm / Argo CD
↓
Kubernetes resource definition
↓
Scheduling
↓
Pod / Container
↓
Service / Networking
↓
Application
↓
Dependencies
↓
User experience
```

The higher layer can be perfectly healthy while a lower layer is failing.

Next chapter — chapter 37: Kubernetes/OpenShift High Availability, Upgrades & Disaster Recovery The next logical chapter is more platform-oriented.

We✓I learn:

```
Control-plane HA
Multiple API servers
etcd quorum
Worker-node failure
Availability zones
Pod disruption
Cluster upgrades
draining Nodes
cordon / drain
etcd backup
application backup

disaster recovery
RPO / RTO
```

Most importantly, we✓I answer:

If a Kubernetes control-plane Node dies, why doesn't the whole cluster immediately stop?

and:

What exactly must be backed up to recover a Kubernetes/OpenShift environment?

We'll move into the platform side now: how Kubernetes/OpenShift stays available during failures, maintenance, upgrades, and disaster recovery. I'll connect control-plane HA, etcd quorum, node drain, PDBs, backups, RPO/RTO, and recovery into one end-to-end mental model.

Availability and Integrated Production Architecture

Chapter 32 — High Availability, Upgrades, and Disaster Recovery

CHAPTER FOCUS

Understand **High Availability, Upgrades, and Disaster Recovery** through the problem it solves, a visual mental model, internal behavior, practical configuration, and production troubleshooting.

Recovery This chapter is about a different level of Kubernetes knowledge.

Until now, we mostly asked:

How does my application survive a Pod failure?

Now we're asking:

How does the whole platform survive Node failures, control-plane failures, upgrades, and even loss of the cluster?

The most important distinction for this chapter is:

High Availability

"`

Disaster Recovery

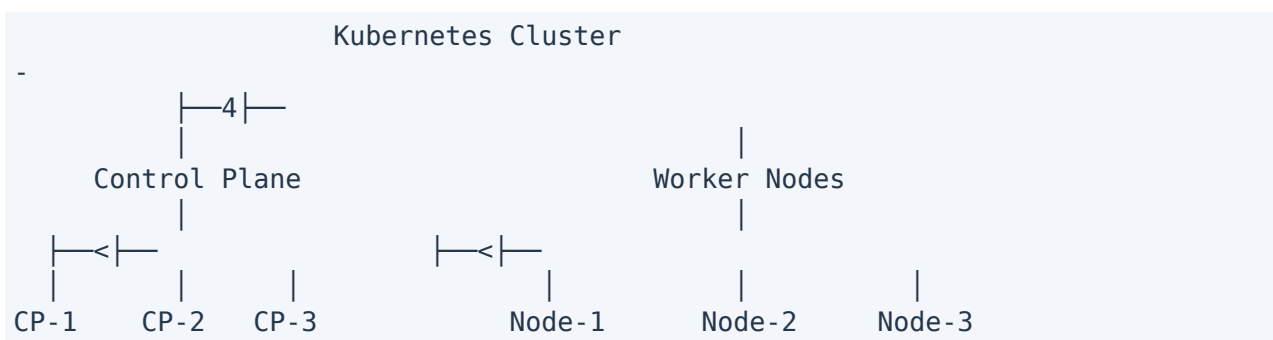
"`

Backup

They solve different problems.

Start with a production cluster

Imagine:



Why three control-plane Nodes?

Because running only:

1 control-plane Node

creates a major single point of failure.

If that server dies:

API Server ×
 Scheduler ×
 Controllers ×
 etcd potentially ×

The cluster's management plane can become unavailable.

Production clusters therefore typically use multiple control-plane instances.

2. What does Control Plane HA mean?

Recall the components:

API Server
 Scheduler
 Controller Manager
 etcd

They don't all achieve HA in exactly the same way.

That distinction is important.

API Server HA

The API Server can run on multiple control-plane Nodes.

Example:

CP-1 → API Server
 CP-2 → API Server
 CP-3 → API Server

Clients usually reach them through some form of load-balanced API endpoint:

```
kubectl / oc
-
  | 1/4
API Load Balancer
-
|<|
| 1/4 | 1/4 | 1/4
API 1 API 2 API 3
```

If API Server 1 dies:


```
API-1 x
API-2 '
API-3 '
```

requests can still be served by the others.

| Scheduler HA is different

Suppose three control-plane Nodes all run scheduler processes.

You don't want:

```
Scheduler 1
Scheduler 2
Scheduler 3
```

all independently making conflicting decisions for the same Pod.

Instead, Kubernetes components use leader election.

Conceptually:

```
Scheduler-1 → LEADER
Scheduler-2 → standby
Scheduler-3 → standby
```

The leader performs the active scheduling work.

If it fails:

```
Scheduler-1 x
      ↓
leader election
      ↓
Scheduler-2 becomes leader
```

Same broad idea applies to controller-manager HA.

| API Server vs Scheduler HA

Remember:

```
API Servers
→ multiple can serve API requests
```

while:

```
Scheduler / Controller Manager
→ multiple instances can exist
→ leader election chooses active leader
```

Good interview distinction.

| Now the most important component: etcd

etcd stores Kubernetes cluster state.

Remember:

```
API Server
  ↓
etcd
  ↓
Deployments
Services
Secrets
ConfigMaps
Nodes
Pods
RBAC
etc.
```

Because etcd stores such critical state, its availability is fundamental.

| etcd uses quorum

Suppose you have three etcd members:

```
etcd-1
etcd-2

etcd-3
```

A majority must be available to maintain quorum.

For three members:

```
Quorum = 2
```

Therefore:

```
etcd-1 '
etcd-2 '
etcd-3 x
```

still works.

But:

```
etcd-1 '
etcd-2 x
etcd-3 x
```

loses quorum.

The control plane can no longer safely perform normal state changes.

| Quorum formula

A useful simplified formula is:

```
Quorum = floor(N / 2) + 1
```

Examples:

etcd members Quorum Failures tolerated

1 1 0

3 2 1

5 3 2

This is why odd numbers such as:

```
3
5
```

are normally preferred.

Adding a fourth etcd member does not increase failure tolerance over three in the same way you'd expect.

9. Why is etcd different from a normal stateless component?

API Server instance dies:

```
another API Server can answer
```

But etcd contains consensus-based state.

It cannot simply say:

“I have one surviving copy, so let's continue making changes.”

Otherwise network partitions could lead to conflicting cluster states.

Quorum protects consistency.

10. What happens if etcd loses quorum?

Conceptually:

```
etcd quorum lost
  ↓
Control plane cannot reliably persist/update state
  ↓
Deployments cannot be normally changed
New Pods cannot be normally scheduled
Controllers cannot reliably reconcile state
```

But an important point:

Running Pods on worker Nodes don't instantly disappear just because etcd/control plane becomes unavailable.

| Control plane failure vs running workloads

Suppose your application Pods are already running:

```
Node 1 → employee-1
Node 2 → employee-2
Node 3 → employee-3
```

Then the control plane becomes unavailable.

The worker Nodes don't immediately say:

```
"API Server is gone. Kill everything."
```

Existing containers can continue running.

So:

```
Existing traffic
→ may continue
```

depending on networking and application dependencies.

But Kubernetes loses much of its management/self-healing capability.

| Example

Control plane unavailable:

```
employee-1 '
employee-2 '
employee-3 '
```

Traffic may continue.

But then:

```
employee-2 crashes
```

Normally:

```
Deployment
↓
ReplicaSet notices
↓
replacement Pod
↓
Scheduler
```

↓
new Pod

Without a functioning control plane:

replacement may not happen

So existing workloads can survive temporarily, but the cluster cannot properly adapt.

High Availability isn't "nothing ever fails"

This is the correct HA mindset:

Failure happens
↓
Enough redundant components remain
↓
Service continues

Not:

Failure never happens

HA engineering assumes components will fail.

Worker Node failure

Suppose:

Node A
├─ employee-1
├─ payroll-1
└─ gateway-1

Node A dies.

The control plane eventually recognizes the Node as unavailable.

Managed workloads can be recreated elsewhere:

employee-1 replacement → Node B
payroll-1 replacement → Node C
gateway-1 replacement → Node B

provided:

other Nodes have capacity
scheduling rules permit it
storage can move

Kubernetes can't create capacity out of nowhere

Suppose:

3 Nodes

are all already near full capacity.

One fails.

Now surviving Nodes don't have enough:

CPU
memory

for replacement Pods.

Kubernetes may create replacement Pod objects, but they remain:

Pending

with:

Insufficient cpu

or:

Insufficient memory

So HA requires spare capacity planning.

Multi-zone availability

Now imagine cloud/infrastructure topology:

```
Region
-
├─ Zone A
│   ├── Node 1
│   └── Node 2
-
├─ Zone B
│   ├── Node 3
│   └── Node 4
-
└─ Zone C
    ├── Node 5
    └── Node 6
```

If all three replicas of your application are in Zone A:

```
Zone A
├─ employee-1
```

```
|— employee-2  
|— employee-3
```

then Zone A failure means:

```
all replicas ×
```

Better:

```
Zone A → employee-1  
Zone B → employee-2  
Zone C → employee-3
```

This connects directly to the topology/anti-affinity concepts you already learned.

HA needs more than Pod replicas

You might say:

```
replicas: 3
```

and assume:

Highly available.

Not necessarily.

All three might be placed poorly.

True availability depends on:

```
replica distribution  
node diversity  
zone diversity  
load balancing  
storage design  
dependency HA  
capacity
```

For example:

```
Application = highly available  
Database = single instance
```

still leaves a single point of failure.

Voluntary vs involuntary disruption

This distinction becomes important.

Involuntary disruption Something unexpectedly fails:

```
Node crash
hardware failure
zone outage
kernel failure
```

Voluntary disruption You intentionally disrupt workloads:

```
Node maintenance
cluster upgrade
drain
planned replacement
```

Kubernetes provides mechanisms to make voluntary disruptions safer.

| **cordon**

Suppose you want to perform maintenance on:

```
worker-3
```

First:

```
kubectl cordon worker-3
```

Conceptually:

```
worker-3
  ↓
Unschedulable for new normal workloads
```

Existing Pods generally keep running.

So:

```
cordon
=
Don't place new Pods here
```

Not:

```
delete existing Pods
```

| **drain**

Next you may run:

```
kubectl drain worker-3 ...
```

Conceptually:


```
Node
↓
evict movable workloads
↓
controllers create replacements elsewhere
↓
Node becomes ready for maintenance
```

Think:

```
cordons
→ stop new scheduling

drains
→ safely evacuate appropriate workloads
```

21. Why not simply shut down the Node?

Suppose:

```
employee-service
replicas = 3
```

and Node contains two replicas.

Instant shutdown:

```
2 replicas vanish simultaneously
```

A controlled drain allows Kubernetes to coordinate workload movement before maintenance.

That is much safer.

DaemonSets behave differently

Remember:

```
DaemonSet
=

one Pod per eligible Node
```

During normal drain operations, DaemonSet-managed Pods receive special handling rather than being treated like ordinary Deployment Pods.

Why?

Because their purpose is tied to the Node itself.

Examples:

```
logging agent
monitoring agent
CNI agent
```

When the Node returns, the DaemonSet ensures its Pod exists again.

| PodDisruptionBudget — PDB

This is an important concept we skipped earlier.

Suppose:

```
employee-service
replicas = 5
```

You don't want planned maintenance to take down too many simultaneously.

You can define:

```
minAvailable: 4
```

Conceptually:

```
5 Pods
```

```
During voluntary disruption:
at least 4 should remain available
```

That is:

PodDisruptionBudget

| PDB example

```
apiVersion: policy/v1
kind: PodDisruptionBudget

metadata:
  name: employee-pdb

spec:
  minAvailable: 4

  selector:
    matchLabels:
      app: employee
```

Think:

```
PDB
=
availability guardrail during voluntary disruption
```

| minAvailable vs maxUnavailable

You can express either style.

Example

```
minAvailable = 4
```

means:

Keep at least 4 available.

Or:

```
maxUnavailable = 1
```

means:

Allow at most 1 unavailable due to covered voluntary disruptions.

For five replicas these may express similar intent.

| Important: PDB does NOT stop hardware failure

Suppose all five Pods happen to be on a Node that suddenly loses power.

PDB cannot tell the failed hardware:

“Please wait, only one Pod may go down.”

PDB primarily helps coordinate voluntary disruptions, such as eviction during maintenance.

So:

```
PDB
" `
protection against every failure
```

You still need good replica placement.

| Now cluster upgrades

Production Kubernetes/OpenShift needs upgrades for:

```
security fixes
bug fixes
```

```
platform improvements
supported-version lifecycle
```

But you can't simply:

```
turn cluster off
upgrade everything
turn it back on
```

for critical environments.

Upgrades should preserve availability.

| Simplified upgrade flow

Conceptually:

1. Validate cluster health
2. Ensure backups/recovery readiness
3. Upgrade control-plane/platform components
4. Upgrade worker Nodes progressively
5. Verify workloads
6. Continue to next Node/group

Exact procedures are platform/version specific, but the HA principle is stable:

Upgrade incrementally rather than disrupting everything simultaneously.

| Worker Node rolling upgrade

Imagine:

```
Node 1
Node 2
Node 3
```

Upgrade:

```
Node 1
↓
cordn
↓
drain
↓
upgrade
↓
return Node
↓
uncordon
```

Then:

Node 2

then:

Node 3

This allows workloads to remain on surviving Nodes.

30. What does uncordon do?

After maintenance:

```
kubectl uncordon worker-3
```

means:

Node can receive new scheduled workloads again

So lifecycle:

```
Normal
↓
cordon
↓
drain
↓
maintenance
↓
uncordon
↓
Normal
```

| Application readiness matters during upgrades

Imagine Kubernetes moves a Pod:

```
old Pod deleted
↓
new Pod starts
```

If the new Spring Boot Pod takes:

60 seconds

to become operational, you don't want traffic immediately.

This is where:

```
startupProbe
readinessProbe
```

protect availability.

So upgrades connect to probes too.

| Graceful shutdown matters too

When draining a Node, a Pod may receive termination.

Your application should stop accepting new traffic and finish reasonable in-flight work before exiting.

Conceptually:

```
Pod termination begins
    ↓
Pod becomes unavailable for new traffic
    ↓
application receives termination signal
    ↓
finish/stop work
    ↓
container exits
```

This is why graceful shutdown and `terminationGracePeriodSeconds` matter.

We skipped their dedicated chapter earlier, but this is where their value becomes clear.

| High Availability vs Disaster Recovery

Now an extremely important distinction.

High Availability Handles expected component failures:

```
one Pod fails
one Node fails
one control-plane instance fails
```

Goal:

```
keep service running
```

Disaster Recovery Handles major catastrophic events:

```
entire cluster lost
datacenter lost
region lost
critical storage corruption
severe operator mistake
```

Goal:

```
recover service and data
```

| HA is NOT DR

Example:

You have:

```
3 control-plane Nodes
10 worker Nodes
```

Very good HA.

But someone accidentally destroys the entire cluster.

Your redundant Nodes don't help because:

```
all gone
```

Now you need:

```
backups
recovery procedures
external copies
rebuild automation
```

That is DR.

35. What must actually be backed up?

This is one of the most important questions in the course.

A production Kubernetes/OpenShift environment has multiple categories of state:

```
Cluster state
Application configuration
Application data
Container artifacts
External systems
```

You need to think about each separately.

| Category 1 — Kubernetes cluster state

Critical Kubernetes API state is stored in etcd.

An etcd backup can include resources such as:

```
Namespaces
Deployments
Services
RBAC
```

```
ConfigMaps  
Secrets  
Custom Resources  
etc.
```

So:

```
etcd backup  
=  
cluster control-plane state backup
```

But that's only one part of DR.

37. Does etcd backup contain database data?

No.

Suppose:

```
PostgreSQL Pod  
↓  
PVC  
↓  
500GB database
```

etcd contains the Kubernetes objects describing things such as the PVC.

It does not contain the actual 500 GB database contents.

So:

```
etcd snapshot  
" "  
persistent volume data backup
```

Extremely important.

Category 2 — Application persistent data

Your application might store data in:

```
PostgreSQL  
Oracle  
Kafka  
PVC  
object storage
```

These require their own backup/replication strategies.

For example:


```
database-native backup
volume snapshot
storage replication
object-store versioning
```

depending on architecture.

| Category 3 — Git/GitOps configuration

If you're using GitOps:

```
Git
↓
Helm/Kustomize
↓
Argo CD
```

much of your declarative configuration is already preserved in Git.

This is extremely helpful for rebuilding a cluster.

Conceptually:

```
New cluster
↓
Install platform services
↓
Install Argo CD
↓
Connect Git
↓
Applications reconstructed
```

But application data still needs restoration separately.

| Category 4 — Container images

Your application images live in a registry:

```
employee-service:2.7
payroll-service:4.1
```

If the cluster is destroyed but the registry remains available:

```
new cluster
↓
pull same images
```

Good.

If your registry is also destroyed and there is no replicated/backup artifact repository, recovery becomes harder.

So artifacts are part of recovery planning too.

| Category 5 — External systems

Your Kubernetes application may depend on:

```
external Oracle DB
Vault
cloud database

external Kafka
DNS
certificate authority
identity provider
```

Kubernetes backup doesn't automatically recover those systems.

A real DR plan covers the entire application dependency graph.

| The recovery stack

Think of DR as layers:

```
Application
-
  |  $\frac{1}{4}$ 
Kubernetes resources
-
  |  $\frac{1}{4}$ 
Persistent data
-
  |  $\frac{1}{4}$ 
Platform infrastructure
-
  |  $\frac{1}{4}$ 
External services
```

Recovery fails if one critical layer is missing.

| RPO

Now two essential DR terms.

RPO — Recovery Point Objective Question:

How much data can we afford to lose?

Example:

```
RPO = 15 minutes
```

means your recovery design should aim for no more than roughly 15 minutes of data

loss.

| RPO example

Database backup:

12:00
12:15
12:30
12:45

Disaster:

12:52

If your last recoverable point is:

12:45

you might lose:

7 minutes

That is within a 15-minute RPO.

| RTO

RTO — Recovery Time Objective Question:

How long can the service remain unavailable before it needs to be recovered?

Example:

RT0 = 1 hour

means:

We aim to restore service within one hour of the disaster.

| RPO vs RTO

Memorize:

RPO
=
How much DATA can I lose?

RT0
=
How much TIME can I be down?

Very common interview question.

| Example business requirements

Critical payment system:

```
RP0 = 1 minute
RT0 = 10 minutes
```

Internal reporting system:

```
RP0 = 24 hours
RT0 = 8 hours
```

The first architecture will be much more expensive and complex.

DR requirements are business decisions, not just Kubernetes settings.

| Backup isn't useful unless restore works

An extremely important lesson:

```
"We have backups"
```

isn't sufficient.

You need:

```
Can we restore them?
How long does restore take?
Is the backup corrupted?
Does the application work afterward?
```

So organizations perform:

```
restore testing
DR drills
```

| Example cluster-loss recovery

Imagine the entire cluster is lost.

A possible conceptual recovery path:

```
Infrastructure recreated
  ↓
Kubernetes/OpenShift restored/reinstalled
  ↓
Control-plane state restored where applicable
  ↓
Platform services restored
  ↓
GitOps configuration reapplied
  ↓
```

```
Persistent application data restored
↓
External integrations validated
↓
Applications verified
↓
Traffic restored
```

The exact method depends heavily on your platform and DR design.

| etcd restore vs rebuilding through GitOps

Interesting distinction.

If you have a valid etcd backup, you may restore control-plane state.

But organizations may also design clusters to be largely reproducible from:

```
Infrastructure as Code
+
GitOps
+
external data backups
```

Conceptually:

```
Create clean cluster
↓
GitOps rebuilds apps/resources
↓
restore persistent data
```

Neither approach means Git replaces etcd.

They solve different recovery layers.

| OpenShift context

In OpenShift, the same principles apply:

```
Control-plane redundancy
etcd quorum
worker replacement
Projects
Routes
PVCs
Operators
GitOps
```

OpenShift provides platform-specific procedures/tooling for backup, upgrade, and recovery, but the mental model remains:

```
Cluster state
+
application state
+
persistent data
+
external dependencies
```

all need consideration.

52. Failure scenarios — what protects you?

Failure Protection

One container crashes kubelet restart

One Pod disappears workload controller

One worker Node dies replicas + scheduler + spare capacity

One zone dies multi-zone placement + dependency HA

One API server dies multiple API servers

Scheduler leader dies leader election

One etcd member dies in 3-node cluster remaining quorum

Planned Node maintenance cordon + drain + PDB

Entire cluster destroyed DR/backups/rebuild process

Database corruption database backup/replication

Accidental config change Git/history/rollback

This table ties most of Kubernetes together.

A tricky interview scenario

We have three control-plane Nodes. One control-plane Node dies. Does the cluster stop?

Generally:

```
No
```

if the remaining control-plane components and etcd quorum are healthy.

For three etcd members:

```
3 → lose 1 → 2 remain
```

Quorum remains.

Therefore control-plane operations can continue.

| Another scenario

Two of three etcd members die.

Now:

```
1 / 3 remains
```

But quorum requires:

```
2
```

So:

```
etcd quorum lost
```

and the control plane cannot normally continue managing state.

Running application containers may continue temporarily, but the Kubernetes management plane is severely impaired.

| Another interview question

Does having three Pods guarantee HA?

No.

You should explain:

Availability also depends on where those Pods are scheduled, Node/zone failure domains, Service/load-balancer availability, storage, application dependencies, database architecture, capacity and disruption policies. Excellent answer.

| Another interview question

What's the difference between cordon and drain?

```
cordon
```

```
→ stop new normal Pods from being scheduled onto the Node
```

```
drain
```

```
→ evict appropriate existing workloads so the Node can be maintained
```

Then:

```
uncordon
```

```
→ make the Node schedulable again
```

| Another interview question

What is a PodDisruptionBudget?

A PDB limits how many Pods of a selected workload may be voluntarily disrupted at the same time, using `minAvailable` or `maxUnavailable` to help maintain application availability during operations such as Node drain.

And add:

It does not prevent unexpected infrastructure failures.

Another interview question

Does an etcd backup back up my PVC data?

No.

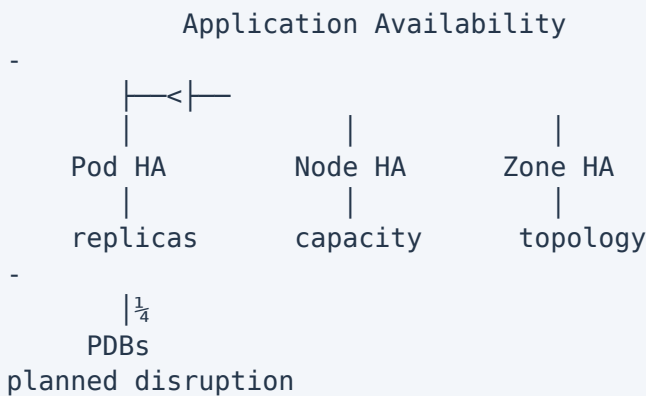
etcd
→ Kubernetes API/cluster state

PVC's actual database/files
→ separate storage/data backup

Very important.

Your HA/DR mental model

I want you to remember this hierarchy:



Then platform:

Control Plane HA
-
| API Server redundancy
| Scheduler leader election
| Controller leader election
| etcd quorum

Then disaster recovery:

DR
-
| etcd/cluster state

- |— Git configuration
- |— application data
- |— storage backups
- |— container artifacts
- |— external dependencies

chapter 37 Summary

The most important concepts:

HA

→ survive component failures

DR

→ recover from major disasters

etcd quorum

→ majority required

3 etcd members

→ tolerate 1 member failure

cordon

→ stop new scheduling

drain

→ evacuate appropriate workloads

PDB

→ protect availability during voluntary disruption

RP0

→ acceptable data loss

RT0

→ acceptable downtime

And perhaps the biggest lesson:

Backing up Kubernetes cluster state is not the same as backing up application data.

Next chapter — chapter 38: OpenShift Architecture in Depth This is a good point to shift from generic Kubernetes to the platform you're specifically learning.

We✓I take everything you already know and map it into OpenShift:

Kubernetes Namespace → OpenShift Project

Ingress Controller → OpenShift Ingress/Router

Ingress → Route

kubectl → oc

RBAC → OpenShift RBAC

Pod Security → SCC

Container Registry → OpenShift image capabilities

Kubernetes controllers

+

Operators

+

OpenShift platform components

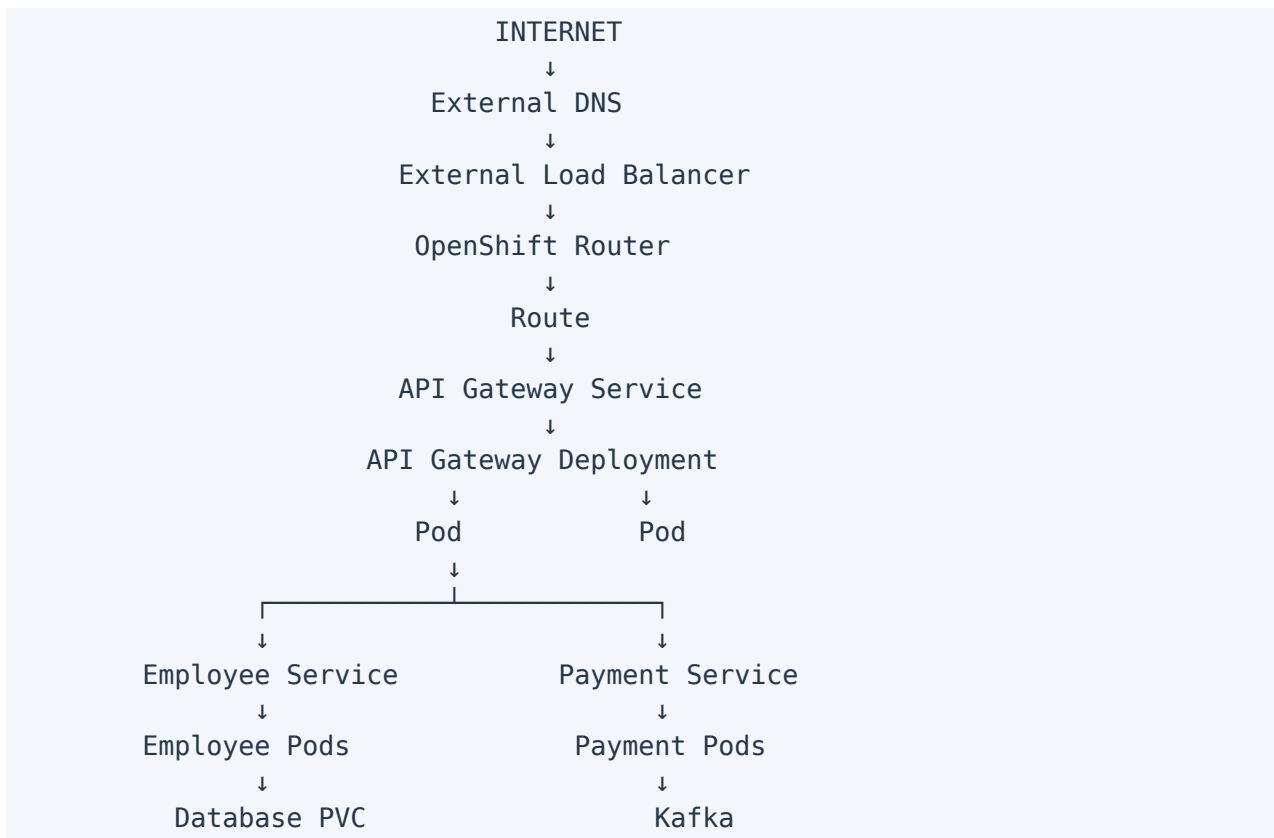
Most importantly, we✓I answer:

What exactly does OpenShift add on top of Kubernetes, and why would an enterprise use OpenShift instead of plain Kubernetes?

We'll now shift from "Kubernetes concepts" to how OpenShift packages and operates those concepts as an enterprise platform. I'll keep the focus on the architecture and on what OpenShift adds, rather than re-teaching Pods, Services, RBAC, or storage.

Chapter 33 — Complete Production Architecture

This final architecture connects the individual concepts into one operating system for applications.



Delivery lifecycle

Developer → Git → CI build and test → Container image
→ Registry / ImageStream → GitOps repository
→ Argo CD → Kubernetes API → Deployment
→ ReplicaSet → Pods → Service → Route → Users

The CI system proves and packages a version. GitOps records the version that should run. Argo CD reconciles Git with the cluster. Kubernetes controllers create workload objects, the scheduler chooses Nodes, kubelets ask the runtime to start containers, and networking objects connect users to ready endpoints.

Cross-cutting controls

ConfigMaps hold non-sensitive settings such as log levels and service URLs. Secrets or an external manager such as Vault supply credentials. Probes control startup, restart, and traffic eligibility. Requests influence placement; limits constrain runtime consumption. HPA changes replicas in response to metrics. NetworkPolicies restrict traffic paths. RBAC controls API actions, while SecurityContext and OpenShift SCC constrain workload privileges.

Persistent data follows the mount-to-storage chain: application path → Pod volume → PVC → PV → storage backend. StatefulSets add stable identity and per-replica claims when application membership depends on identity.

Metrics describe trends and resource behavior, logs describe events, and traces follow one request across services. Together they provide the evidence used by production troubleshooting.

A layer-by-layer diagnostic path

When a user receives a 503, do not begin by restarting random Pods. Follow the request path:

```
User → DNS → Load Balancer → Router/Ingress → Route/Ingress
      → Service → Endpoint → Pod → Container → Application → Dependency
```

At each layer, compare desired state with current evidence. Confirm DNS resolution, external reachability, routing rules, Service selectors, ready endpoints, container health, application listeners, and dependency connectivity.

Final production questions

1. Where is desired application configuration stored in GitOps, and where is live cluster state stored?
2. Which component decides the Node, and which component starts the container there?
3. Why can a Service remain stable while its Pods change?
4. Which controls protect the API, the network, and the container process respectively?
5. Which evidence source—metrics, logs, or traces—best follows one request across microservices?

Answers and explanations

1. Git stores the reviewed declarative configuration; etcd stores Kubernetes API state. Argo CD connects the two reconciliation systems.
2. The scheduler selects the Node; the Node's kubelet coordinates with the container runtime.

3. The Service selects Pods by labels and routes through a changing endpoint set rather than binding its identity to one Pod.
4. RBAC protects API actions, NetworkPolicy controls allowed network paths, and SecurityContext/SCC constrain process privileges.
5. A distributed trace, composed of correlated spans, follows the request end to end.

Kubernetes & OpenShift Interview Guide

Beginner

What is Kubernetes? A control-plane-driven platform that reconciles declared application and infrastructure-related resources across a cluster of Nodes.

What is a Pod? The smallest schedulable Kubernetes workload unit. Its containers share the Pod network namespace and may share volumes.

Why use a Deployment? It owns rollout and rollback behavior and manages ReplicaSets, which maintain Pods.

Why use a Service? It gives clients stable discovery and routing in front of replaceable Pods.

Intermediate

How does a Service discover Pods? Its selector matches Pod labels; the control plane maintains endpoint data for eligible Pods.

Deployment versus StatefulSet? A Deployment treats replicas as interchangeable. A StatefulSet provides stable ordinal identity and can provide per-replica persistent claims.

PV versus PVC? A PV represents provisioned storage; a PVC is a workload's request for storage. A StorageClass can enable dynamic provisioning.

Requests versus limits? Requests guide scheduling and resource guarantees; limits constrain runtime consumption. Excess memory can cause OOMKilled, while excess CPU is normally throttled.

Advanced

What happens after a Deployment is applied? The API server authenticates, authorizes, validates, and persists the object. Controllers create a ReplicaSet and Pods, the scheduler binds pending Pods to Nodes, and kubelets coordinate container startup.

What happens during a Node failure? The control plane eventually marks the Node unhealthy. Controllers create replacement Pods elsewhere when their workload policy and storage constraints allow it. The failed Node's local data is not magically recovered.

How does GitOps differ from traditional CI/CD? CI builds and verifies artifacts. GitOps uses a controller to continuously reconcile reviewed declarations in Git with the cluster, including drift detection.

How do Operators work? They extend the API and run domain-specific controllers that reconcile Custom Resources into application and infrastructure actions.

| Troubleshooting

Pod Pending: inspect events, requests, taints, affinity, quotas, and PVC binding.

CrashLoopBackOff: inspect current and previous logs, exit codes, configuration, dependencies, and probes.

OOMKilled: compare observed memory with requests and limits, then investigate application allocation and leaks before merely increasing the limit.

Service not routing: verify selector, Pod labels, EndpointSlices, ports, readiness, NetworkPolicy, and the process listener.

PVC Pending: inspect the claim, StorageClass, provisioner, requested size/access mode, topology, and events.

Route returns 503: trace router → Route → Service → ready endpoints → target port → application.

| OpenShift

Project versus Namespace: a Project is OpenShift's developer-facing abstraction around a Kubernetes Namespace with integrated policy and user experience.

Route versus Ingress: both express external routing intent; Route is an OpenShift API consumed by the OpenShift Router and includes OpenShift-specific TLS and traffic features.

What is SCC? An OpenShift policy mechanism that determines which security settings a Pod may use and under which service account.

Image versus ImageStream: an image is the packaged artifact; an ImageStream tracks image references/tags inside OpenShift and can drive triggers.

Kubernetes & OpenShift Final Revision Guide

One-page Kubernetes mental model

```
kubectl/oc → API Server → etcd
                        ↓
                    Controllers
                        ↓
Pending Pod → Scheduler → Node
                        ↓
                kubelet → runtime → container
```

Desired state enters through the API. Controllers create and repair resources. The scheduler chooses placement. The kubelet realizes assigned Pod state through the runtime.

One-page networking mental model

```
External user → DNS → Load Balancer → Ingress Controller / Router
                → Ingress / Route → Service → ready Pod endpoint
```

Pod IPs identify individual ephemeral workloads. Service IP/DNS provides stable in-cluster identity. Node IP identifies a machine. NetworkPolicy limits allowed Pod traffic but does not replace routing.

One-page storage mental model

```
Application path → volumeMount → Pod volume → PVC → PV → storage backend
```

The writable container layer follows container lifetime. emptyDir follows Pod lifetime. Persistent storage is intentionally externalized from both.

One-page scheduling mental model

```
Pending Pod → feasible Nodes → requests + labels + affinity + taints +
↪ topology
                → score candidates → bind selected Node
```


| One-page security mental model

Human/workload identity → authentication → RBAC authorization → API action
Pod admission → Pod Security / SCC → permitted runtime identity and privileges
NetworkPolicy → permitted network paths

| One-page OpenShift mental model

Kubernetes core
+ developer Projects and Routes
+ integrated authentication, registry, monitoring, and console
+ Operators and platform lifecycle management

| Resource hierarchy

```
Cluster
├── Control plane: API server, etcd, scheduler, controllers
├── Nodes
│   └── Pods
│       └── Containers
├── Workload controllers
│   ├── Deployments → ReplicaSets → Pods
│   ├── StatefulSets → stable Pods + PVCs
│   ├── DaemonSets → node-local Pods
│   └── Jobs / CronJobs
├── Networking: Services, EndpointSlices, Ingress, NetworkPolicies
├── Configuration: ConfigMaps, Secrets
├── Storage: PVCs, PVs, StorageClasses
└── Security: ServiceAccounts, Roles, Bindings, admission policies
```

Glossary

Affinity: A scheduling rule that attracts a Pod to Nodes or other Pods matching selected properties.

API Server: The authenticated, authorized, validated entry point to the Kubernetes API.

Cluster: A Kubernetes control plane and its registered Nodes and resources.

ClusterIP: The default Service type, reachable through a stable virtual IP inside the cluster.

ConfigMap: A Kubernetes object for non-sensitive configuration data.

Container: An isolated application process environment created from an image and sharing the host kernel.

Controller: A control loop that reconciles current state toward desired state.

CRD: A CustomResourceDefinition that extends the Kubernetes API with a new resource type.

DaemonSet: A controller that places a Pod on each eligible Node.

Deployment: A controller that manages ReplicaSets and declarative rollouts for interchangeable Pods.

etcd: The consistent key-value store holding Kubernetes API state.

GitOps: Operations through version-controlled declarations continuously reconciled by software.

HPA: Horizontal Pod Autoscaler; a controller that changes workload replica counts from metrics.

Image: An immutable package of application files, runtime, dependencies, and metadata.

ImageStream: An OpenShift abstraction that tracks image tags and can participate in triggers.

Ingress: A Kubernetes resource expressing HTTP/HTTPS routing rules into Services.

kubelet: The Node agent responsible for realizing assigned Pod specifications.

Kustomize: A manifest customization system based on bases, overlays, patches, and transformations.

LoadBalancer: A Service type that requests externally reachable load balancing from integrated infrastructure.

Namespace: A logical scope for names, access, policy, and resource governance.

NetworkPolicy: A policy selecting Pods and declaring allowed ingress and/or egress traffic.

Node: A registered machine that runs Pods.

Operator: A domain-specific controller, often using Custom Resources, that automates operational knowledge.

Pod: The smallest schedulable Kubernetes unit, containing one or more tightly coupled containers.

PVC: A PersistentVolumeClaim: a workload's request for persistent storage.

PV: A PersistentVolume: a cluster storage resource bound to a claim.

RBAC: Role-Based Access Control for authorizing Kubernetes API actions.

ReplicaSet: A controller maintaining a desired count of matching Pods.

Route: An OpenShift object that configures external HTTP/HTTPS exposure through the Router.

SCC: Security Context Constraints, OpenShift policy controlling workload security settings.

Scheduler: The control-plane component that selects a Node for each unscheduled Pod.

Secret: A Kubernetes object for sensitive values, with protection dependent on cluster and application controls.

Service: A stable discovery and routing abstraction for a changing set of Pod endpoints.

StatefulSet: A controller providing stable ordinal identity and storage patterns for stateful replicas.

StorageClass: A storage profile that can drive dynamic PV provisioning.

Taint: A Node property that repels Pods without a matching toleration.

Toleration: A Pod rule allowing—but not forcing—placement on a matching tainted Node.

Source Learning Map

Original learning discussion	Reconstructed book location
Containers and why Kubernetes exists	Chapters 1-3
Pods, ReplicaSets, Deployments, labels, selectors	Chapters 4-5
Jobs and CronJobs	Chapter 6
Networking, Services, DNS, Ingress, Routes, policy	Chapters 7-9
ConfigMaps and Secrets	Chapter 10
Volumes, PV, PVC, StorageClass, StatefulSets	Chapters 11-13
Probes and graceful shutdown	Chapters 14-15
CPU, memory, quotas, autoscaling, scheduling	Chapters 16-19
RBAC, ServiceAccounts, SecurityContext, SCC	Chapters 20-21 and 27
Helm, Kustomize, GitOps, Argo CD	Chapters 22-24
OpenShift architecture, Projects, Routes, Builds, ImageStreams	Chapters 25-27
Operators, CRDs, OLM	Chapter 28
Monitoring, logging, tracing, troubleshooting	Chapters 29-32
High availability, upgrades, disaster recovery	Chapter 32
End-to-end production architecture	Chapter 33

The source was treated as the primary authority. The Operator bridge and the final integrated revision material are explicitly supplementary where they connect concepts that the source introduced but did not develop to equal depth.